In [4]:
```
print("Mohammed Abdul Rahman")
print("23BDS0286")
```

```
Mohammed Abdul Rahman
23BDS0286
```

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Crime.csv"
df = pd.read_csv(url)

print(df.head())
print(df.shape)
print(df.columns.tolist())
```

```
   rownames  county  year    crmrte    prbarr   prbconv   prbpris  avgsen  \
0         1       1    81  0.039885  0.289696  0.402062  0.472222    5.61
1         2       1    82  0.038345  0.338111  0.433005  0.506993    5.59
2         3       1    83  0.030305  0.330449  0.525703  0.479705    5.80
3         4       1    84  0.034726  0.362525  0.604706  0.520104    6.89
4         5       1    85  0.036573  0.325395  0.578723  0.497059    6.55

      polpc   density  ...     lwfir     lwser     lwmfg     lwfed     lwsta  \
0  0.001787  2.307159  ...  5.607452  5.374044  5.434246  6.014619  5.464848
1  0.001767  2.330254  ...  5.706707  5.444911  5.482013  6.039540  5.536862
2  0.001836  2.341801  ...  5.736475  5.481292  5.597310  6.084157  5.522900
3  0.001886  2.346420  ...  5.858180  5.531204  5.640985  6.129421  5.568077
4  0.001924  2.364896  ...  5.948220  5.564850  5.700042  6.195282  5.639919

      lwloc  lpctymle   lpctmin    ltaxpc      lmix
0  5.444450 -2.433870  3.006608  3.246399 -2.303407
1  5.467174 -2.449038  3.006608  3.213833 -2.272549
2  5.515765 -2.464036  3.006608  3.275311 -2.517281
3  5.577387 -2.478925  3.006608  3.289981 -2.544612
4  5.664972 -2.497306  3.006608  3.337204 -2.372487

[5 rows x 45 columns]
(630, 45)
['rownames', 'county', 'year', 'crmrte', 'prbarr', 'prbconv', 'prbpris', 'avgsen',
'polpc', 'density', 'taxpc', 'region', 'smsa', 'pctmin', 'wcon', 'wtuc', 'wtrd', 'wf
ir', 'wser', 'wmfg', 'wfed', 'wsta', 'wloc', 'mix', 'pctymle', 'lcrmrte', 'lprbarr',
'lprbconv', 'lprbpris', 'lavgsen', 'lpolpc', 'ldensity', 'lwcon', 'lwtuc', 'lwtrd',
'lwfir', 'lwser', 'lwmfg', 'lwfed', 'lwsta', 'lwloc', 'lpctymle', 'lpctmin', 'ltaxp
c', 'lmix']
```

In [5]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew, kurtosis, pearsonr, spearmanr
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
```

In [6]:
```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nDataset information:")
df.info()
```

```
Number of rows: 630
Number of columns: 45

Column names:
['rownames', 'county', 'year', 'crmrte', 'prbarr', 'prbconv', 'prbpris', 'avgsen',
'polpc', 'density', 'taxpc', 'region', 'smsa', 'pctmin', 'wcon', 'wtuc', 'wtrd', 'wf
ir', 'wser', 'wmfg', 'wfed', 'wsta', 'wloc', 'mix', 'pctymle', 'lcrmrte', 'lprbarr',
'lprbconv', 'lprbpris', 'lavgsen', 'lpolpc', 'ldensity', 'lwcon', 'lwtuc', 'lwtrd',
'lwfir', 'lwser', 'lwmfg', 'lwfed', 'lwsta', 'lwloc', 'lpctymle', 'lpctmin', 'ltaxp
c', 'lmix']

Data types:
rownames      int64
county        int64
year          int64
crmrte      float64
prbarr      float64
prbconv     float64
prbpris     float64
avgsen      float64
polpc       float64
density     float64
taxpc       float64
region          str
smsa            str
pctmin      float64
wcon        float64
wtuc        float64
wtrd        float64
wfir        float64
wser        float64
wmfg        float64
wfed        float64
wsta        float64
wloc        float64
mix         float64
pctymle     float64
lcrmrte     float64
lprbarr     float64
lprbconv    float64
lprbpris    float64
lavgsen     float64
lpolpc      float64
ldensity    float64
lwcon       float64
lwtuc       float64
lwtrd       float64
lwfir       float64
lwser       float64
lwmfg       float64
lwfed       float64
lwsta       float64
lwloc       float64
lpctymle    float64
lpctmin     float64
ltaxpc      float64
```

```
lmix        float64
dtype: object

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 630 entries, 0 to 629
Data columns (total 45 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   rownames  630 non-null    int64
 1   county    630 non-null    int64
 2   year      630 non-null    int64
 3   crmrte    630 non-null    float64
 4   prbarr    630 non-null    float64
 5   prbconv   630 non-null    float64
 6   prbpris   630 non-null    float64
 7   avgsen    630 non-null    float64
 8   polpc     630 non-null    float64
 9   density   630 non-null    float64
 10  taxpc     630 non-null    float64
 11  region    630 non-null    str
 12  smsa      630 non-null    str
 13  pctmin    630 non-null    float64
 14  wcon      630 non-null    float64
 15  wtuc      630 non-null    float64
 16  wtrd      630 non-null    float64
 17  wfir      630 non-null    float64
 18  wser      630 non-null    float64
 19  wmfg      630 non-null    float64
 20  wfed      630 non-null    float64
 21  wsta      630 non-null    float64
 22  wloc      630 non-null    float64
 23  mix       630 non-null    float64
 24  pctymle   630 non-null    float64
 25  lcrmrte   630 non-null    float64
 26  lprbarr   630 non-null    float64
 27  lprbconv  630 non-null    float64
 28  lprbpris  630 non-null    float64
 29  lavgsen   630 non-null    float64
 30  lpolpc    630 non-null    float64
 31  ldensity  630 non-null    float64
 32  lwcon     630 non-null    float64
 33  lwtuc     630 non-null    float64
 34  lwtrd     630 non-null    float64
 35  lwfir     630 non-null    float64
 36  lwser     630 non-null    float64
 37  lwmfg     630 non-null    float64
 38  lwfed     630 non-null    float64
 39  lwsta     630 non-null    float64
 40  lwloc     630 non-null    float64
 41  lpctymle  630 non-null    float64
 42  lpctmin   630 non-null    float64
 43  ltaxpc    630 non-null    float64
 44  lmix      630 non-null    float64
dtypes: float64(40), int64(3), str(2)
memory usage: 221.6 KB
```

In [7]:
```
display(df.describe(include="all").T)
```

	count	unique	top	freq	mean	std	min	25%	5
rownames	630.0	NaN	NaN	NaN	315.5	182.009615	1.0	158.25	3
county	630.0	NaN	NaN	NaN	100.6	58.03627	1.0	51.0	1(
year	630.0	NaN	NaN	NaN	84.0	2.001589	81.0	82.0	8
crmrte	630.0	NaN	NaN	NaN	0.031588	0.018121	0.001812	0.018352	0.028
prbarr	630.0	NaN	NaN	NaN	0.307368	0.171205	0.058824	0.217902	0.27
prbconv	630.0	NaN	NaN	NaN	0.688618	1.690345	0.068376	0.347692	0.474
prbpris	630.0	NaN	NaN	NaN	0.425518	0.087245	0.148936	0.374403	0.428
avgsen	630.0	NaN	NaN	NaN	8.95454	2.658082	4.22	7.16	8.
polpc	630.0	NaN	NaN	NaN	0.001917	0.002735	0.000458	0.001191	0.001
density	630.0	NaN	NaN	NaN	1.386062	1.439703	0.197719	0.532944	0.952
taxpc	630.0	NaN	NaN	NaN	30.239194	11.454695	14.30256	23.425595	27.79
region	630	3	other	245	NaN	NaN	NaN	NaN	N
smsa	630	2	no	574	NaN	NaN	NaN	NaN	N
pctmin	630.0	NaN	NaN	NaN	25.712852	16.903536	1.28365	10.0046	24.85
wcon	630.0	NaN	NaN	NaN	245.666079	121.98372	65.62158	201.660775	236.4
wtuc	630.0	NaN	NaN	NaN	406.1028	266.513781	28.8577	317.598325	358.2
wtrd	630.0	NaN	NaN	NaN	192.823065	88.407259	16.87376	168.04955	185.4
wfir	630.0	NaN	NaN	NaN	272.059255	55.768084	3.51568	235.704775	264.4
wser	630.0	NaN	NaN	NaN	224.670455	104.8667	1.843794	191.31915	216.4
wmfg	630.0	NaN	NaN	NaN	285.170095	82.368065	101.83	234.03	271.
wfed	630.0	NaN	NaN	NaN	403.895889	63.066692	255.4	361.525	40.
wsta	630.0	NaN	NaN	NaN	296.90746	53.431614	173.02	258.1575	28
wloc	630.0	NaN	NaN	NaN	257.976206	41.358021	163.59	226.785	253.
mix	630.0	NaN	NaN	NaN	0.139396	0.219755	0.002457	0.075324	0.102
pctymle	630.0	NaN	NaN	NaN	0.088974	0.024349	0.062158	0.078594	0.083
lcrmrte	630.0	NaN	NaN	NaN	-3.609225	0.572808	-6.31355	-3.998036	-3.559
lprbarr	630.0	NaN	NaN	NaN	-1.274264	0.415897	-2.833214	-1.523711	-1.279
lprbconv	630.0	NaN	NaN	NaN	-0.692919	0.609595	-2.682732	-1.056439	-0.745
lprbpris	630.0	NaN	NaN	NaN	-0.878632	0.230514	-1.904239	-0.982423	-0.847
lavgsen	630.0	NaN	NaN	NaN	2.153344	0.273729	1.439835	1.96851	2.139

	count	unique	top	freq	mean	std	min	25%	5
lpolpc	630.0	NaN	NaN	NaN	-6.490637	0.526654	-7.687507	-6.732704	-6.535
ldensity	630.0	NaN	NaN	NaN	-0.015927	0.774735	-1.62091	-0.629341	-0.048
lwcon	630.0	NaN	NaN	NaN	5.462869	0.248178	4.183905	5.306587	5.465
lwtuc	630.0	NaN	NaN	NaN	5.915883	0.370219	3.362377	5.760787	5.881
lwtrd	630.0	NaN	NaN	NaN	5.232423	0.214391	2.82576	5.124259	5.222
lwfir	630.0	NaN	NaN	NaN	5.579433	0.277204	1.257233	5.462579	5.57
lwser	630.0	NaN	NaN	NaN	5.364625	0.360098	0.611825	5.253943	5.377
lwmfg	630.0	NaN	NaN	NaN	5.615181	0.272747	4.623305	5.45545	5.604
lwfed	630.0	NaN	NaN	NaN	5.988757	0.158761	5.542831	5.89033	6.001
lwsta	630.0	NaN	NaN	NaN	5.677787	0.176131	5.153407	5.553569	5.667
lwloc	630.0	NaN	NaN	NaN	5.540139	0.159691	5.097363	5.424002	5.533
lpctymle	630.0	NaN	NaN	NaN	-2.443015	0.196784	-2.77808	-2.543456	-2.48
lpctmin	630.0	NaN	NaN	NaN	2.913361	0.954615	0.249708	2.303045	3.212
ltaxpc	630.0	NaN	NaN	NaN	3.356385	0.308224	2.660439	3.153829	3.324
lmix	630.0	NaN	NaN	NaN	-2.234233	0.649565	-6.008814	-2.585961	-2.281

In [8]:
```
print("Assessment 1 — DataFrame operations, merging, and normalization")
```

```
Assessment 1 — DataFrame operations, merging, and normalization
```

In [9]:
```
display(df.select_dtypes(include=np.number).head())

numeric = df.select_dtypes(include=np.number)

if not numeric.empty:
    column = numeric.columns[0]
    display(df.loc[df[column] > df[column].median()].head())
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	dens
0	1	1	81	0.039885	0.289696	0.402062	0.472222	5.61	0.001787	2.307
1	2	1	82	0.038345	0.338111	0.433005	0.506993	5.59	0.001767	2.330
2	3	1	83	0.030305	0.330449	0.525703	0.479705	5.80	0.001836	2.341
3	4	1	84	0.034726	0.362525	0.604706	0.520104	6.89	0.001886	2.346
4	5	1	85	0.036573	0.325395	0.578723	0.497059	6.55	0.001924	2.364

5 rows × 43 columns

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	de
315	316	105	81	0.062679	0.255328	0.437396	0.465649	7.51	0.001844	1.4
316	317	105	82	0.057929	0.278614	0.523732	0.521875	4.61	0.001770	1.4
317	318	105	83	0.048286	0.266919	0.476673	0.519149	7.60	0.001804	1.4
318	319	105	84	0.035448	0.300000	0.459854	0.486772	4.32	0.001811	1.5
319	320	105	85	0.044593	0.268571	0.493617	0.495690	6.91	0.001835	1.5

5 rows × 45 columns

In [10]:
```
if not numeric.empty:
    column = numeric.columns[0]
    display(df.sort_values(column, ascending=False).head())

    df["Scaled " + column] = (
        (df[column] - df[column].min()) /
        (df[column].max() - df[column].min())
    )

    display(df[[column, "Scaled " + column]].head())
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	de
629	630	197	87	0.014193	0.207595	1.182930	0.360825	12.23	0.001186	0.8
628	629	197	86	0.012874	0.112676	2.250000	0.244444	7.68	0.001088	0.8
627	628	197	85	0.013086	0.180556	0.830769	0.333333	5.96	0.001054	0.8
626	627	197	84	0.013662	0.204188	1.410260	0.372727	10.11	0.001109	0.8
625	626	197	83	0.015575	0.226667	0.480392	0.428571	7.77	0.001073	0.8

5 rows × 45 columns

	rownames	Scaled rownames
0	1	0.000000
1	2	0.001590
2	3	0.003180
3	4	0.004769
4	5	0.006359

In [11]:
```
categorical = df.select_dtypes(include=["object", "category"])

if not categorical.empty and not numeric.empty:
    category = categorical.columns[0]
    value = numeric.columns[0]

    summary = df.groupby(category, dropna=False)[value].agg(
        ["count", "mean", "median", "min", "max"]
    )

    display(summary)
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\3024388624.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = df.select_dtypes(include=["object", "category"])
```

region	count	mean	median	min	max
central	238	310.558824	308.5	1	588
other	245	314.800000	312.0	43	623
west	147	324.666667	340.0	15	630

In [12]:
```
if len(df.columns) > 0:
    column = df.columns[0]

    first = df.iloc[:, :min(3, len(df.columns))].copy()
    second = df.iloc[:, :min(3, len(df.columns))].copy()

    merged = pd.merge(first, second, on=column, how="inner")

    display(merged.head())
```

	rownames	county_x	year_x	county_y	year_y
0	1	1	81	1	81
1	2	1	82	1	82
2	3	1	83	1	83
3	4	1	84	1	84
4	5	1	85	1	85

In [13]:
```
if not numeric.empty:
    values = numeric.dropna()

    if not values.empty:
        standard = StandardScaler().fit_transform(values)
        normalized = MinMaxScaler().fit_transform(values)

        standarddf = pd.DataFrame(
            standard, columns=values.columns, index=values.index
        )

        normalizeddf = pd.DataFrame(
            normalized, columns=values.columns, index=values.index
        )

        print("Standardized data")
        display(standarddf.head())

        print("Min-max normalized data")
        display(normalizeddf.head())
```

```
Standardized data
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polp
0	-1.729304	-1.717532	-1.5	0.458251	-0.103305	-0.169660	0.535740	-1.259253	-0.04756
1	-1.723805	-1.717532	-1.0	0.373198	0.179710	-0.151339	0.934600	-1.266783	-0.05496
2	-1.718307	-1.717532	-0.5	-0.070847	0.134921	-0.096456	0.621577	-1.187716	-0.02963
3	-1.712808	-1.717532	0.0	0.173325	0.322425	-0.049681	1.084997	-0.777320	-0.01130
4	-1.707309	-1.717532	0.5	0.275338	0.105377	-0.065065	0.820646	-0.905333	0.00278

5 rows × 43 columns

```
Min-max normalized data
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc
0	0.000000	0.0	0.000000	0.234986	0.085789	0.009035	0.610394	0.064322	0.037822
1	0.001590	0.0	0.166667	0.225482	0.103779	0.009873	0.676045	0.063397	0.037247
2	0.003180	0.0	0.333333	0.175859	0.100932	0.012383	0.624523	0.073114	0.039217
3	0.004769	0.0	0.500000	0.203145	0.112851	0.014522	0.700800	0.123554	0.040644
4	0.006359	0.0	0.666667	0.214546	0.099054	0.013819	0.657289	0.107820	0.041740

5 rows × 43 columns

In [14]:
```
print("Assessment 2 – Data inspection, cleaning, and descriptive statistics")
```

```
Assessment 2 – Data inspection, cleaning, and descriptive statistics
```

In [15]:
```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nDataset information:")
df.info()

print("\nFirst five rows:")
display(df.head())
```

```
Dataset shape: (630, 46)

Column names:
['rownames', 'county', 'year', 'crmrte', 'prbarr', 'prbconv', 'prbpris', 'avgsen',
'polpc', 'density', 'taxpc', 'region', 'smsa', 'pctmin', 'wcon', 'wtuc', 'wtrd', 'wf
ir', 'wser', 'wmfg', 'wfed', 'wsta', 'wloc', 'mix', 'pctymle', 'lcrmrte', 'lprbarr',
'lprbconv', 'lprbpris', 'lavgsen', 'lpolpc', 'ldensity', 'lwcon', 'lwtuc', 'lwtrd',
'lwfir', 'lwser', 'lwmfg', 'lwfed', 'lwsta', 'lwloc', 'lpctymle', 'lpctmin', 'ltaxp
c', 'lmix', 'Scaled rownames']

Data types:
rownames             int64
county               int64
year                 int64
crmrte             float64
prbarr             float64
prbconv            float64
prbpris            float64
avgsen             float64
polpc              float64
density            float64
taxpc              float64
region                 str
smsa                   str
pctmin             float64
wcon               float64
wtuc               float64
wtrd               float64
wfir               float64
wser               float64
wmfg               float64
wfed               float64
wsta               float64
wloc               float64
mix                float64
pctymle            float64
lcrmrte            float64
lprbarr            float64
lprbconv           float64
lprbpris           float64
lavgsen            float64
lpolpc             float64
ldensity           float64
lwcon              float64
lwtuc              float64
lwtrd              float64
lwfir              float64
lwser              float64
lwmfg              float64
lwfed              float64
lwsta              float64
lwloc              float64
lpctymle           float64
lpctmin            float64
ltaxpc             float64
lmix               float64
```

```
Scaled rownames    float64
dtype: object

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 630 entries, 0 to 629
Data columns (total 46 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   rownames         630 non-null    int64
 1   county           630 non-null    int64
 2   year             630 non-null    int64
 3   crmrte           630 non-null    float64
 4   prbarr           630 non-null    float64
 5   prbconv          630 non-null    float64
 6   prbpris          630 non-null    float64
 7   avgsen           630 non-null    float64
 8   polpc            630 non-null    float64
 9   density          630 non-null    float64
 10  taxpc            630 non-null    float64
 11  region           630 non-null    str
 12  smsa             630 non-null    str
 13  pctmin           630 non-null    float64
 14  wcon             630 non-null    float64
 15  wtuc             630 non-null    float64
 16  wtrd             630 non-null    float64
 17  wfir             630 non-null    float64
 18  wser             630 non-null    float64
 19  wmfg             630 non-null    float64
 20  wfed             630 non-null    float64
 21  wsta             630 non-null    float64
 22  wloc             630 non-null    float64
 23  mix              630 non-null    float64
 24  pctymle          630 non-null    float64
 25  lcrmrte          630 non-null    float64
 26  lprbarr          630 non-null    float64
 27  lprbconv         630 non-null    float64
 28  lprbpris         630 non-null    float64
 29  lavgsen          630 non-null    float64
 30  lpolpc           630 non-null    float64
 31  ldensity         630 non-null    float64
 32  lwcon            630 non-null    float64
 33  lwtuc            630 non-null    float64
 34  lwtrd            630 non-null    float64
 35  lwfir            630 non-null    float64
 36  lwser            630 non-null    float64
 37  lwmfg            630 non-null    float64
 38  lwfed            630 non-null    float64
 39  lwsta            630 non-null    float64
 40  lwloc            630 non-null    float64
 41  lpctymle         630 non-null    float64
 42  lpctmin          630 non-null    float64
 43  ltaxpc           630 non-null    float64
 44  lmix             630 non-null    float64
 45  Scaled rownames  630 non-null    float64
dtypes: float64(41), int64(3), str(2)
```

```
memory usage: 226.5 KB
```

```
First five rows:
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	dens
0	1	1	81	0.039885	0.289696	0.402062	0.472222	5.61	0.001787	2.307
1	2	1	82	0.038345	0.338111	0.433005	0.506993	5.59	0.001767	2.330
2	3	1	83	0.030305	0.330449	0.525703	0.479705	5.80	0.001836	2.341
3	4	1	84	0.034726	0.362525	0.604706	0.520104	6.89	0.001886	2.346
4	5	1	85	0.036573	0.325395	0.578723	0.497059	6.55	0.001924	2.364

5 rows × 46 columns

In [16]:
```
missing = df.isnull().sum()

print("Missing values in each column:")
display(missing.to_frame("Missing Values"))

print("Total missing values:", df.isnull().sum().sum())

print("Missing value percentage:")
display((df.isnull().mean() * 100).round(2).to_frame("Missing Percentage"))
```

```
Missing values in each column:
```

	Missing Values
rownames	0
county	0
year	0
crmrte	0
prbarr	0
prbconv	0
prbpris	0
avgsen	0
polpc	0
density	0
taxpc	0
region	0
smsa	0
pctmin	0
wcon	0
wtuc	0
wtrd	0
wfir	0
wser	0
wmfg	0
wfed	0
wsta	0
wloc	0
mix	0
pctymle	0
lcrmrte	0
lprbarr	0
lprbconv	0
lprbpris	0
lavgsen	0

	Missing Values
lpolpc	0
ldensity	0
lwcon	0
lwtuc	0
lwtrd	0
lwfir	0
lwser	0
lwmfg	0
lwfed	0
lwsta	0
lwloc	0
lpctymle	0
lpctmin	0
ltaxpc	0
lmix	0
Scaled rownames	0

```
Total missing values: 0
Missing value percentage:
```

	Missing Percentage
rownames	0.0
county	0.0
year	0.0
crmrte	0.0
prbarr	0.0
prbconv	0.0
prbpris	0.0
avgsen	0.0
polpc	0.0
density	0.0
taxpc	0.0
region	0.0
smsa	0.0
pctmin	0.0
wcon	0.0
wtuc	0.0
wtrd	0.0
wfir	0.0
wser	0.0
wmfg	0.0
wfed	0.0
wsta	0.0
wloc	0.0
mix	0.0
pctymle	0.0
lcrmrte	0.0
lprbarr	0.0
lprbconv	0.0
lprbpris	0.0
lavgsen	0.0

	Missing Percentage
lpolpc	0.0
ldensity	0.0
lwcon	0.0
lwtuc	0.0
lwtrd	0.0
lwfir	0.0
lwser	0.0
lwmfg	0.0
lwfed	0.0
lwsta	0.0
lwloc	0.0
lpctymle	0.0
lpctmin	0.0
ltaxpc	0.0
lmix	0.0
Scaled rownames	0.0

In [17]:
```
clean = df.copy()

for column in clean.select_dtypes(include=np.number).columns:
    clean[column] = clean[column].fillna(clean[column].median())

for column in clean.select_dtypes(include=["object", "category"]).columns:
    if clean[column].notna().any():
        clean[column] = clean[column].fillna(clean[column].mode()[0])

print("Missing values after cleaning:")
display(clean.isnull().sum().to_frame("Missing Values"))
```

```
Missing values after cleaning:
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\2557237905.py:6: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  for column in clean.select_dtypes(include=["object", "category"]).columns:
```

	Missing Values
rownames	0
county	0
year	0
crmrte	0
prbarr	0
prbconv	0
prbpris	0
avgsen	0
polpc	0
density	0
taxpc	0
region	0
smsa	0
pctmin	0
wcon	0
wtuc	0
wtrd	0
wfir	0
wser	0
wmfg	0
wfed	0
wsta	0
wloc	0
mix	0
pctymle	0
lcrmrte	0
lprbarr	0
lprbconv	0
lprbpris	0
lavgsen	0

	Missing Values
lpolpc	0
ldensity	0
lwcon	0
lwtuc	0
lwtrd	0
lwfir	0
lwser	0
lwmfg	0
lwfed	0
lwsta	0
lwloc	0
lpctymle	0
lpctmin	0
ltaxpc	0
lmix	0
Scaled rownames	0

In [18]:
```
print("Duplicate rows:", clean.duplicated().sum())

clean = clean.drop_duplicates().reset_index(drop=True)

print("Shape after removing duplicates:", clean.shape)
```

```
Duplicate rows: 0
Shape after removing duplicates: (630, 46)
```

In [19]:
```
print("Numerical summary:")
display(clean.describe().T)

print("Categorical summary:")
display(clean.describe(include=["object", "category"]).T)
```

```
Numerical summary:
```

	count	mean	std	min	25%	50%	75%	
rownames	630.0	315.500000	182.009615	1.000000	158.250000	315.500000	472.750000	
county	630.0	100.600000	58.036270	1.000000	51.000000	103.000000	151.000000	
year	630.0	84.000000	2.001589	81.000000	82.000000	84.000000	86.000000	
crmrte	630.0	0.031588	0.018121	0.001812	0.018352	0.028441	0.038406	
prbarr	630.0	0.307368	0.171205	0.058824	0.217902	0.278240	0.352518	
prbconv	630.0	0.688618	1.690345	0.068376	0.347692	0.474375	0.635597	
prbpris	630.0	0.425518	0.087245	0.148936	0.374403	0.428571	0.483189	
avgsen	630.0	8.954540	2.658082	4.220000	7.160000	8.495000	10.197500	
polpc	630.0	0.001917	0.002735	0.000458	0.001191	0.001451	0.001803	
density	630.0	1.386062	1.439703	0.197719	0.532944	0.952595	1.507818	
taxpc	630.0	30.239194	11.454695	14.302560	23.425595	27.792330	33.271223	
pctmin	630.0	25.712852	16.903536	1.283650	10.004600	24.851550	38.223000	
wcon	630.0	245.666079	121.983720	65.621580	201.660775	236.462200	269.693000	2
wtuc	630.0	406.102800	266.513781	28.857700	317.598325	358.202400	411.018800	3
wtrd	630.0	192.823065	88.407259	16.873760	168.049550	185.477700	204.821150	2
wfir	630.0	272.059255	55.768084	3.515680	235.704775	264.423100	302.439900	
wser	630.0	224.670455	104.866700	1.843794	191.319150	216.475300	247.155425	2
wmfg	630.0	285.170095	82.368065	101.830000	234.030000	271.595000	319.952500	
wfed	630.0	403.895889	63.066692	255.400000	361.525000	403.980000	444.550000	
wsta	630.0	296.907460	53.431614	173.020000	258.157500	289.370000	331.492500	
wloc	630.0	257.976206	41.358021	163.590000	226.785000	253.135000	289.272500	
mix	630.0	0.139396	0.219755	0.002457	0.075324	0.102089	0.149009	
pctymle	630.0	0.088974	0.024349	0.062158	0.078594	0.083165	0.089194	
lcrmrte	630.0	-3.609225	0.572808	-6.313550	-3.998036	-3.559938	-3.259543	
lprbarr	630.0	-1.274264	0.415897	-2.833214	-1.523711	-1.279271	-1.042653	
lprbconv	630.0	-0.692919	0.609595	-2.682732	-1.056439	-0.745758	-0.453193	
lprbpris	630.0	-0.878632	0.230514	-1.904239	-0.982423	-0.847299	-0.727346	
lavgsen	630.0	2.153344	0.273729	1.439835	1.968510	2.139477	2.322143	
lpolpc	630.0	-6.490637	0.526654	-7.687507	-6.732704	-6.535785	-6.318168	
ldensity	630.0	-0.015927	0.774735	-1.620910	-0.629341	-0.048567	0.410664	

	count	mean	std	min	25%	50%	75%
lwcon	630.0	5.462869	0.248178	4.183905	5.306587	5.465789	5.597283
lwtuc	630.0	5.915883	0.370219	3.362377	5.760787	5.881098	6.018639
lwtrd	630.0	5.232423	0.214391	2.825760	5.124259	5.222934	5.322137
lwfir	630.0	5.579433	0.277204	1.257233	5.462579	5.577550	5.711883
lwser	630.0	5.364625	0.360098	0.611825	5.253943	5.377477	5.510016
lwmfg	630.0	5.615181	0.272747	4.623305	5.455450	5.604311	5.768172
lwfed	630.0	5.988757	0.158761	5.542831	5.890330	6.001365	6.097062
lwsta	630.0	5.677787	0.176131	5.153407	5.553569	5.667706	5.803605
lwloc	630.0	5.540139	0.159691	5.097363	5.424002	5.533923	5.667369
lpctymle	630.0	-2.443015	0.196784	-2.778080	-2.543456	-2.486930	-2.416943
lpctmin	630.0	2.913361	0.954615	0.249708	2.303045	3.212685	3.643437
ltaxpc	630.0	3.356385	0.308224	2.660439	3.153829	3.324759	3.504693
lmix	630.0	-2.234233	0.649565	-6.008814	-2.585961	-2.281916	-1.903749
Scaled rownames	630.0	0.500000	0.289363	0.000000	0.250000	0.500000	0.750000

```
Categorical summary:
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\345463553.py:5: Pandas4Warn
ing: For backward compatibility, 'str' dtypes are included by select_dtypes when 'ob
ject' dtype is specified. This behavior is deprecated and will be removed in a futur
e version. Explicitly pass 'str' to `include` to select them, or to `exclude` to rem
ove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migrat
ion-select-dtypes for details on how to write code that works with pandas 2 and 3.
  display(clean.describe(include=["object", "category"]).T)
```

	count	unique	top	freq
region	630	3	other	245
smsa	630	2	no	574

In [20]:
```
categorical = clean.select_dtypes(include=["object", "category"])

for column in categorical.columns:
    print("Frequency distribution:", column)
    display(clean[column].value_counts(dropna=False).to_frame("Frequency"))
```

```
Frequency distribution: region
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\3709369366.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

region	Frequency
other	245
central	238
west	147

```
Frequency distribution: smsa
```

smsa	Frequency
no	574
yes	56

In [21]:
```
print("Original dataset shape:", df.shape)
print("Cleaned dataset shape:", clean.shape)

print("Original missing values:", df.isnull().sum().sum())
print("Remaining missing values:", clean.isnull().sum().sum())

print("Original duplicate rows:", df.duplicated().sum())
print("Remaining duplicate rows:", clean.duplicated().sum())
```

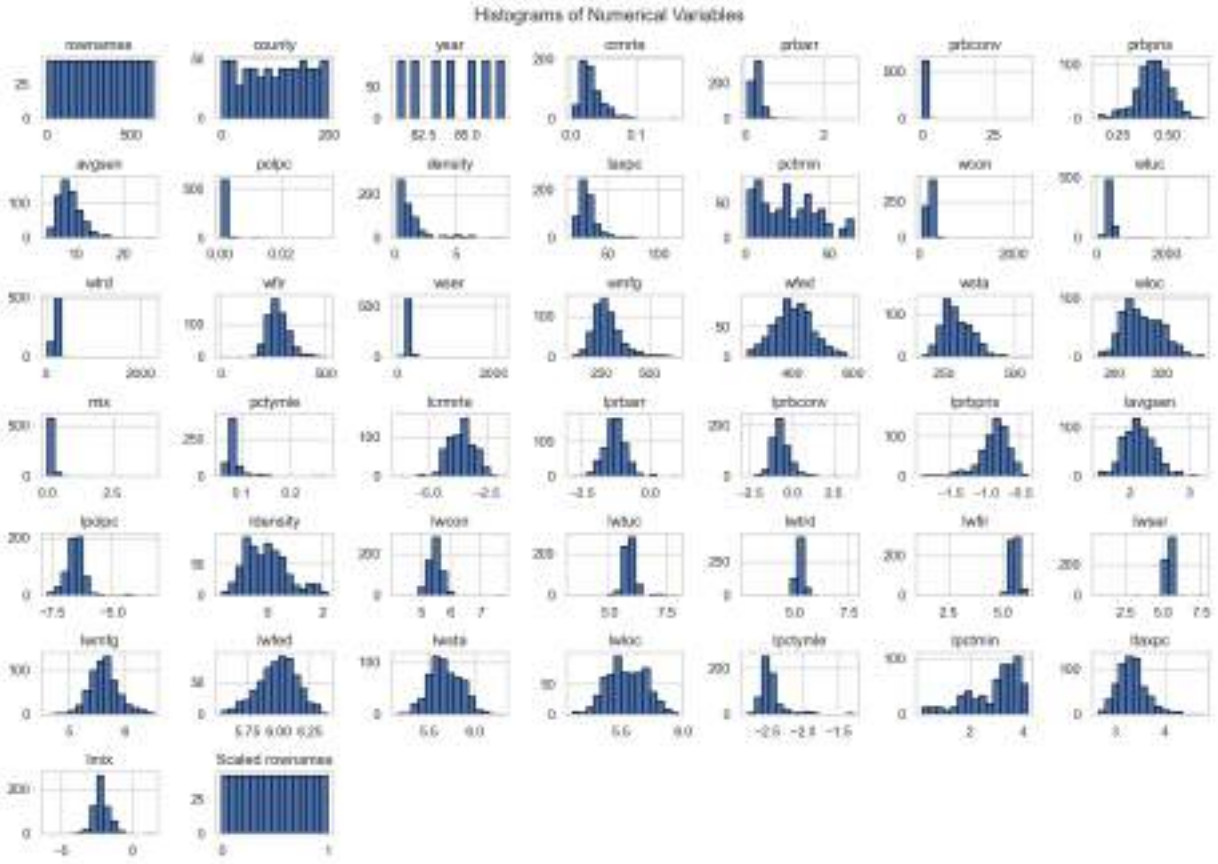
```
Original dataset shape: (630, 46)
Cleaned dataset shape: (630, 46)
Original missing values: 0
Remaining missing values: 0
Original duplicate rows: 0
Remaining duplicate rows: 0
```

In [23]:
```
print("Assessment 3 – Univariate analysis and outlier detection")
```

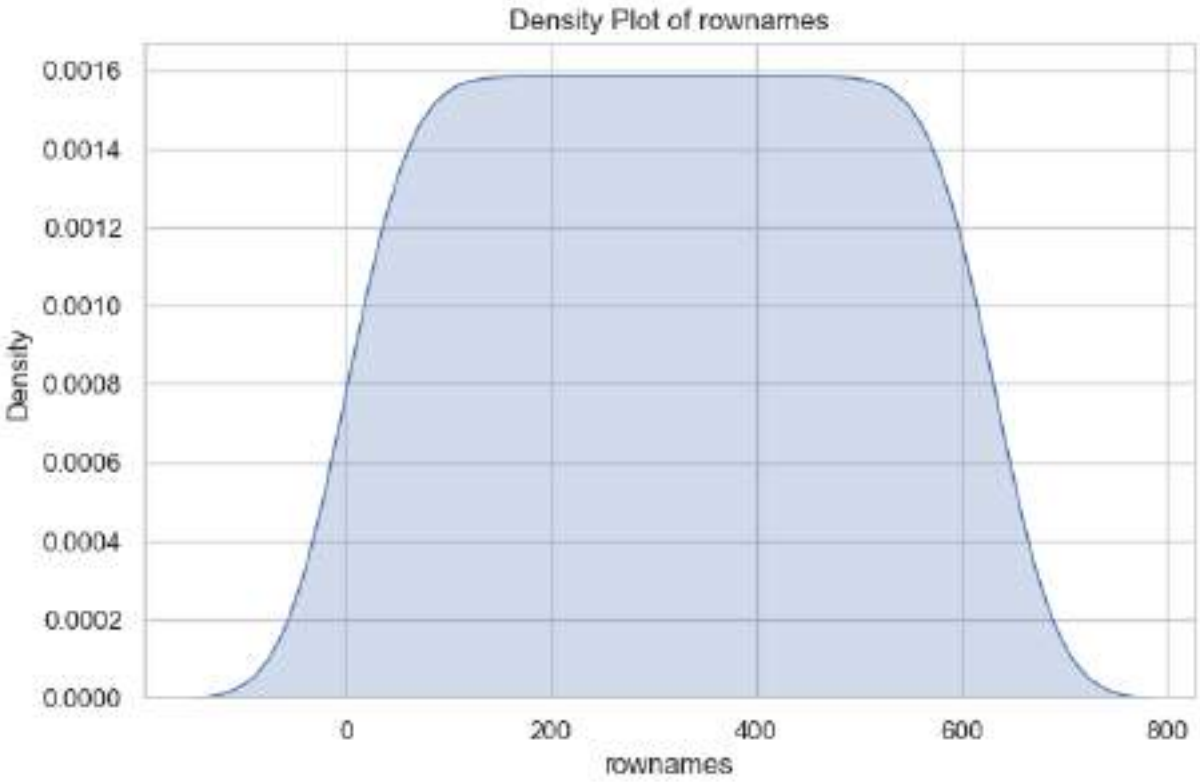
```
Assessment 3 – Univariate analysis and outlier detection
```

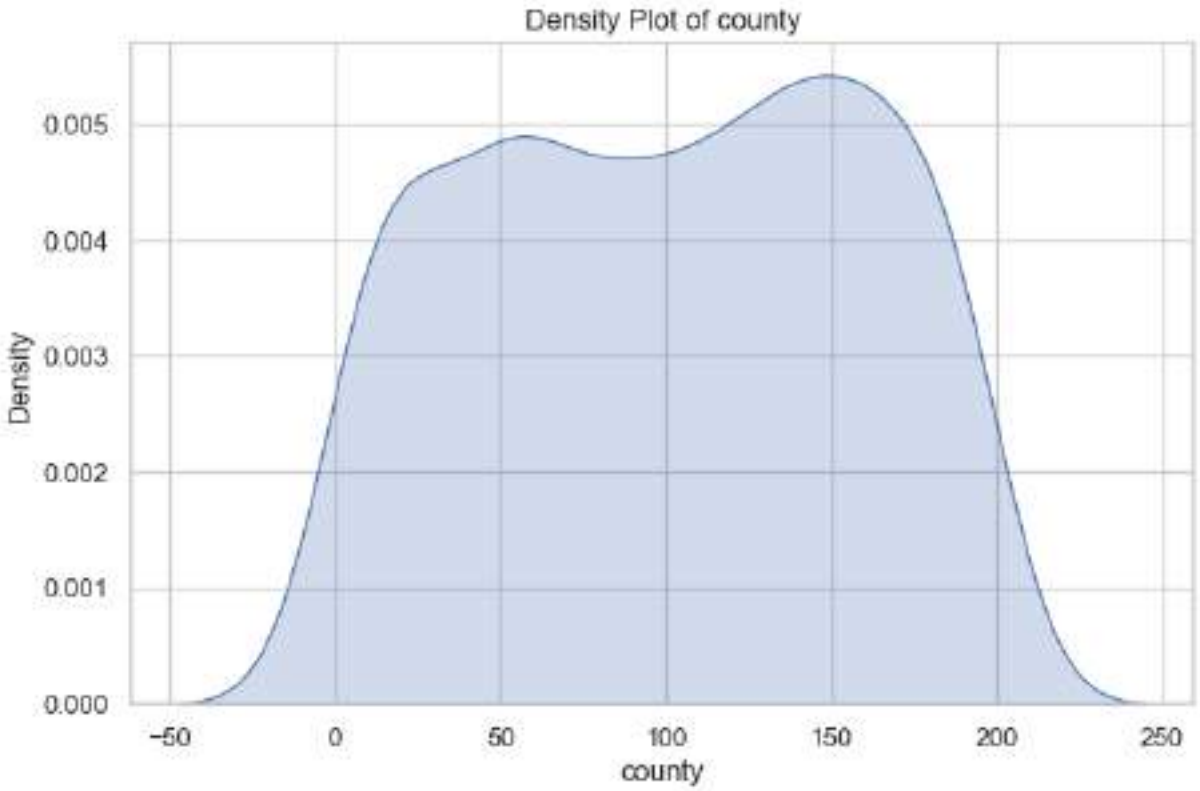
In [24]:
```
numeric = clean.select_dtypes(include=np.number)

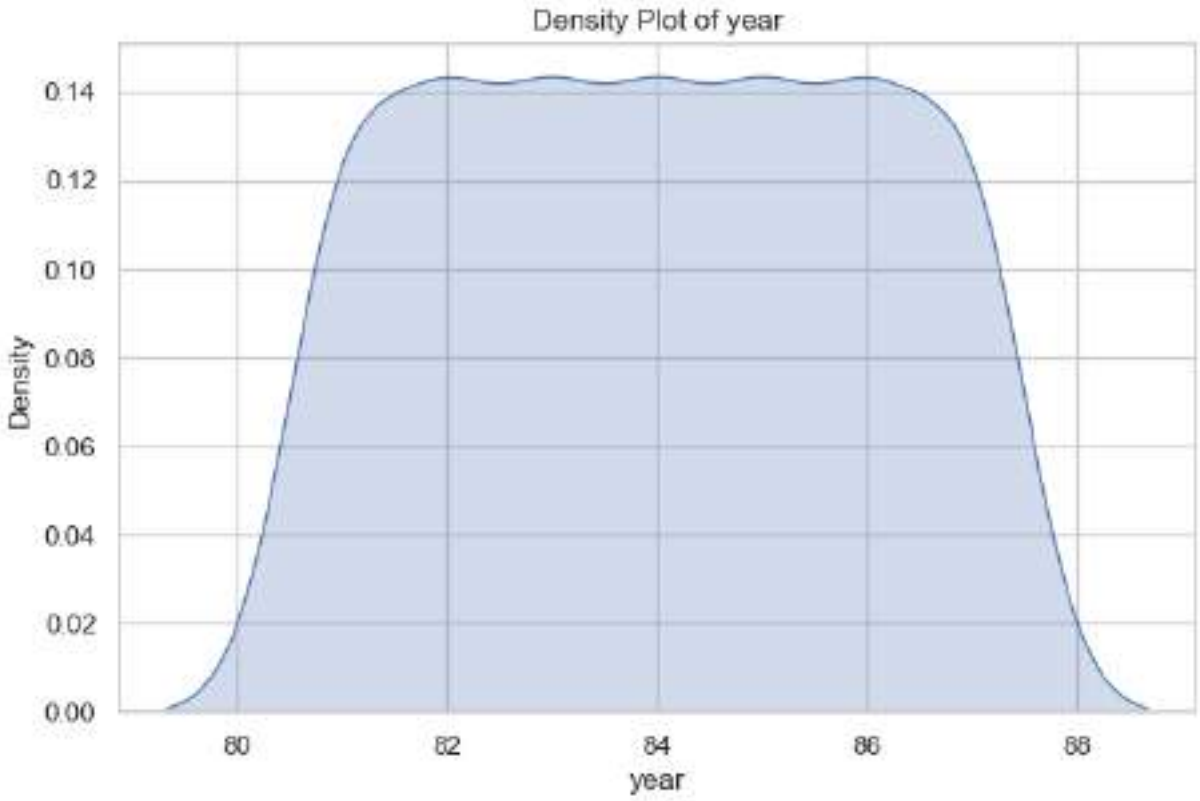
numeric.hist(bins=15, figsize=(14, 10), edgecolor="black")
plt.suptitle("Histograms of Numerical Variables")
plt.tight_layout()
plt.show()
```

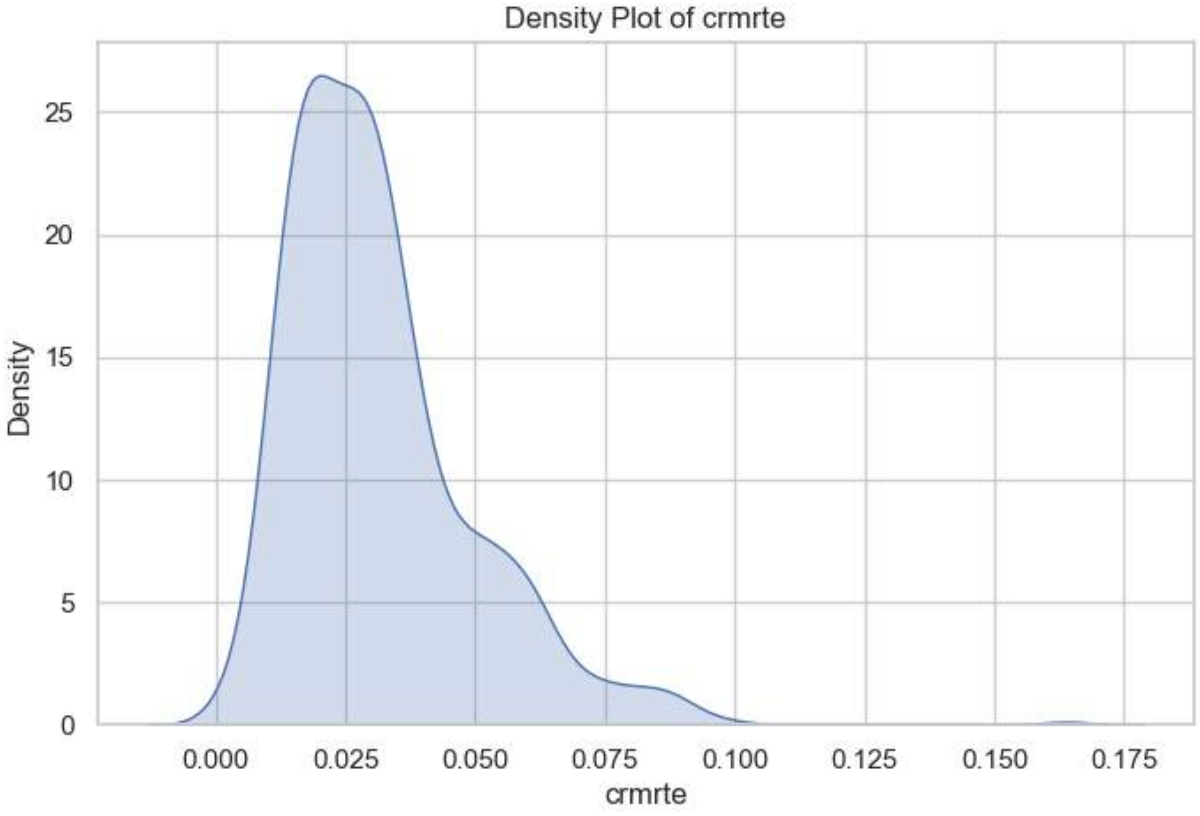


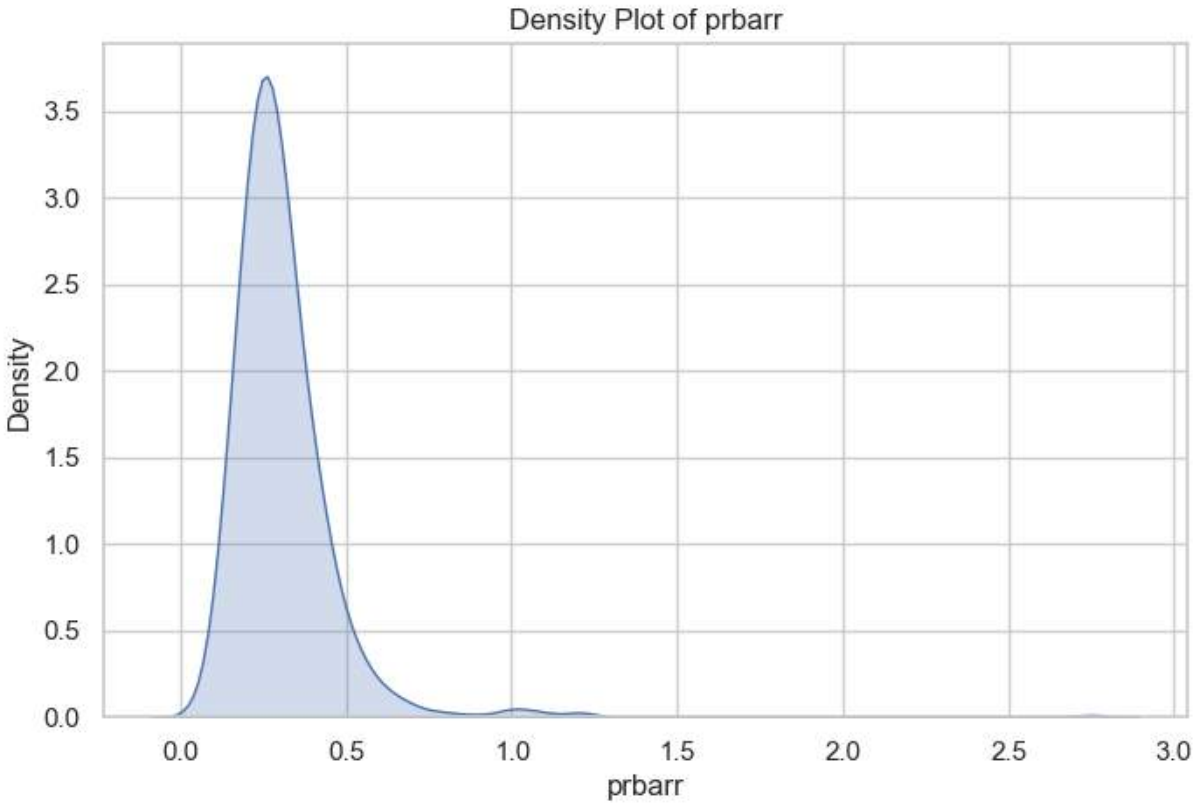
In [25]:
```
for column in numeric.columns:
    if numeric[column].nunique() > 1:
        plt.figure()
        sns.kdeplot(data=clean, x=column, fill=True)
        plt.title("Density Plot of " + column)
        plt.xlabel(column)
        plt.ylabel("Density")
        plt.show()
```

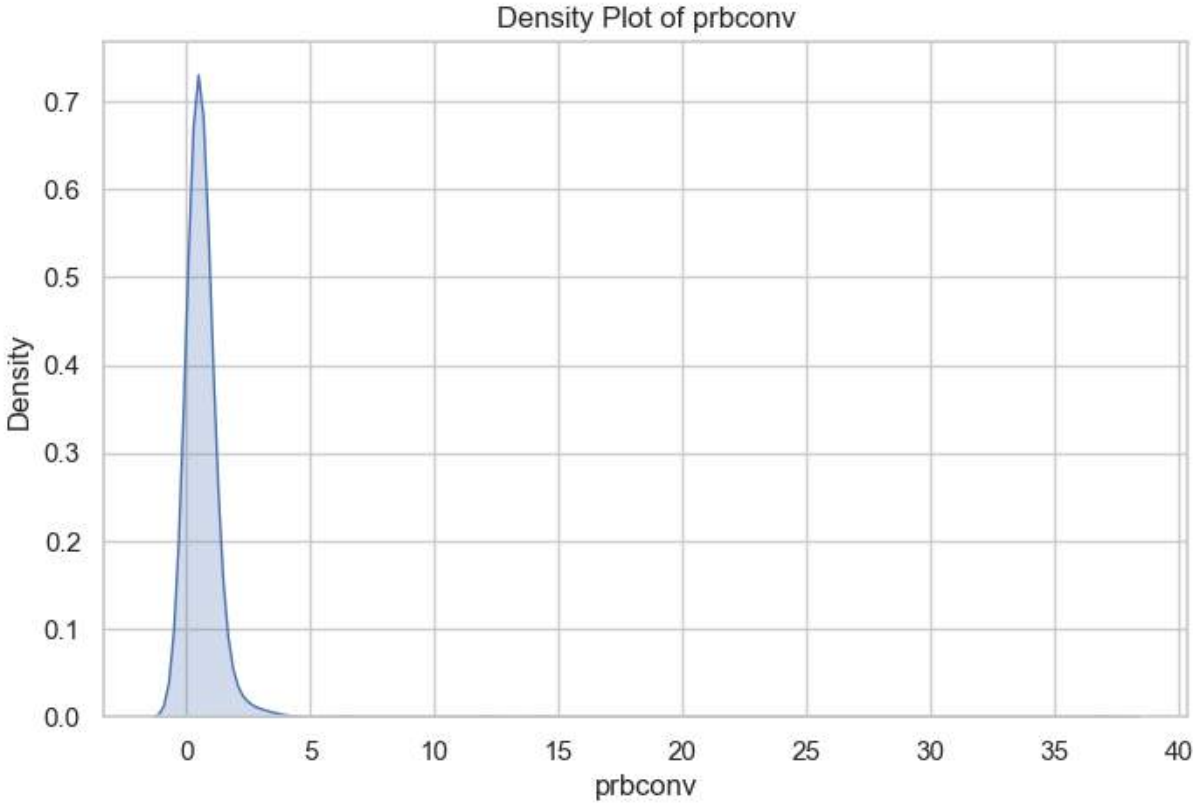

Density Plot of rownames
Density
0.0016
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
0
200
400
600
800
rownames

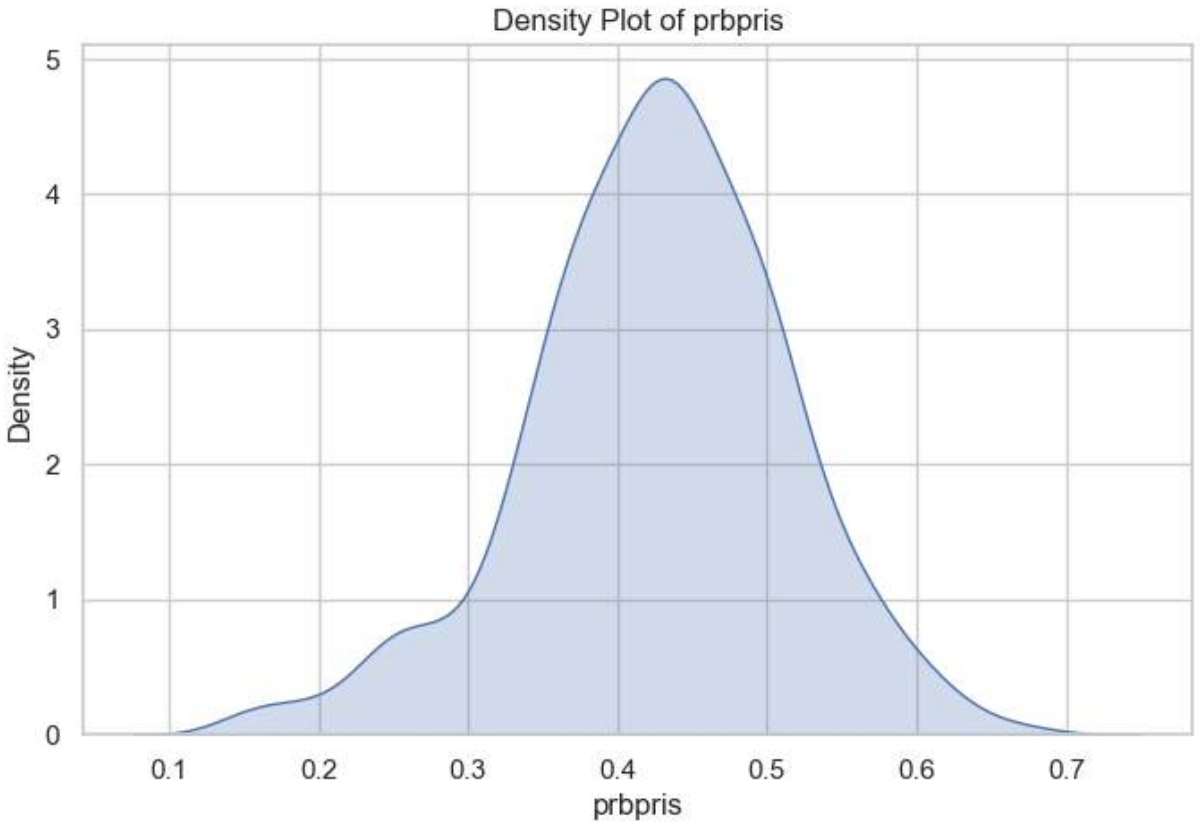

Density Plot of county
Density
0.005
0.004
0.003
0.002
0.001
0.000
−50
0
50
100
150
200
250
county

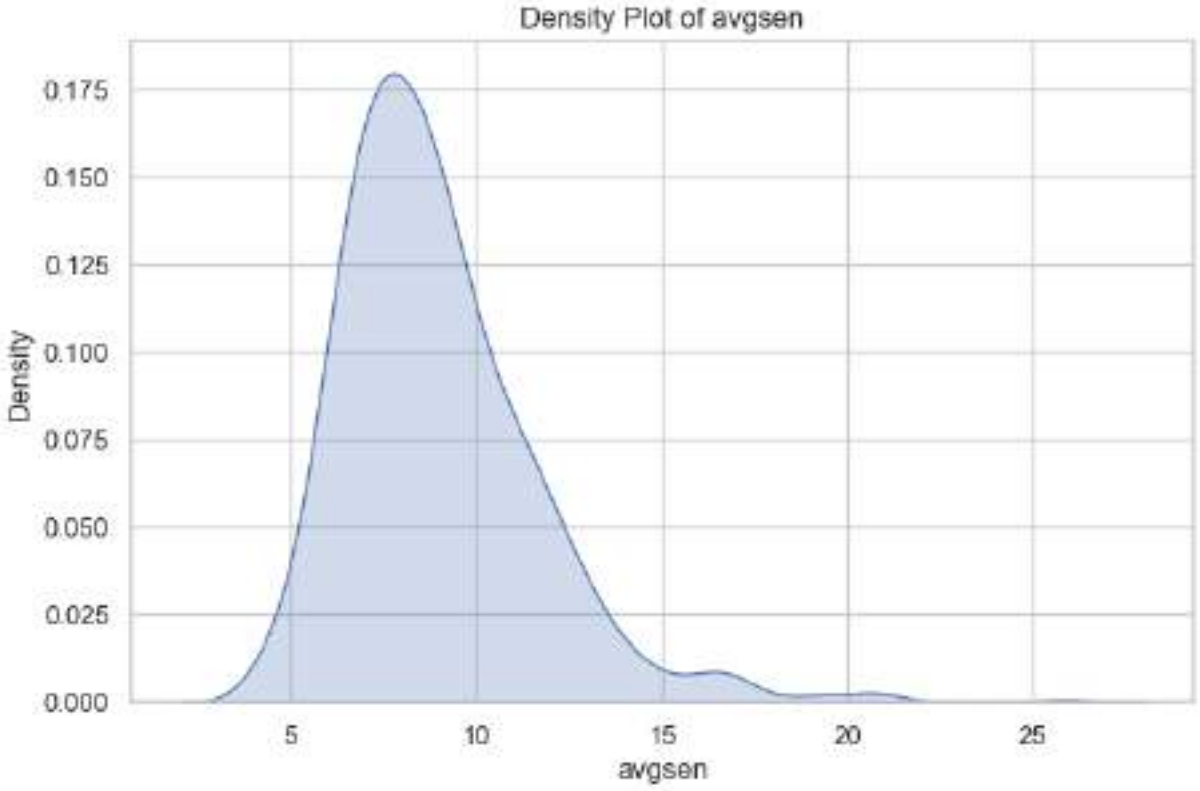

Density Plot of year
Density
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
80
82
84
86
88
year

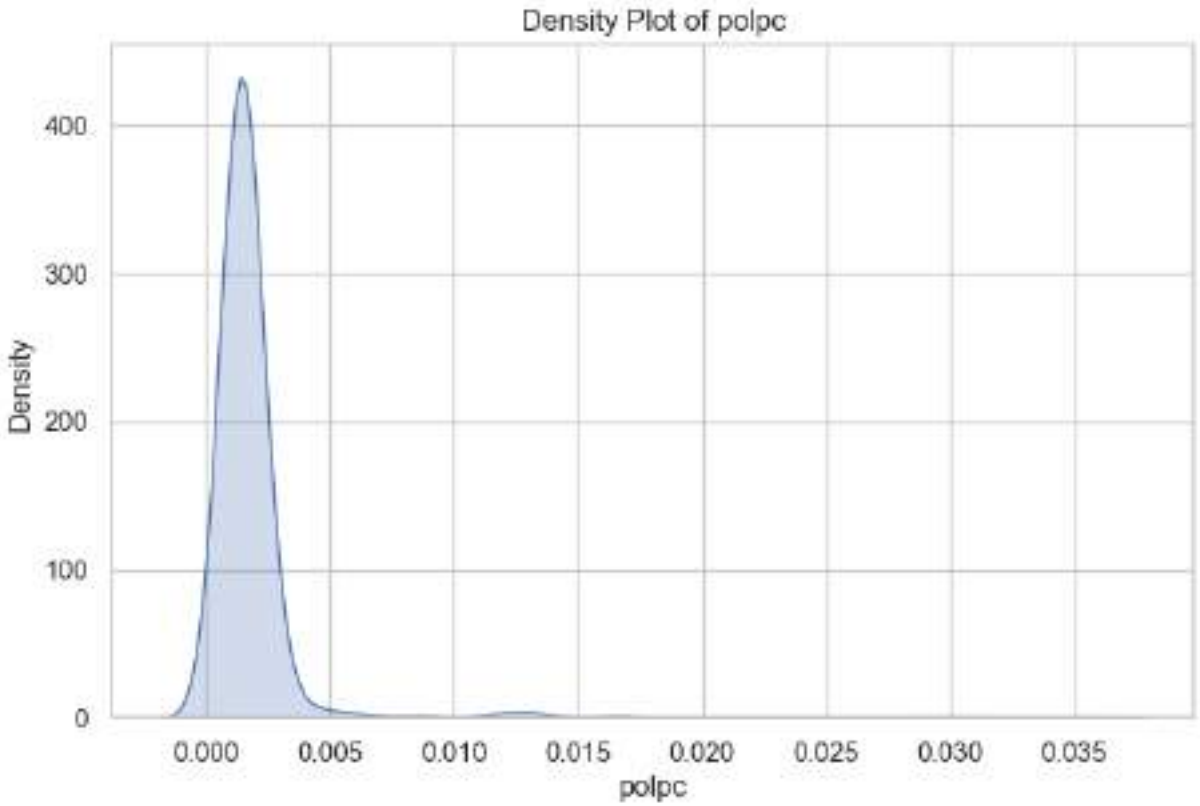

Density Plot of crmrte
Density
25
20
15
10
5
0
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
crmrte

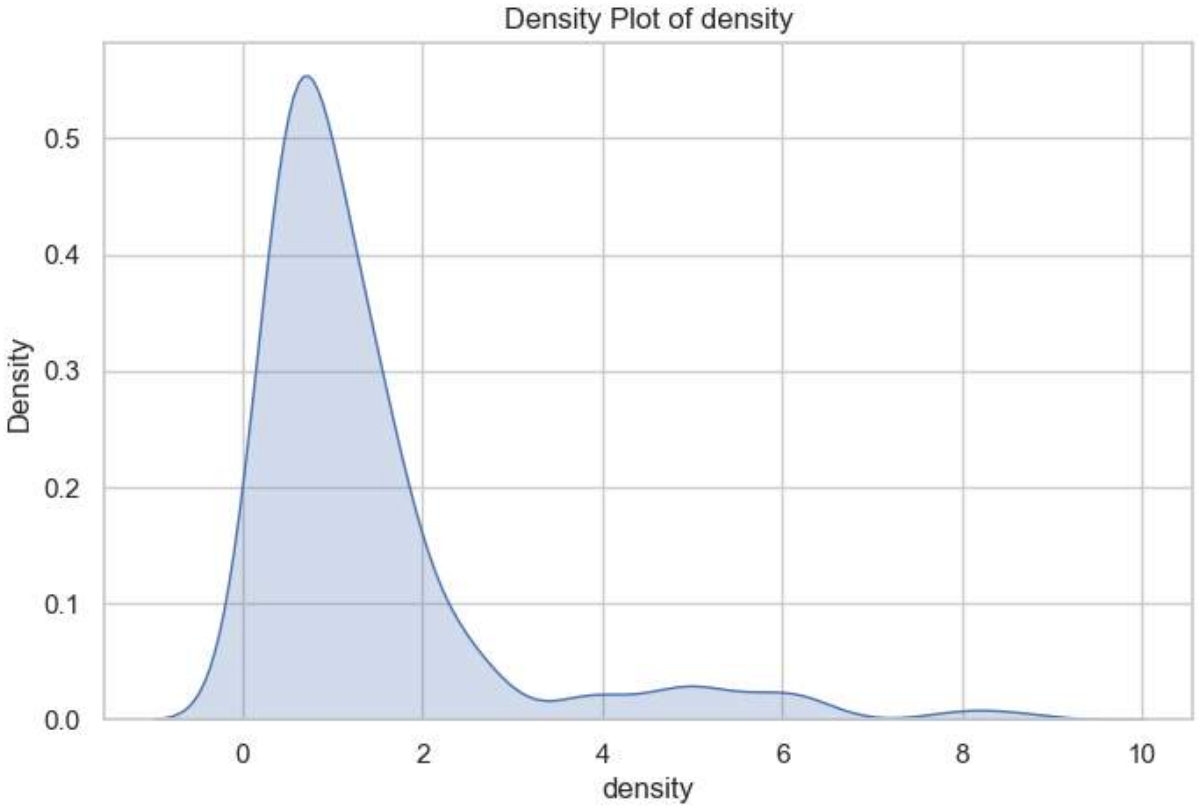

Density Plot of prbarr
Density
prbarr
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5

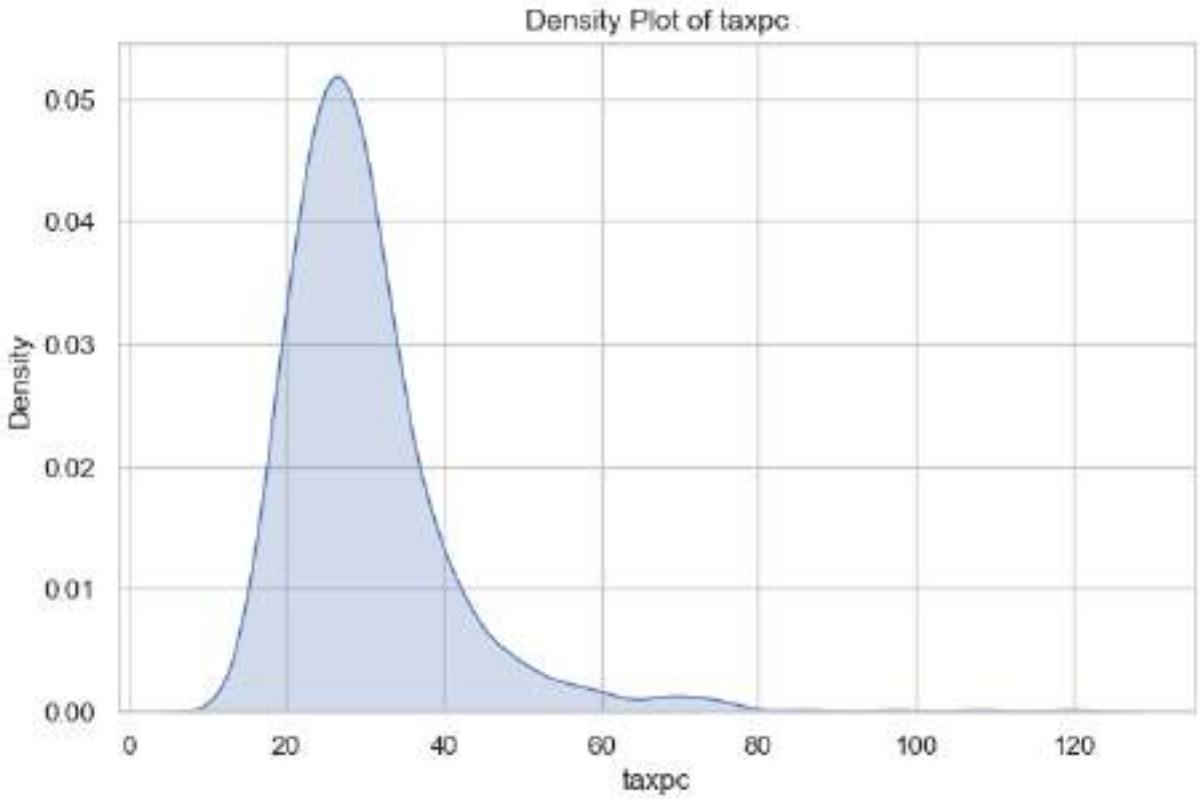

Density Plot of prbconv
Density
prbconv
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
5
10
15
20
25
30
35
40

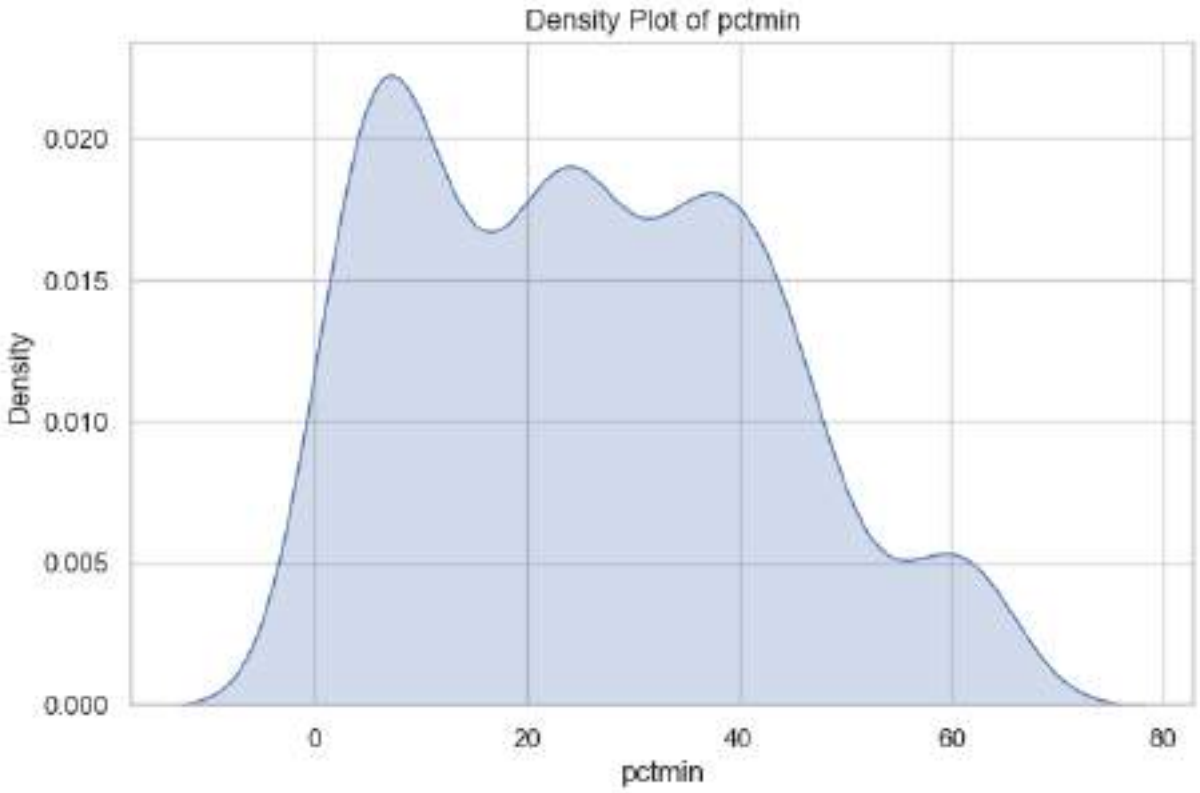

Density Plot of prbpris
Density
prbpris
0
1
2
3
4
5
0.1
0.2
0.3
0.4
0.5
0.6
0.7

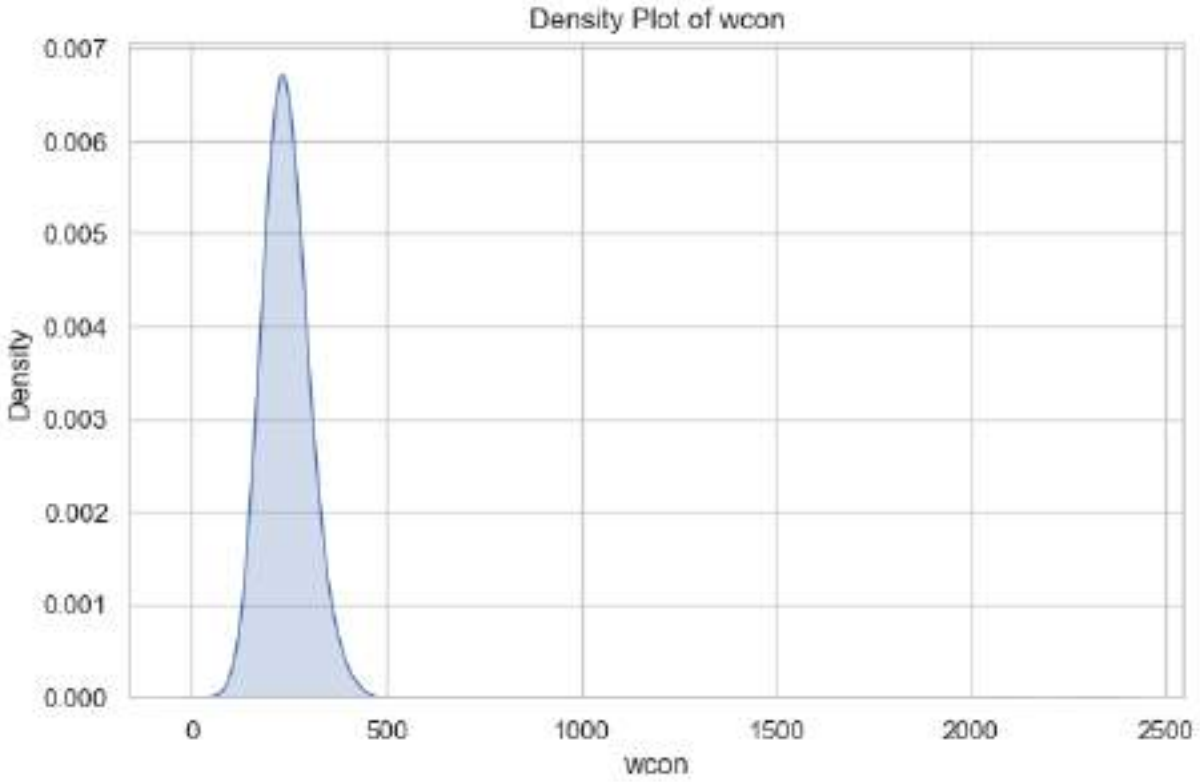

Density Plot of avgsen
Density
avgsen
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
5
10
15
20
25

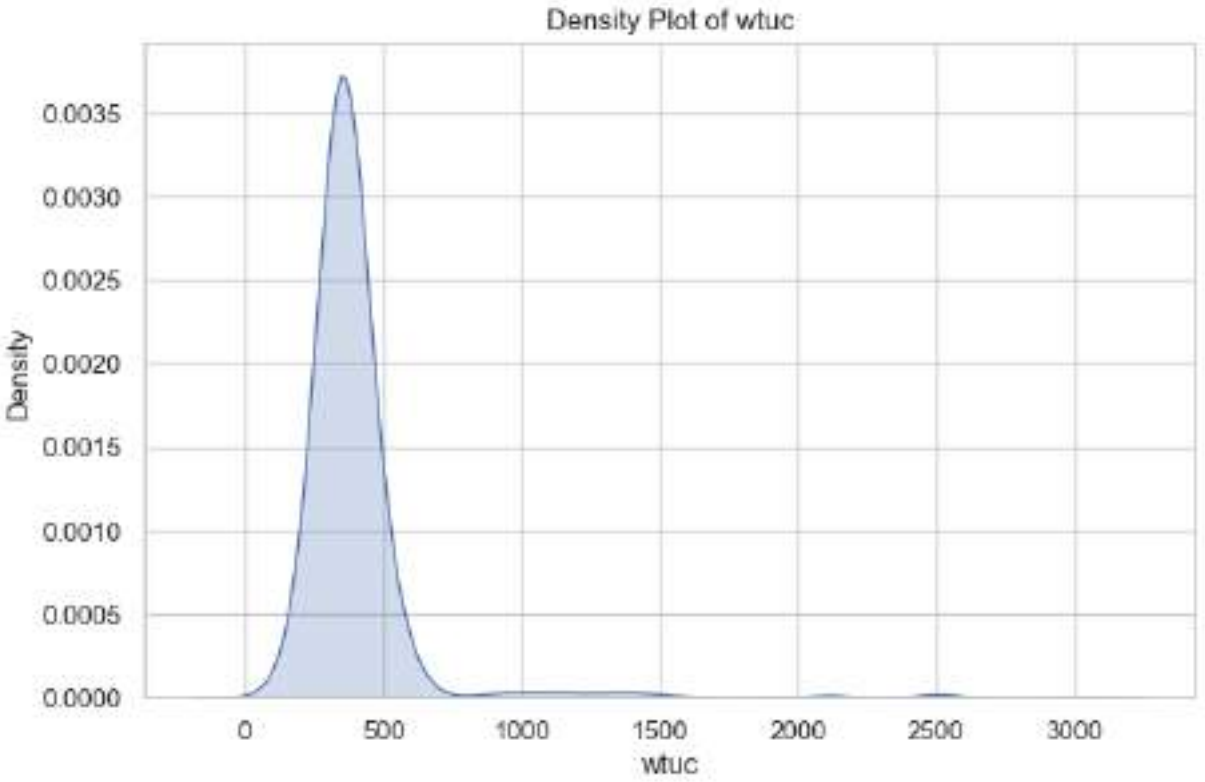

Density Plot of polpc
Density
400
300
200
100
0
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035
polpc

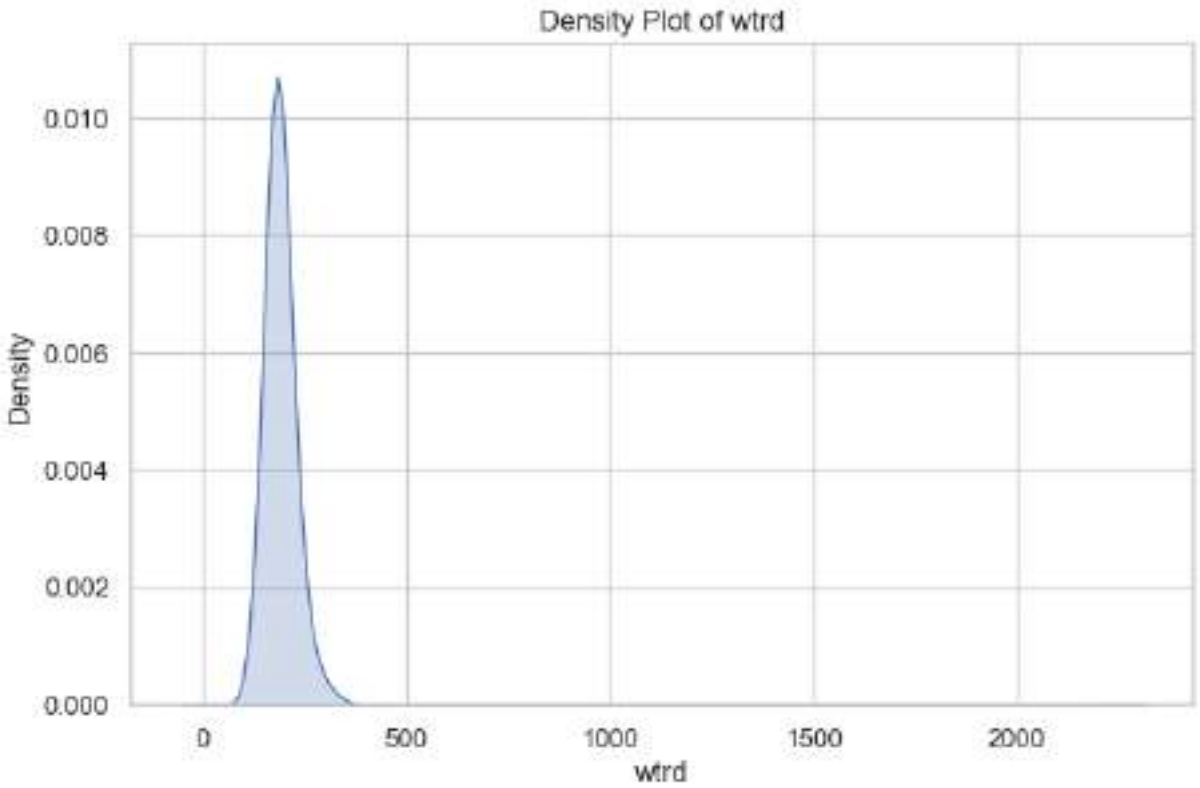

Density Plot of density
Density
0.5
0.4
0.3
0.2
0.1
0.0
0
2
4
6
8
10
density

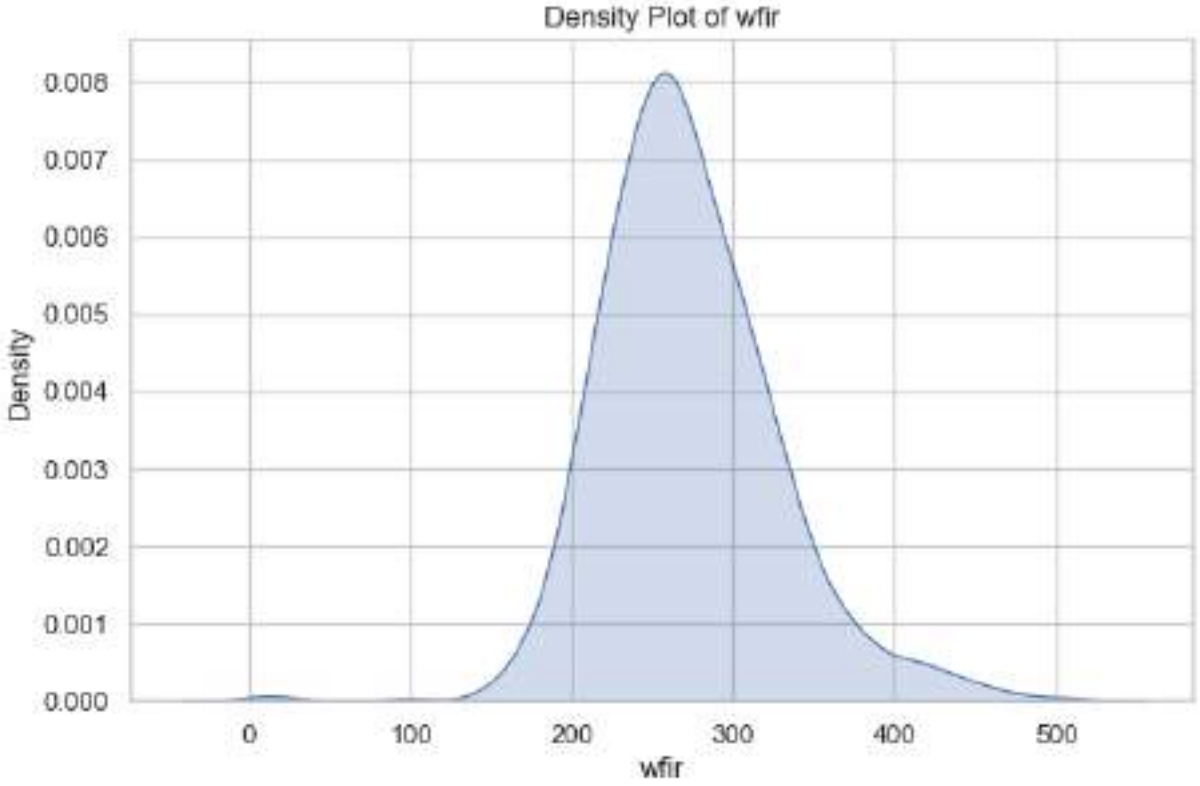

Density Plot of taxpc
Density
0.05
0.04
0.03
0.02
0.01
0.00
0
20
40
60
80
100
120
taxpc

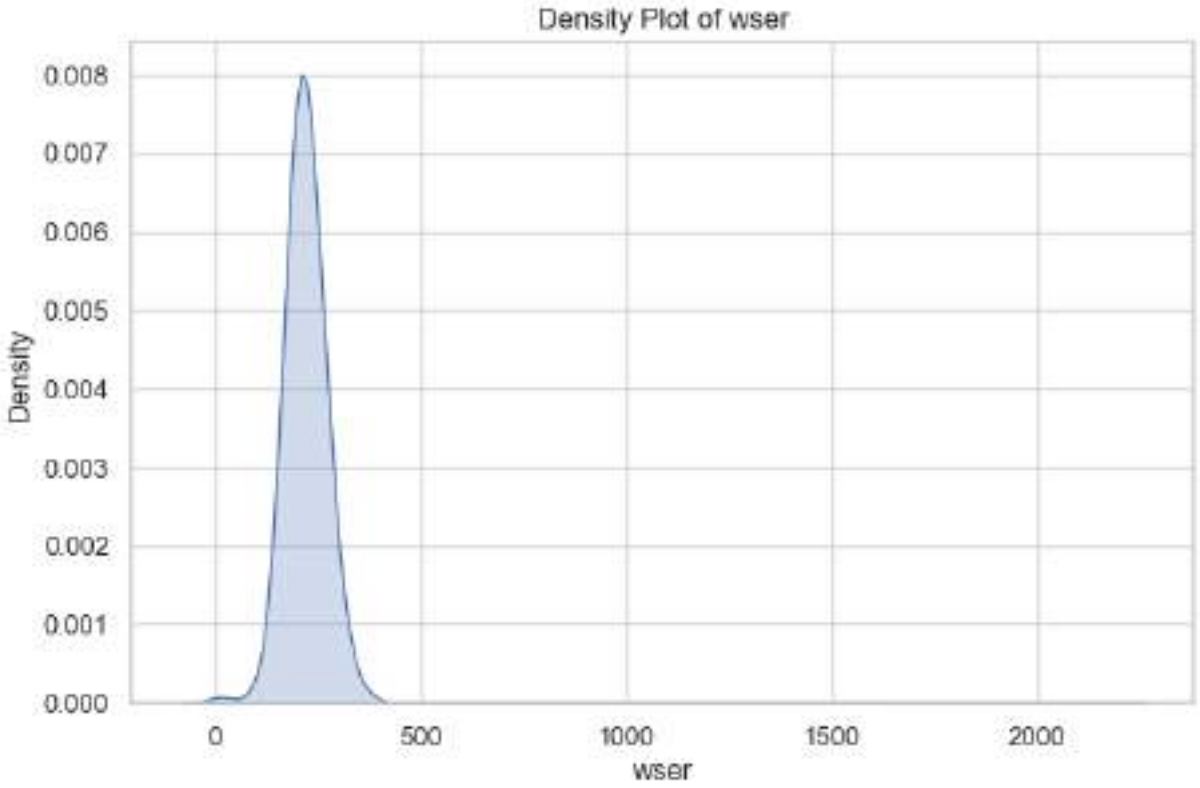

Density Plot of pctmin
Density
0.020
0.015
0.010
0.005
0.000
0
20
40
60
80
pctmin

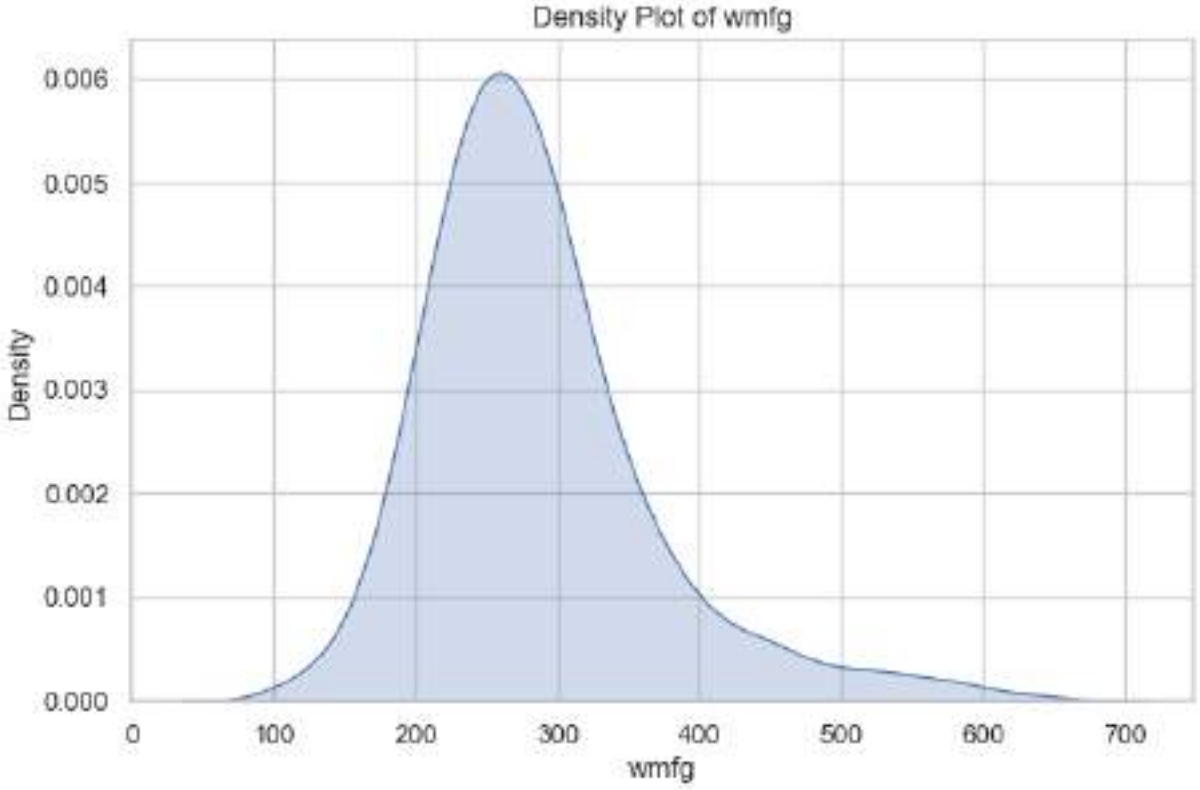

Density Plot of wcon
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
Density
0
500
1000
1500
2000
2500
wcon

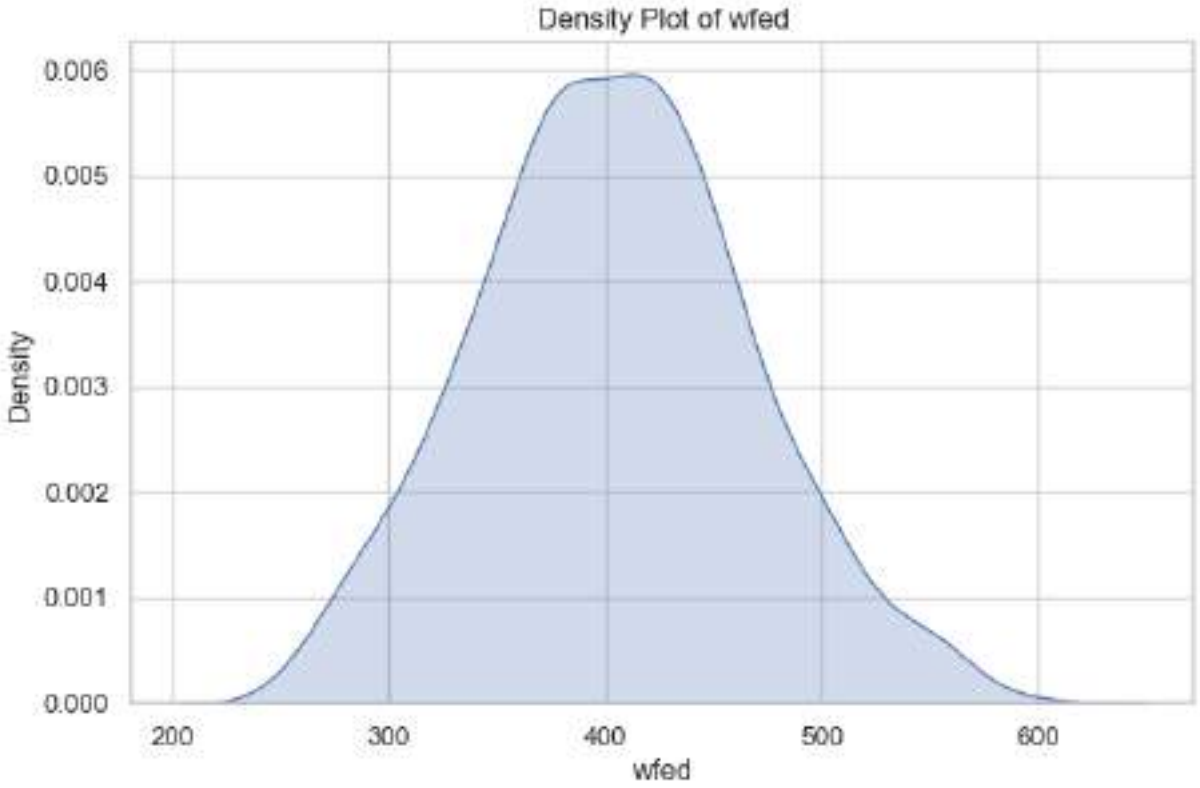

Density Plot of wtuc
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
Density
0
500
1000
1500
2000
2500
3000
wtuc

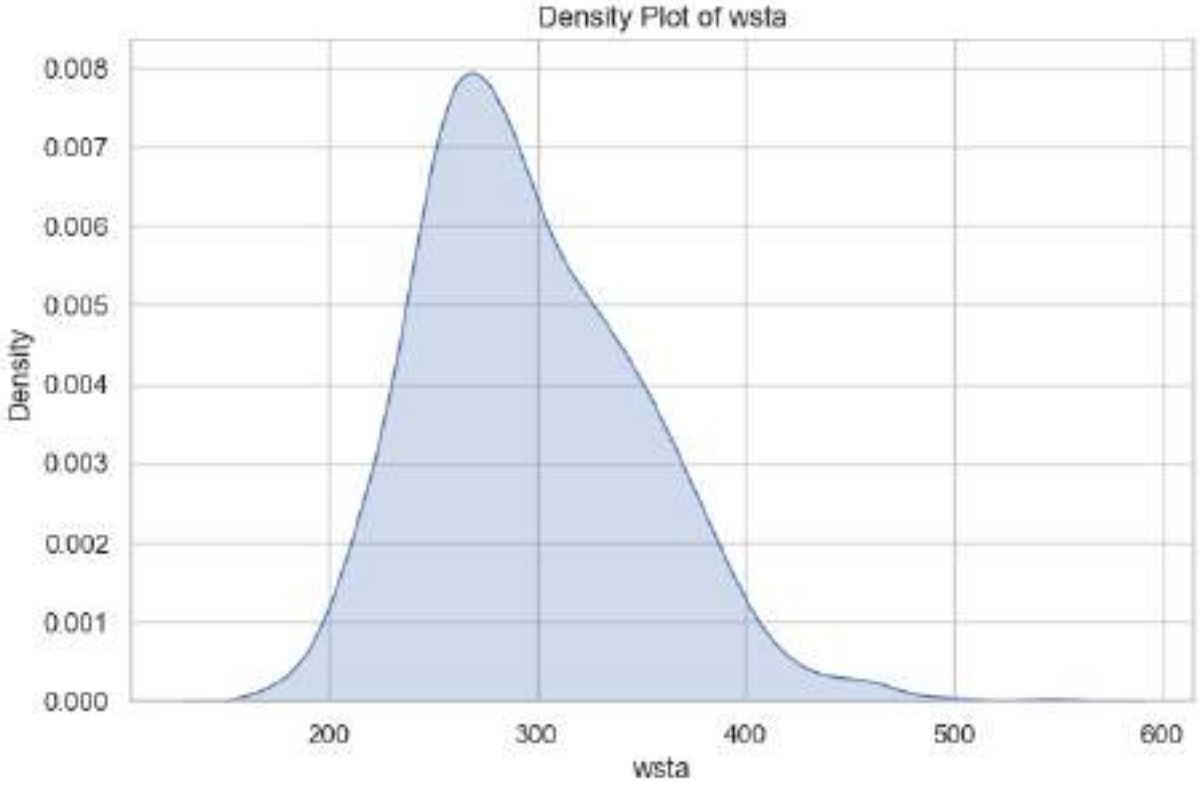

Density Plot of wtrd
Density
0.010
0.008
0.006
0.004
0.002
0.000
0
500
1000
1500
2000
wtrd

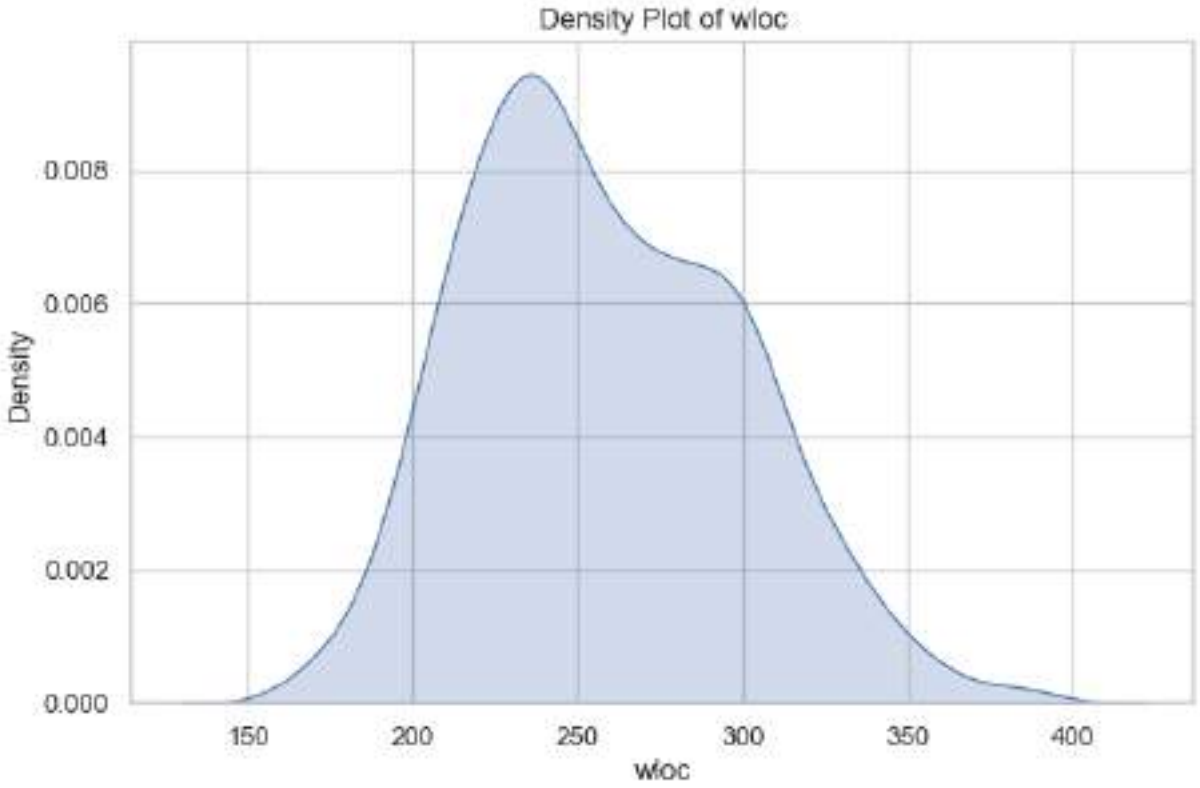

Density Plot of wfir
Density
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
100
200
300
400
500
wfir

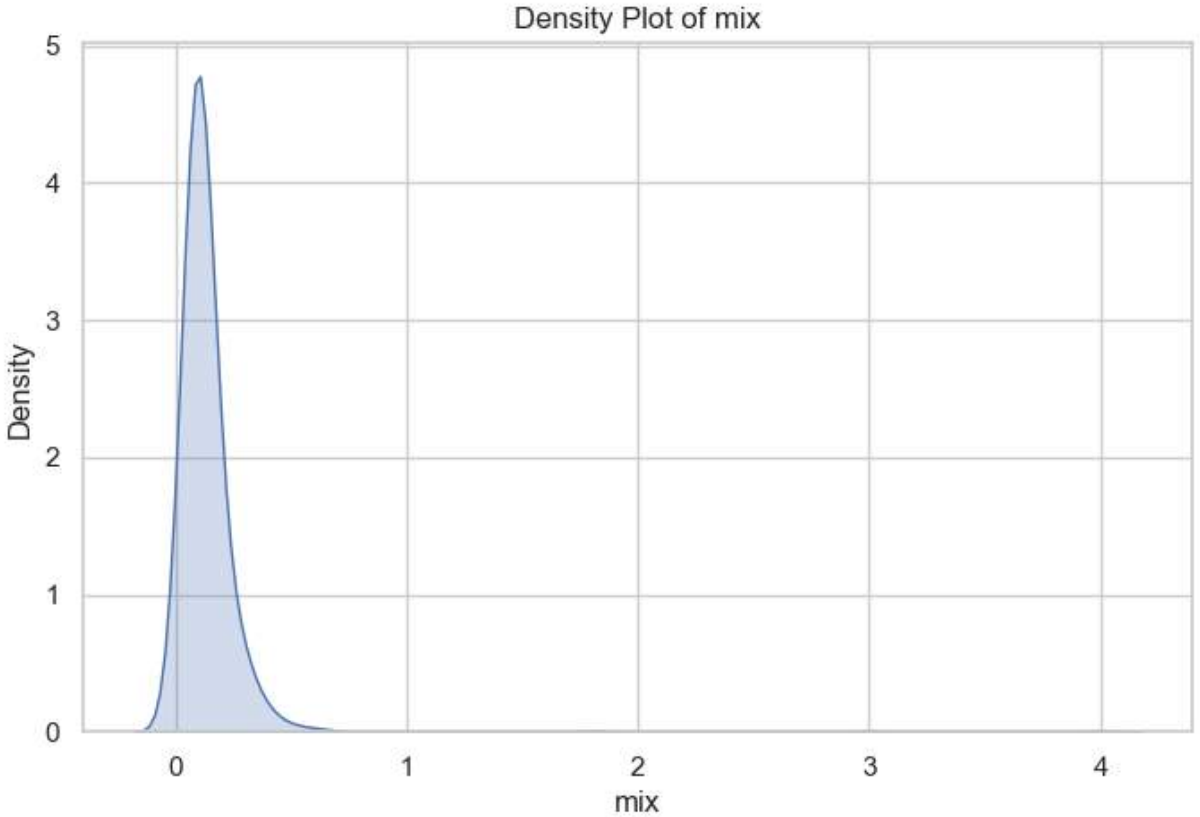

Density Plot of wser
Density
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
500
1000
1500
2000
wser

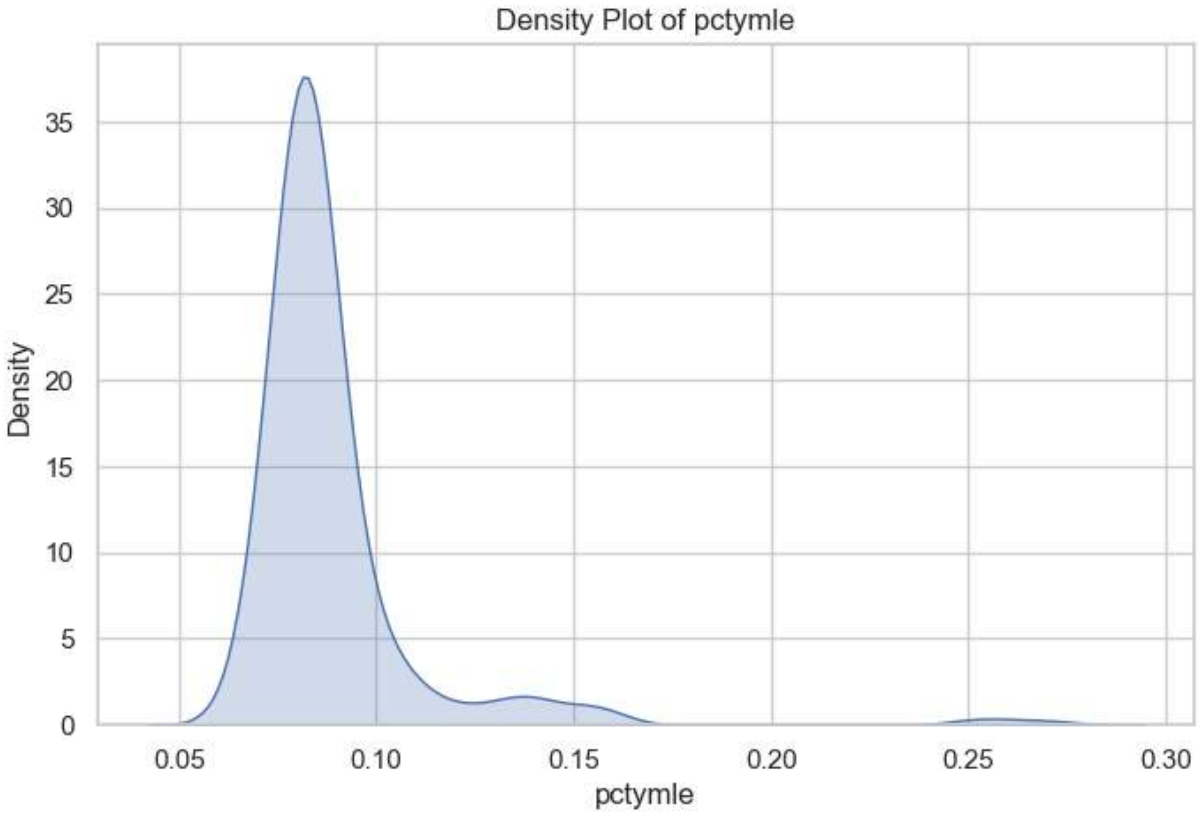

Density Plot of wmfg
Density
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
100
200
300
400
500
600
700
wmfg

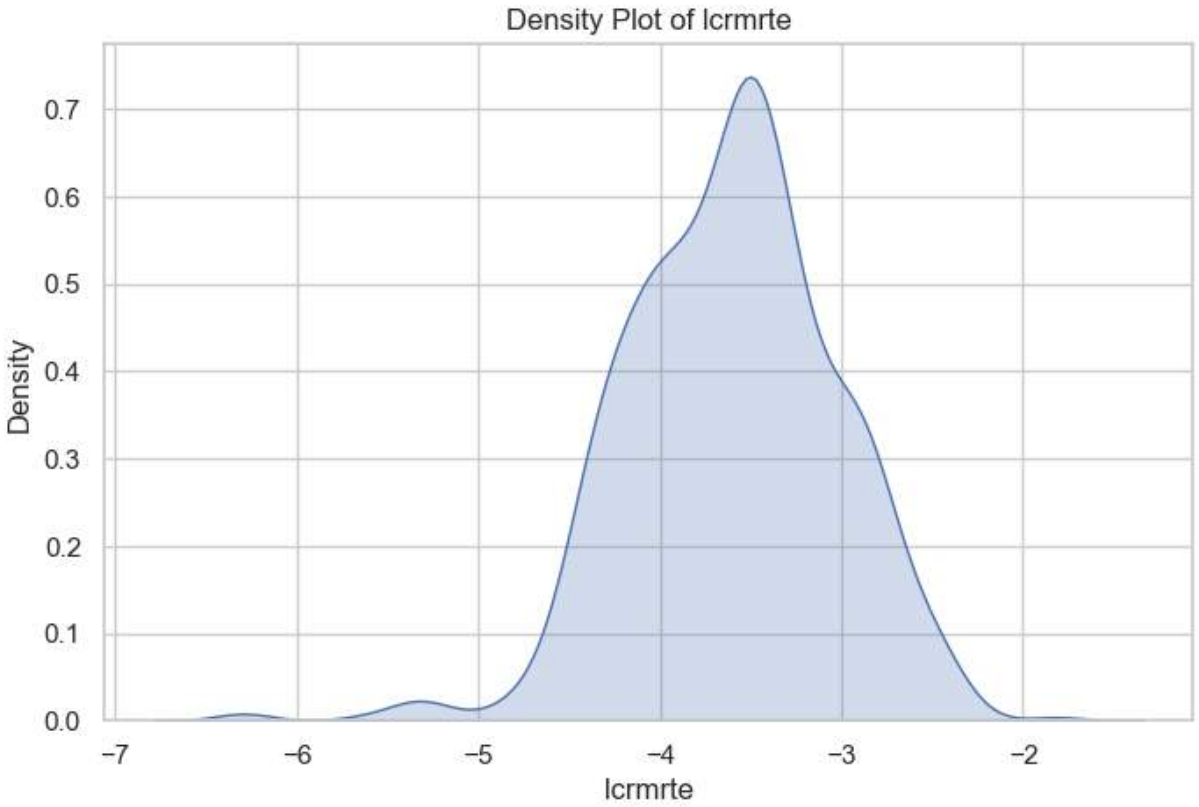

Density Plot of wfed
Density
0.006
0.005
0.004
0.003
0.002
0.001
0.000
200
300
400
500
600
wfed

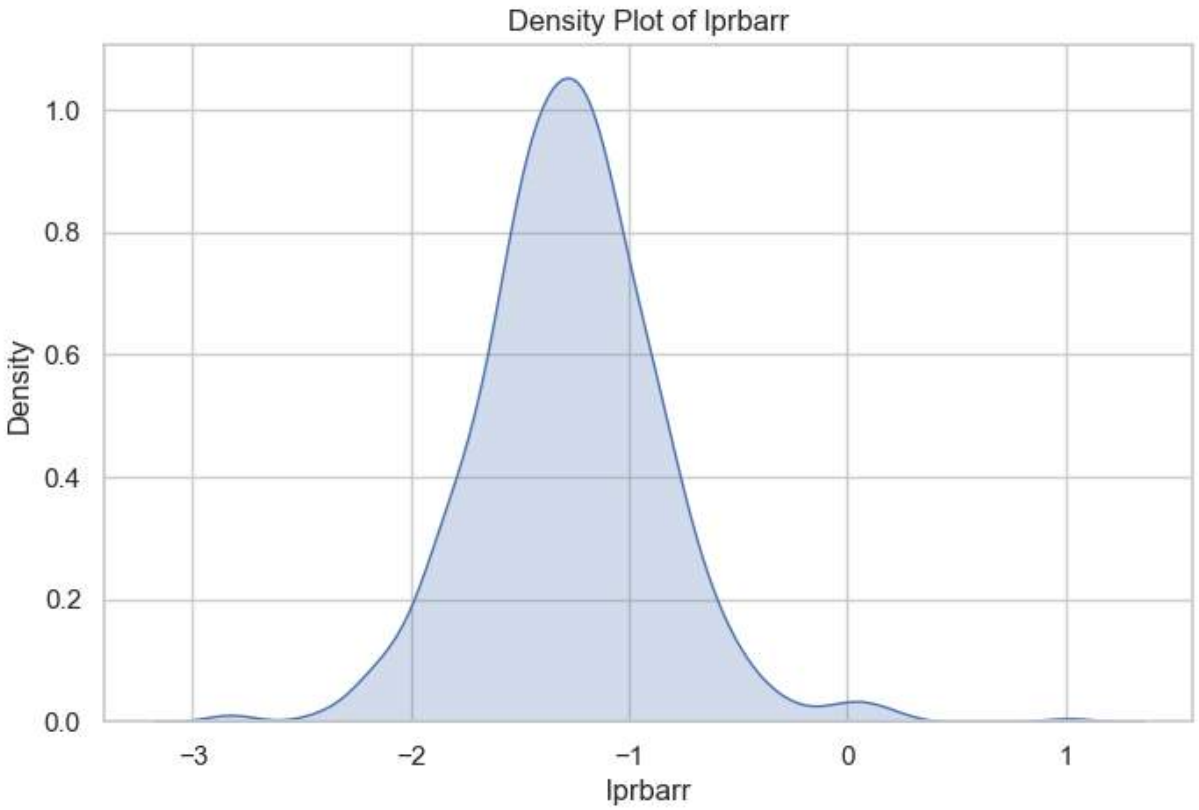

Density Plot of wsta
Density
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
200
300
400
500
600
wsta

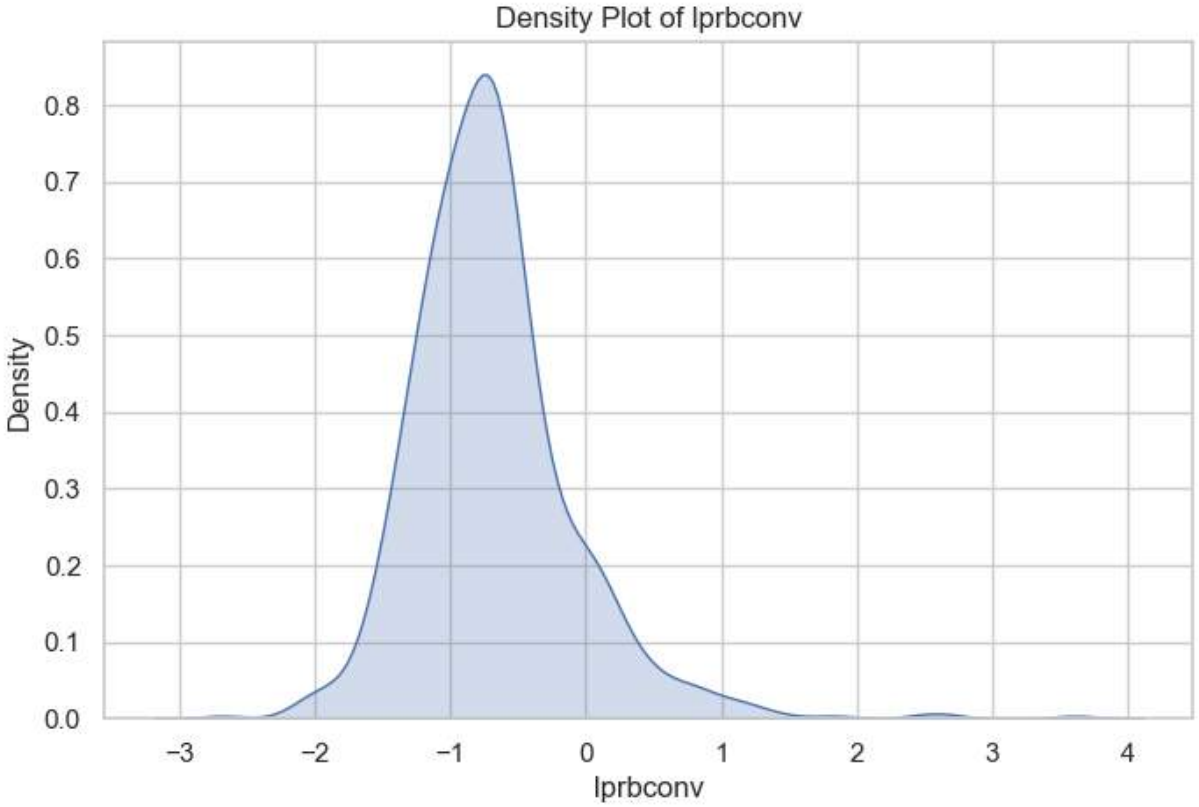

Density Plot of wloc
Density
0.008
0.006
0.004
0.002
0.000
150
200
250
300
350
400
wloc

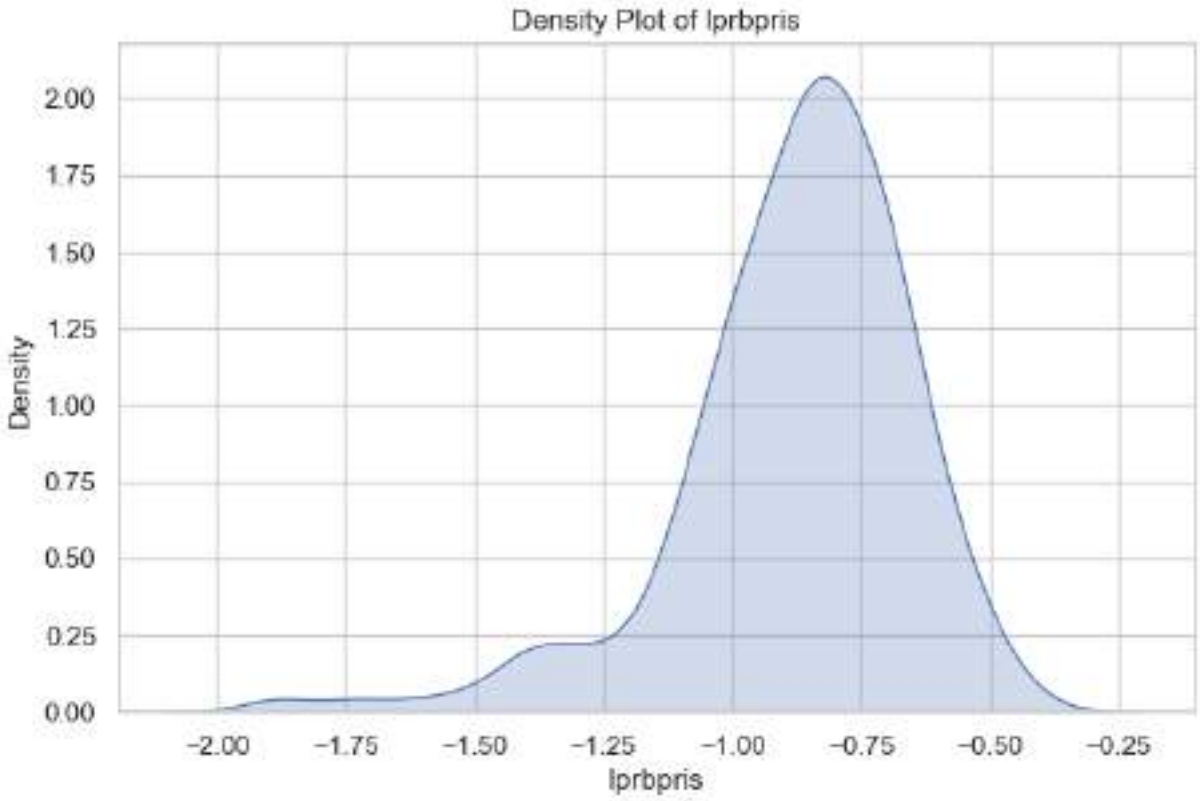

Density Plot of mix
Density
5
4
3
2
1
0
0
1
2
3
4
mix

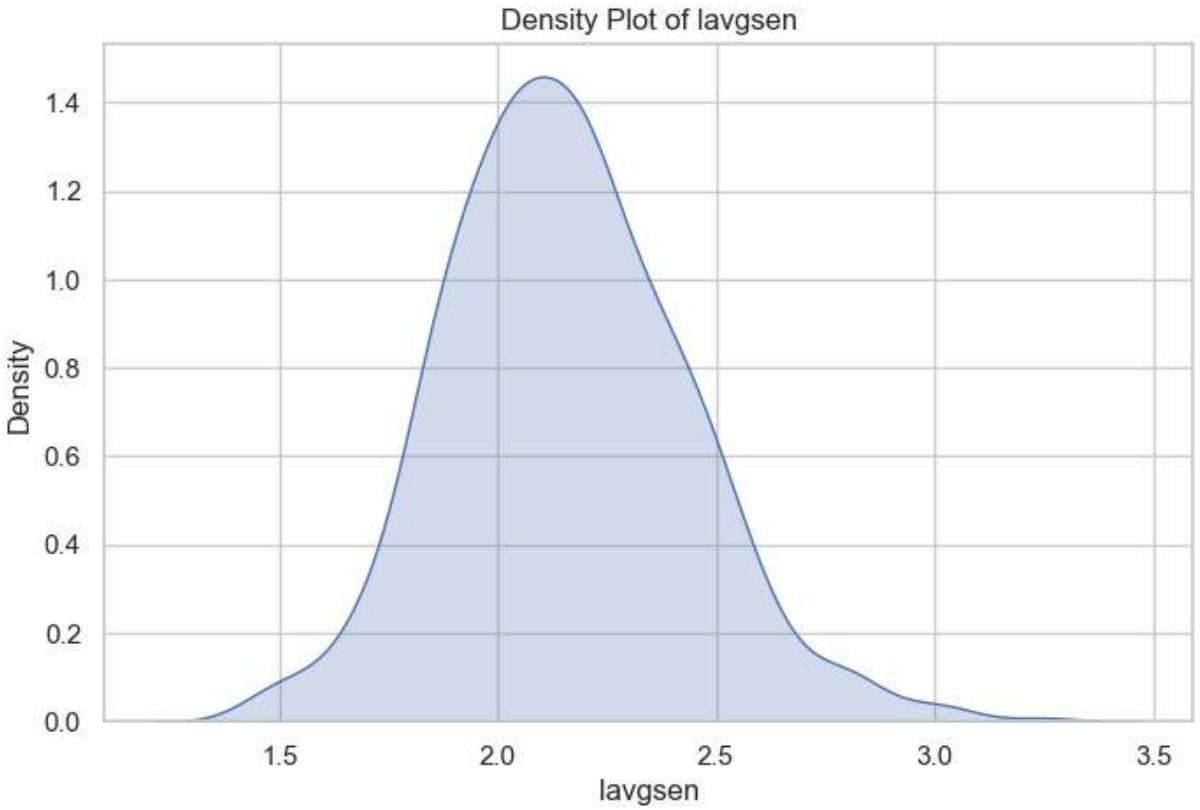

Density Plot of pctymle
Density
pctymle
0
5
10
15
20
25
30
35
0.05
0.10
0.15
0.20
0.25
0.30

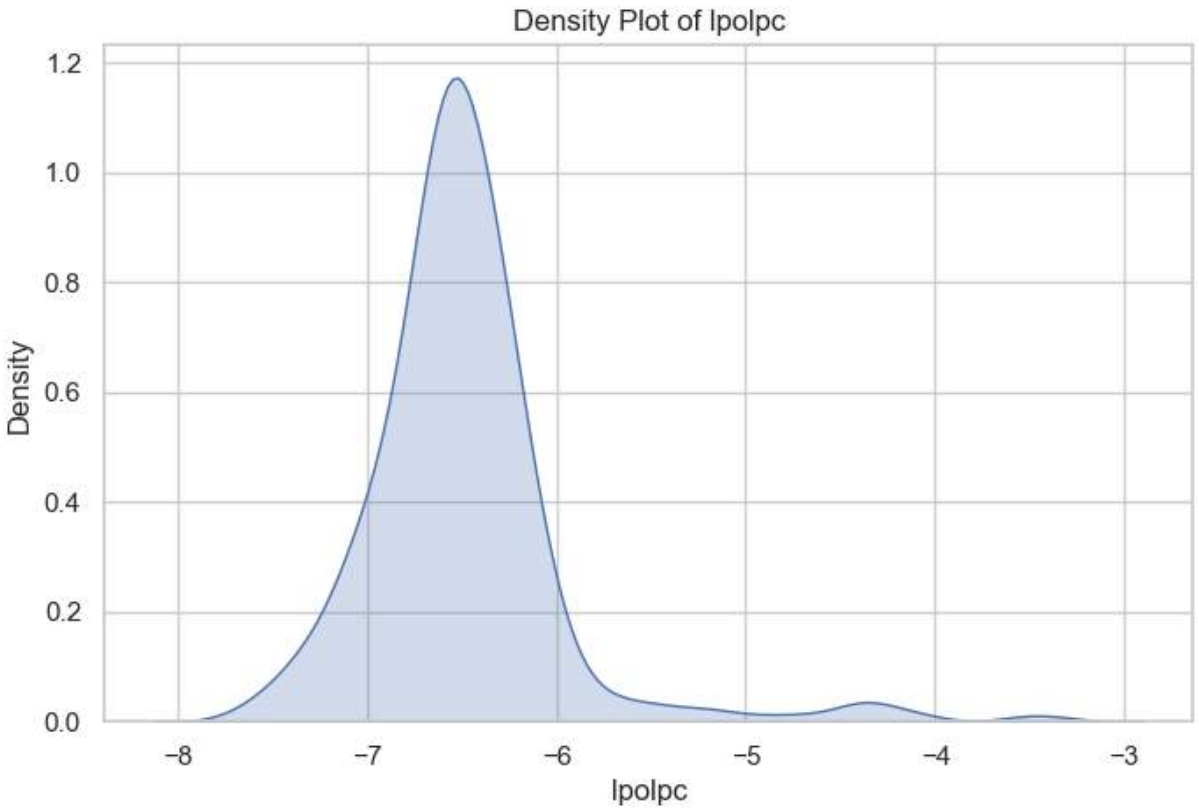

Density Plot of lcrmrte
Density
lcrmrte
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
−7
−6
−5
−4
−3
−2

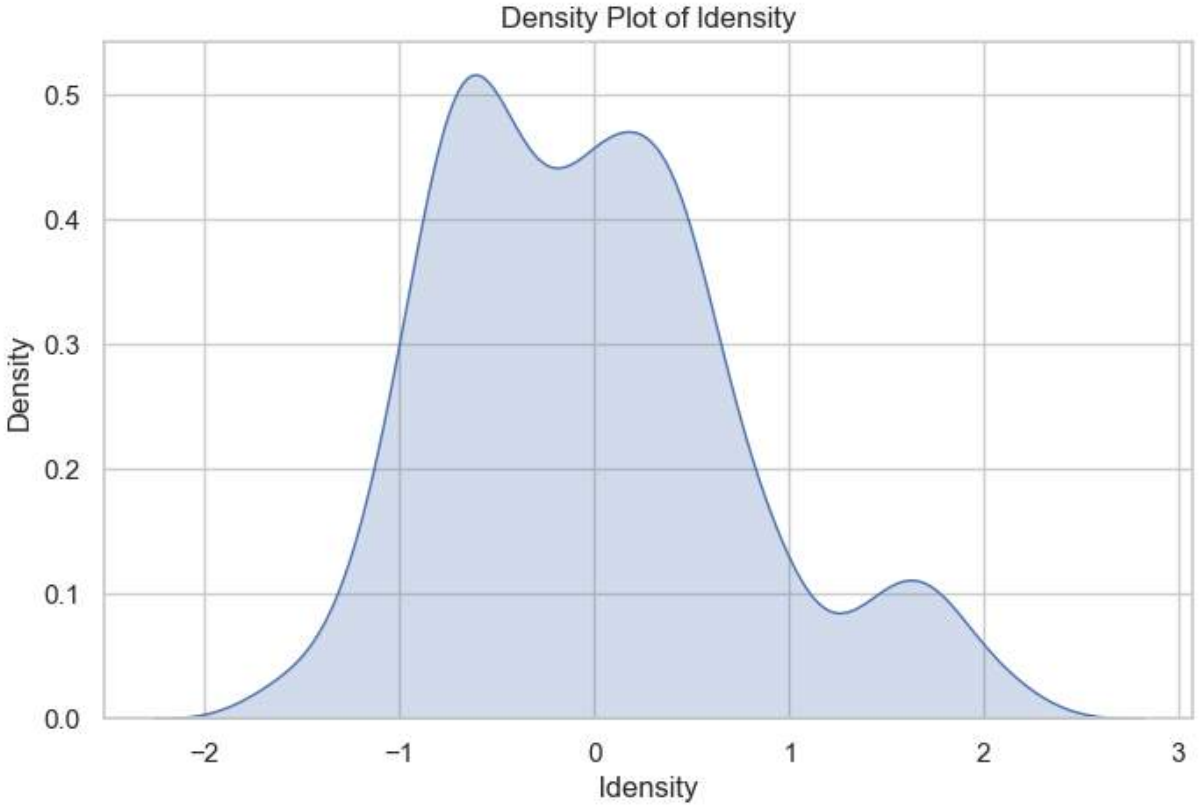

Density Plot of lprbarr
Density
1.0
0.8
0.6
0.4
0.2
0.0
−3
−2
−1
0
1
lprbarr

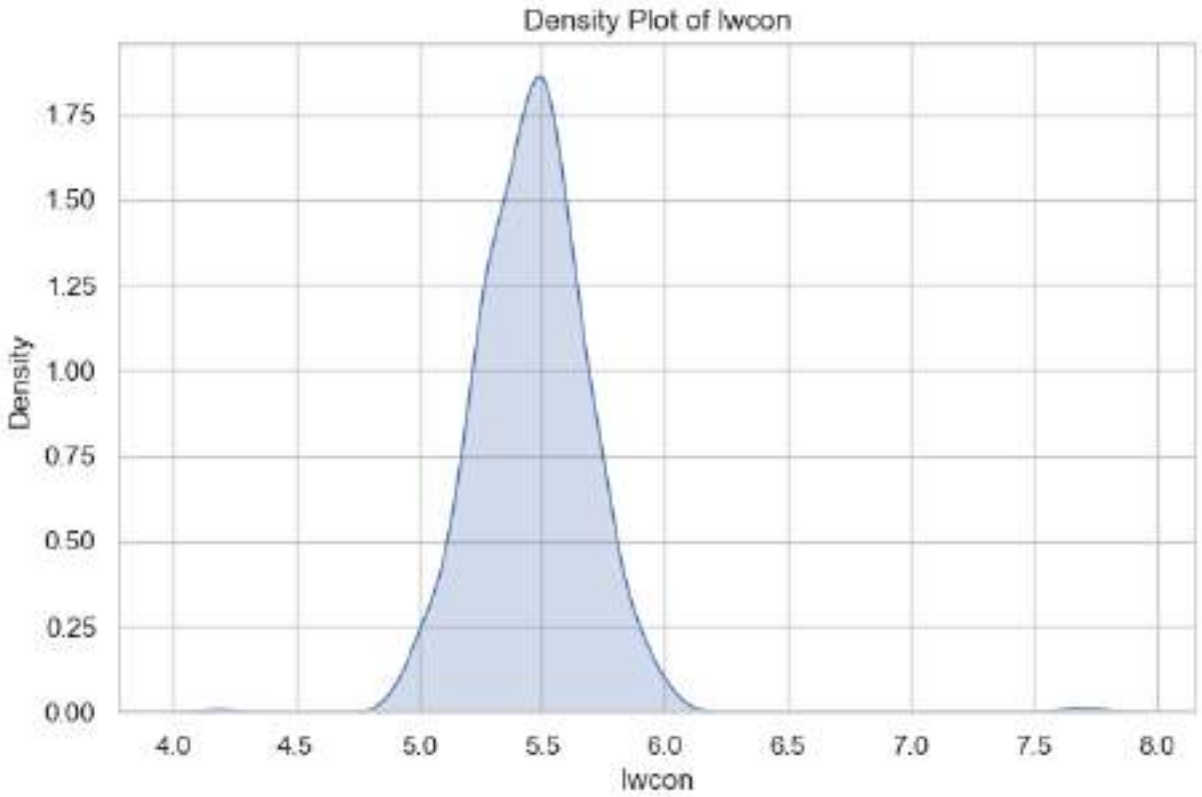

Density Plot of lprbconv
Density
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
−3
−2
−1
0
1
2
3
4
lprbconv

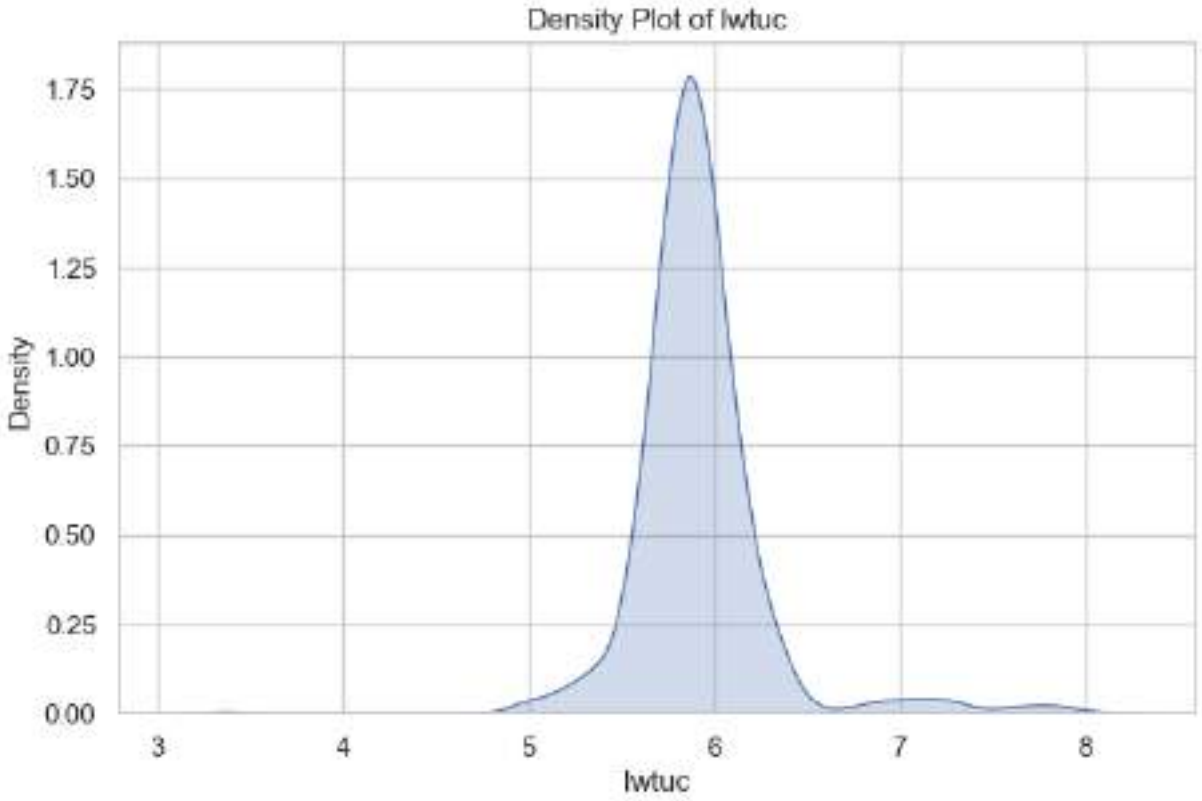

Density Plot of lprbpris
Density
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−2.00
−1.75
−1.50
−1.25
−1.00
−0.75
−0.50
−0.25
lprbpris

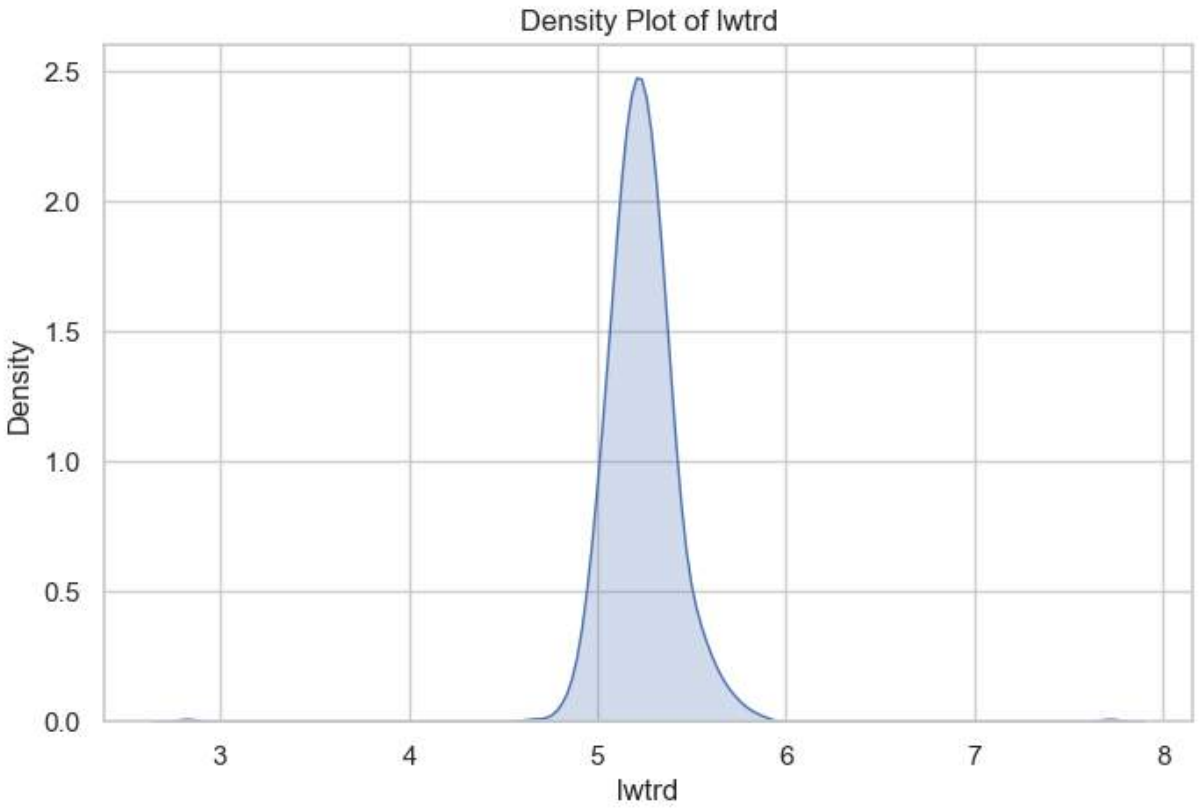

Density Plot of lavgsen
Density
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
1.5
2.0
2.5
3.0
3.5
lavgsen

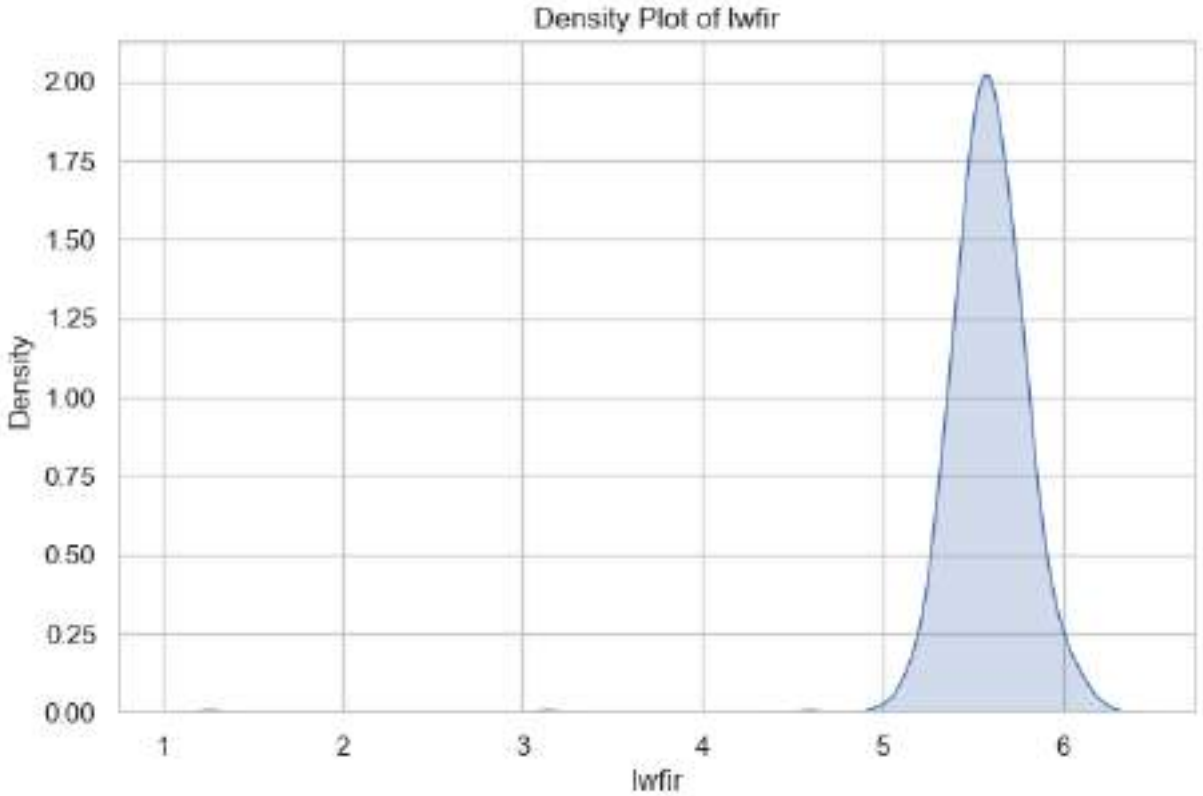

Density Plot of lpolpc
Density
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−8
−7
−6
−5
−4
−3
lpolpc

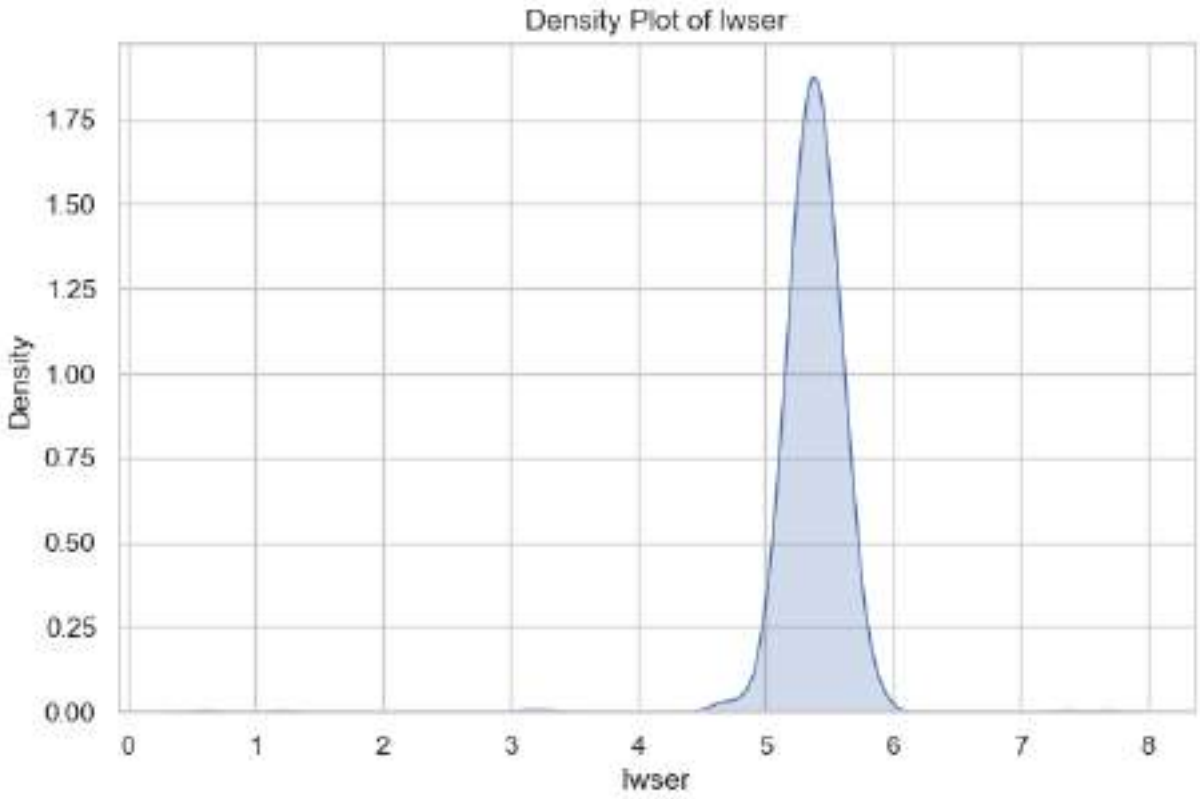

Density Plot of ldensity
Density
0.5
0.4
0.3
0.2
0.1
0.0
−2
−1
0
1
2
3
ldensity

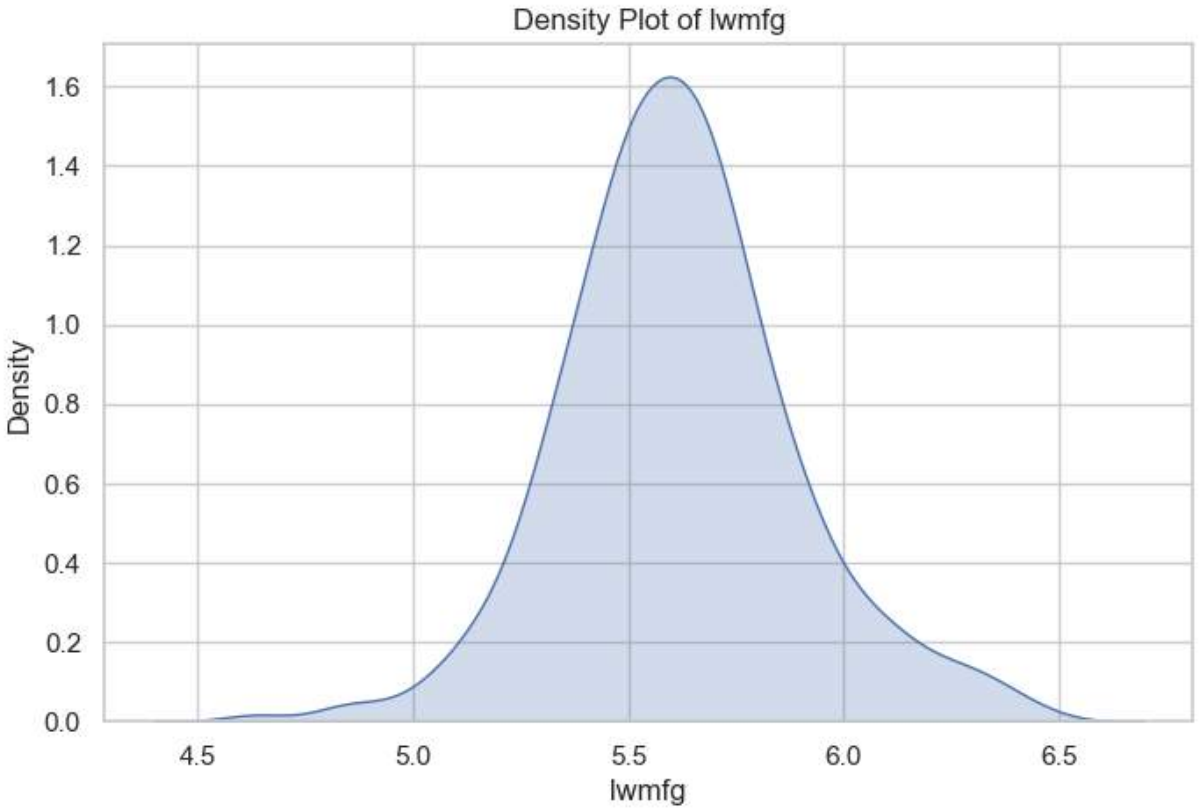

Density Plot of lwcon
Density
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
lwcon

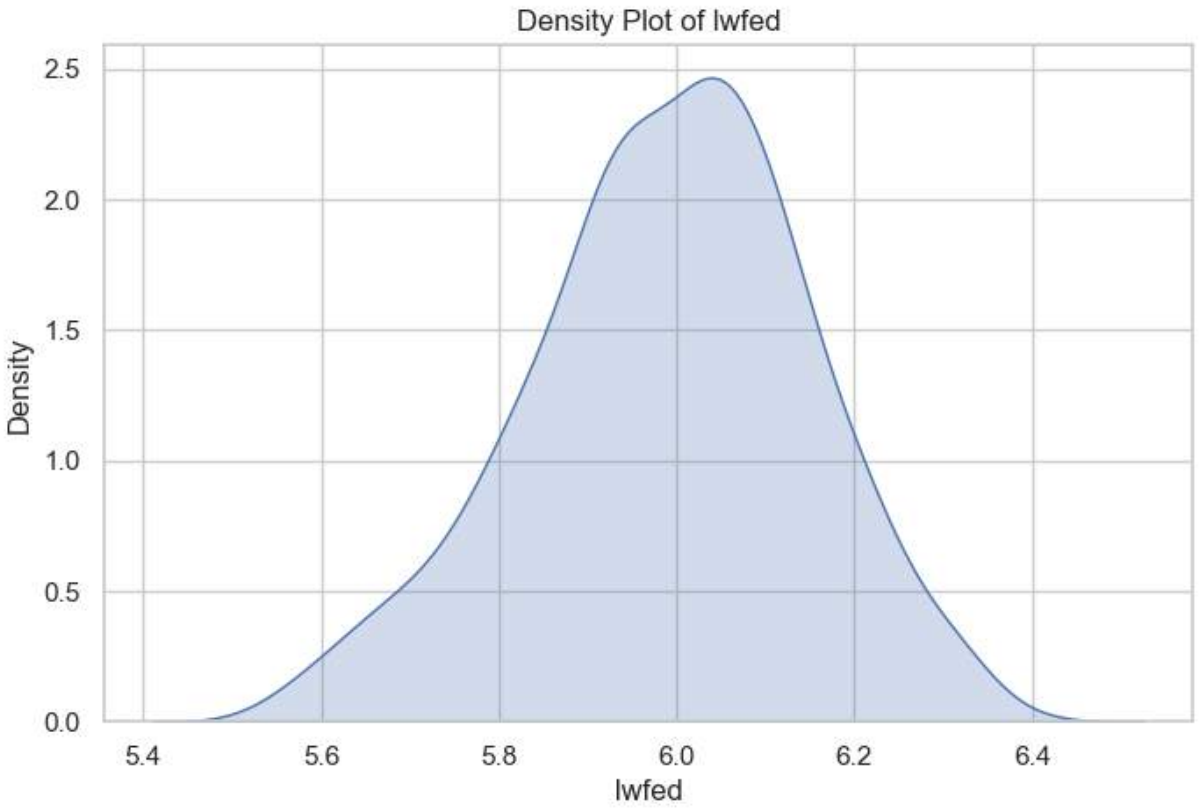

Density Plot of lwtuc
Density
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
3
4
5
6
7
8
lwtuc

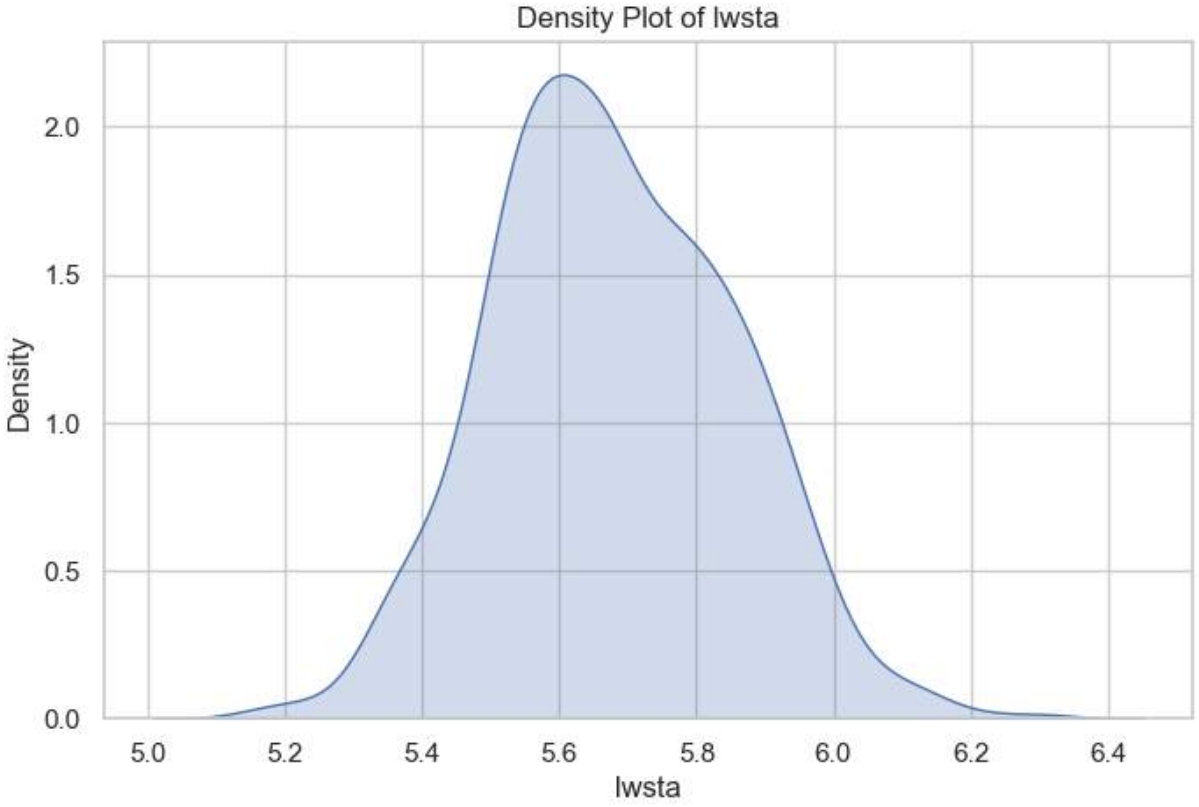

Density Plot of lwtrd
Density
2.5
2.0
1.5
1.0
0.5
0.0
3
4
5
6
7
8
lwtrd

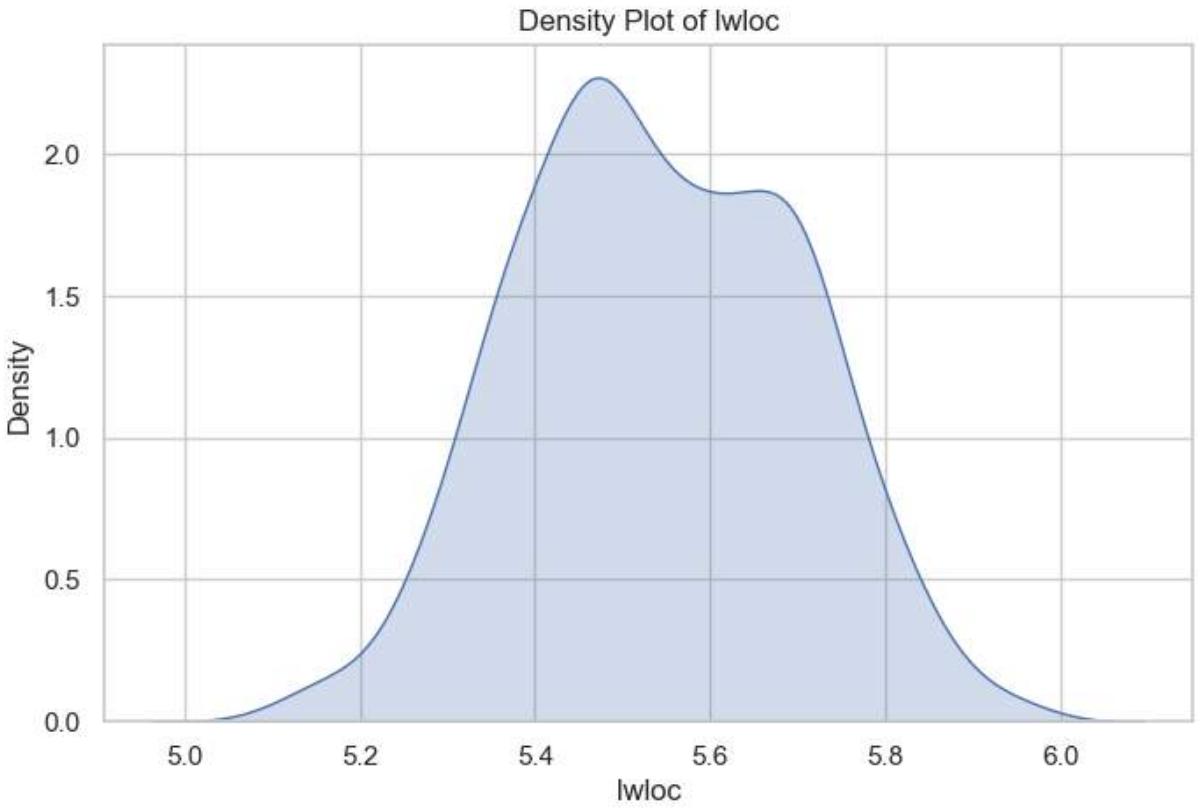

Density Plot of lwfir
Density
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
1
2
3
4
5
6
lwfir

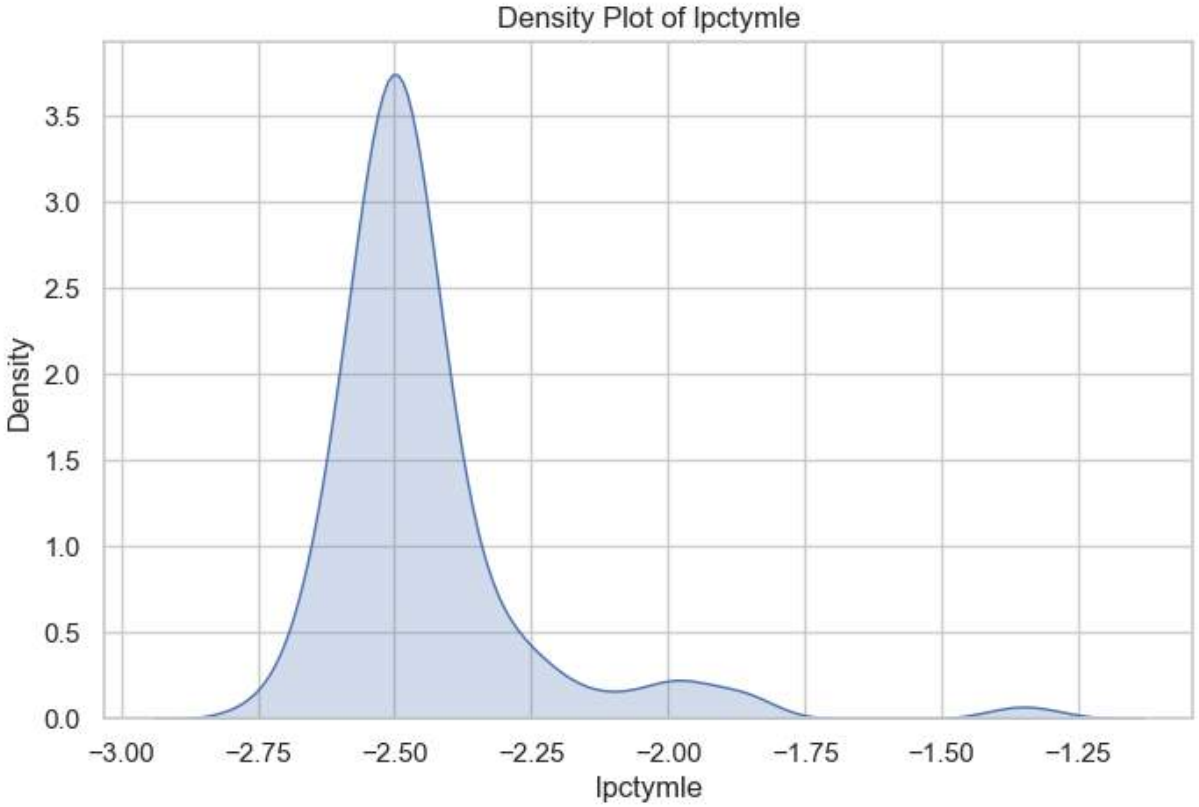

Density Plot of lwser
Density
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
1
2
3
4
5
6
7
8
lwser

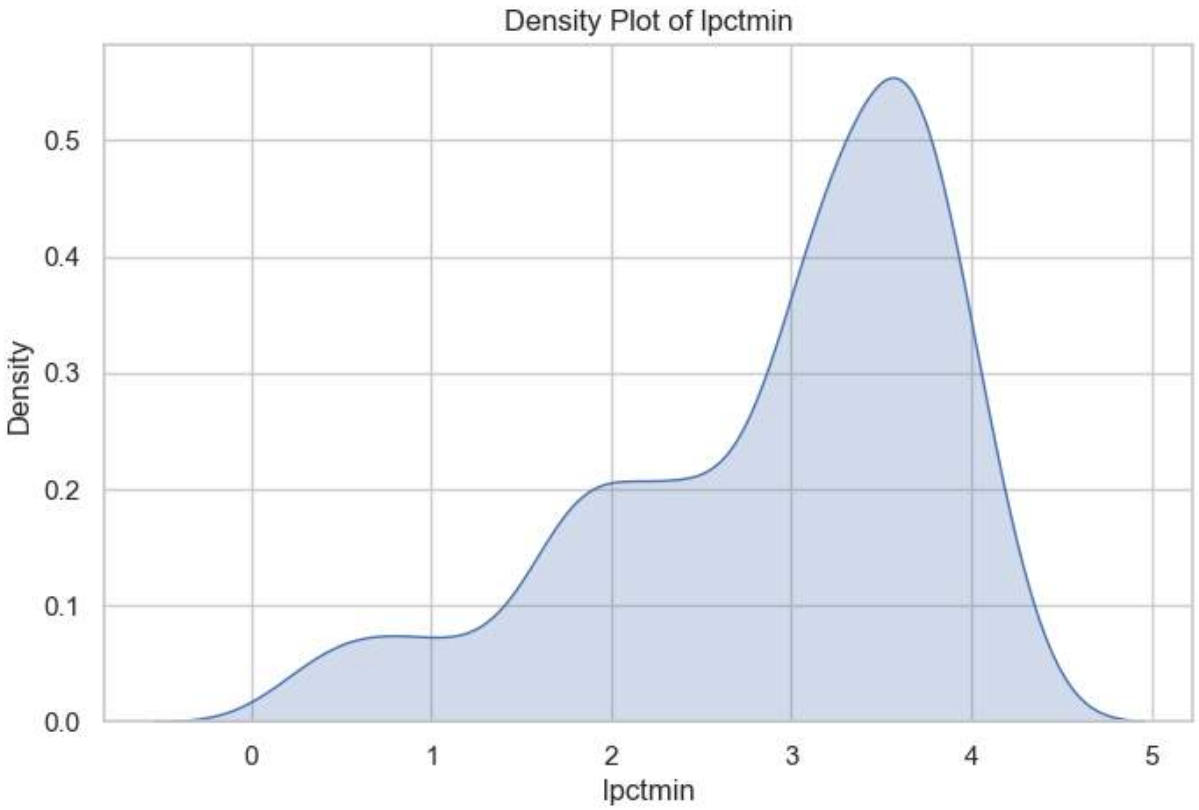

Density Plot of lwmfg
Density
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
4.5
5.0
5.5
6.0
6.5
lwmfg

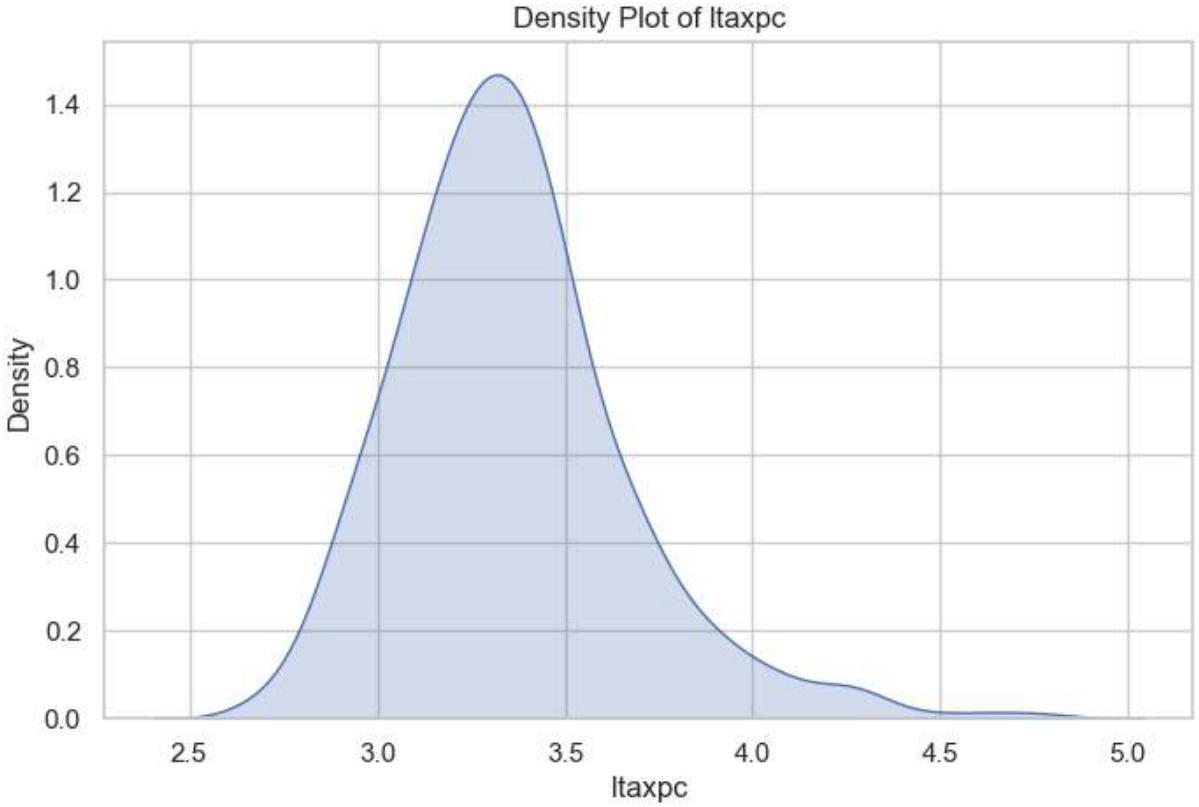

Density Plot of lwfed
Density
lwfed
0.0
0.5
1.0
1.5
2.0
2.5
5.4
5.6
5.8
6.0
6.2
6.4

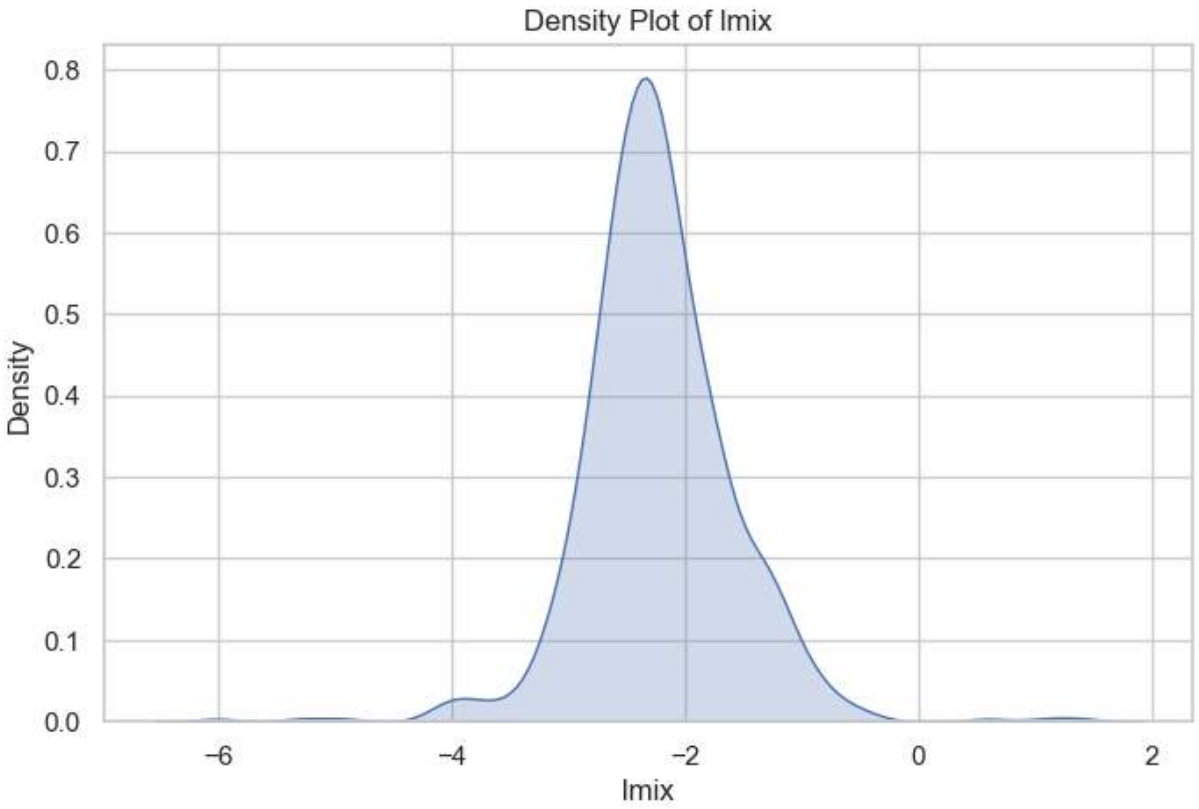

Density Plot of lwsta
Density
lwsta
0.0
0.5
1.0
1.5
2.0
5.0
5.2
5.4
5.6
5.8
6.0
6.2
6.4

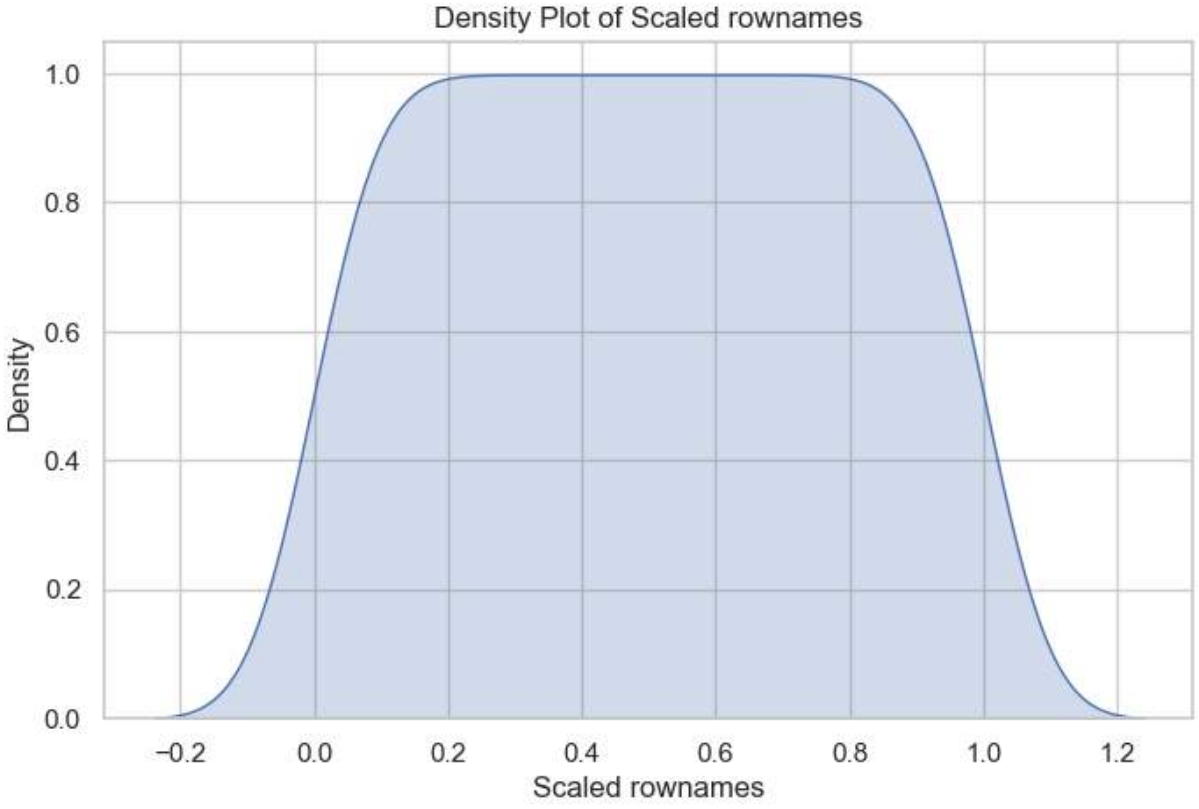

Density Plot of lwloc
Density
0.0
0.5
1.0
1.5
2.0
5.0
5.2
5.4
5.6
5.8
6.0
lwloc


Density Plot of lpctymle
Density
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
−3.00
−2.75
−2.50
−2.25
−2.00
−1.75
−1.50
−1.25
lpctymle

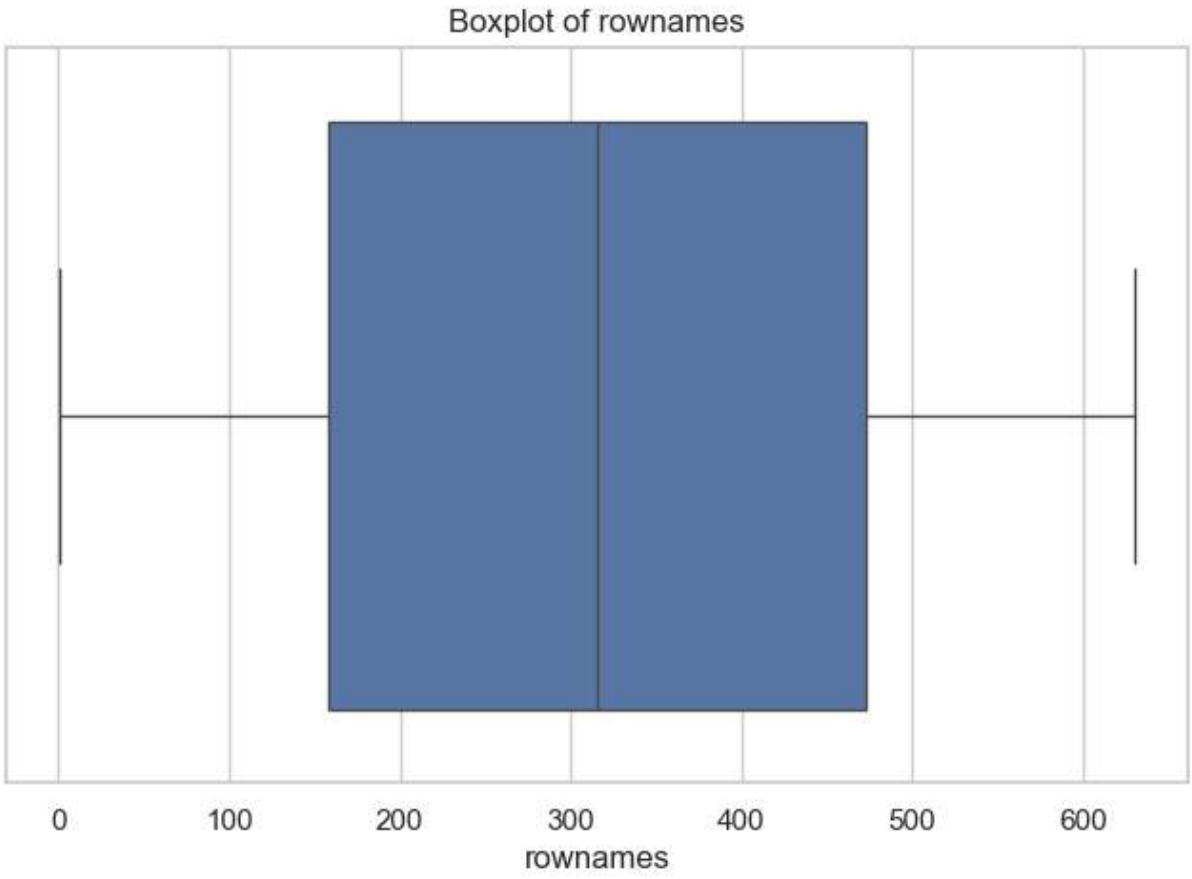

Density Plot of lpctmin
Density
0.0
0.1
0.2
0.3
0.4
0.5
0
1
2
3
4
5
lpctmin

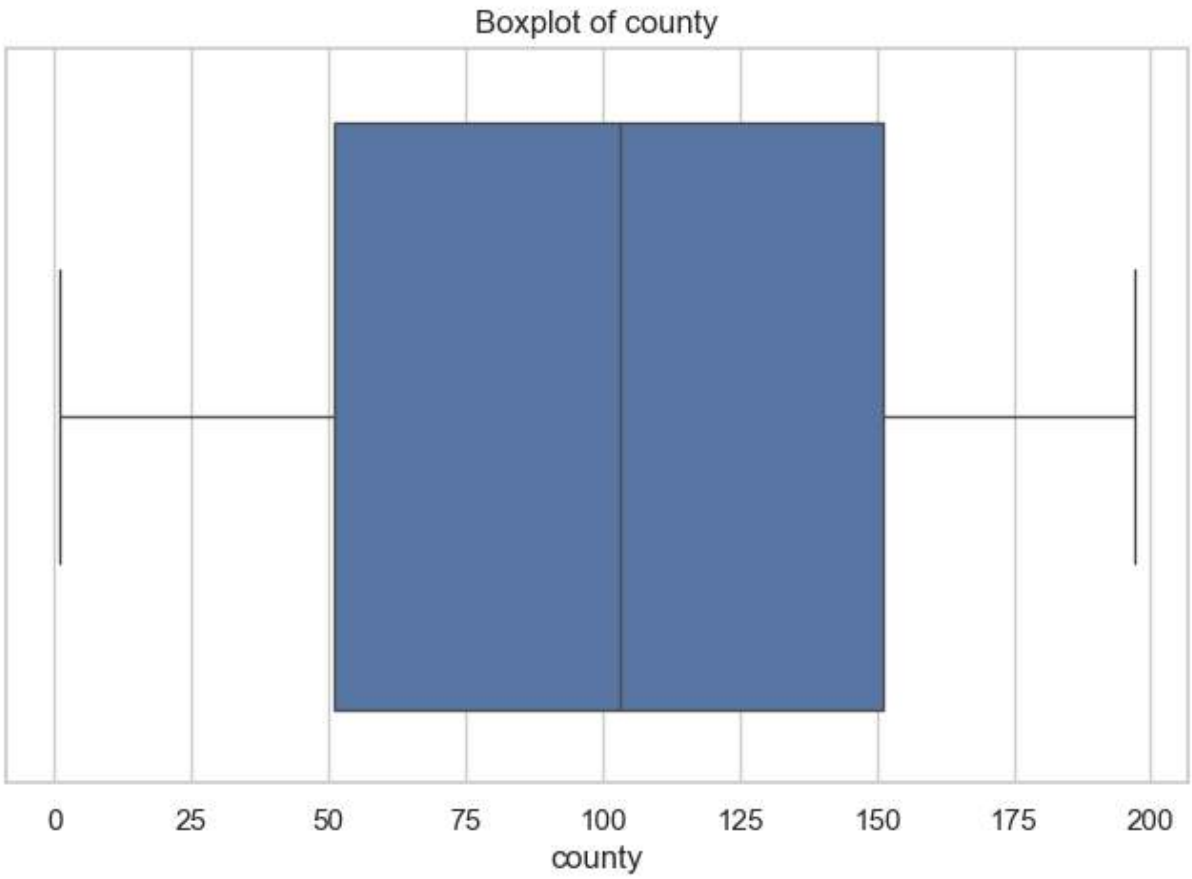

Density Plot of ltaxpc
Density
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
2.5
3.0
3.5
4.0
4.5
5.0
ltaxpc

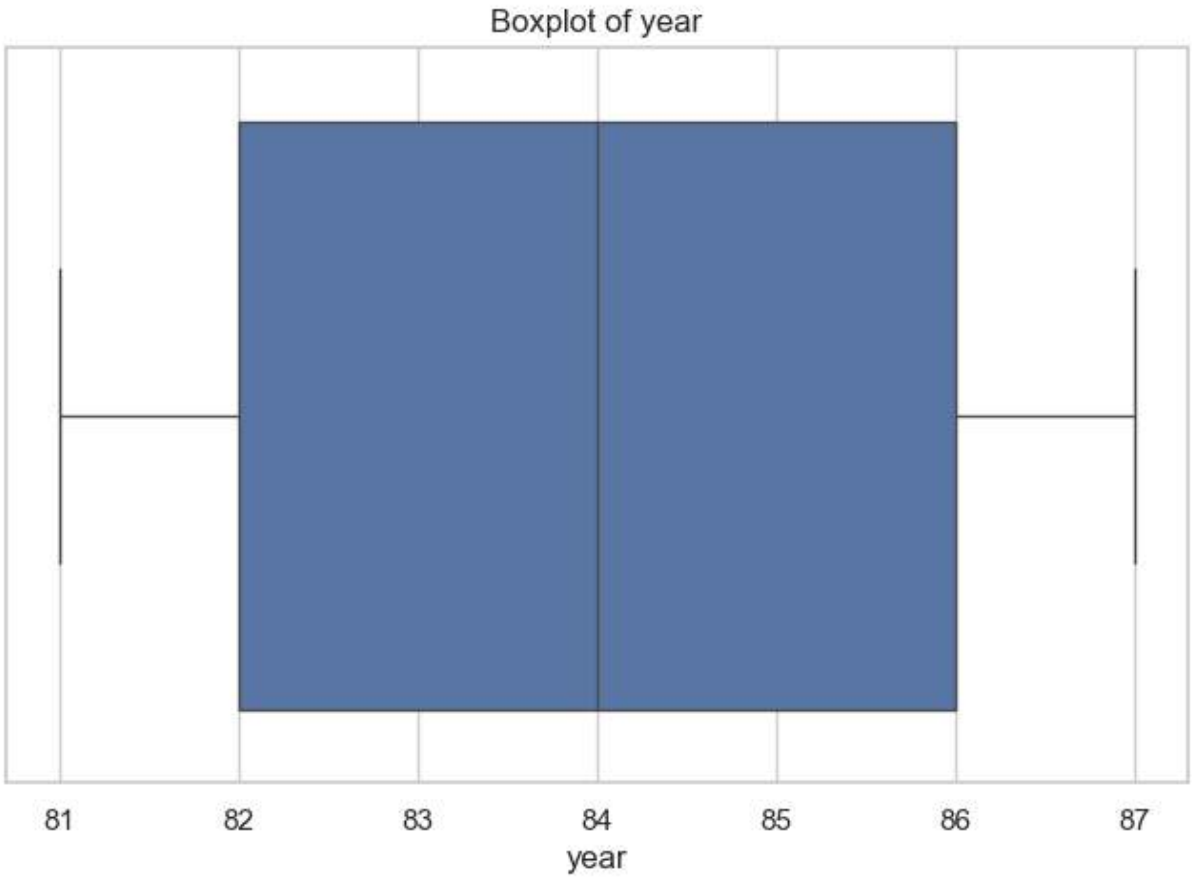


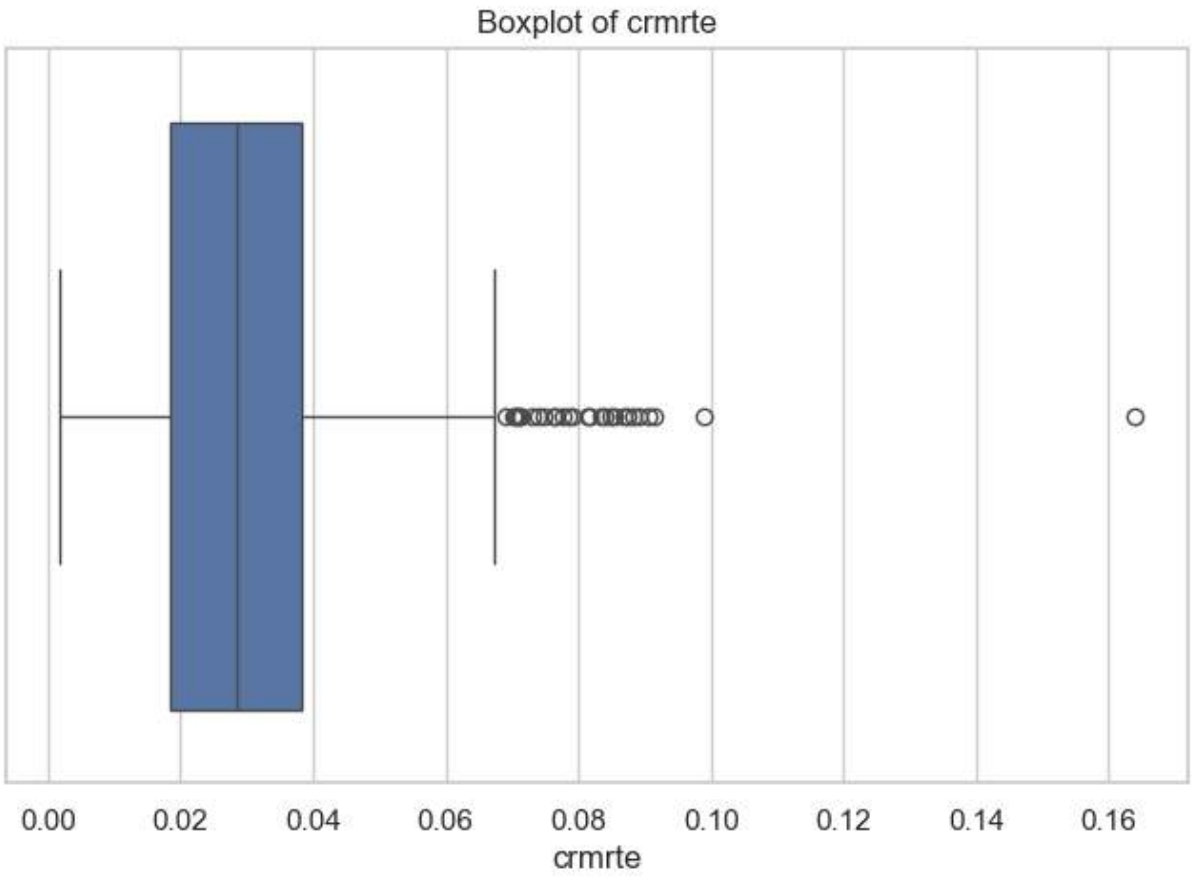


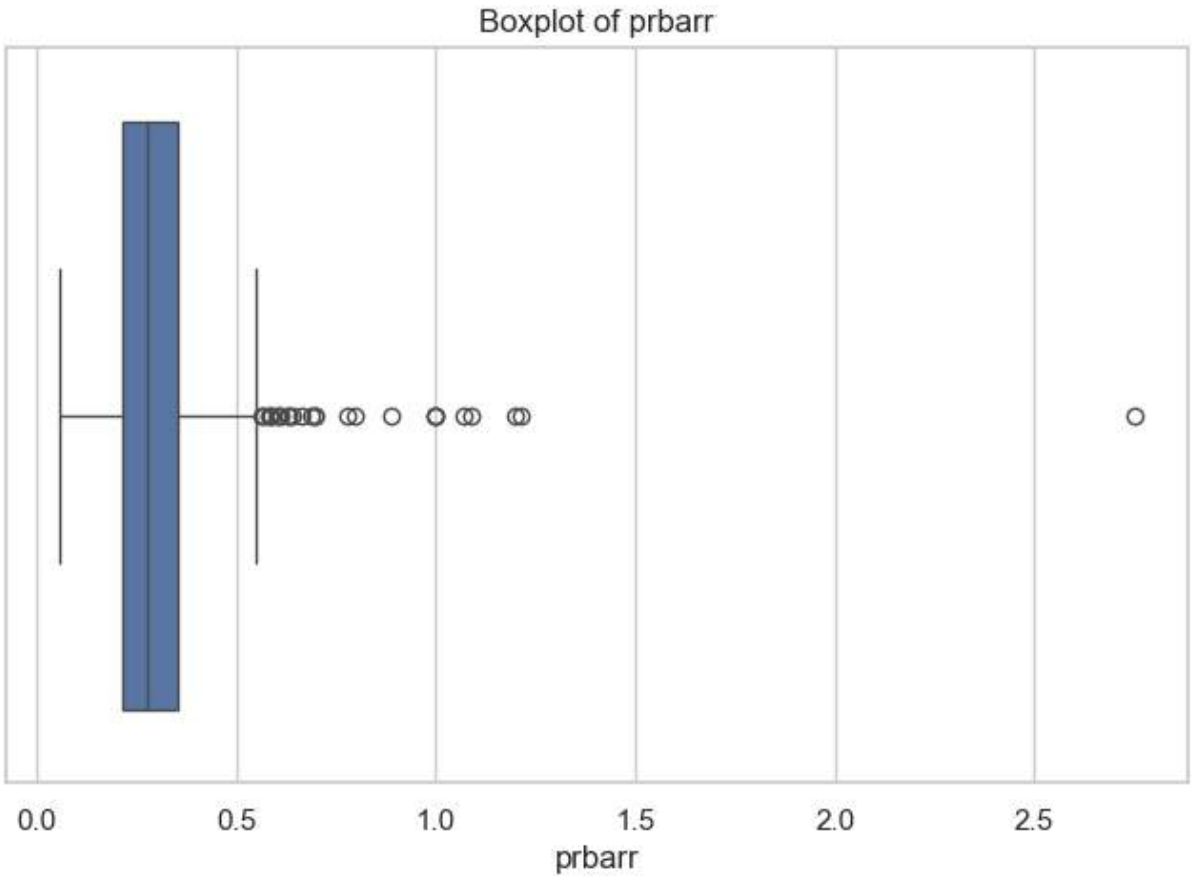
In [26]:
```
for column in numeric.columns:
    plt.figure()
    sns.boxplot(data=clean, x=column)
    plt.title("Boxplot of " + column)
    plt.show()
```

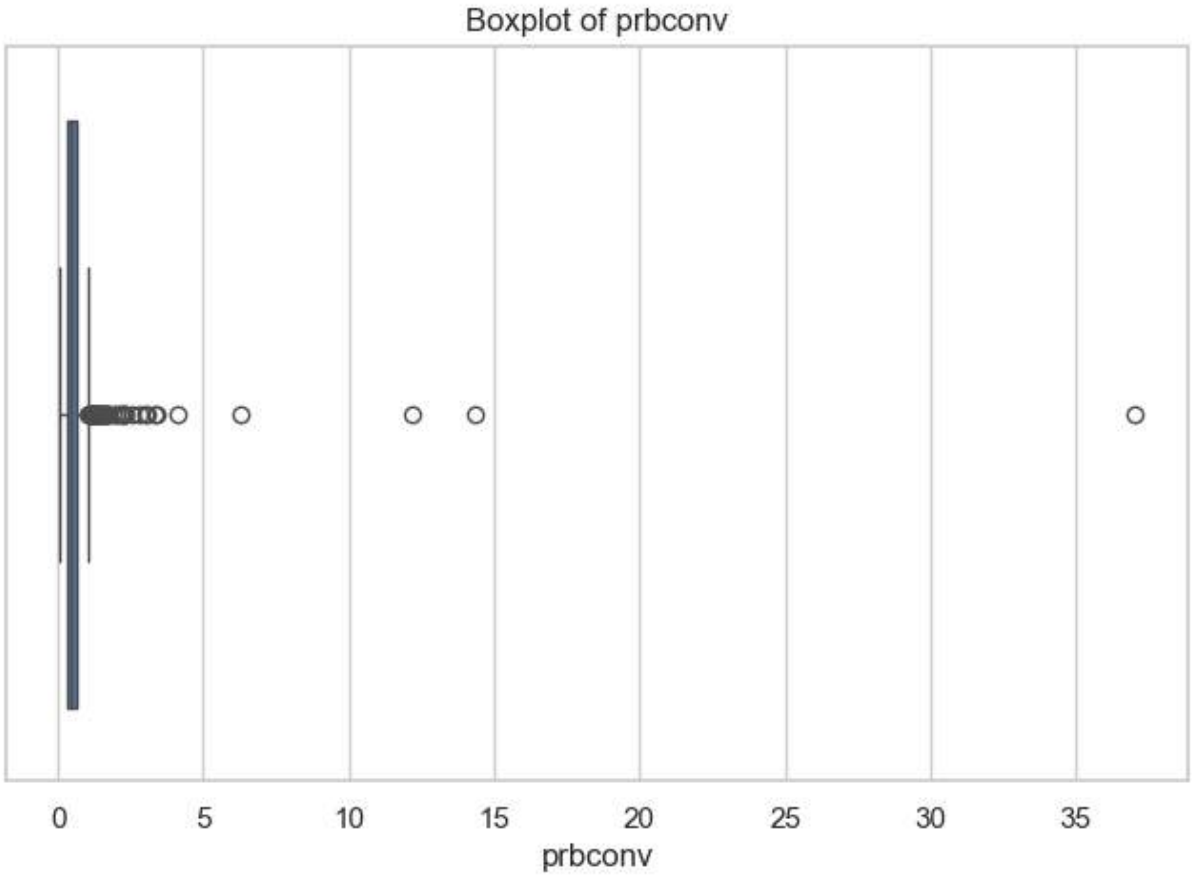

Boxplot of rownames
0
100
200
300
400
500
600
rownames

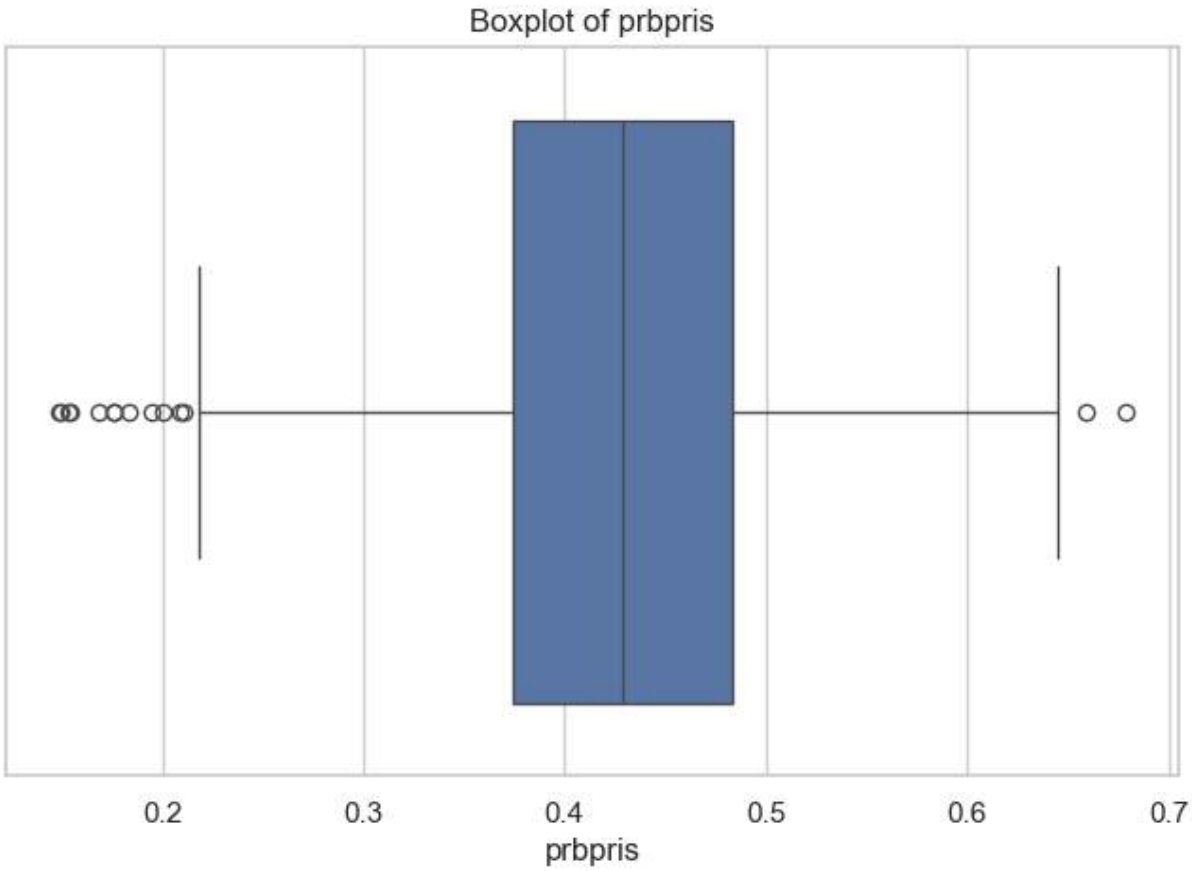

Boxplot of county
0
25
50
75
100
125
150
175
200
county

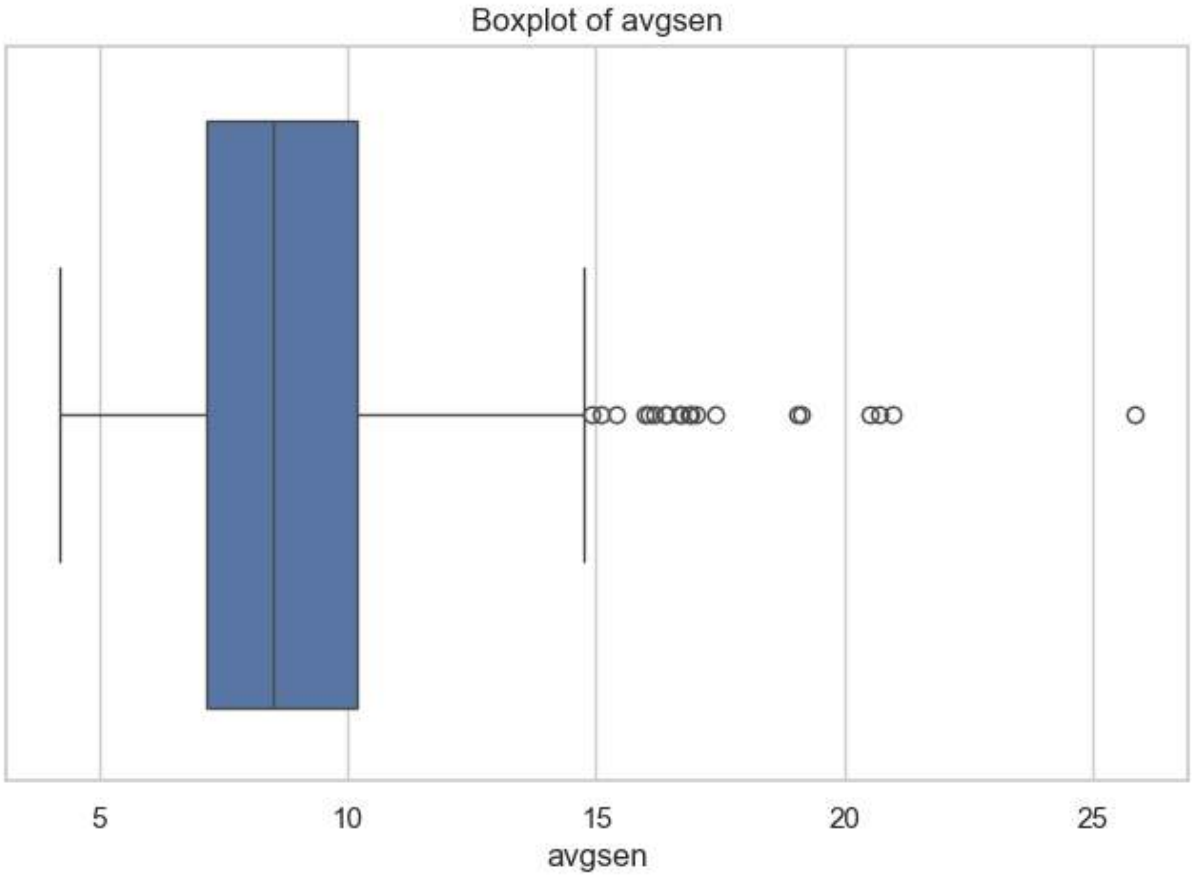

Boxplot of year
81
82
83
84
85
86
87
year

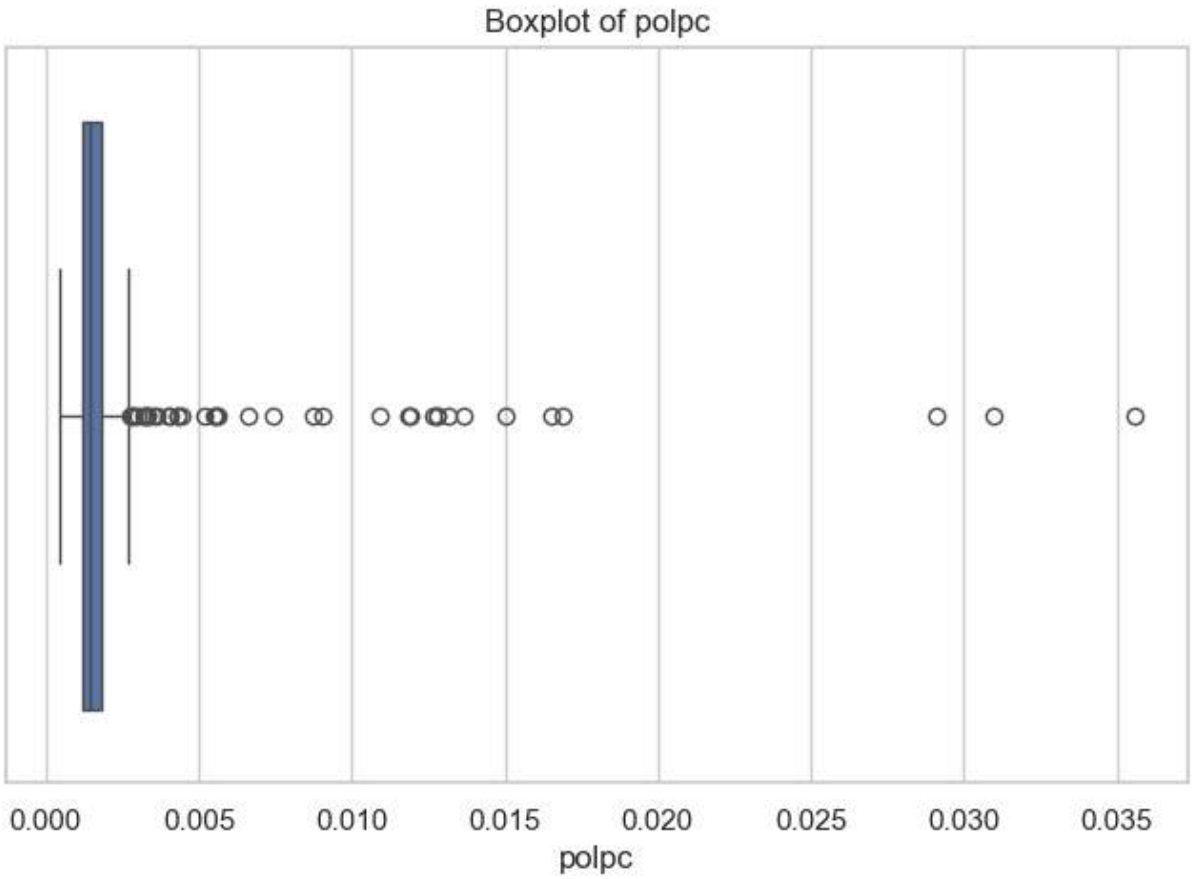

Boxplot of crmrte
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16
crmrte

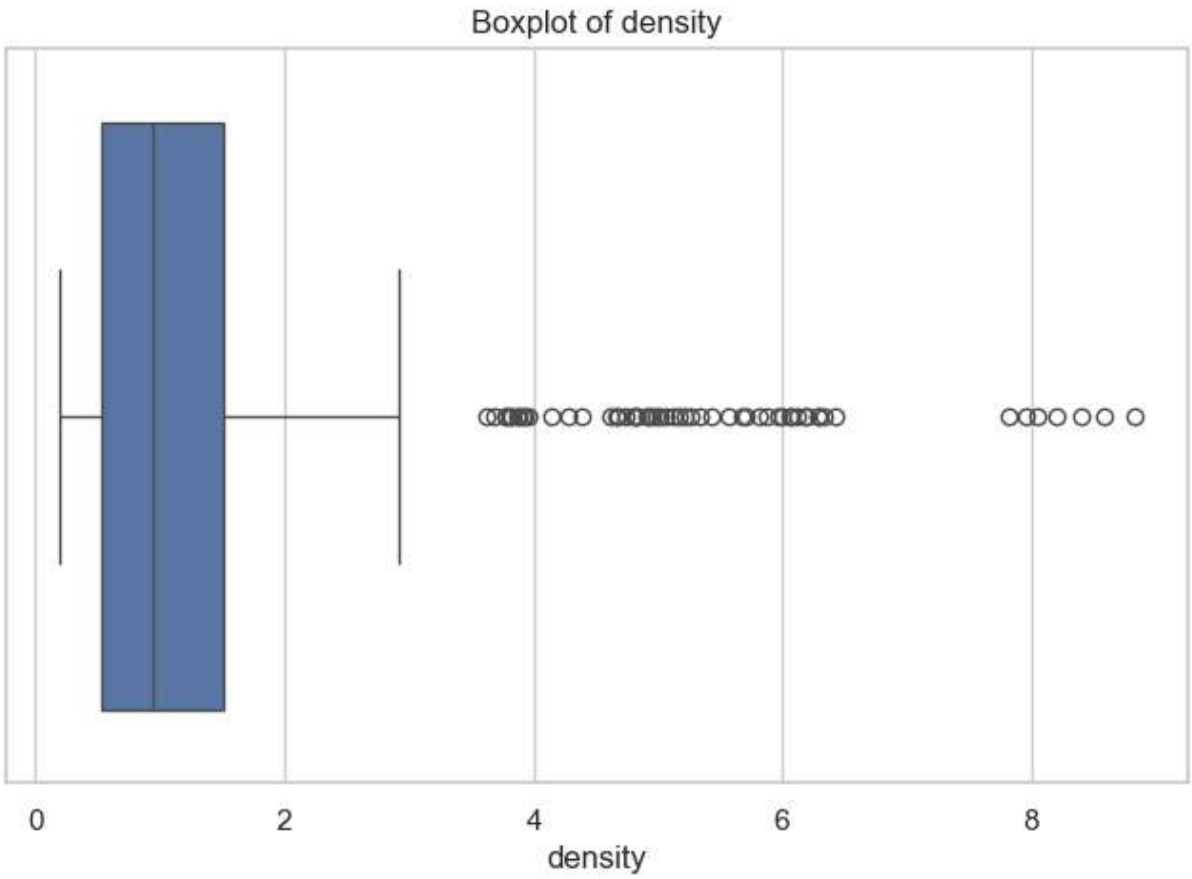

Boxplot of prbarr
0.0
0.5
1.0
1.5
2.0
2.5
prbarr

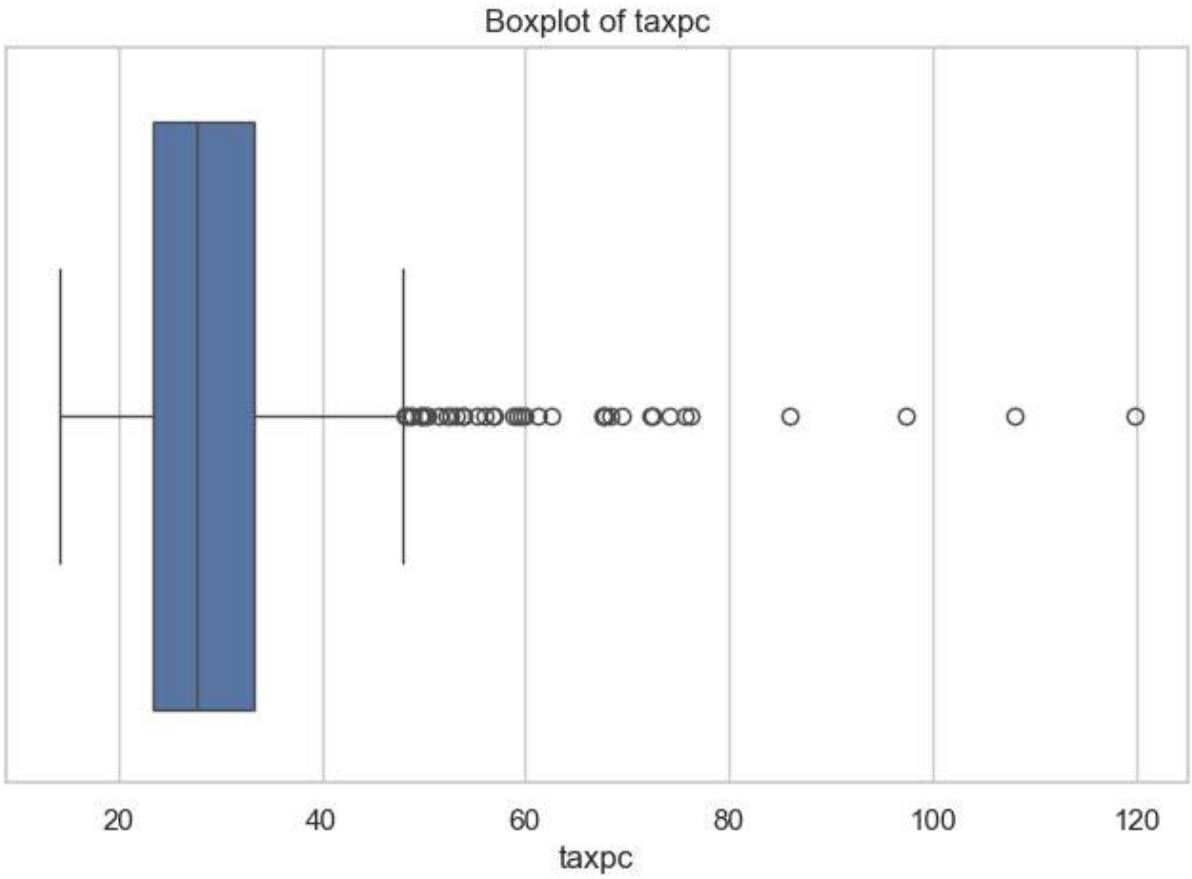

Boxplot of prbconv
0
5
10
15
20
25
30
35
prbconv

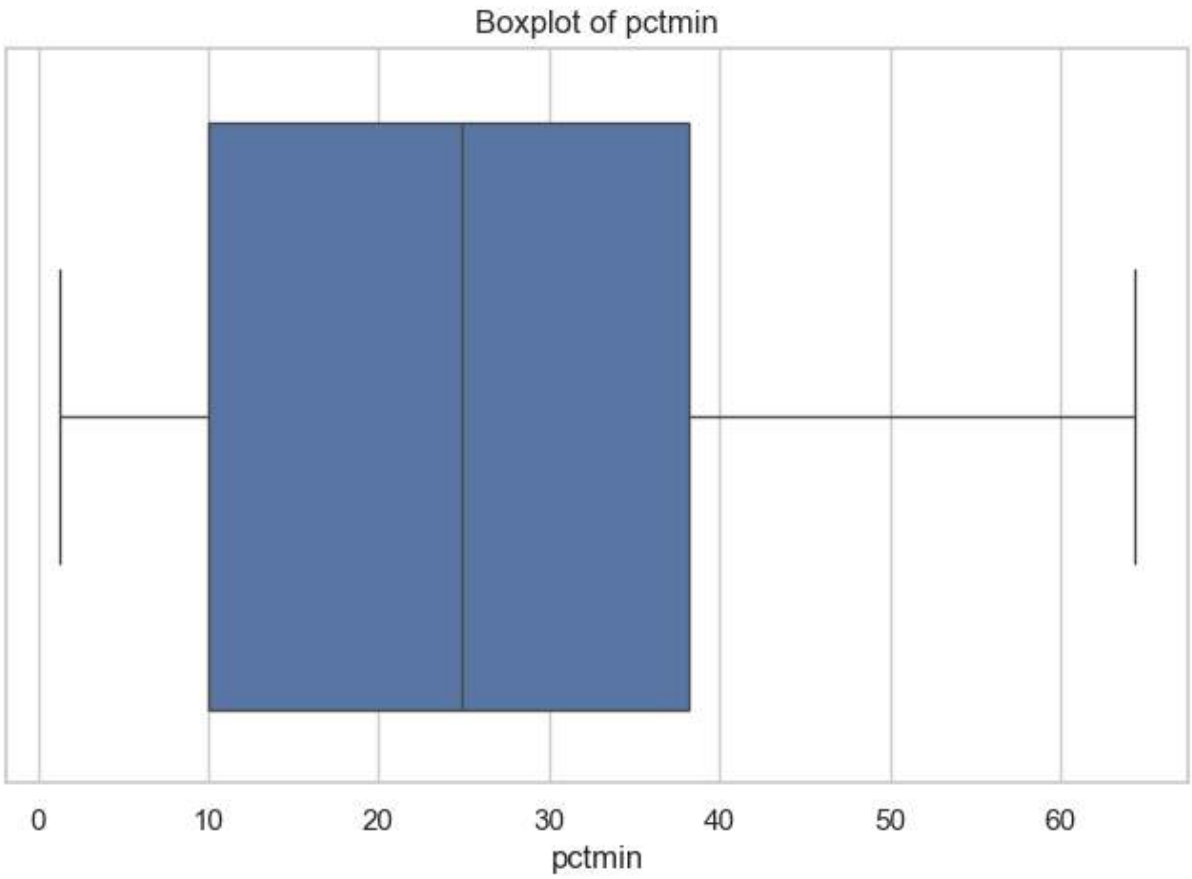

Boxplot of prbpris
0.2
0.3
0.4
0.5
0.6
0.7
prbpris

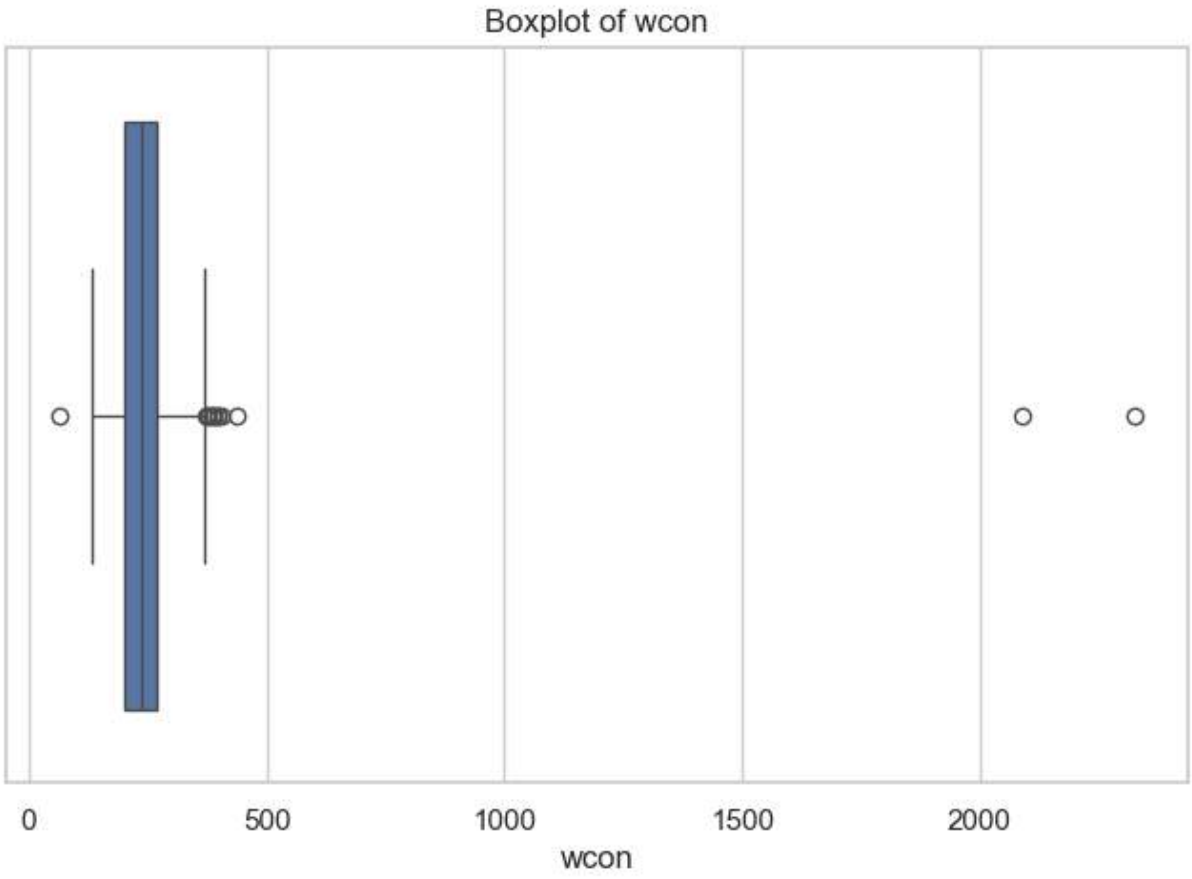

Boxplot of avgsen
5
10
15
20
25
avgsen

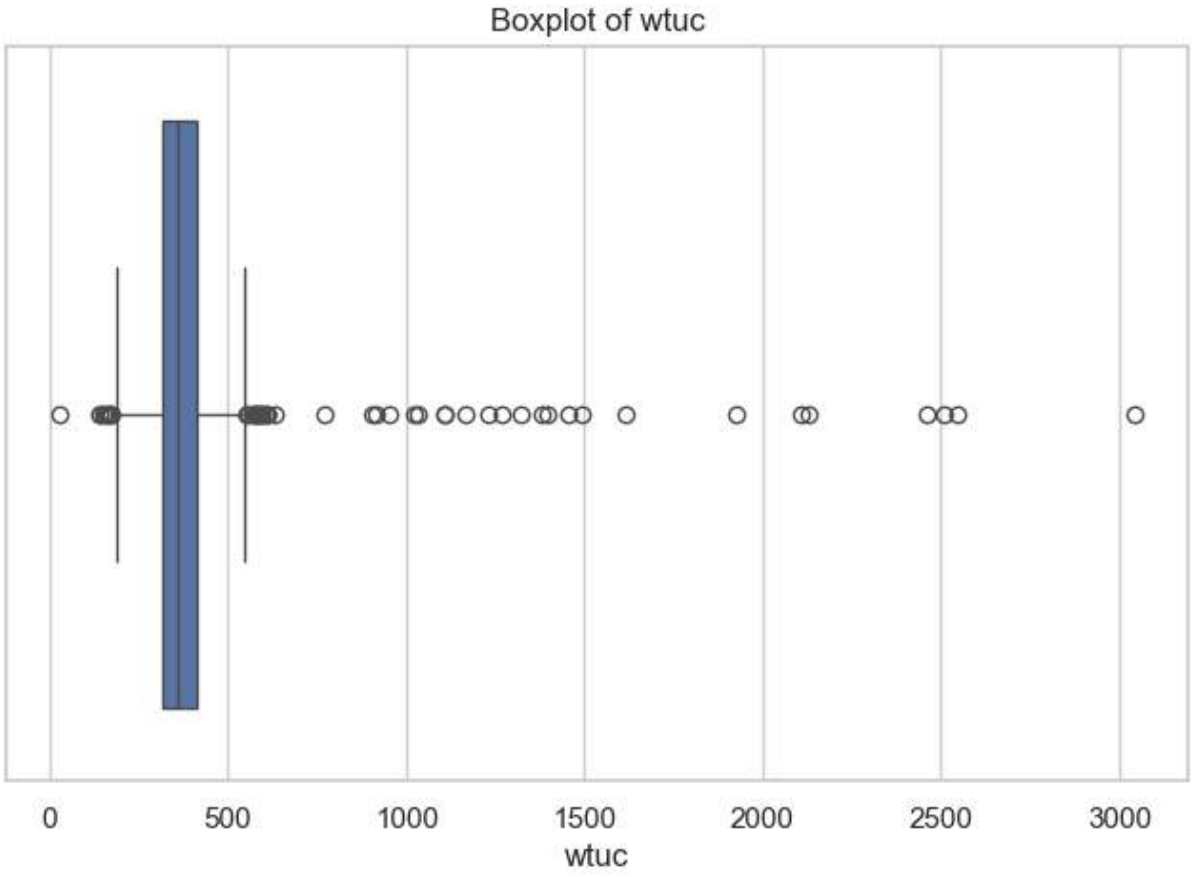

Boxplot of polpc
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035
polpc

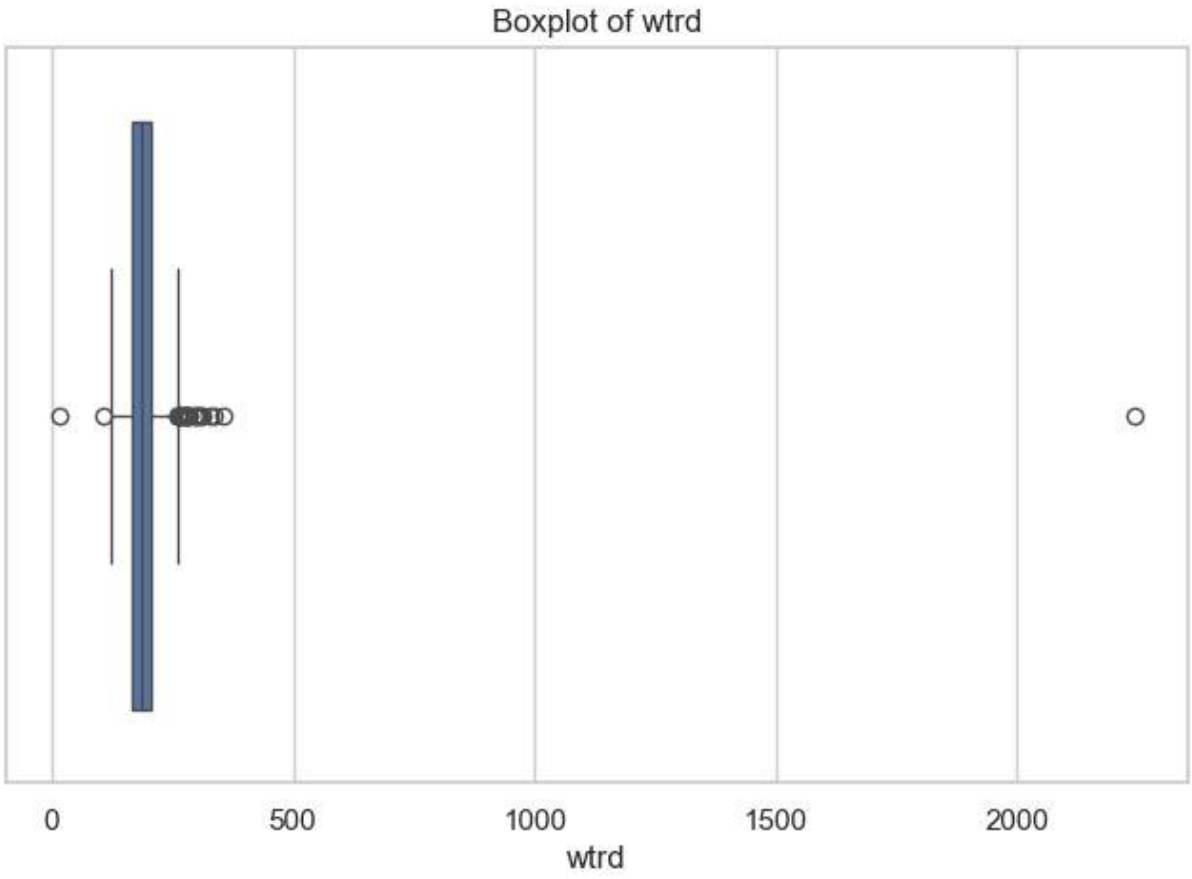

Boxplot of density
0
2
4
6
8
density

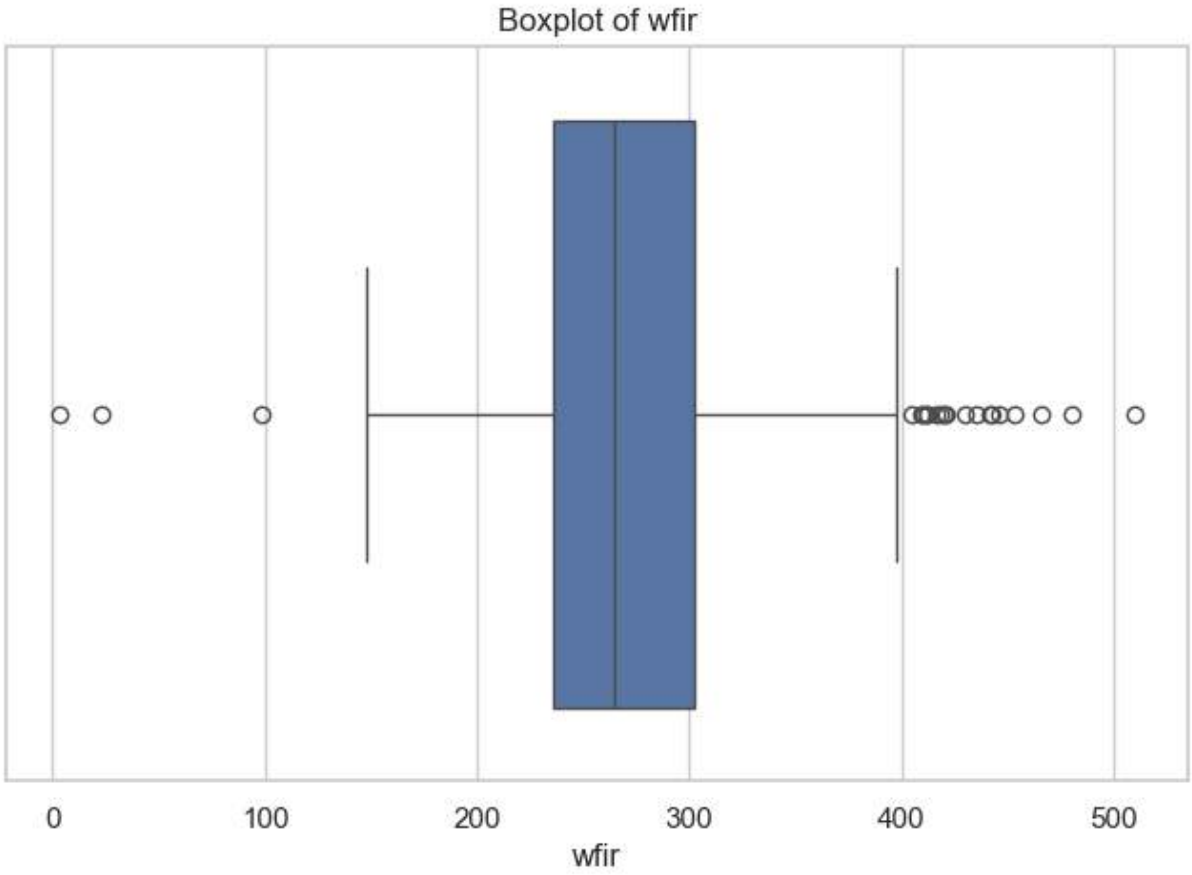

Boxplot of taxpc
20
40
60
80
100
120
taxpc

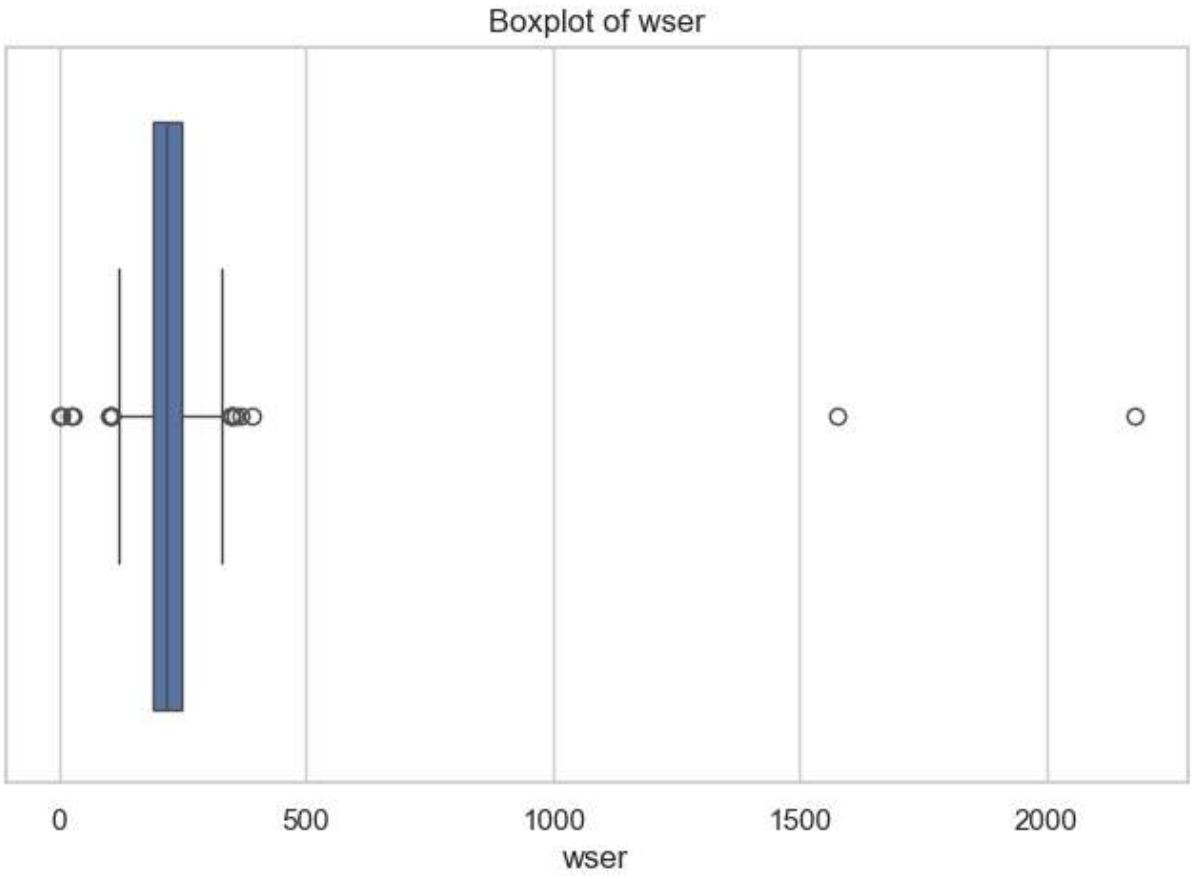

Boxplot of pctmin
0
10
20
30
40
50
60
pctmin

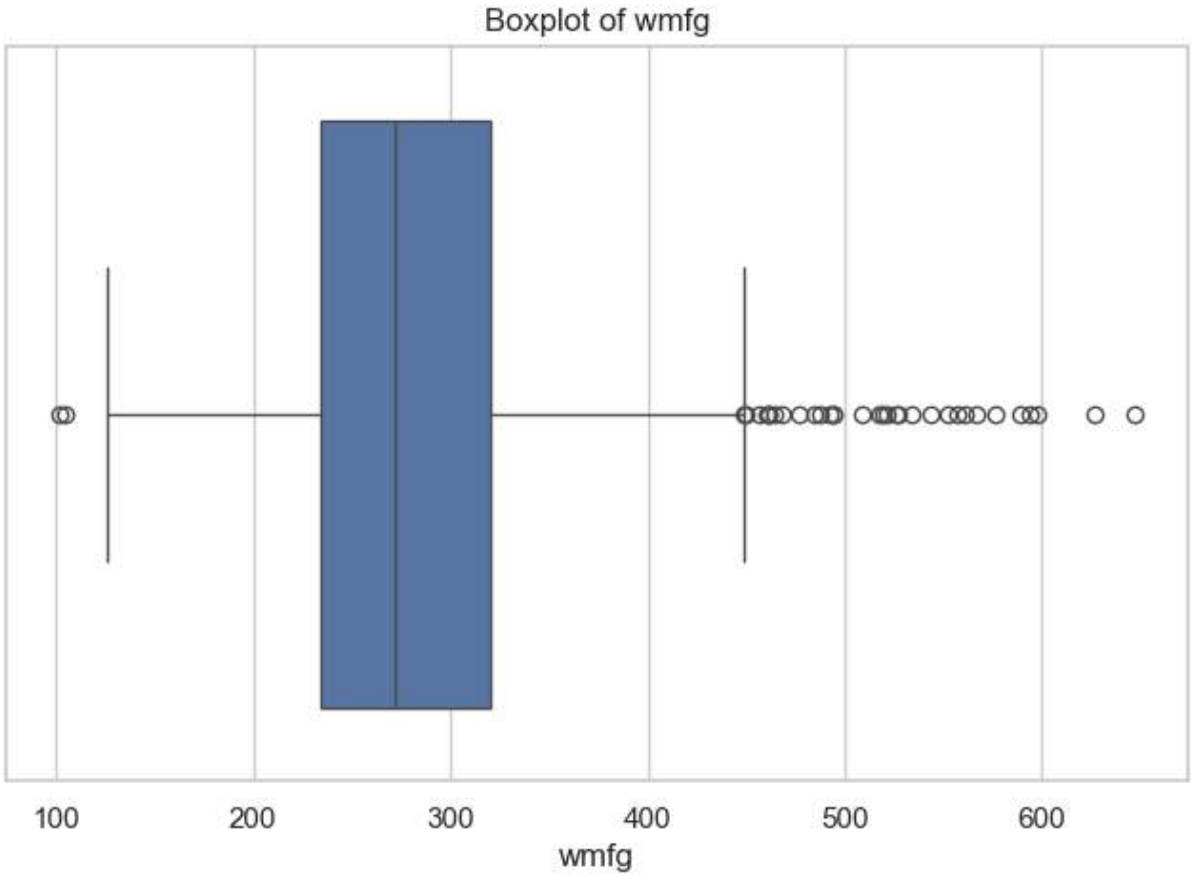

Boxplot of wcon
0
500
1000
1500
2000
wcon

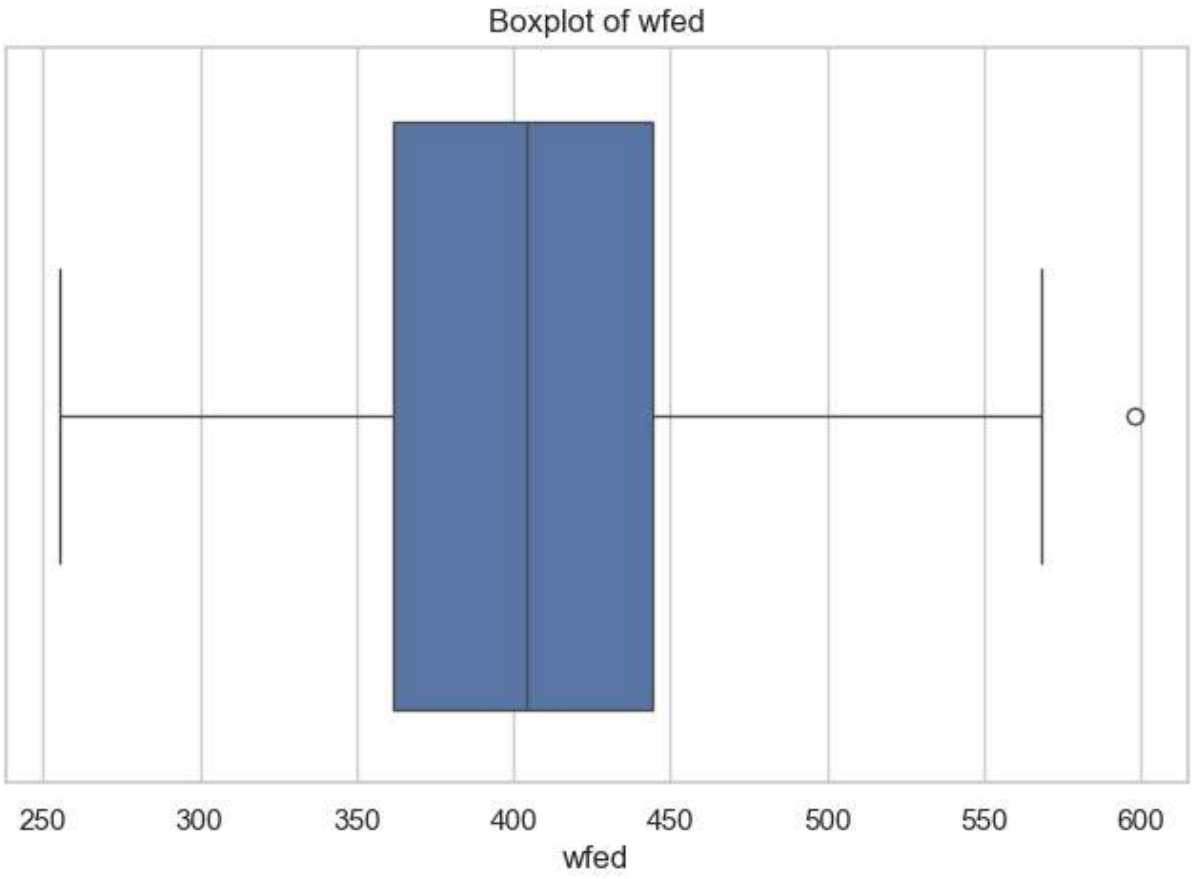

Boxplot of wtuc
0
500
1000
1500
2000
2500
3000
wtuc

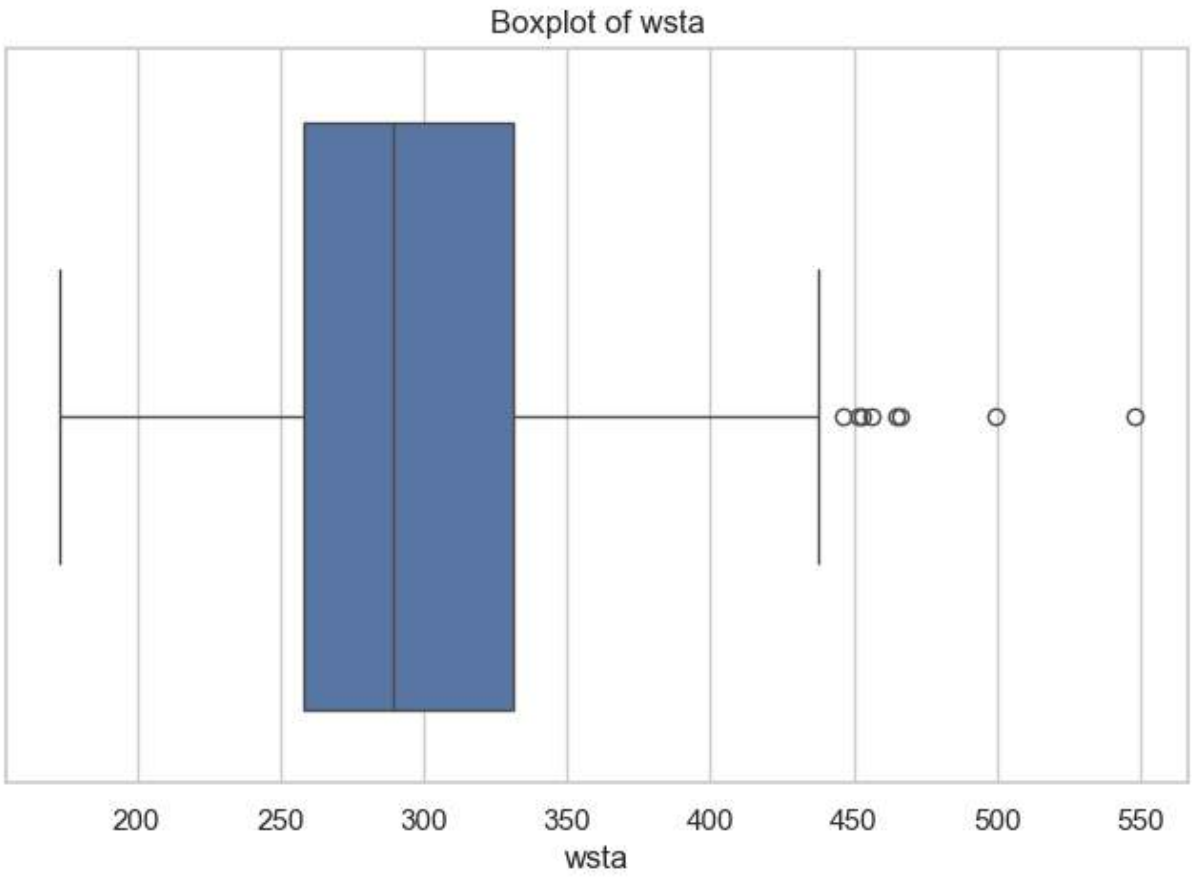

Boxplot of wtrd
0
500
1000
1500
2000
wtrd

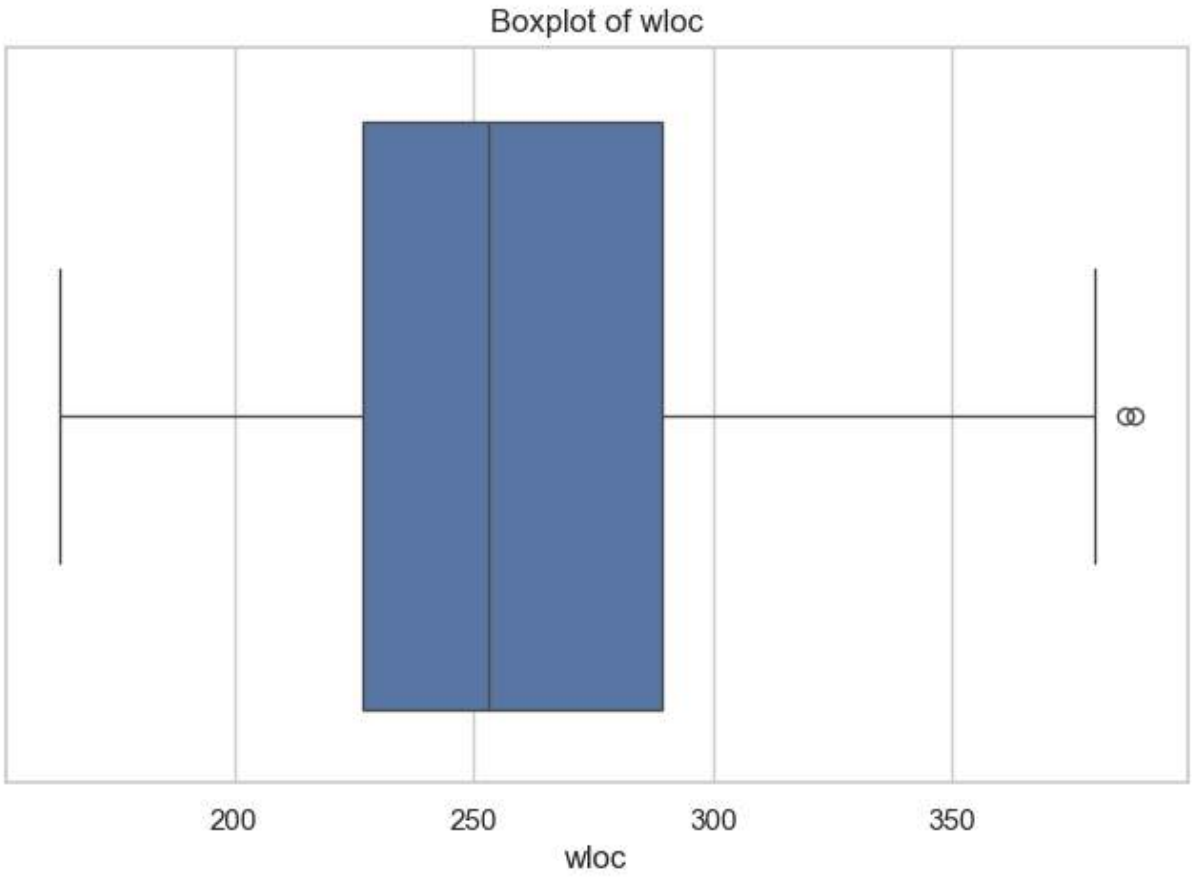

Boxplot of wfir
0
100
200
300
400
500
wfir

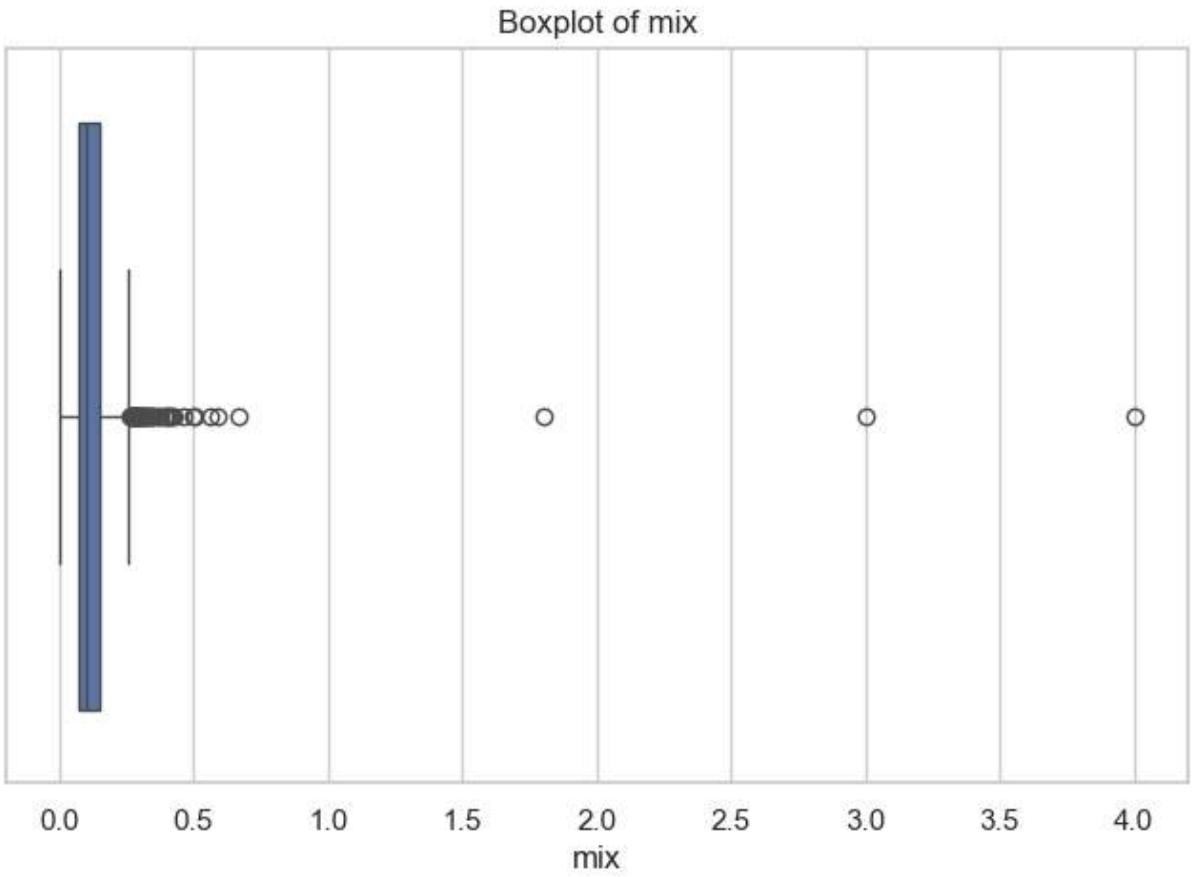

Boxplot of wser
0
500
1000
1500
2000
wser

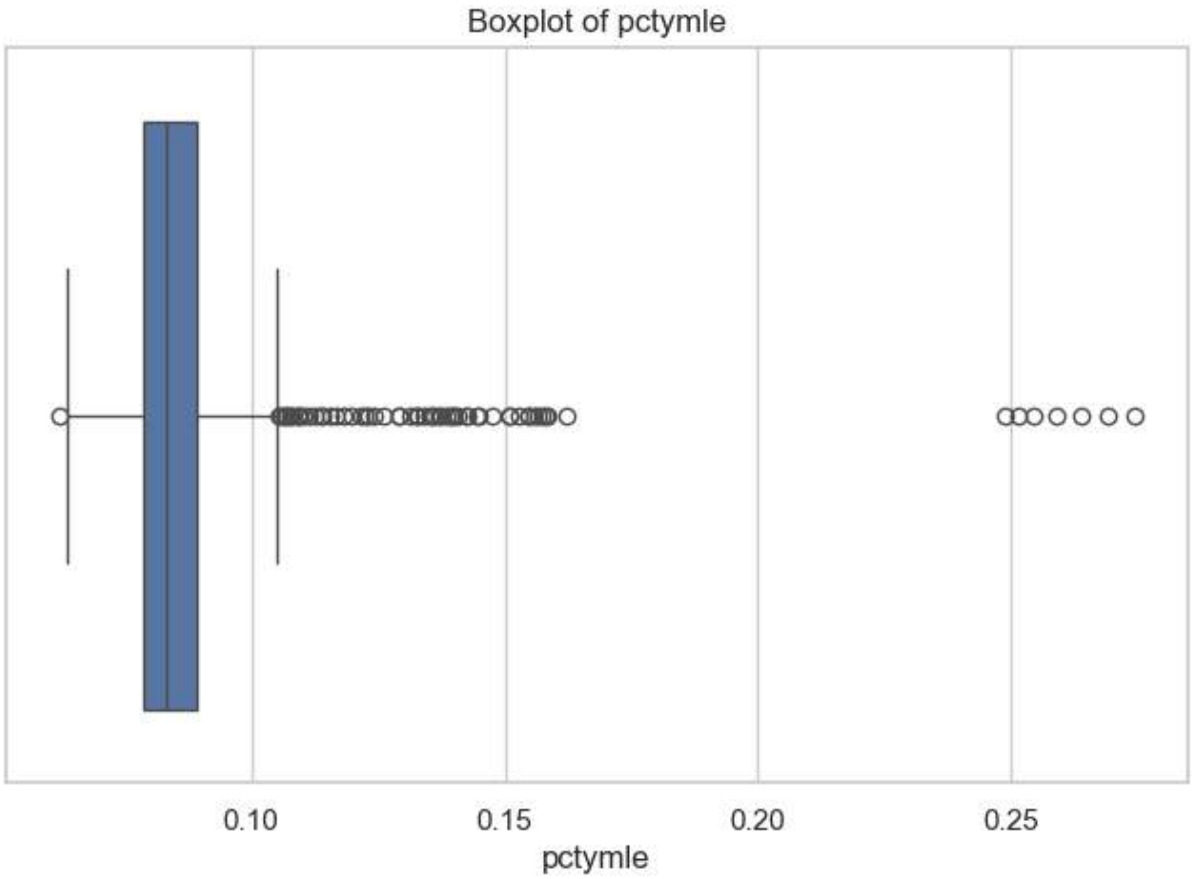

Boxplot of wmfg
100
200
300
400
500
600
wmfg

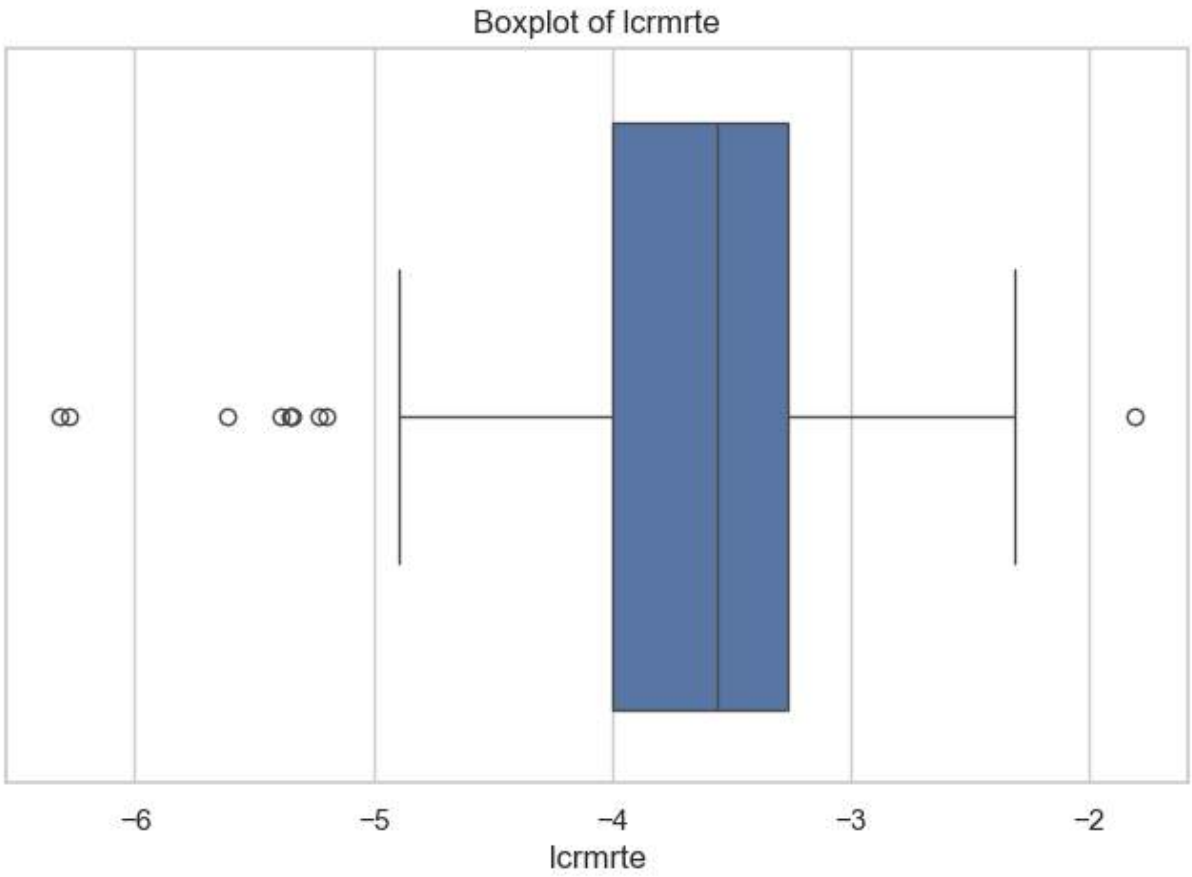

Boxplot of wfed
250
300
350
400
450
500
550
600
wfed

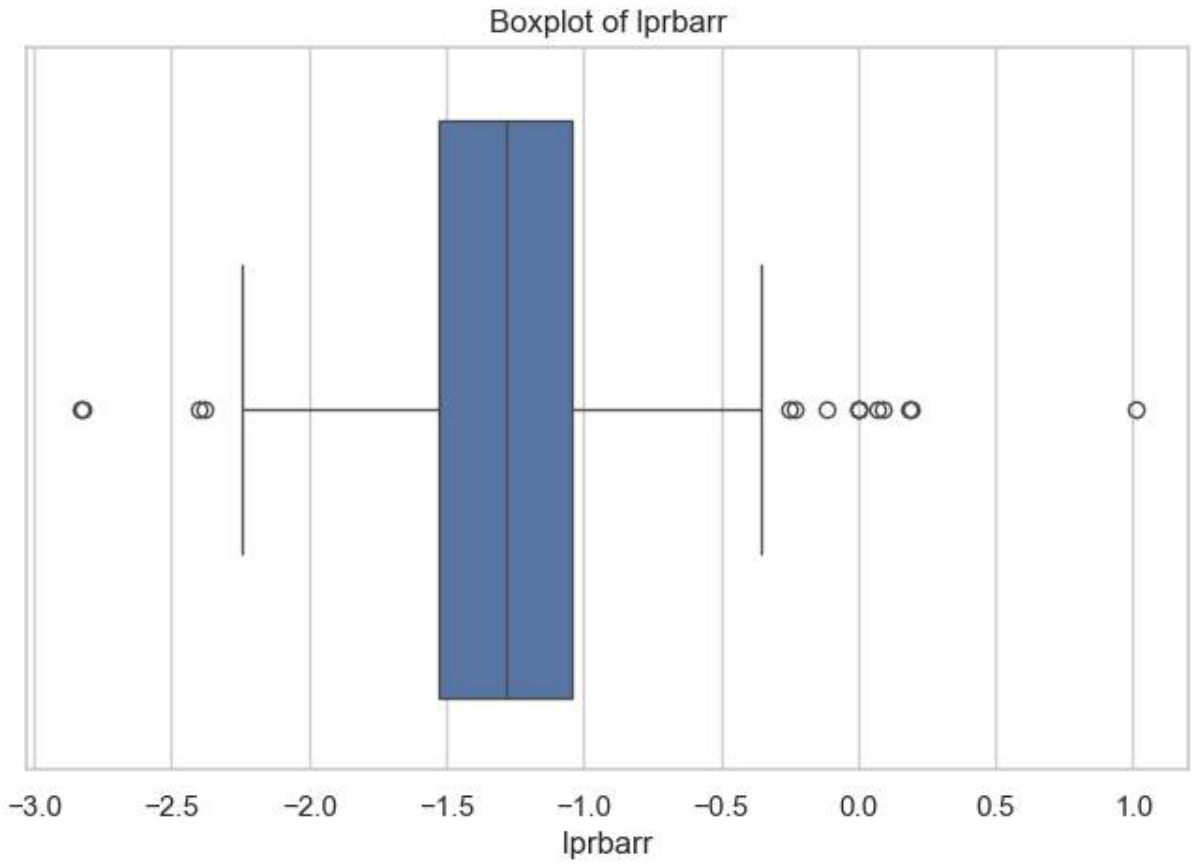

Boxplot of wsta
200
250
300
350
400
450
500
550
wsta

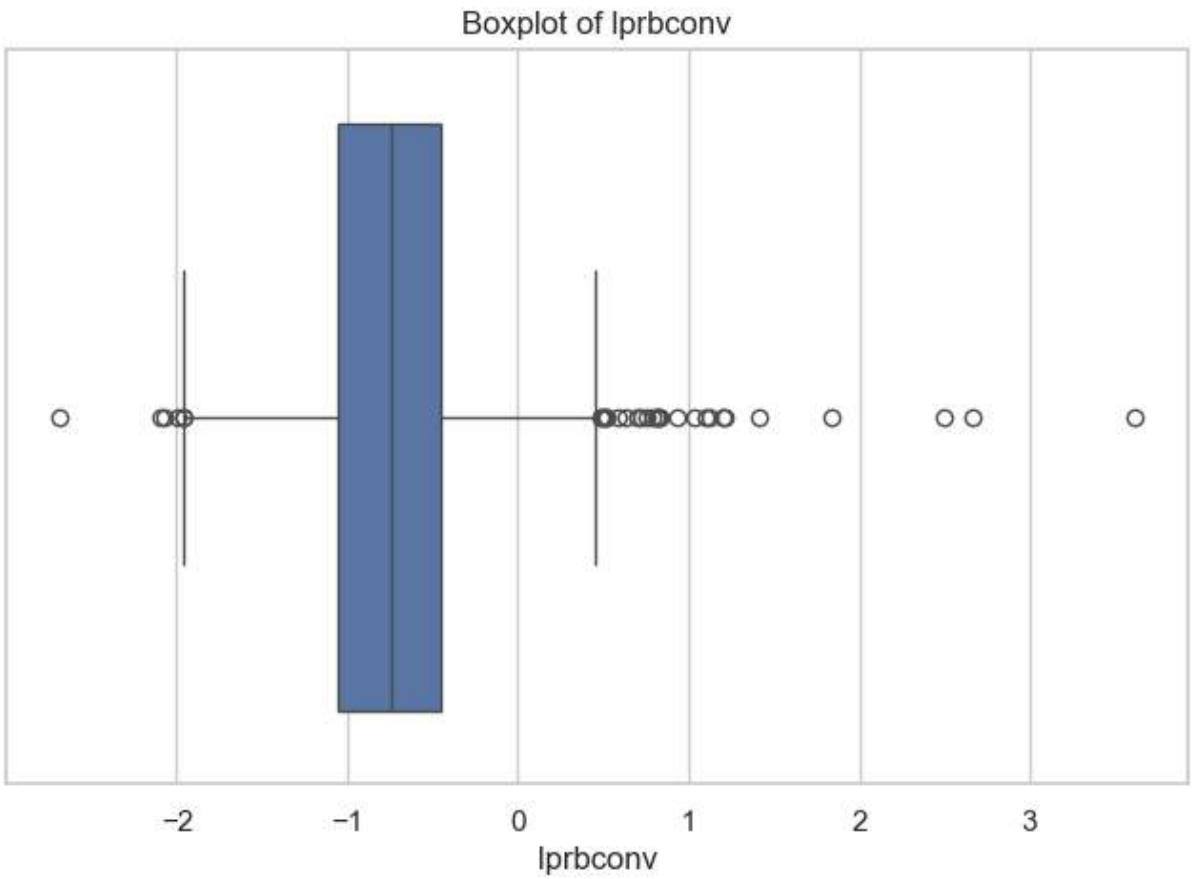

Boxplot of wloc
200
250
300
350
wloc

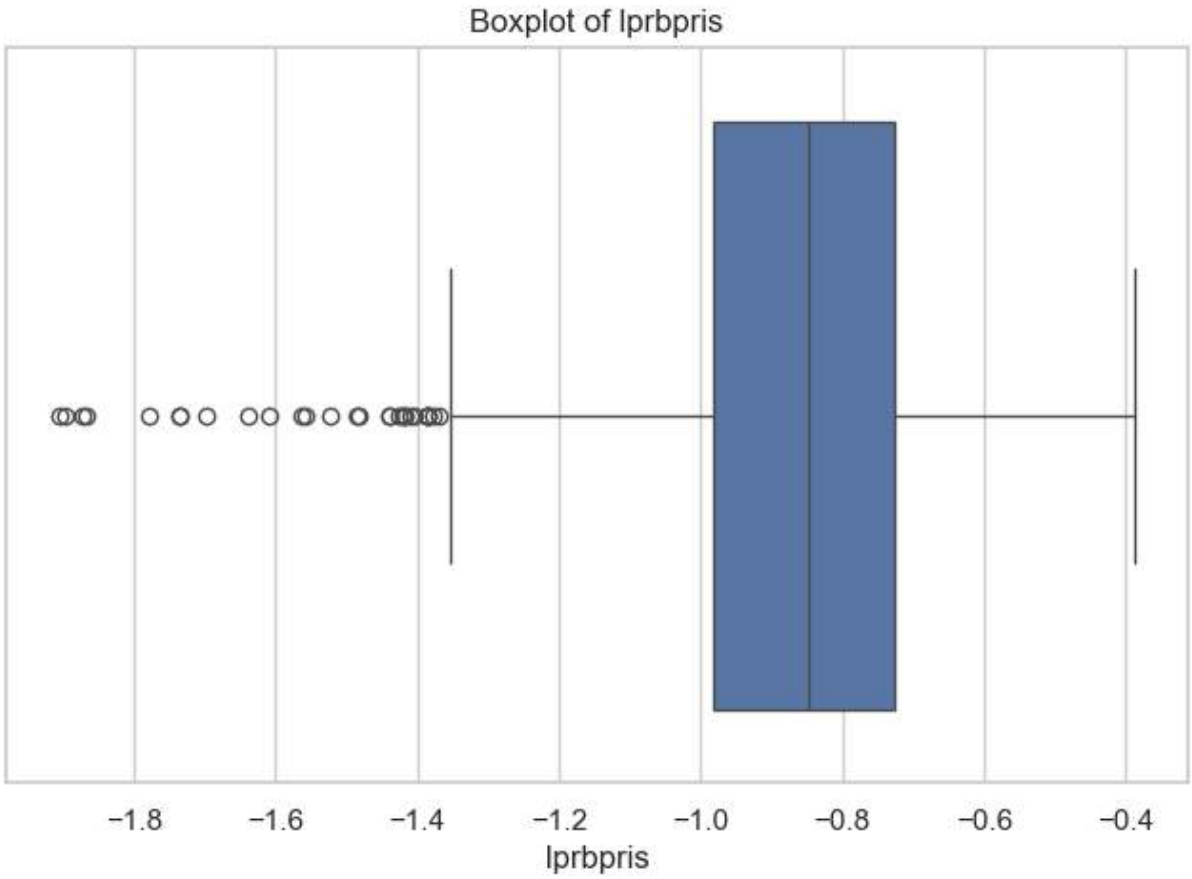

Boxplot of mix
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
mix

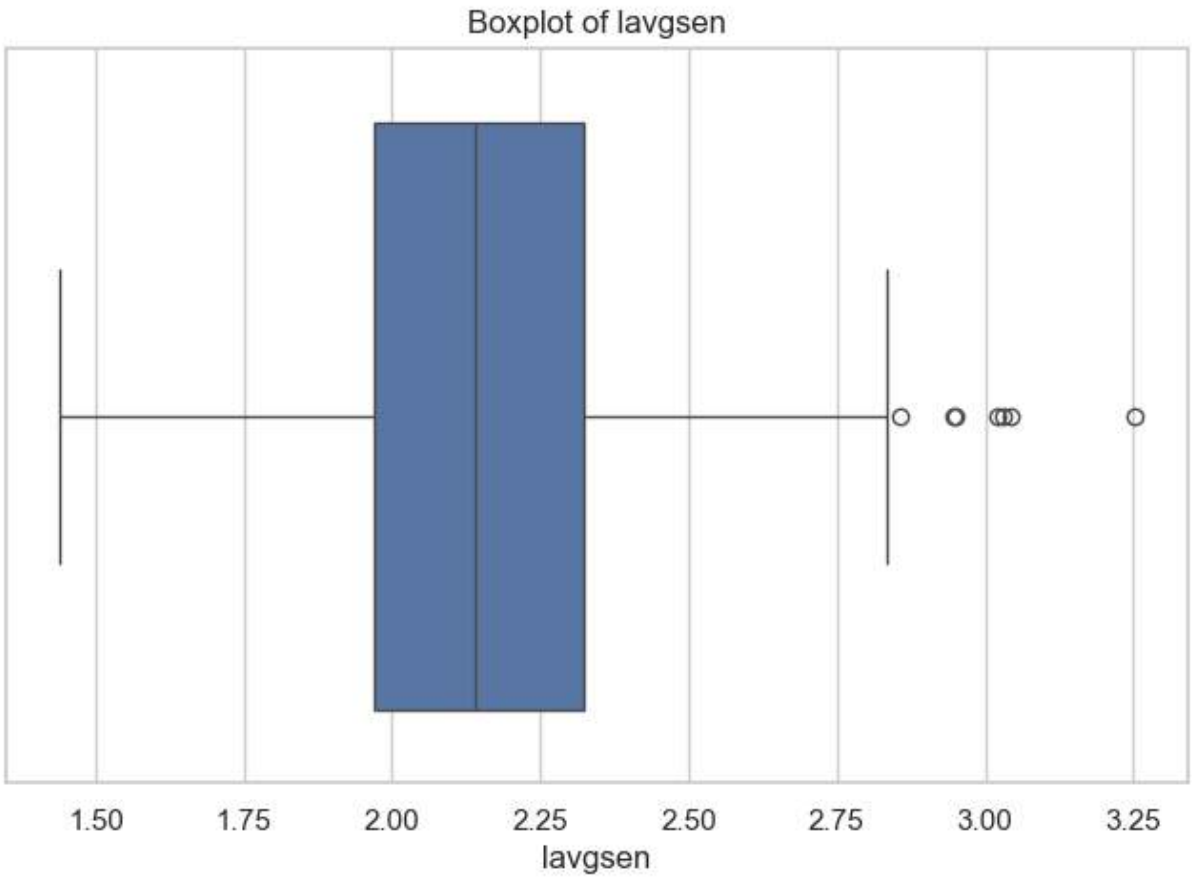

Boxplot of pctymle
0.10
0.15
0.20
0.25
pctymle

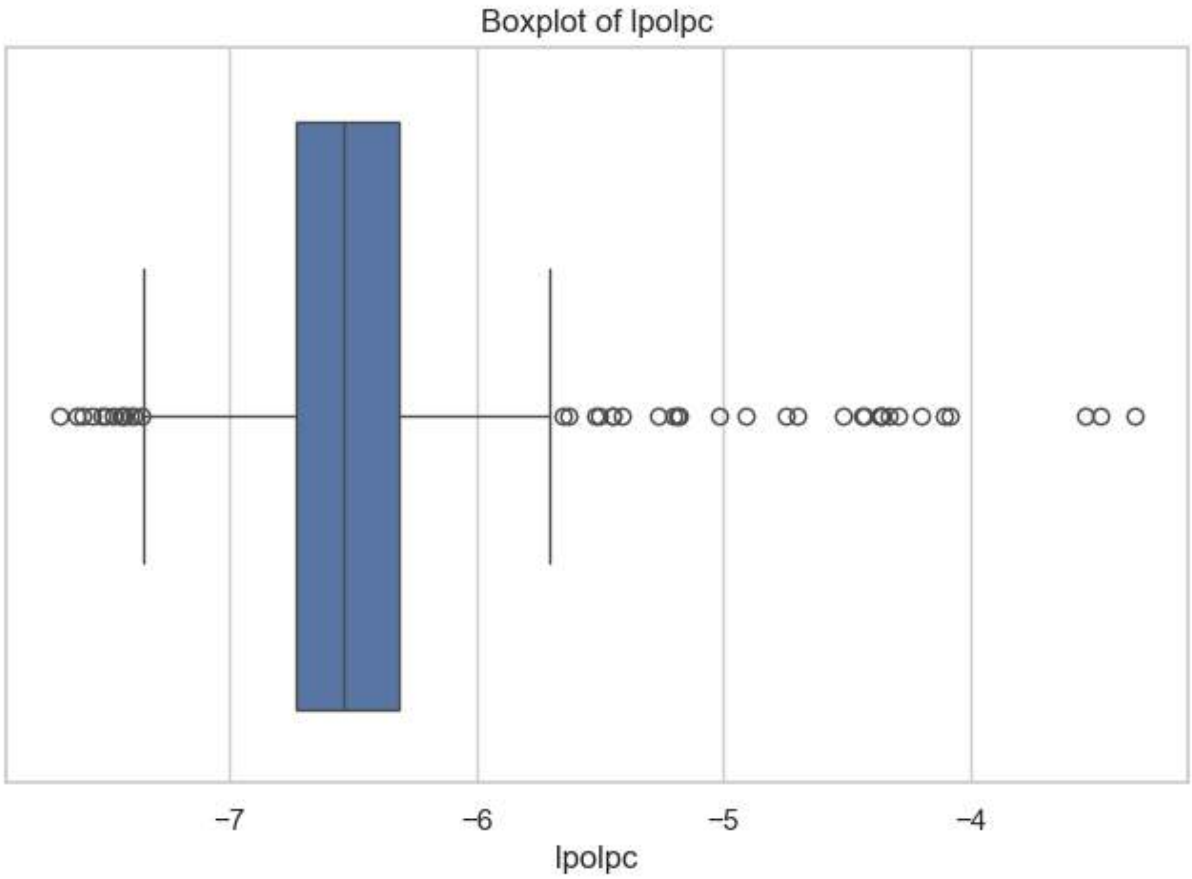

Boxplot of lcrmrte
−6
−5
−4
−3
−2
lcrmrte

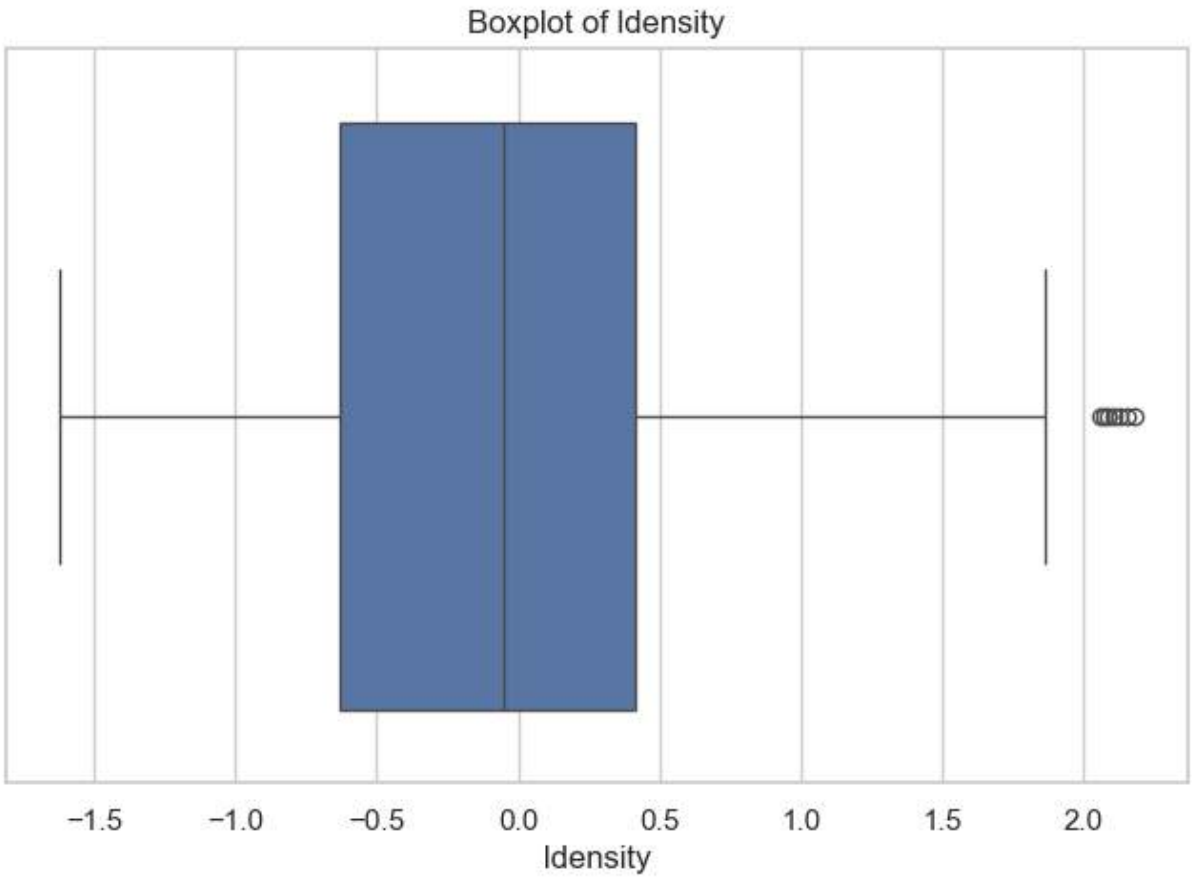

Boxplot of lprbarr
−3.0
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
lprbarr

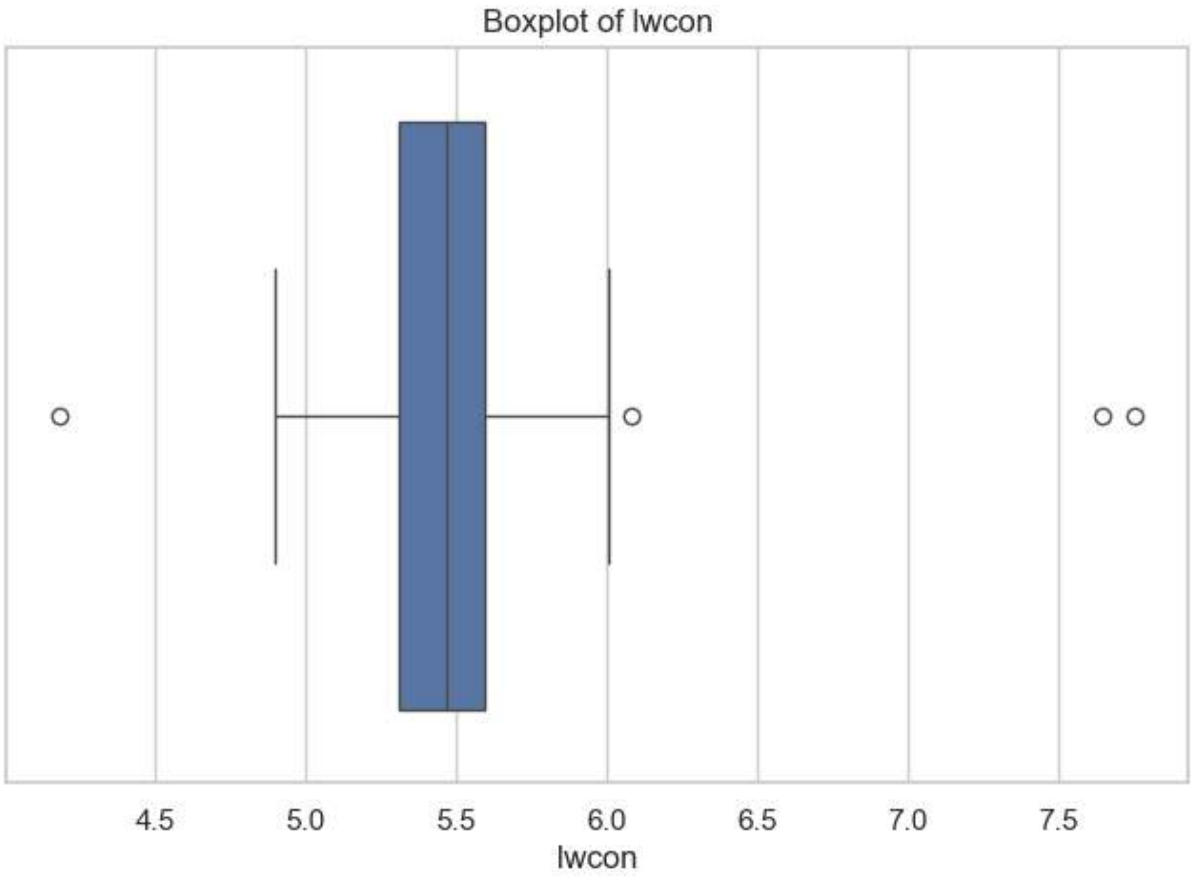

Boxplot of lprbconv
−2
−1
0
1
2
3
lprbconv

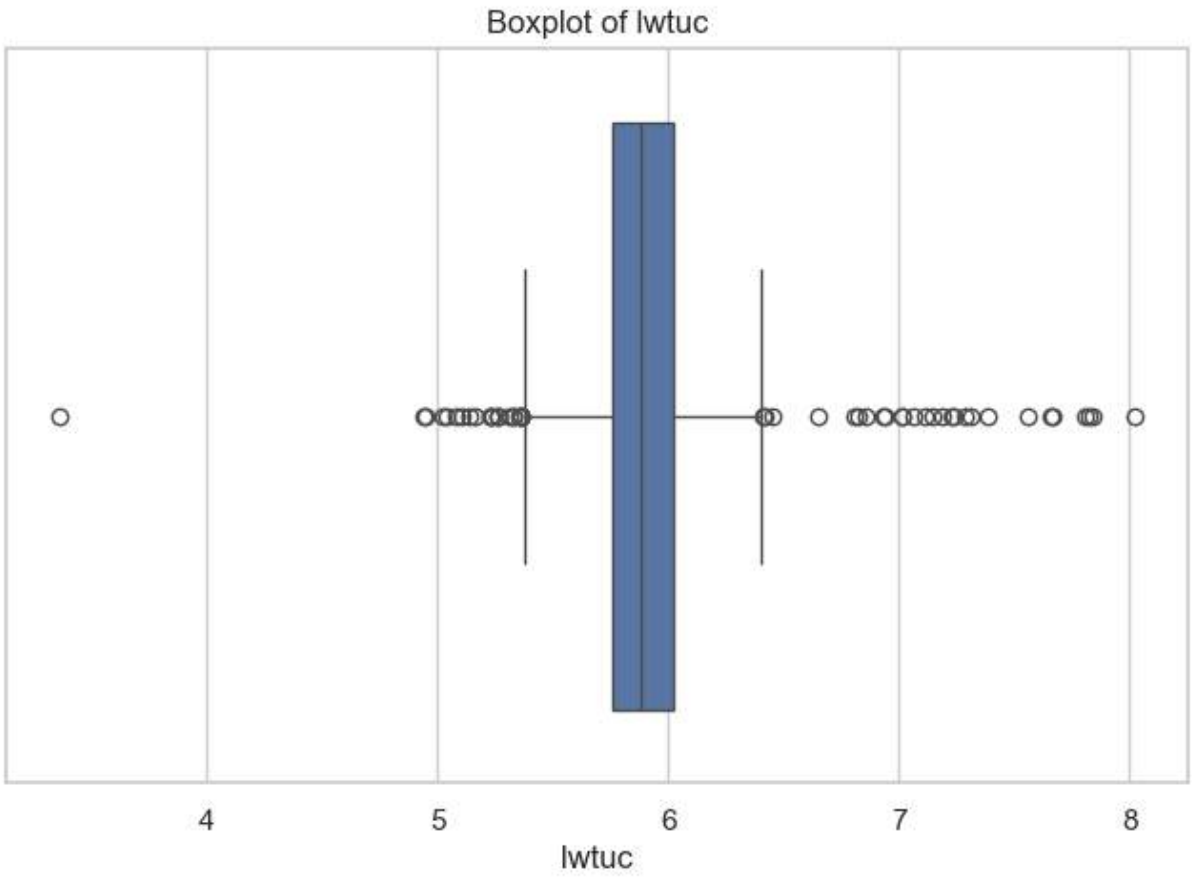

Boxplot of lprbpris
−1.8
−1.6
−1.4
−1.2
−1.0
−0.8
−0.6
−0.4
lprbpris

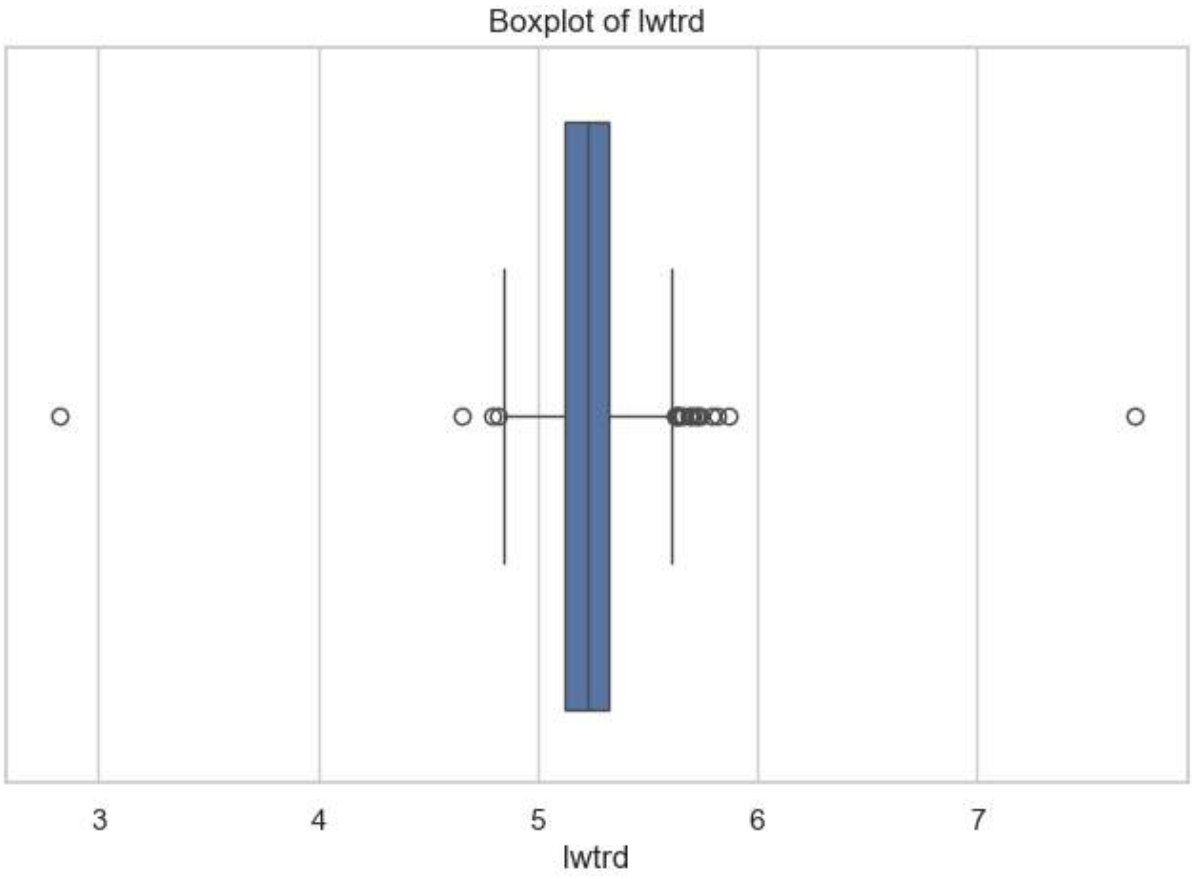

Boxplot of lavgsen
1.50
1.75
2.00
2.25
2.50
2.75
3.00
3.25
lavgsen

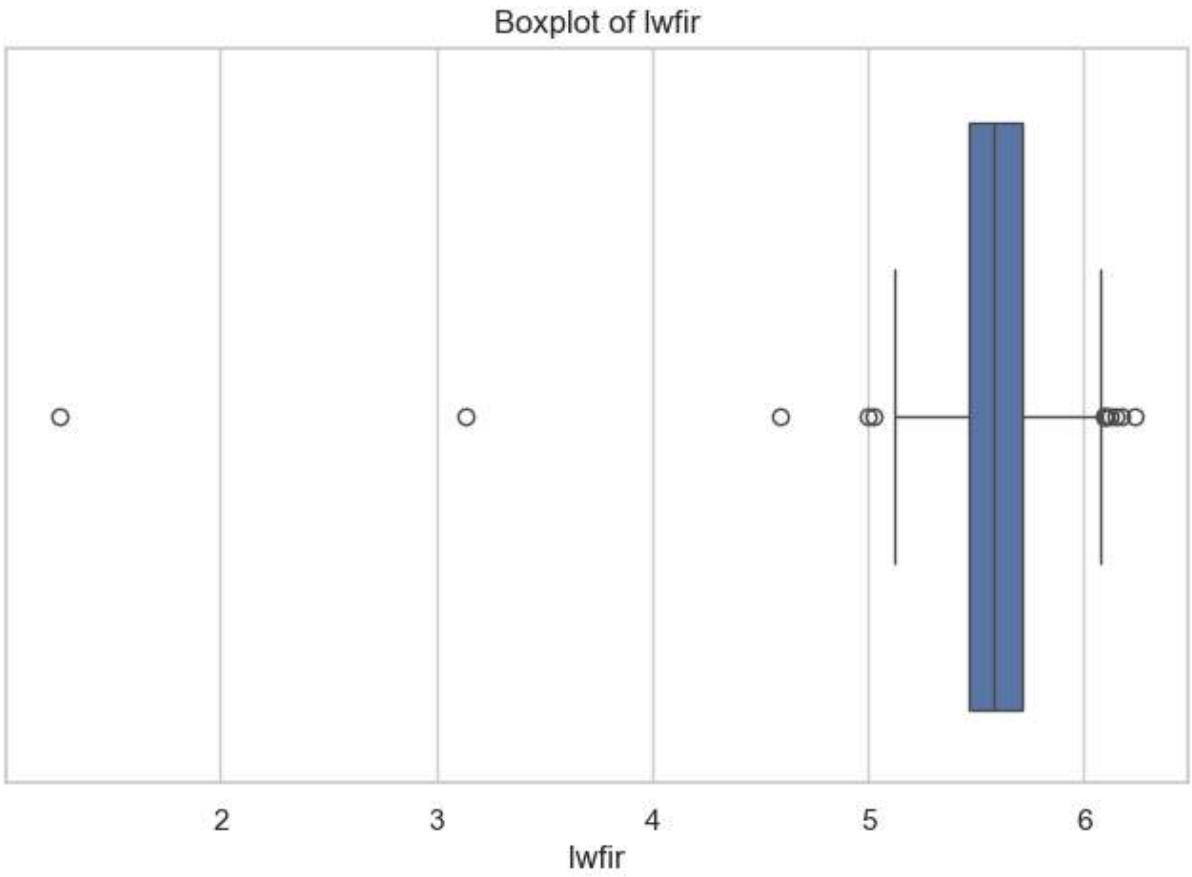

Boxplot of lpolpc
−7
−6
−5
−4
lpolpc

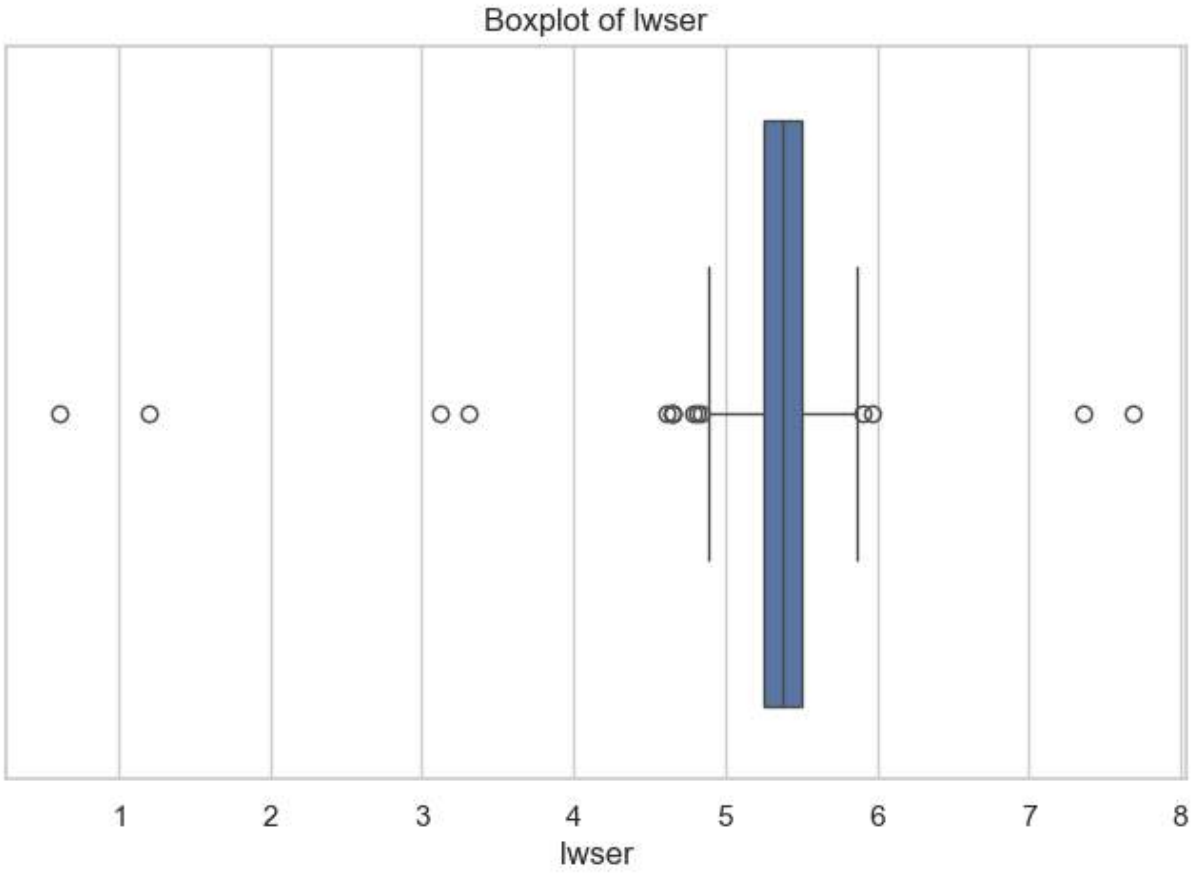

Boxplot of ldensity
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
ldensity

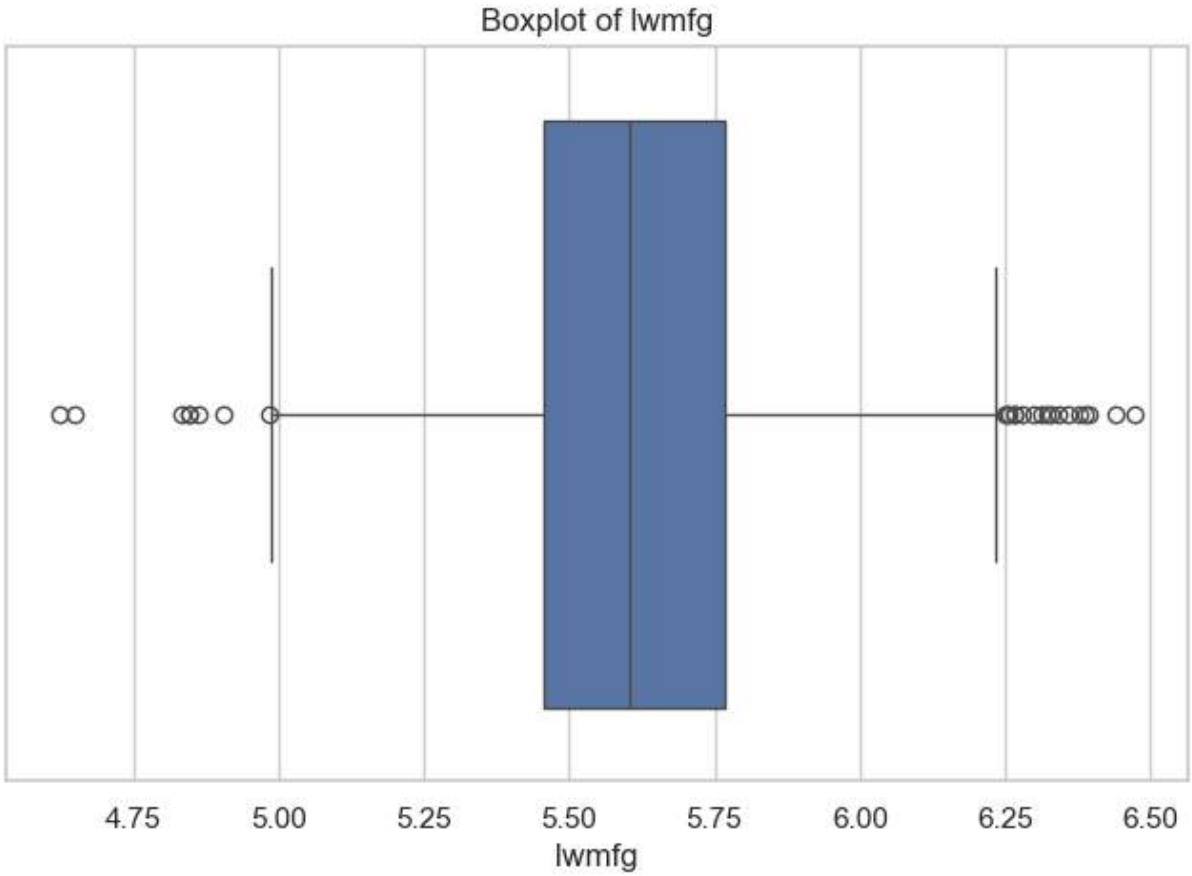

Boxplot of lwcon
4.5
5.0
5.5
6.0
6.5
7.0
7.5
lwcon

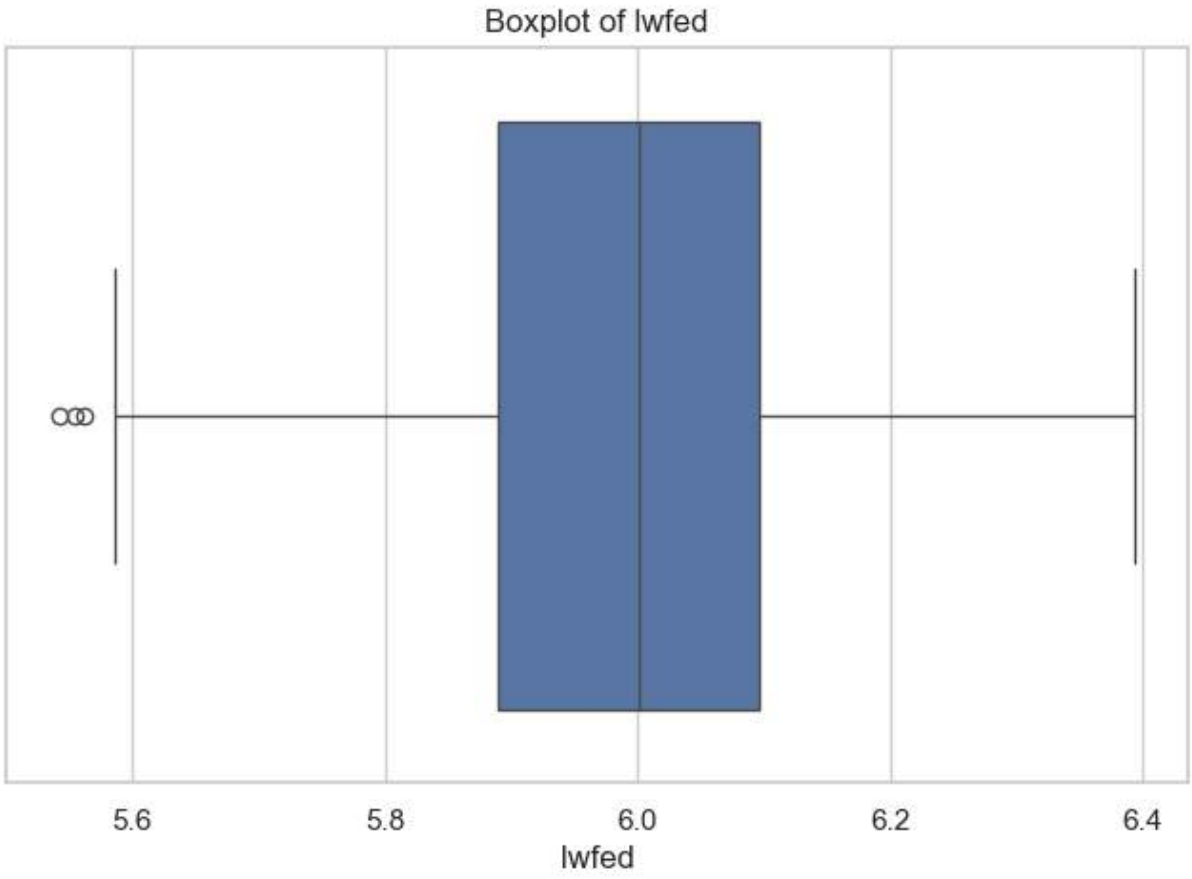

Boxplot of lwtuc
4
5
6
7
8
lwtuc

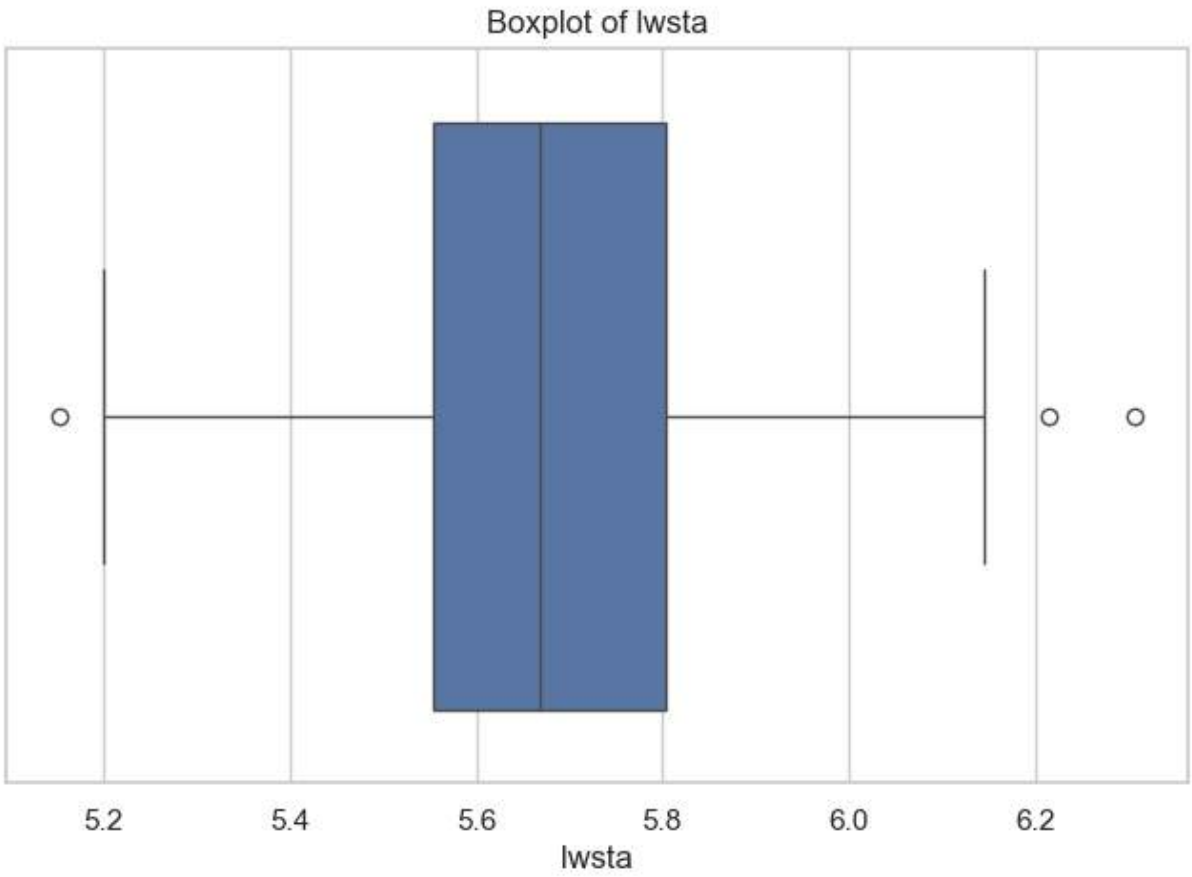

Boxplot of lwtrd
3
4
5
6
7
lwtrd

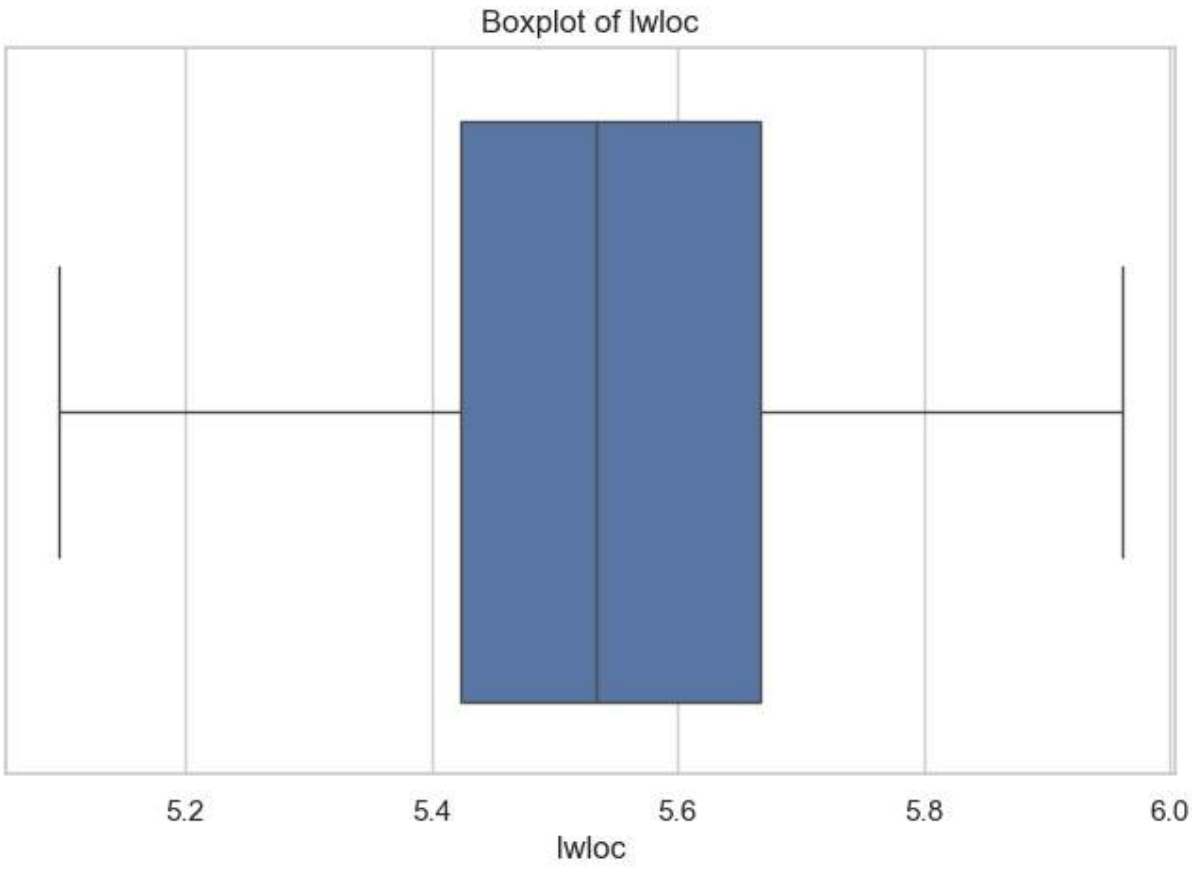

Boxplot of lwfir
2
3
4
5
6
lwfir

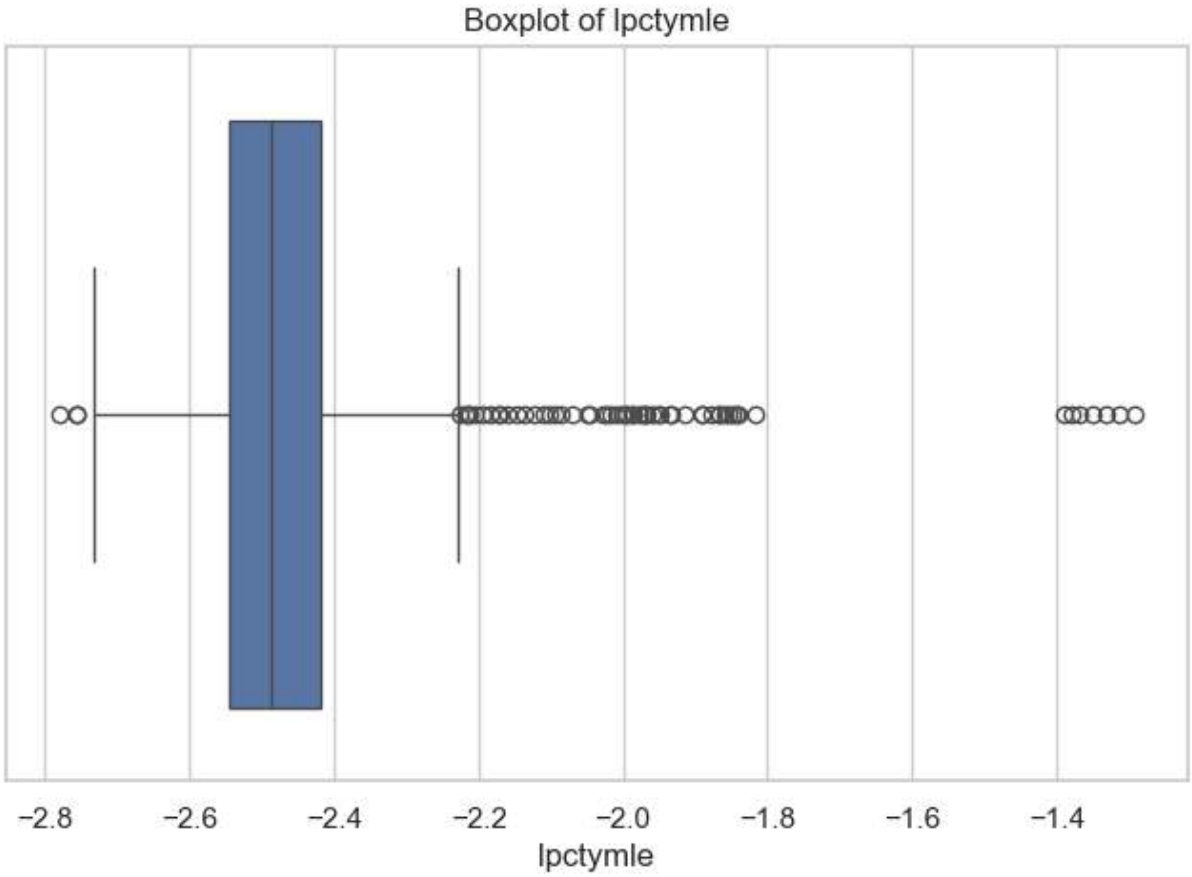

Boxplot of lwser
1
2
3
4
5
6
7
8
lwser

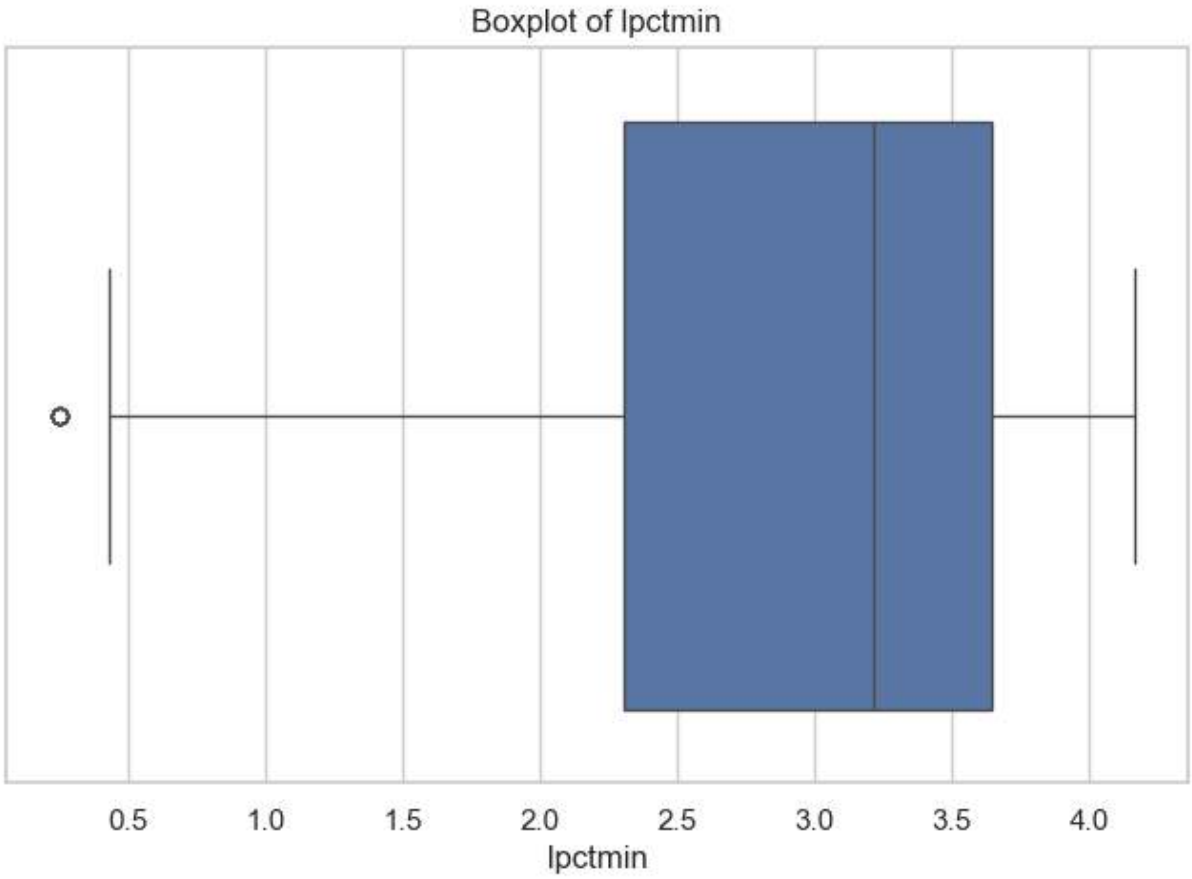

Boxplot of lwmfg
4.75
5.00
5.25
5.50
5.75
6.00
6.25
6.50
lwmfg

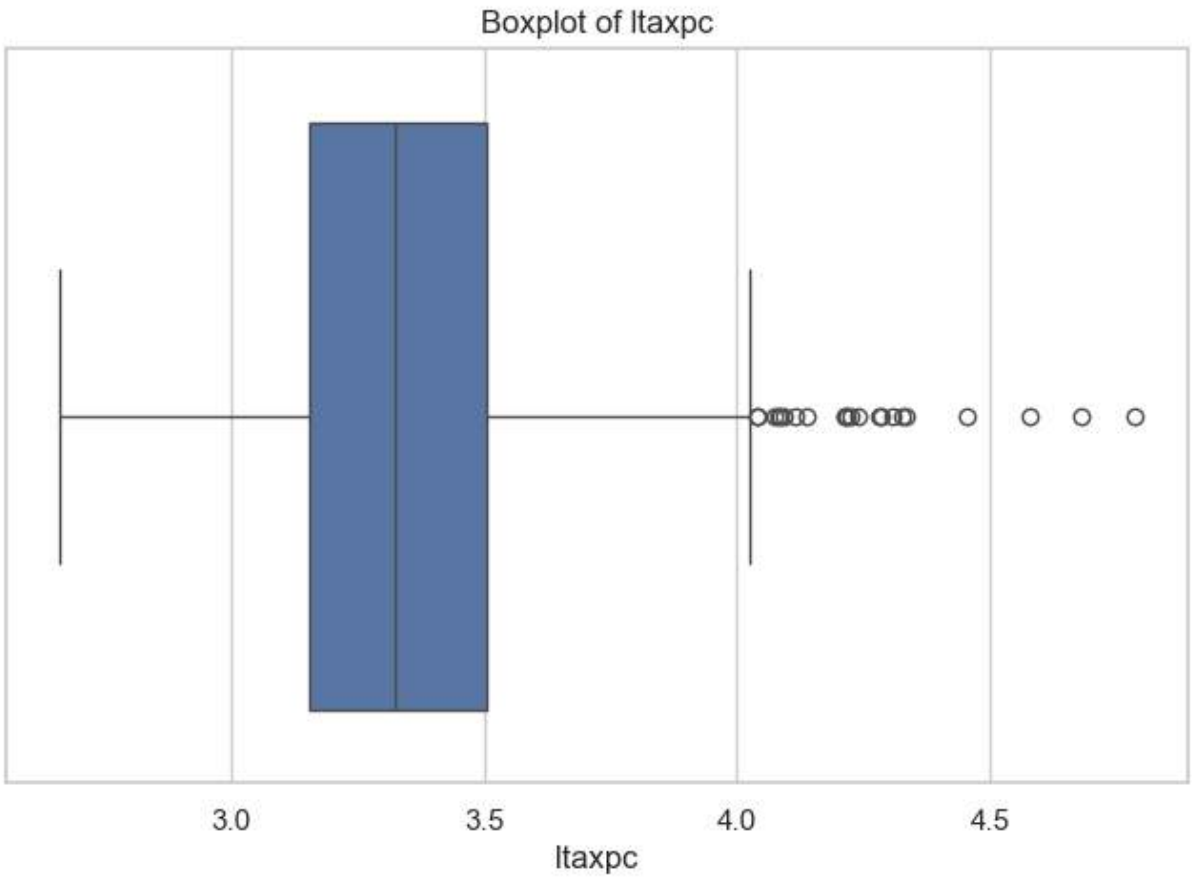

Boxplot of lwfed
5.6
5.8
6.0
6.2
6.4
lwfed

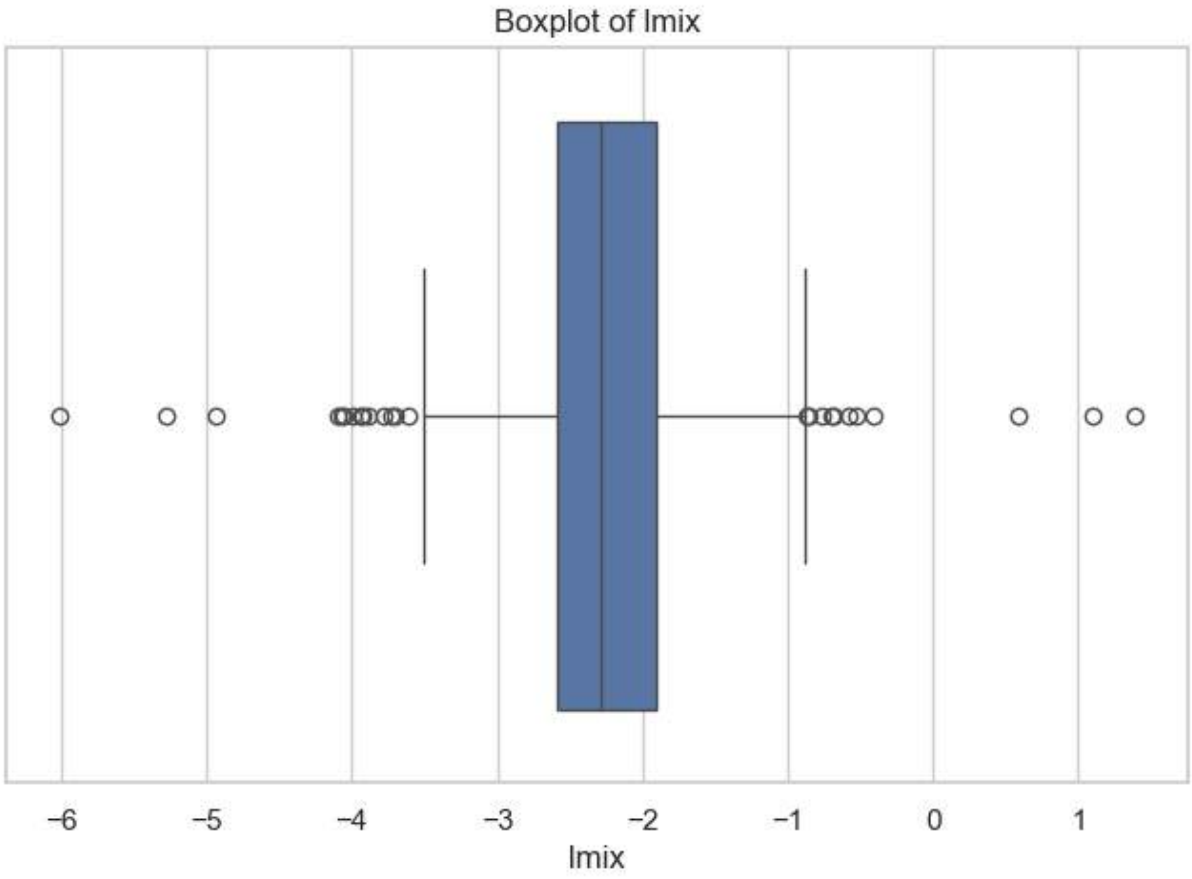

Boxplot of lwsta
5.2
5.4
5.6
5.8
6.0
6.2
lwsta

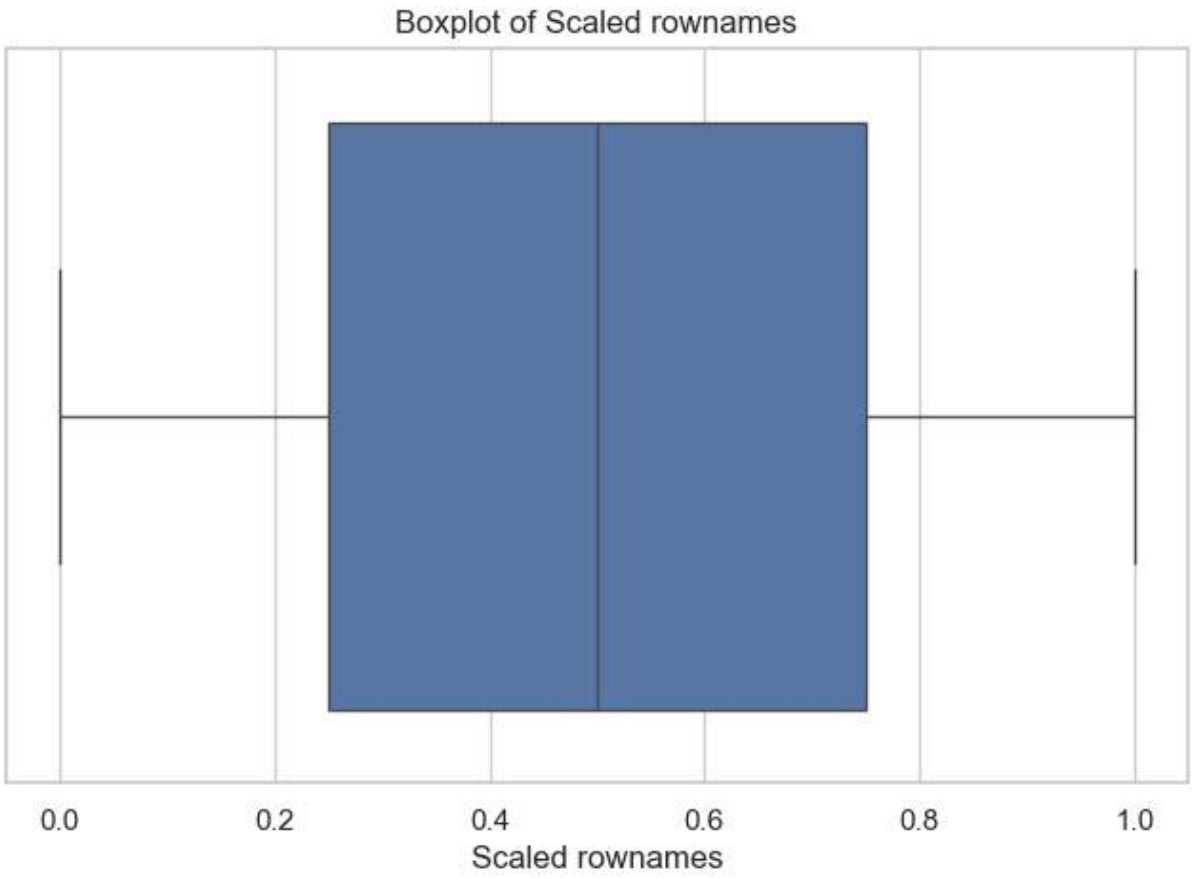

Boxplot of lwloc
5.2
5.4
5.6
5.8
6.0
lwloc


Boxplot of lpctymle
−2.8
−2.6
−2.4
−2.2
−2.0
−1.8
−1.6
−1.4
lpctymle


Boxplot of lpctmin
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
lpctmin


Boxplot of ltaxpc
3.0
3.5
4.0
4.5
ltaxpc





In [27]:
```
outlier_summary = []
```

```
for column in numeric.columns:
    values = clean[column].dropna()

    if not values.empty:
        q1 = values.quantile(0.25)
        q3 = values.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        count = ((values < lower) | (values > upper)).sum()

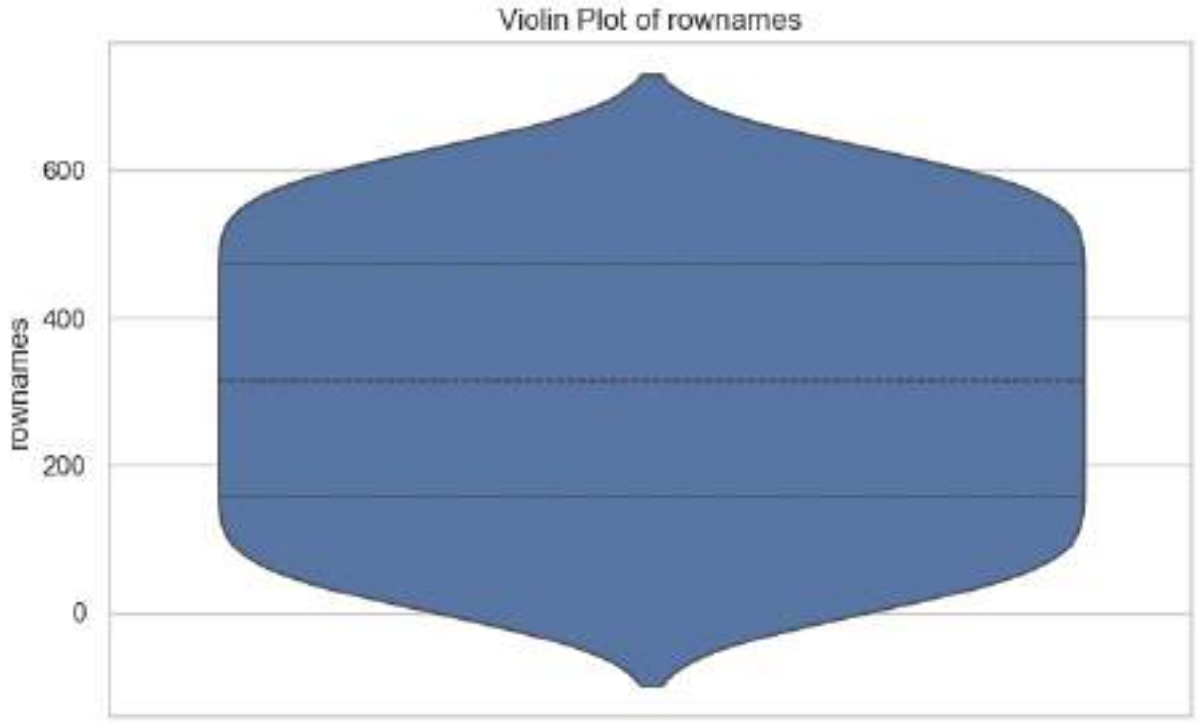
        outlier_summary.append({
            "Variable": column,
            "Lower Bound": lower,
            "Upper Bound": upper,
            "Outlier Count": count
        })

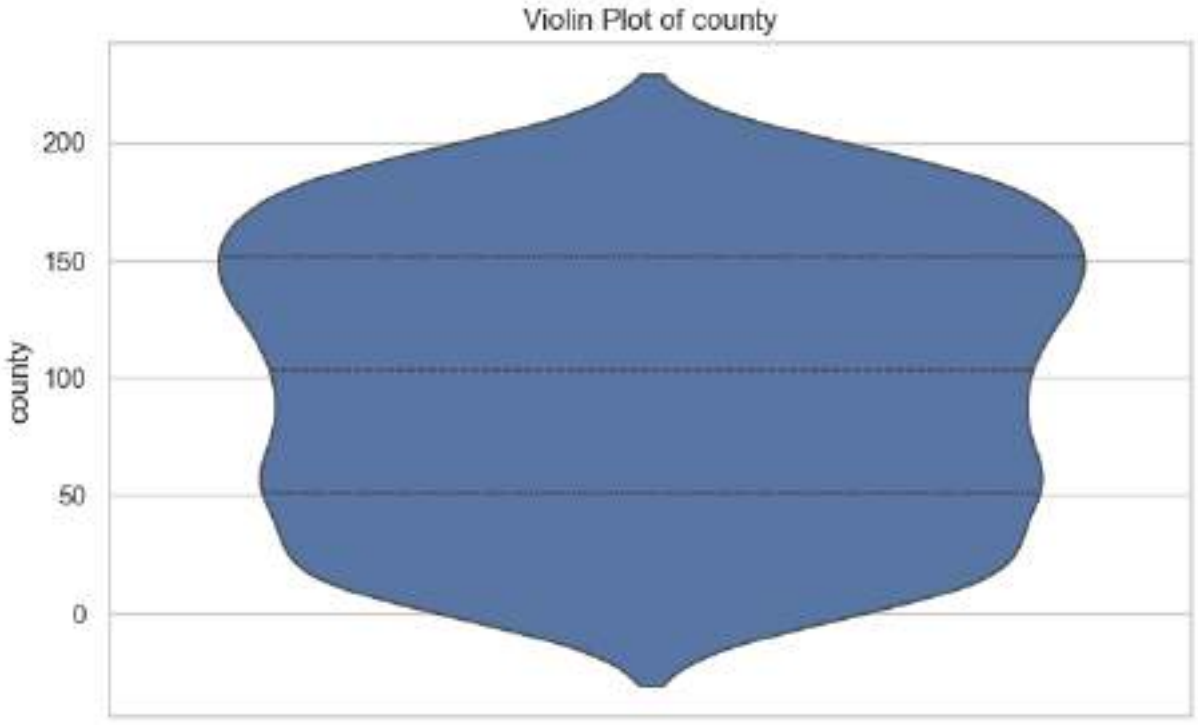
display(pd.DataFrame(outlier_summary))
```

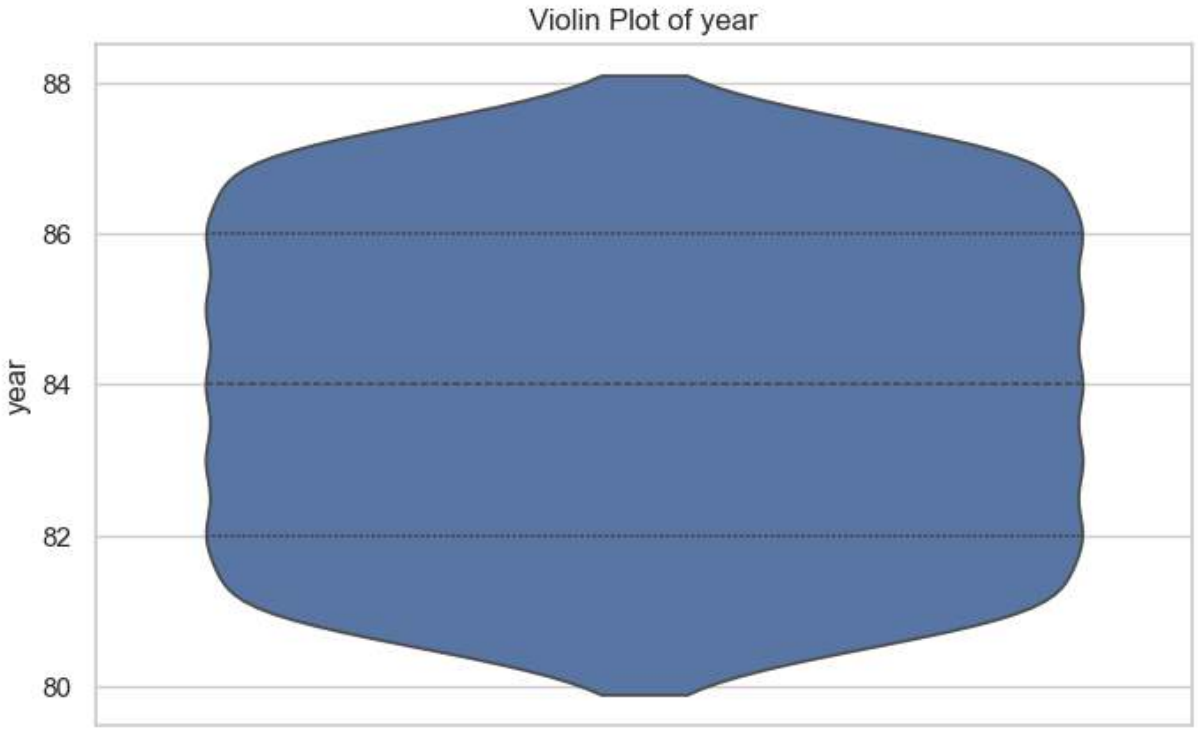
	Variable	Lower Bound	Upper Bound	Outlier Count
0	rownames	-313.500000	944.500000	0
1	county	-99.000000	301.000000	0
2	year	76.000000	92.000000	0
3	crmrte	-0.011730	0.068487	28
4	prbarr	0.015977	0.554443	26
5	prbconv	-0.084165	1.067454	58
6	prbpris	0.211223	0.646369	14
7	avgsen	2.603750	14.753750	20
8	polpc	0.000273	0.002721	37
9	density	-0.929366	2.970128	56
10	taxpc	8.657154	48.039664	39
11	pctmin	-32.323000	80.550600	0
12	wcon	99.612438	371.741337	13
13	wtuc	177.467612	551.149513	47
14	wtrd	112.892150	259.978550	28
15	wfir	135.602087	402.542588	21
16	wser	107.564738	330.909837	16
17	wmfg	105.146250	448.836250	34
18	wfed	236.987500	569.087500	1
19	wsta	148.155000	441.495000	8
20	wloc	133.053750	383.003750	2
21	mix	-0.035203	0.259536	53
22	pctymle	0.062695	0.105093	65
23	lcrmrte	-5.105775	-2.151804	10
24	lprbarr	-2.245299	-0.321065	15
25	lprbconv	-1.961307	0.451675	32
26	lprbpris	-1.365037	-0.344732	29
27	lavgsen	1.438061	2.852592	7
28	lpolpc	-7.354508	-5.696363	42
29	ldensity	-2.189349	1.970671	7

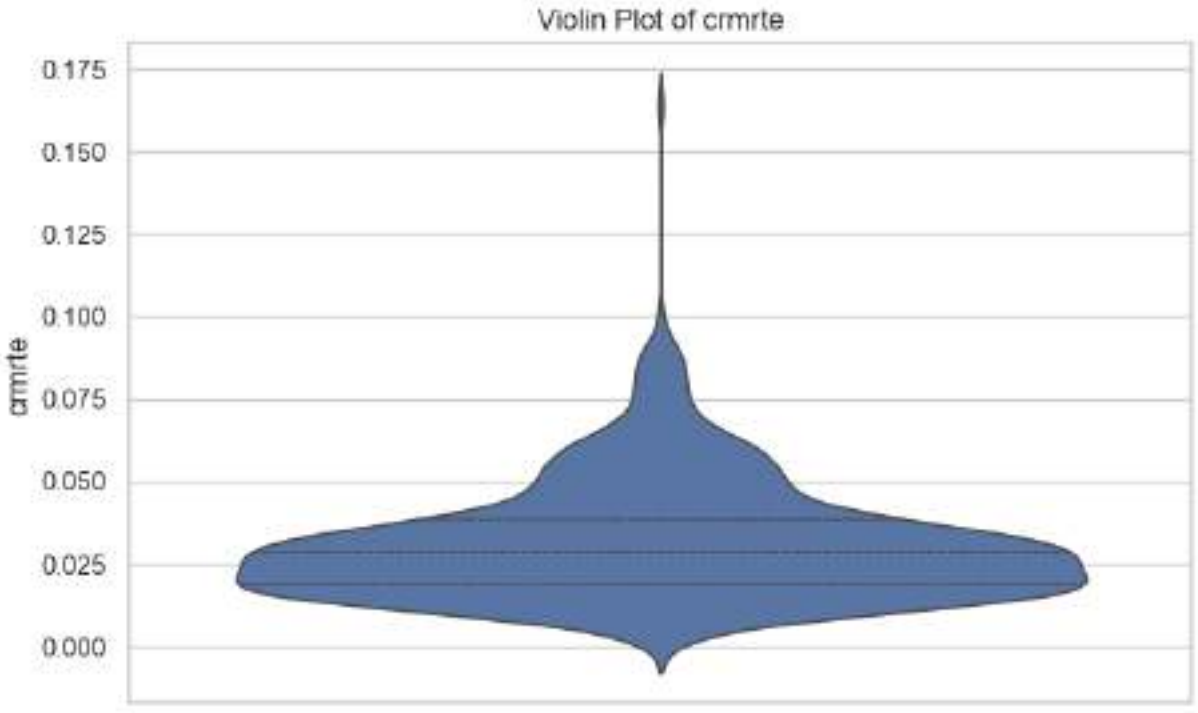
	Variable	Lower Bound	Upper Bound	Outlier Count
30	lwcon	4.870543	6.033328	4
31	lwtuc	5.374010	6.405416	50
32	lwtrd	4.827442	5.618954	21
33	lwfir	5.088625	6.085838	12
34	lwser	4.869833	5.894127	15
35	lwmfg	4.986366	6.237255	25
36	lwfed	5.580233	6.407160	3
37	lwsta	5.178516	6.178659	3
38	lwloc	5.058952	6.032419	0
39	lpctymle	-2.733225	-2.227174	60
40	lpctmin	0.292457	5.654025	7
41	ltaxpc	2.627534	4.030988	22
42	lmix	-3.609280	-0.880430	25
43	Scaled rownames	-0.500000	1.500000	0

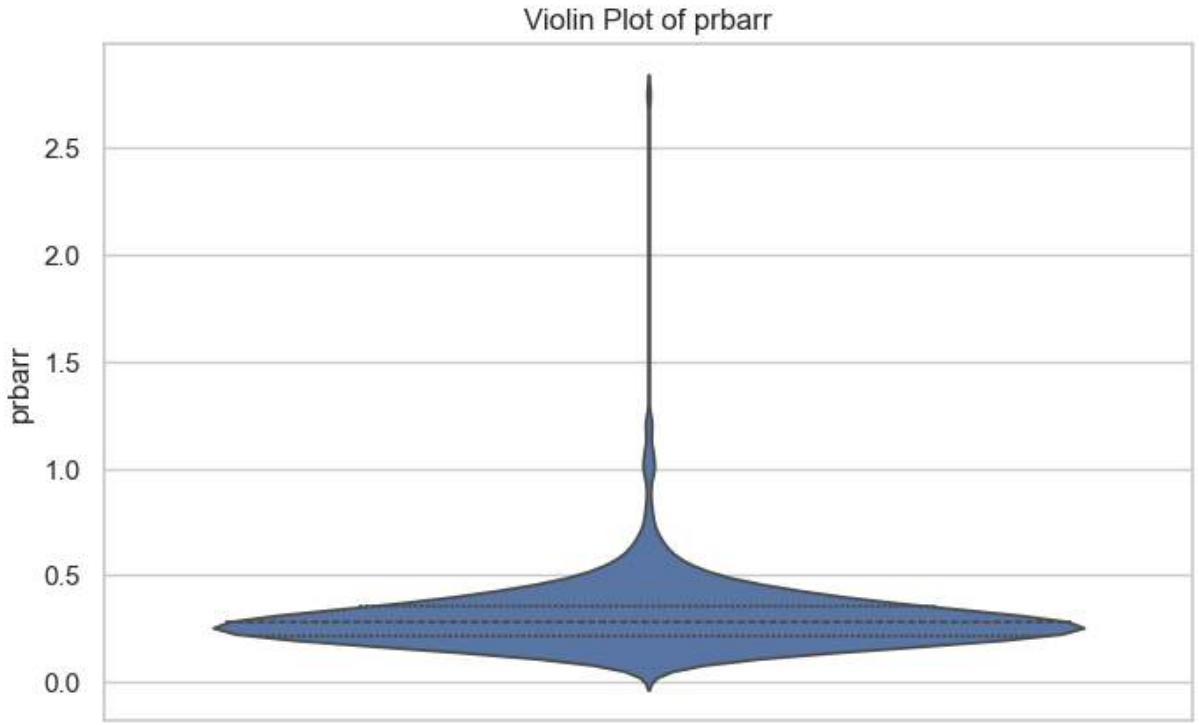
In [28]:
```
for column in numeric.columns:
    if numeric[column].nunique() > 1:
        plt.figure()
        sns.violinplot(data=clean, y=column, inner="quartile")
        plt.title("Violin Plot of " + column)
        plt.show()
```

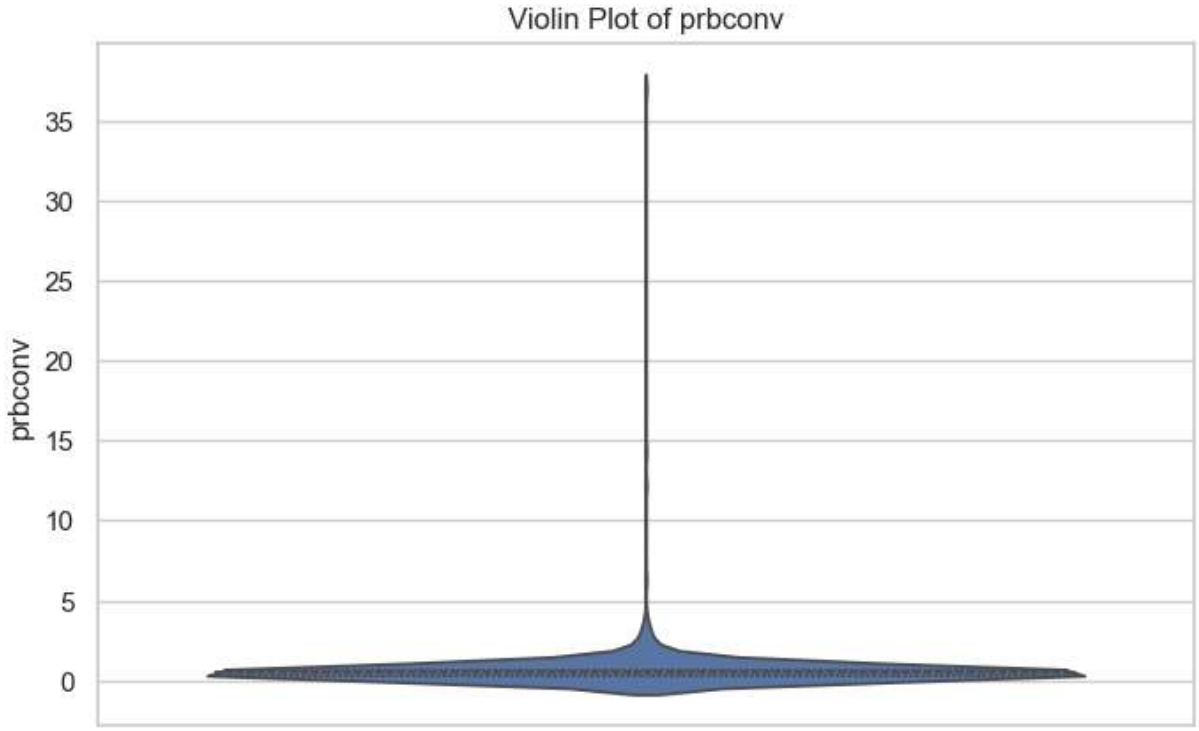

Violin Plot of rownames
rownames
600
400
200
0

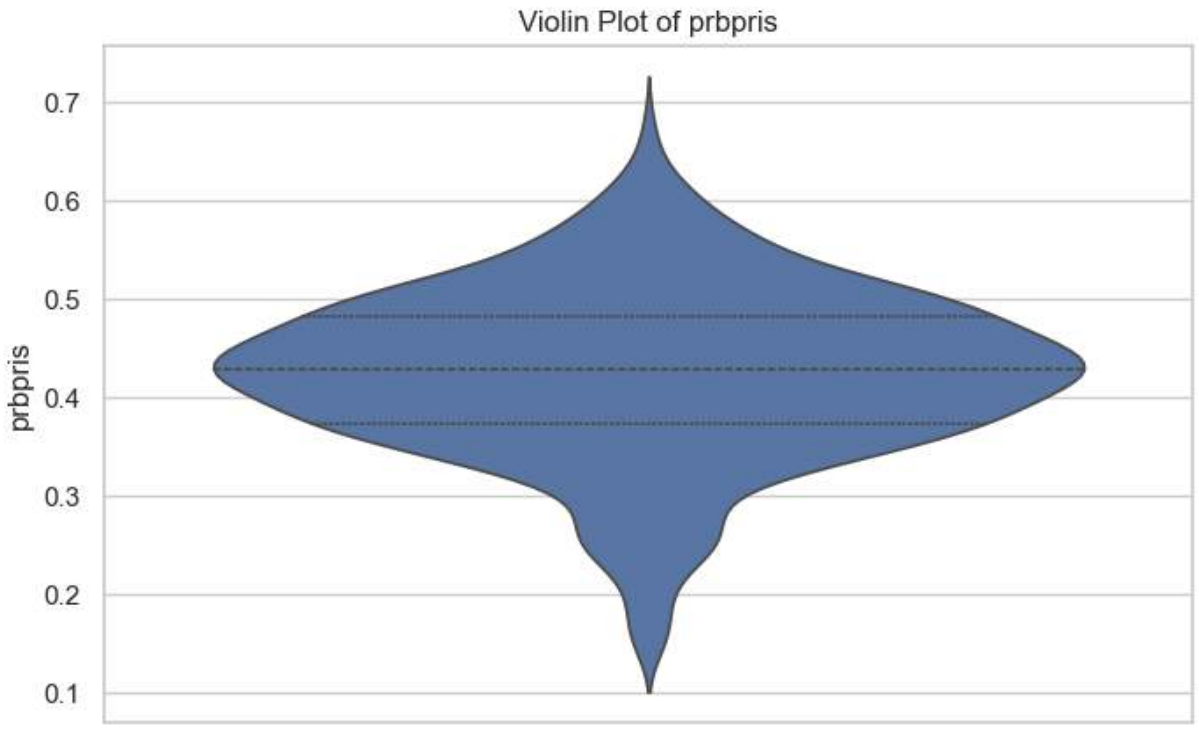

Violin Plot of county
county
200
150
100
50
0

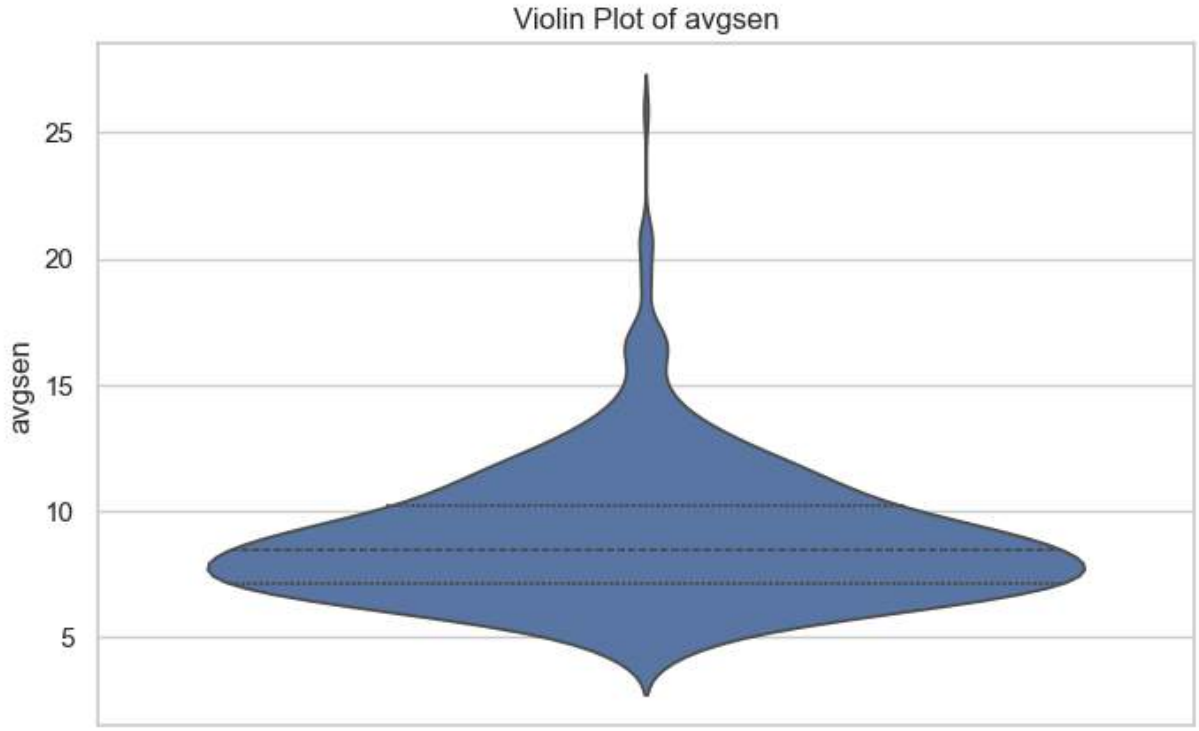

Violin Plot of year
year
88
86
84
82
80

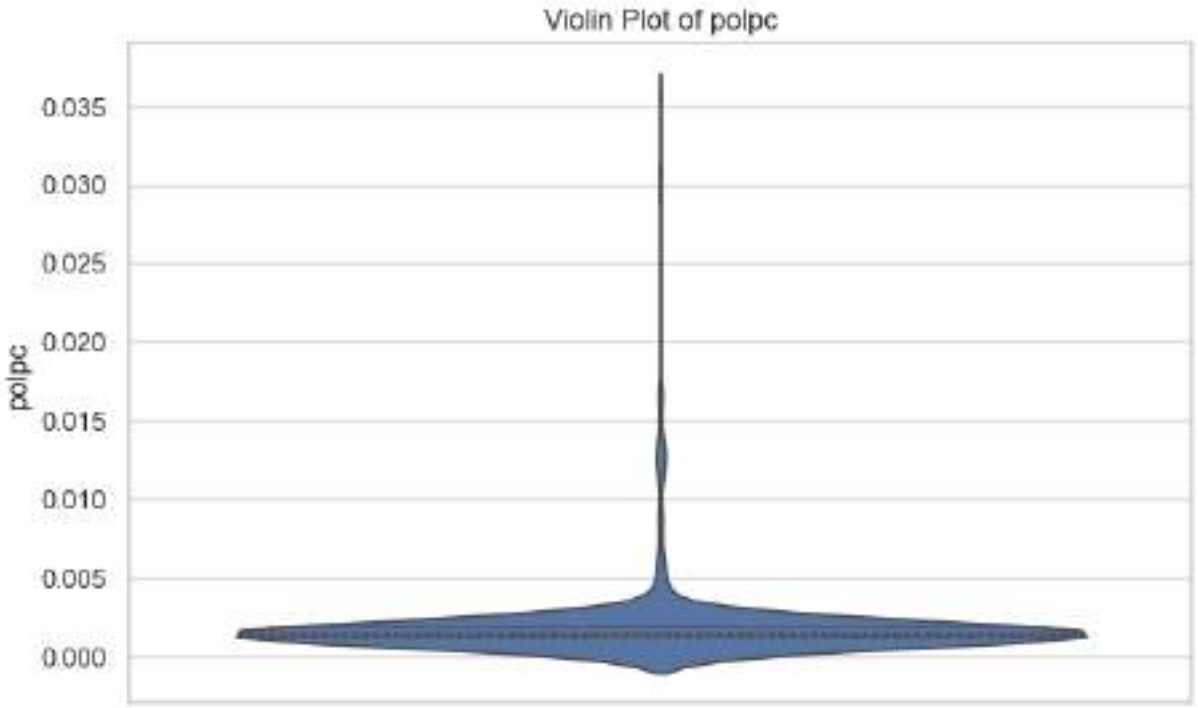

Violin Plot of crmrte
crmrte
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000

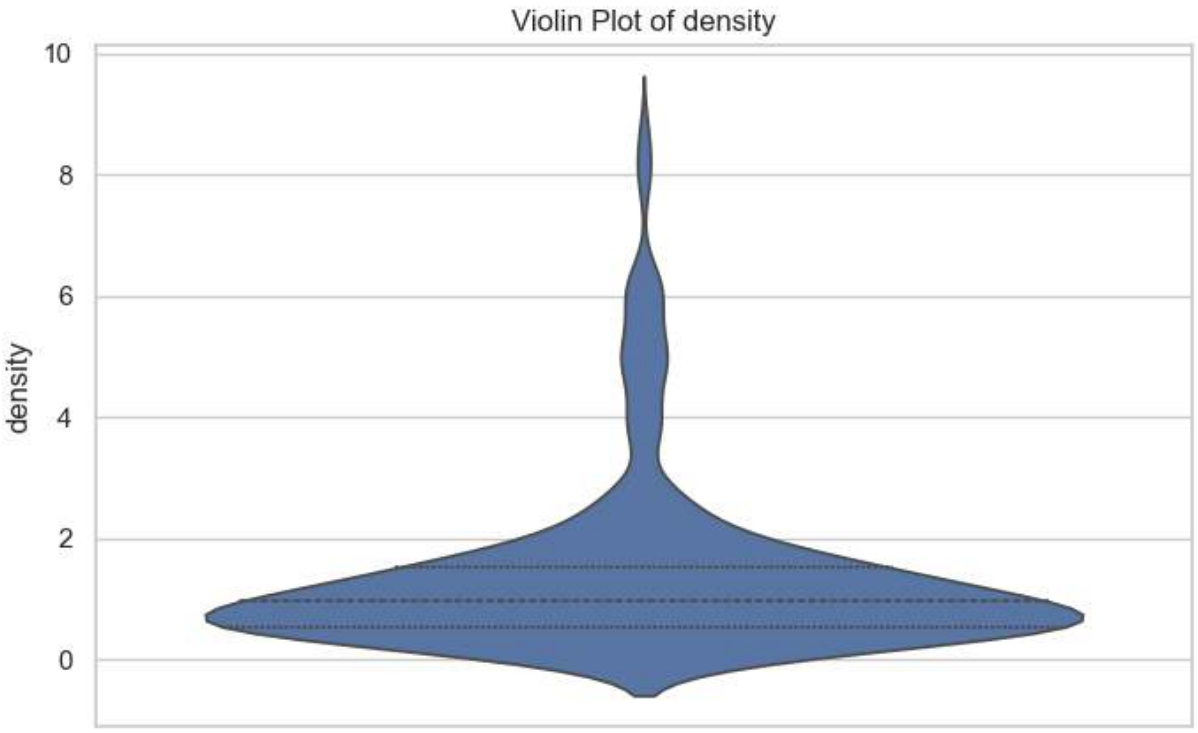

Violin Plot of prbarr
prbarr
2.5
2.0
1.5
1.0
0.5
0.0

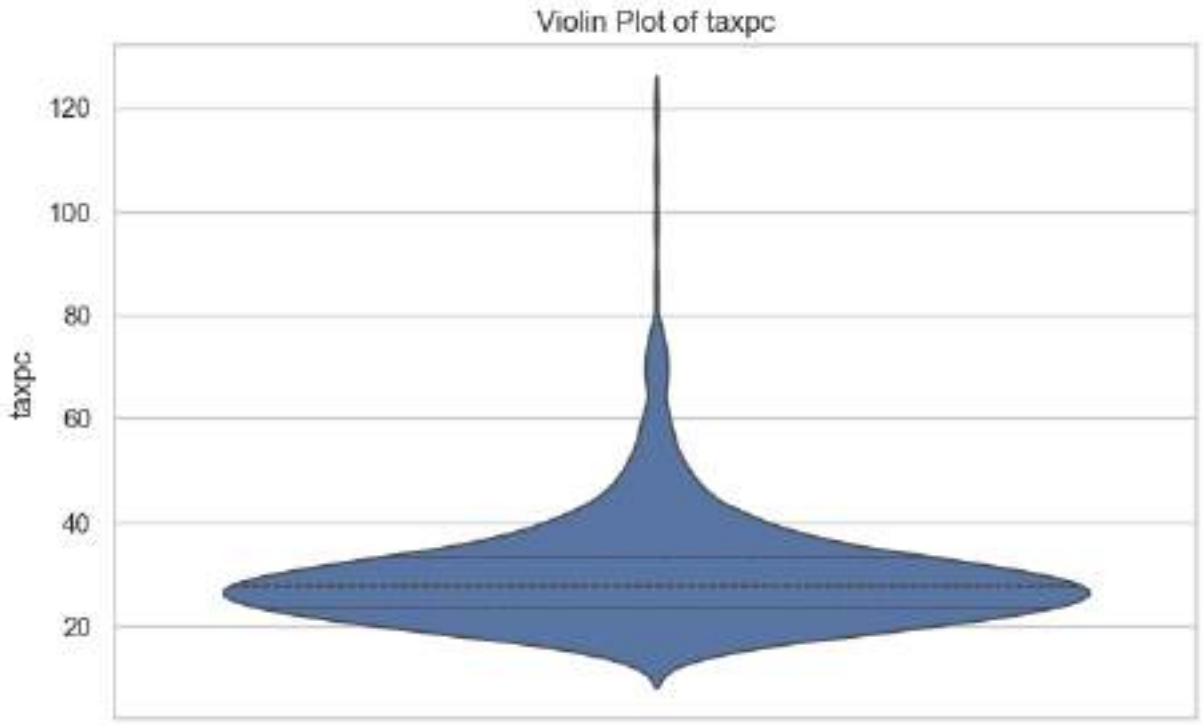

Violin Plot of prbconv
prbconv
35
30
25
20
15
10
5
0

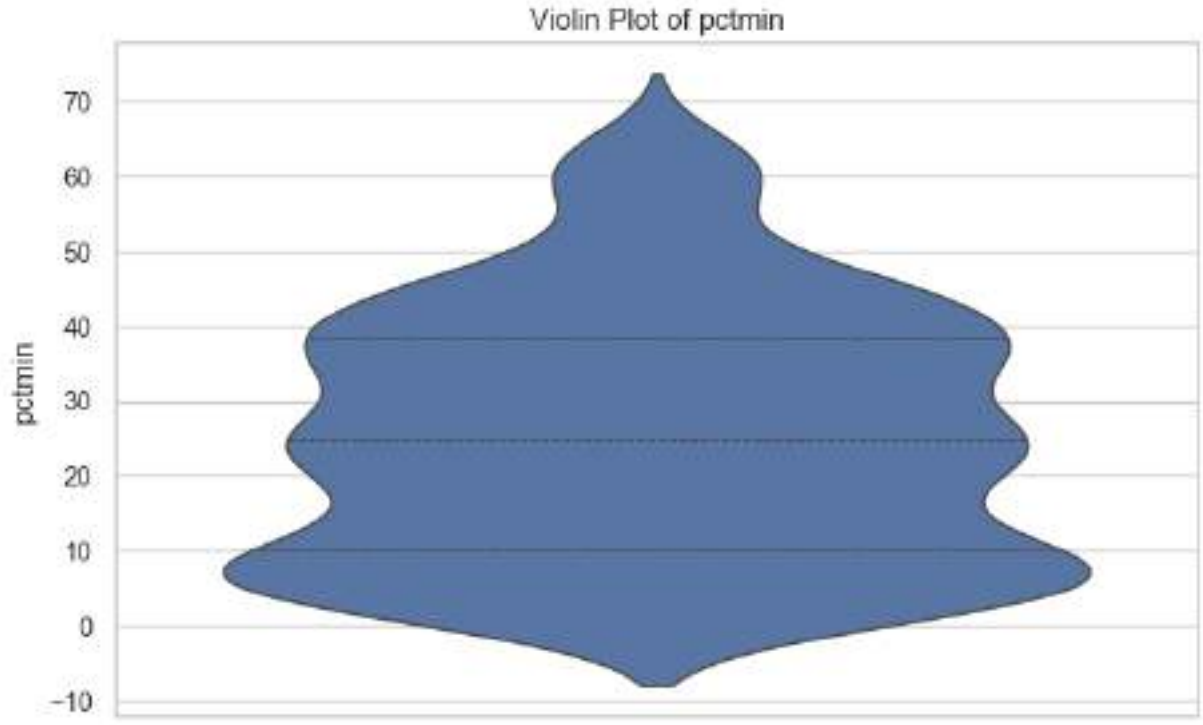

Violin Plot of prbpris
prbpris
0.7
0.6
0.5
0.4
0.3
0.2
0.1

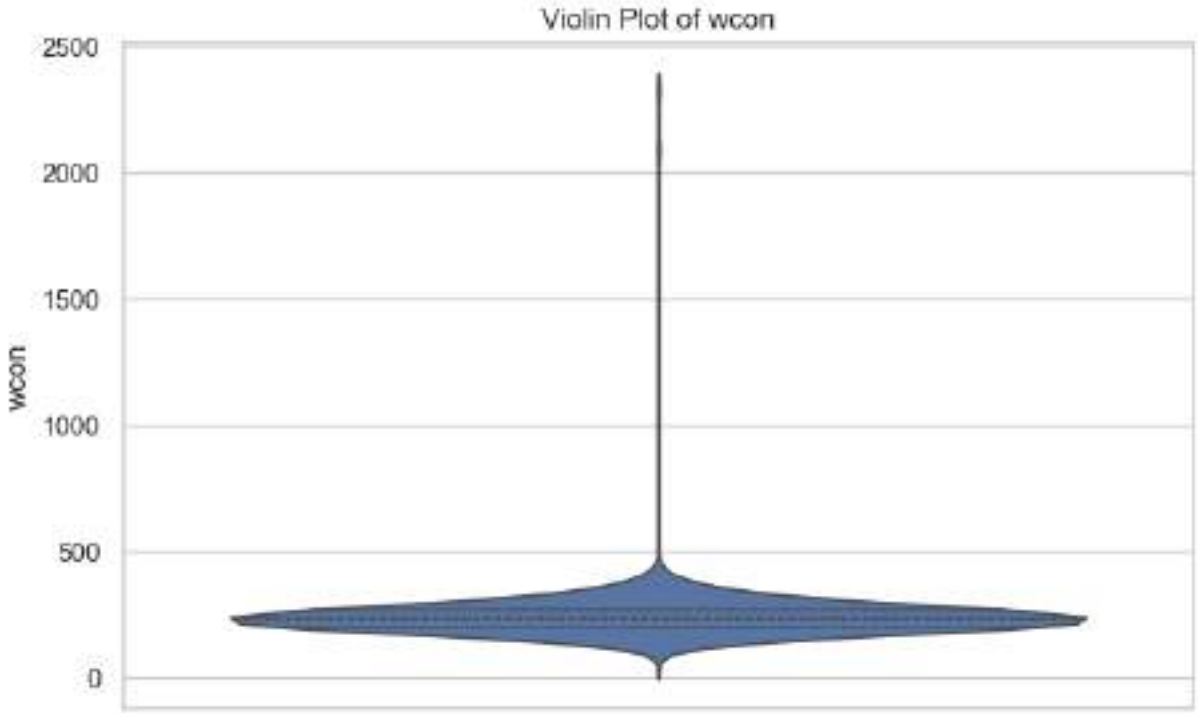

Violin Plot of avgsen
avgsen
25
20
15
10
5

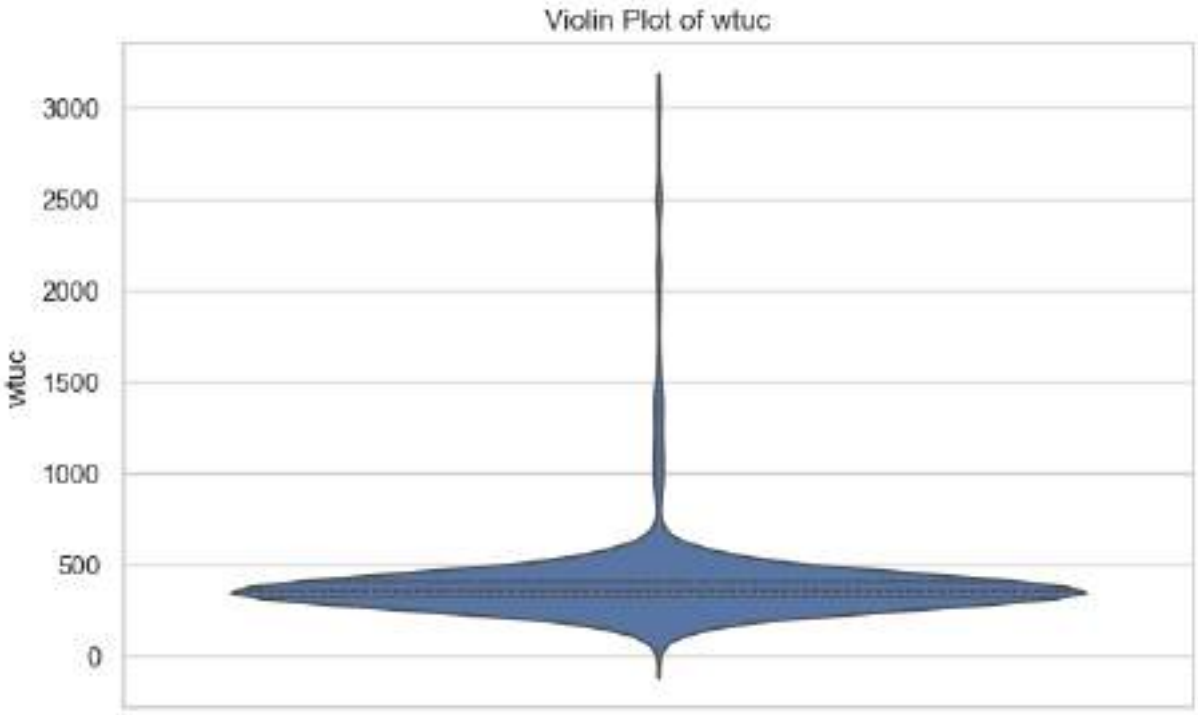

Violin Plot of polpc
polpc
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000

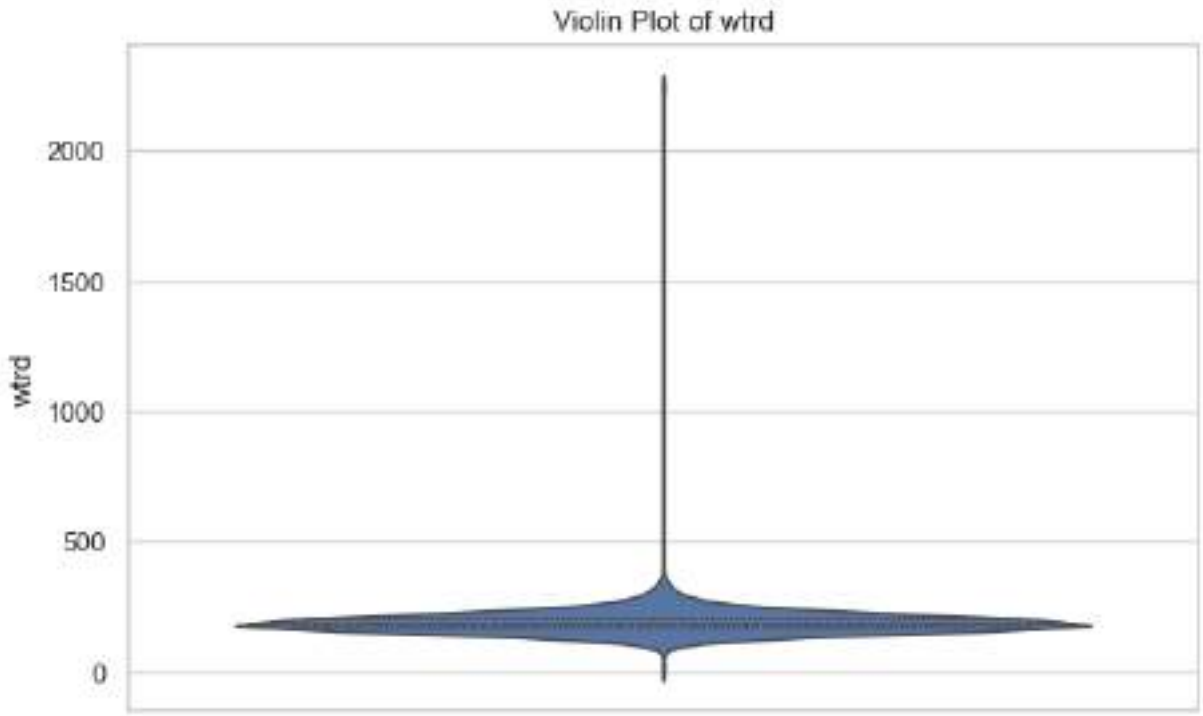

Violin Plot of density
density
10
8
6
4
2
0

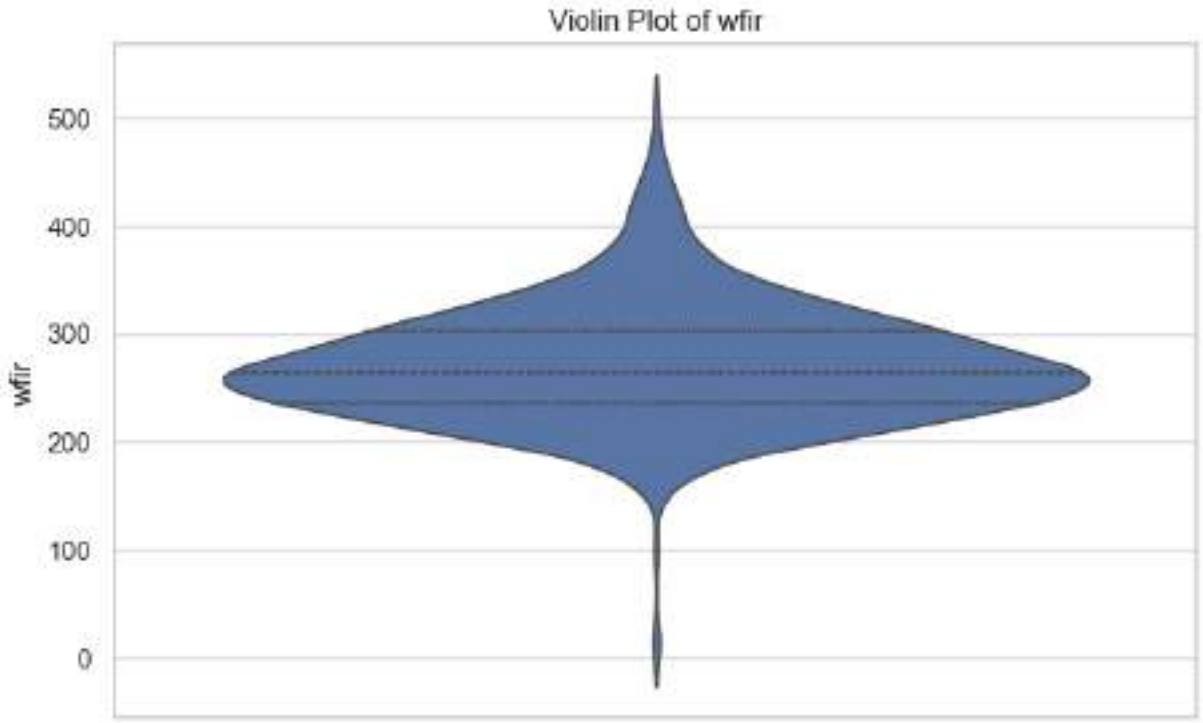

Violin Plot of taxpc
taxpc
120
100
80
60
40
20

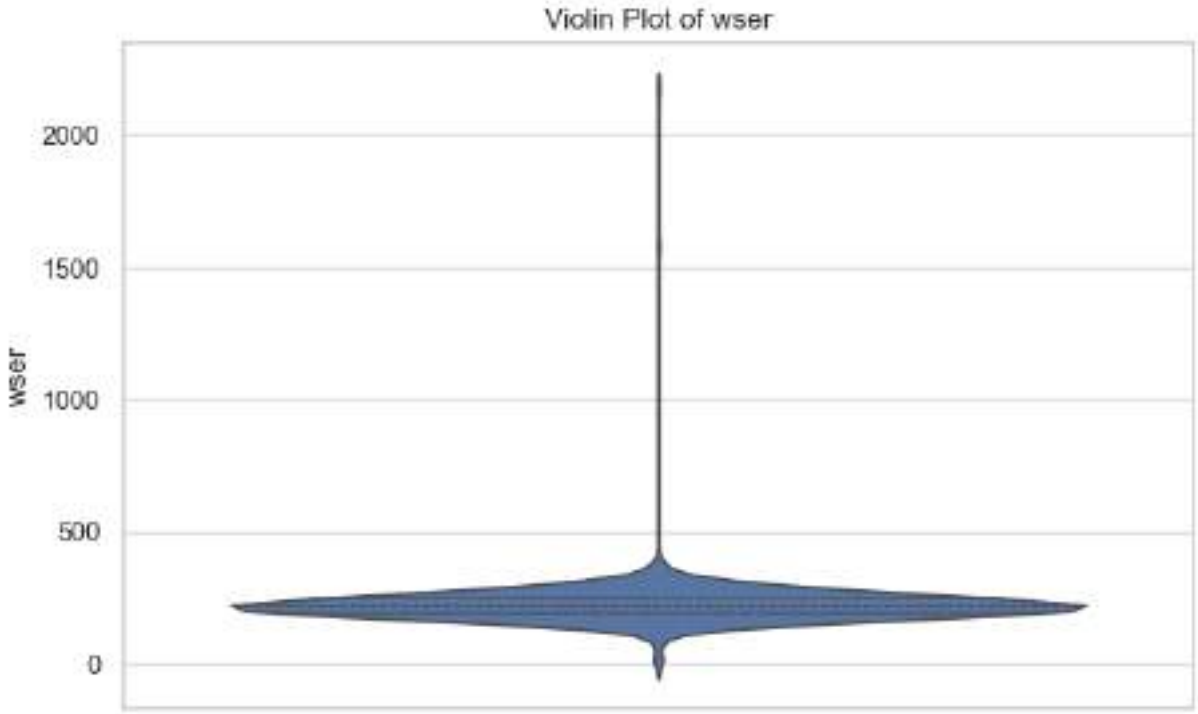

Violin Plot of pctmin
pctmin
70
60
50
40
30
20
10
0
−10

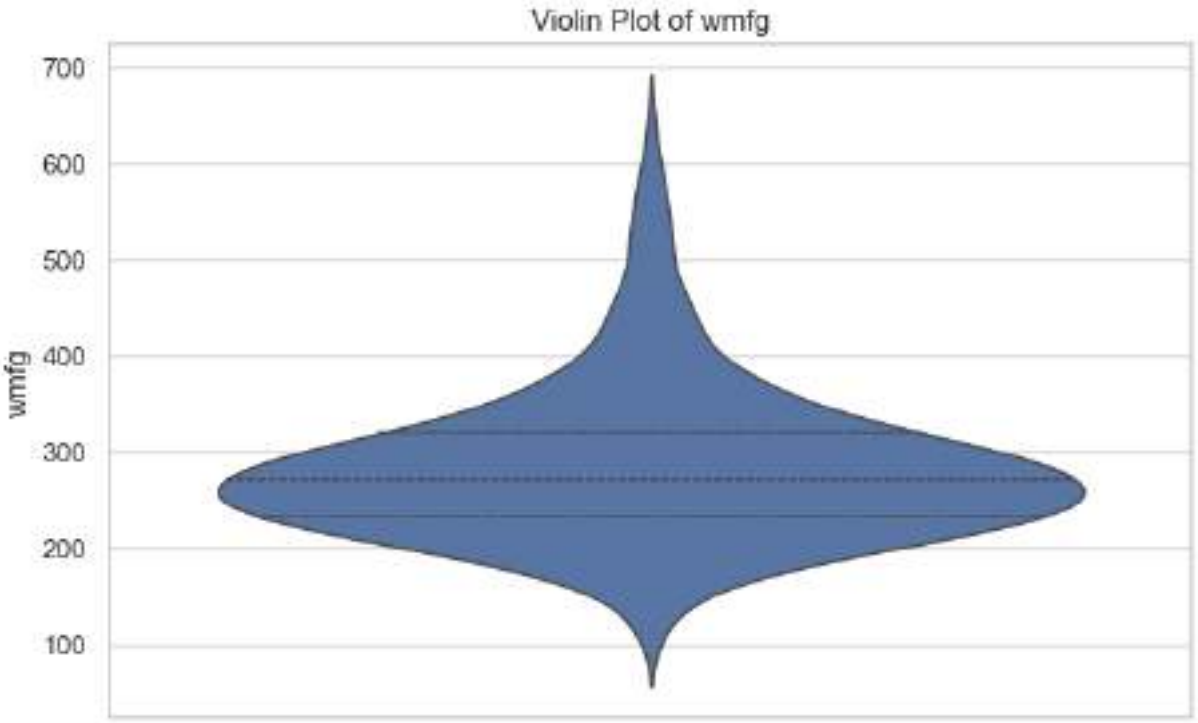

Violin Plot of wcon
2500
2000
1500
1000
500
0
wcon

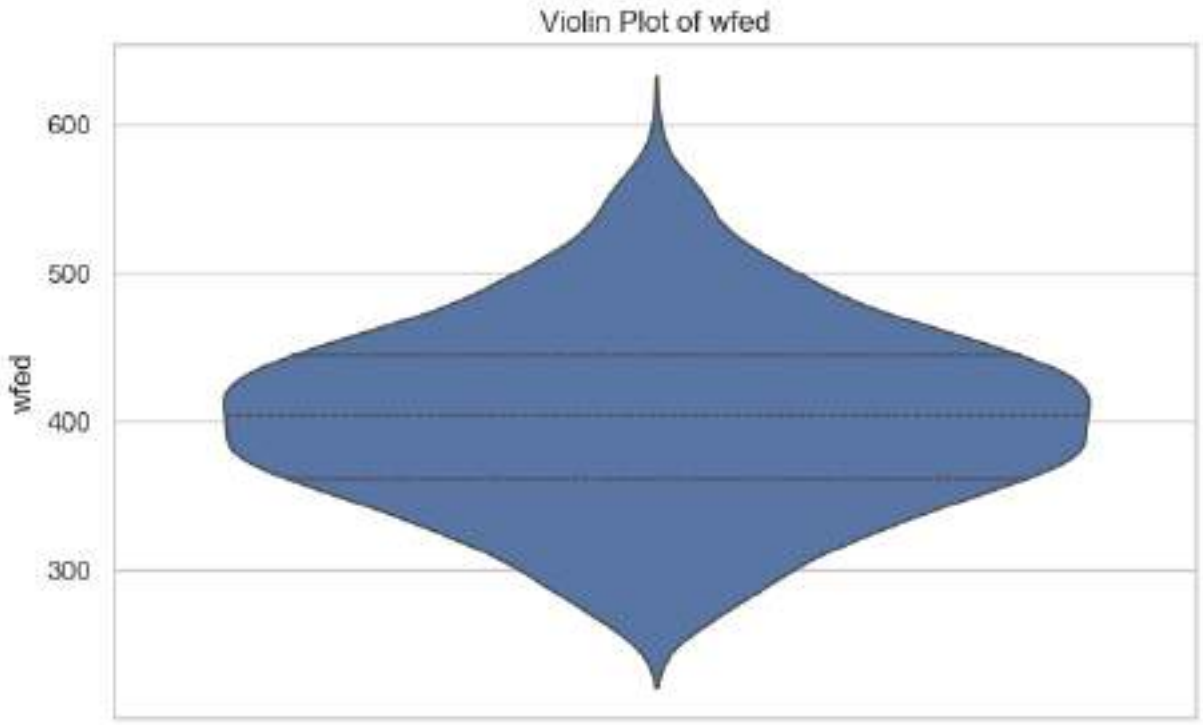

Violin Plot of wtuc
3000
2500
2000
1500
1000
500
0
wtuc

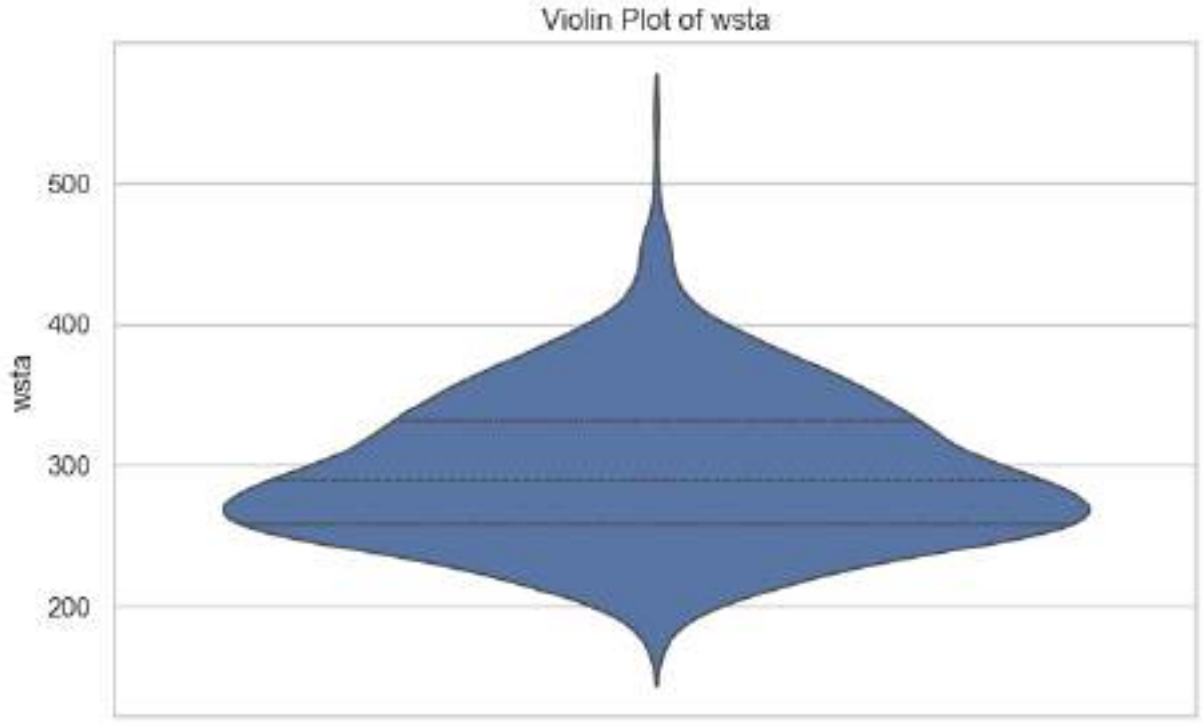

Violin Plot of wtrd
wtrd
2000
1500
1000
500
0

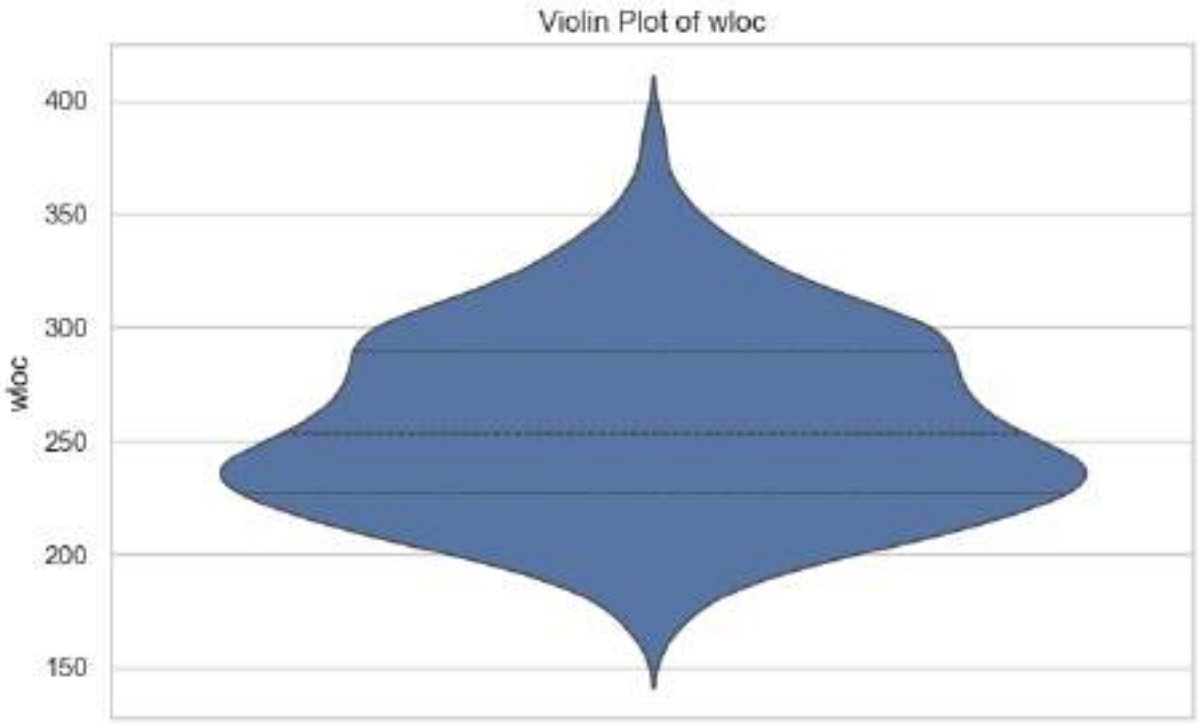

Violin Plot of wfir
wfir
500
400
300
200
100
0

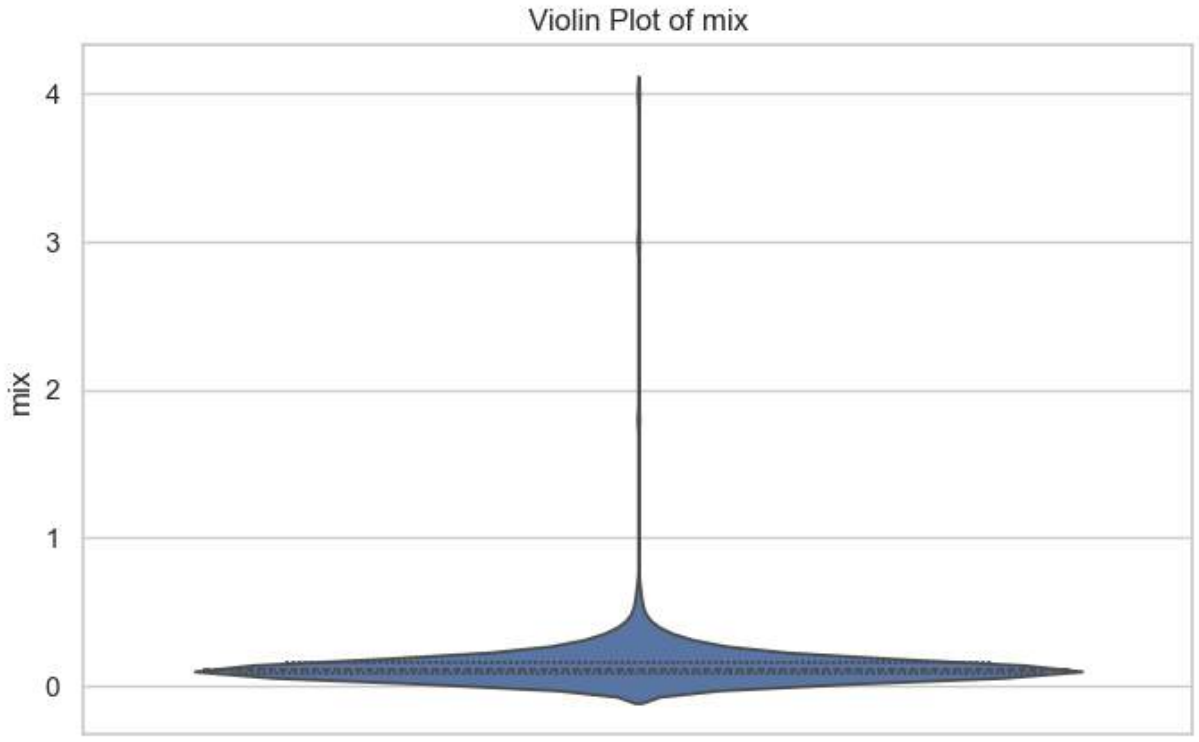

Violin Plot of wser
wser
2000
1500
1000
500
0

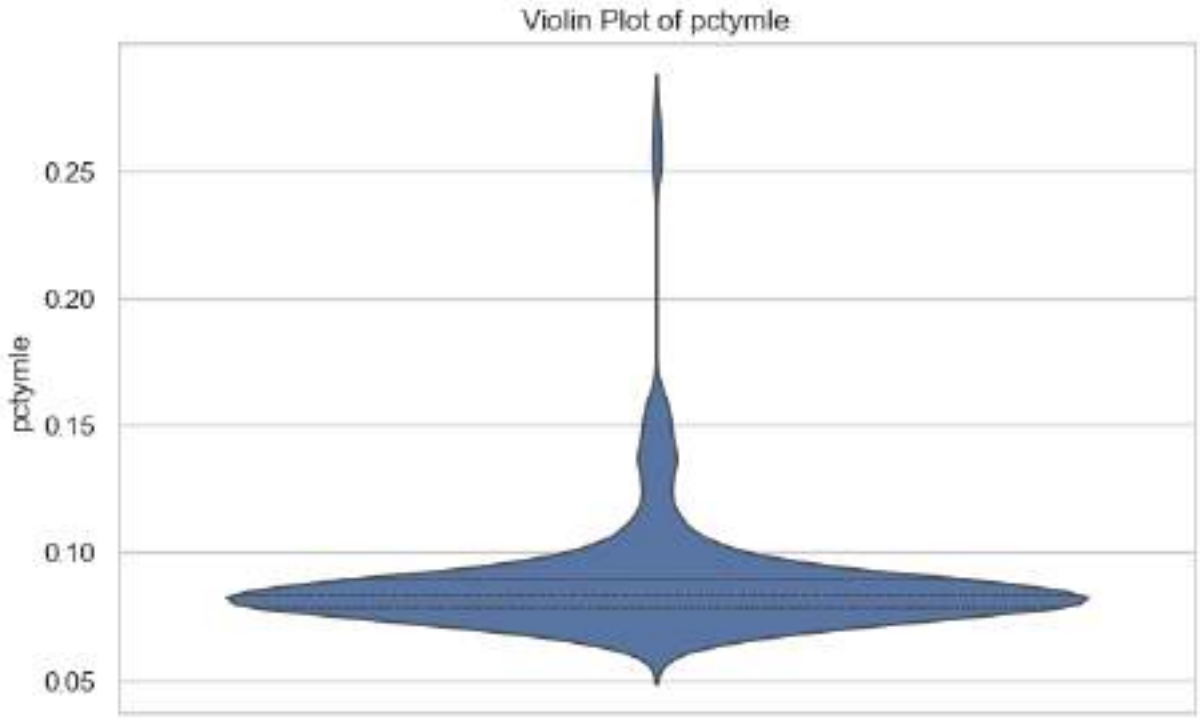

Violin Plot of wmfg
wmfg
700
600
500
400
300
200
100

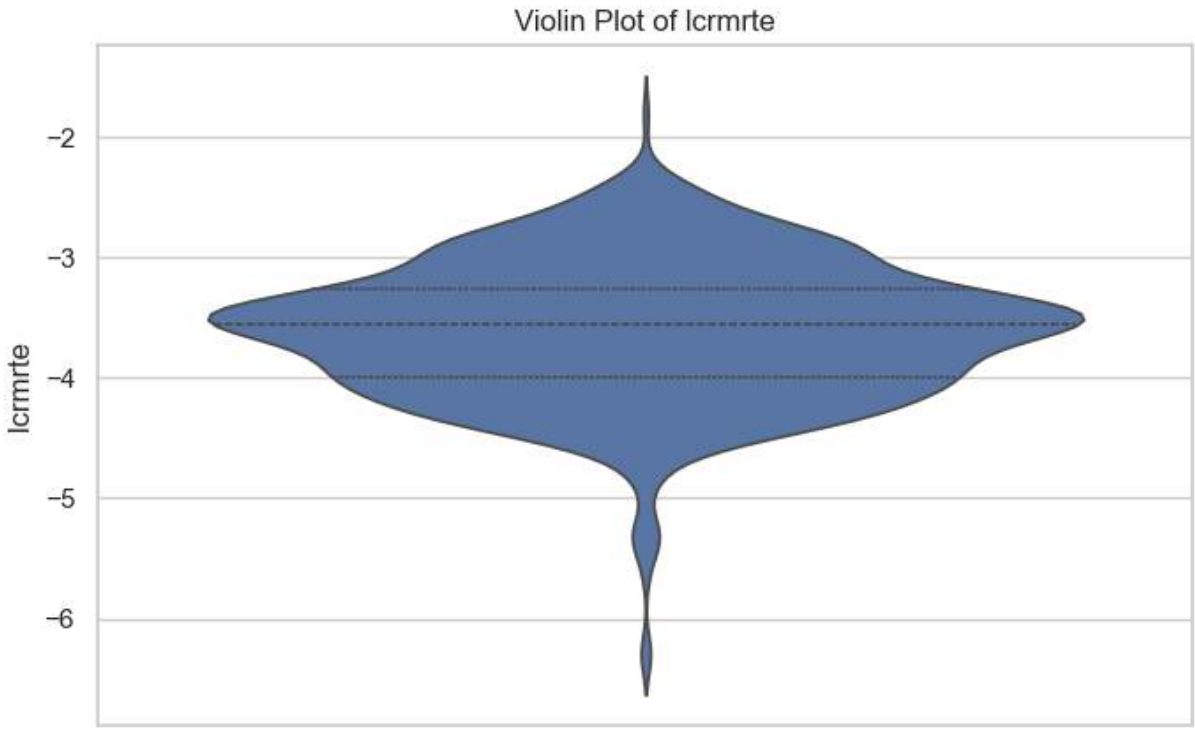

Violin Plot of wfed
wfed
600
500
400
300

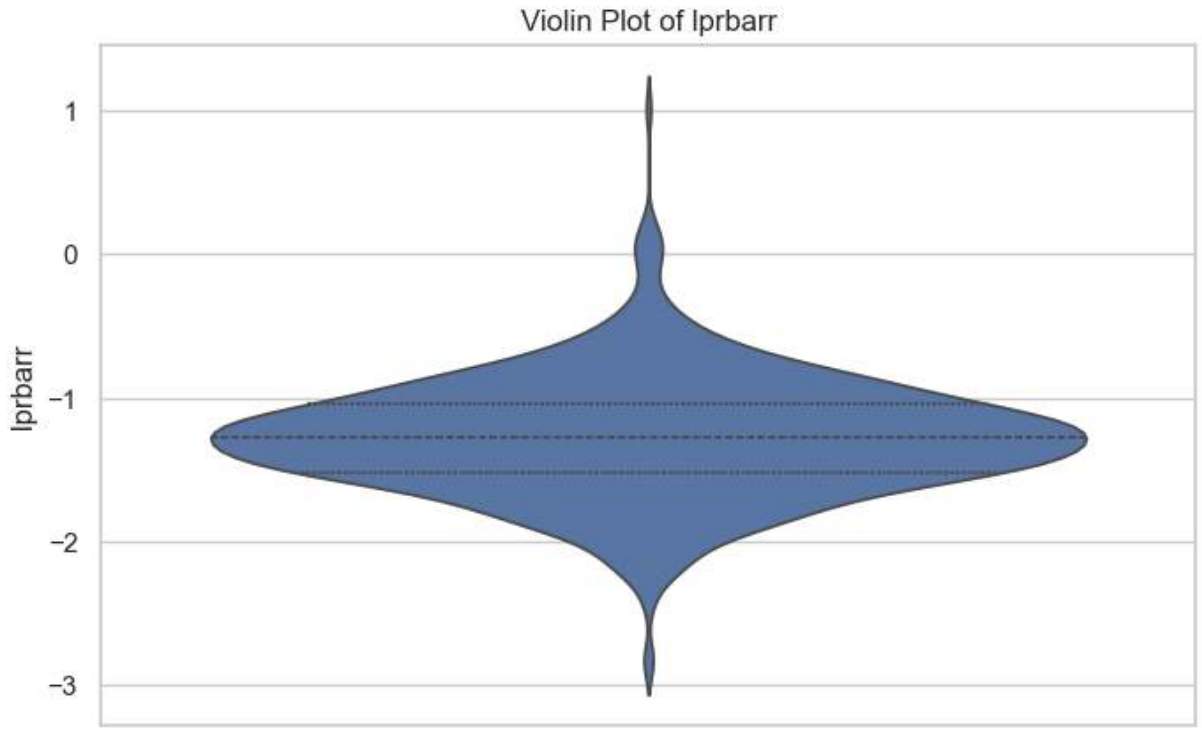

Violin Plot of wsta
wsta
500
400
300
200

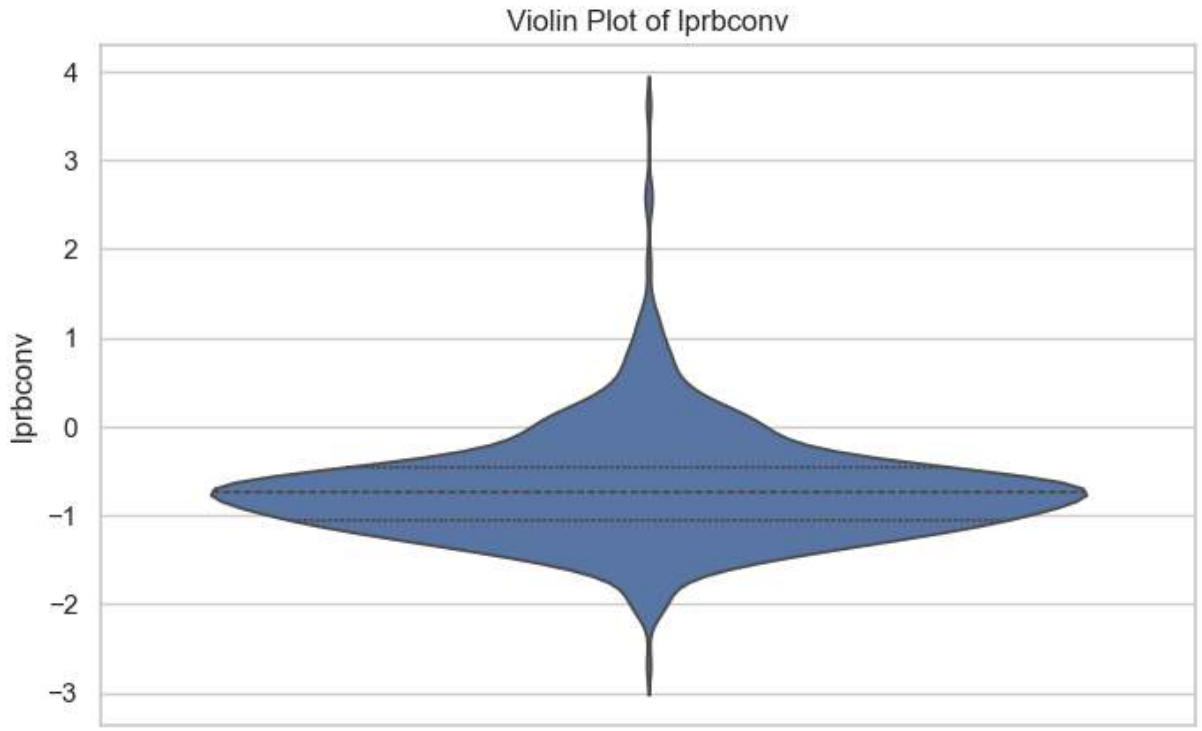

Violin Plot of wloc
wloc
400
350
300
250
200
150

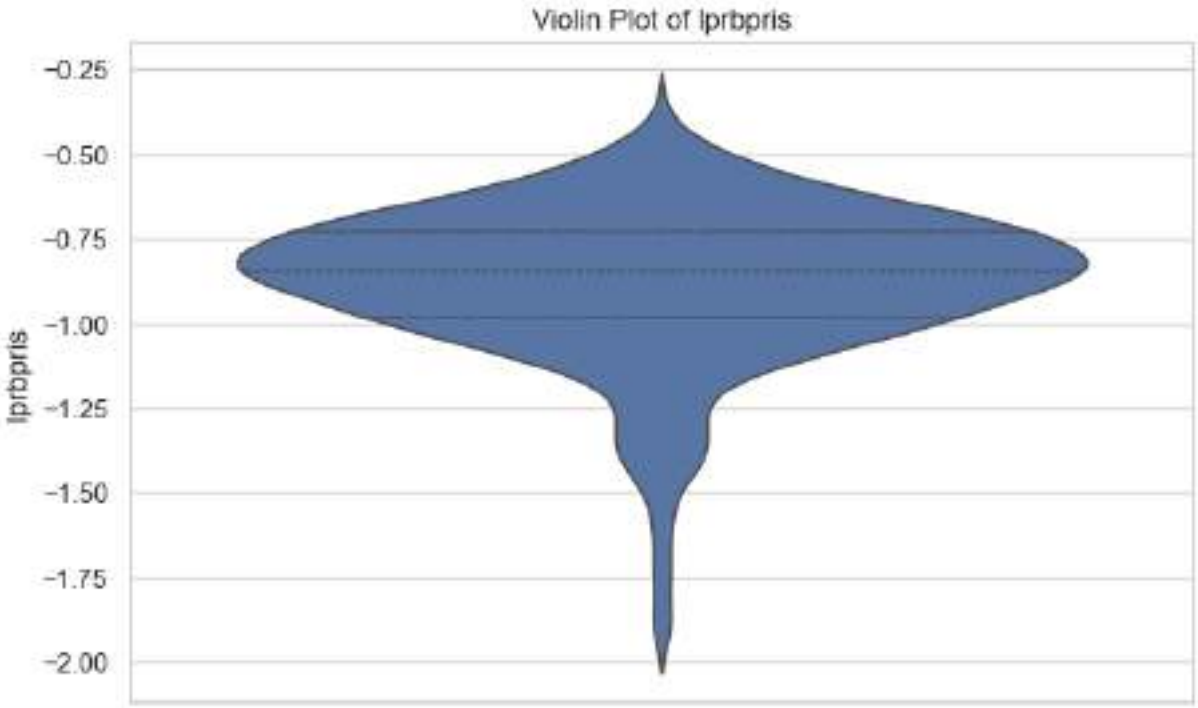

Violin Plot of mix
mix
4
3
2
1
0

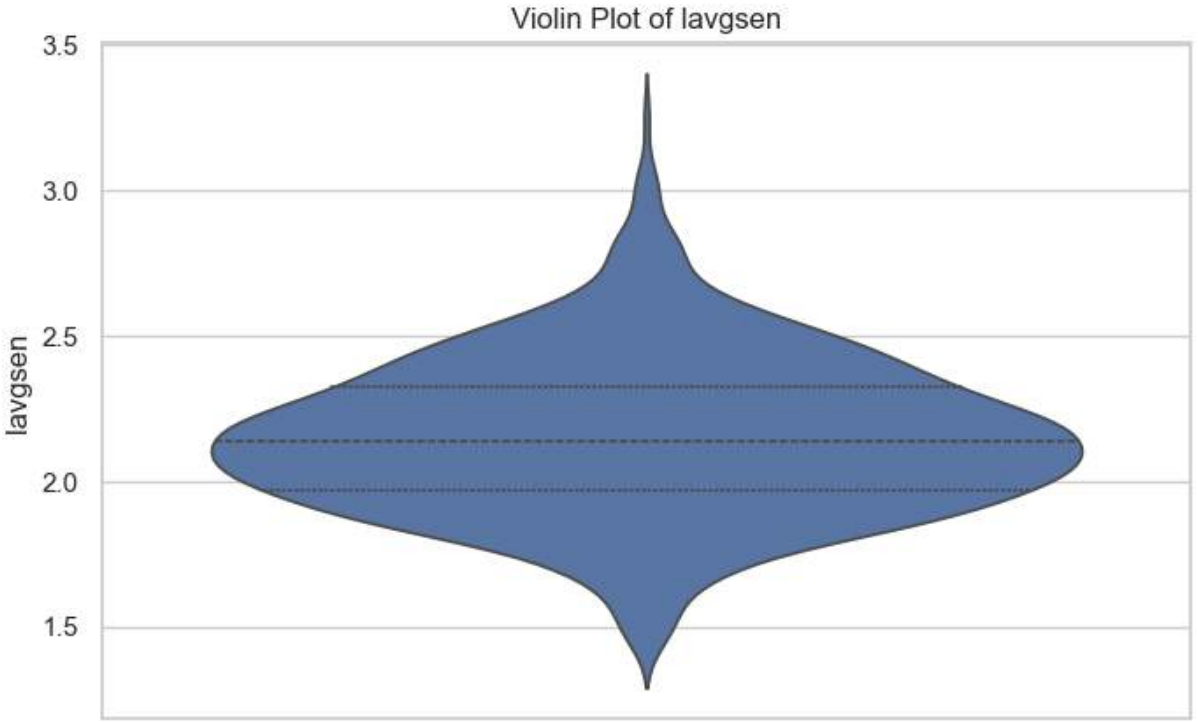

Violin Plot of pctymle
pctymle
0.25
0.20
0.15
0.10
0.05

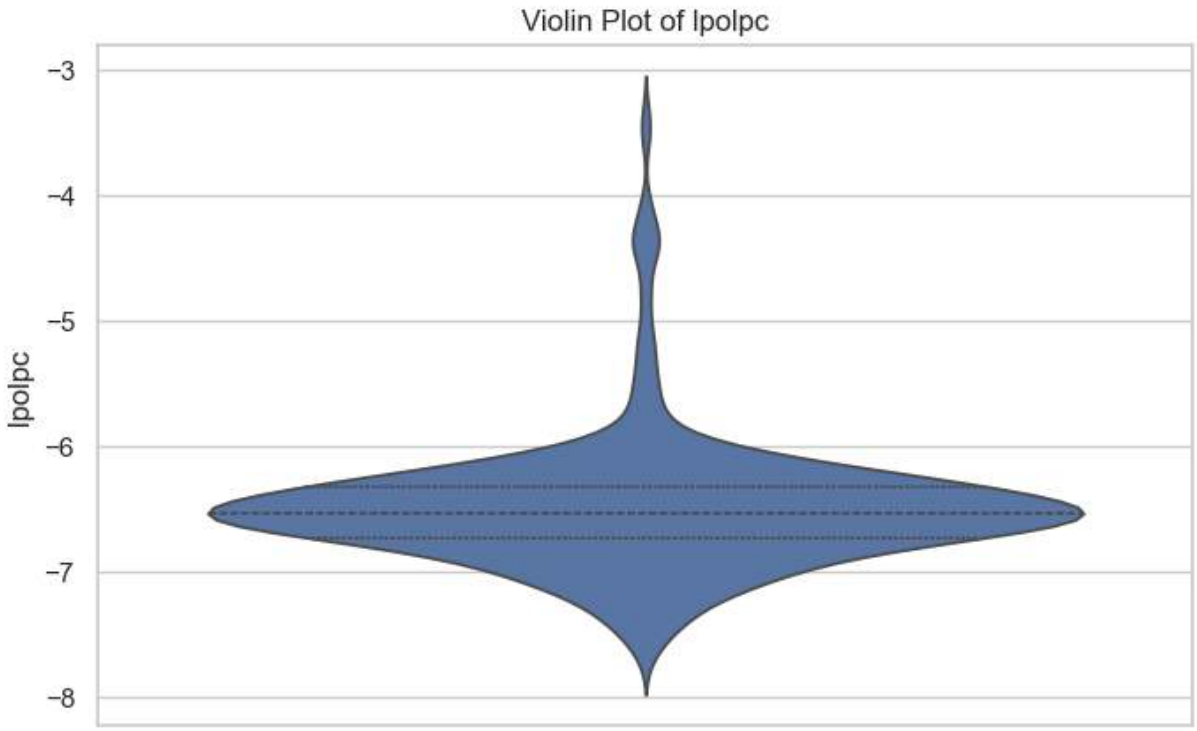

Violin Plot of lcrmrte
lcrmrte
−2
−3
−4
−5
−6

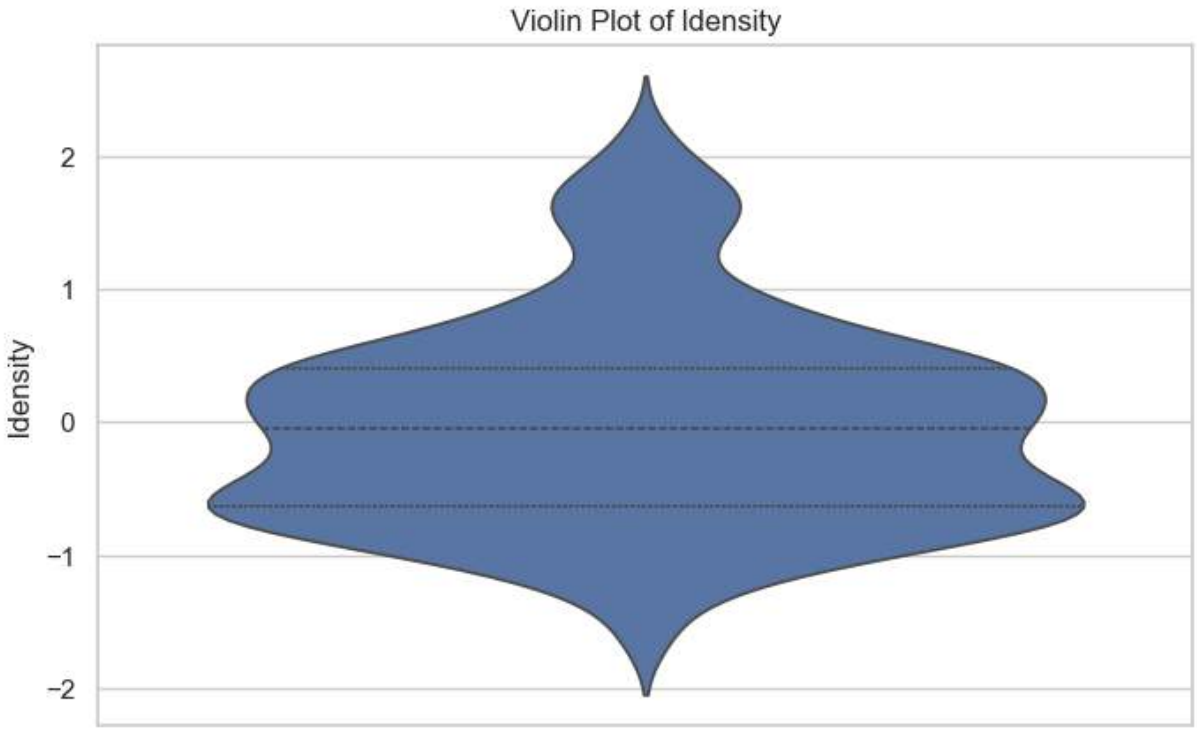

Violin Plot of lprbarr
lprbarr
1
0
−1
−2
−3

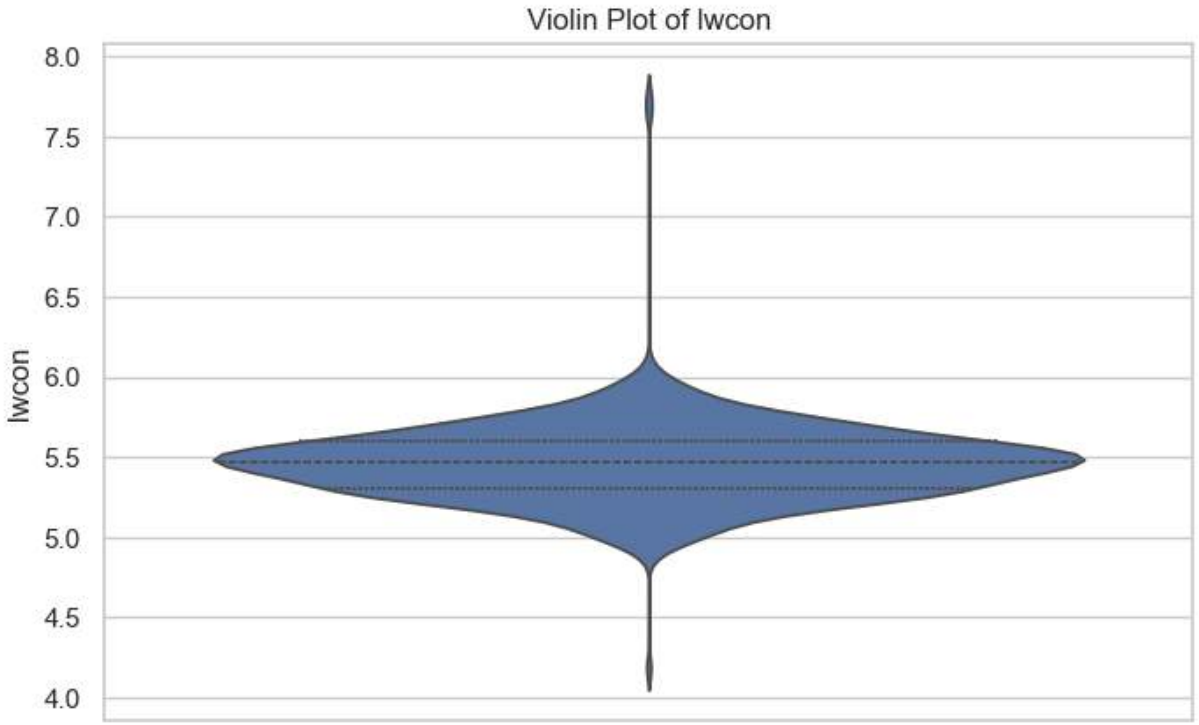

Violin Plot of lprbconv
lprbconv
4
3
2
1
0
−1
−2
−3

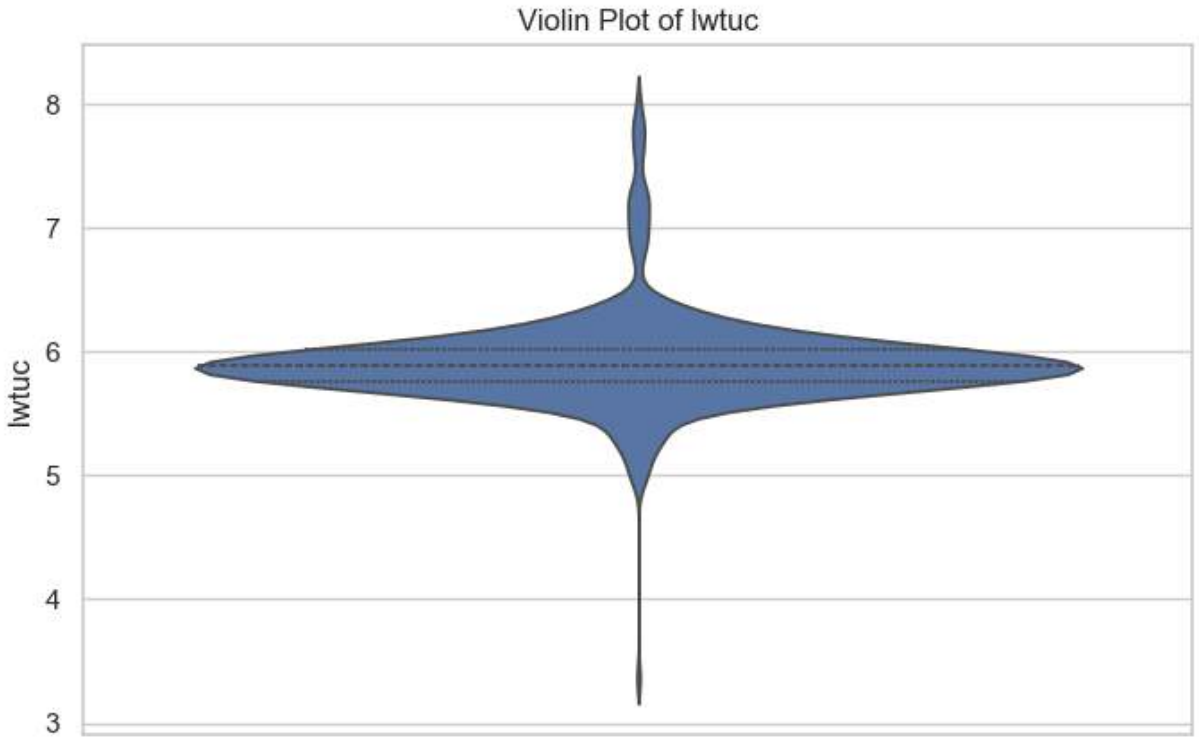

Violin Plot of lprbpris
lprbpris
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00

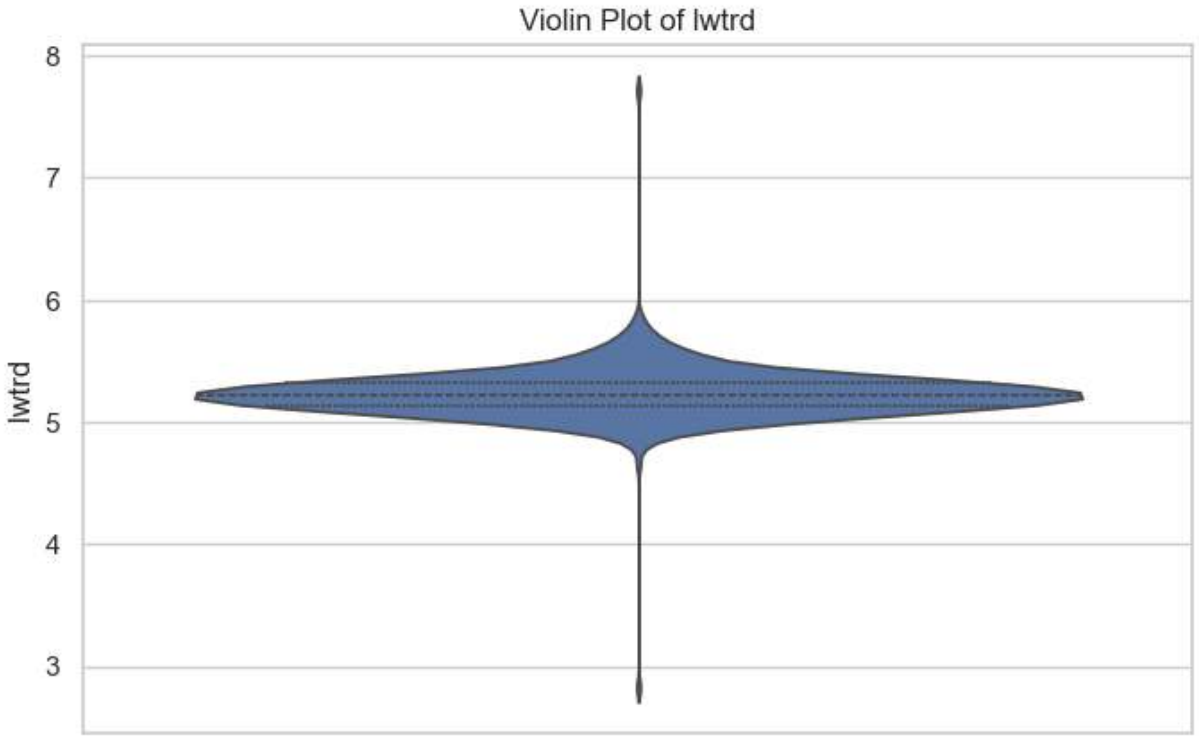

Violin Plot of lavgsen
lavgsen
3.5
3.0
2.5
2.0
1.5

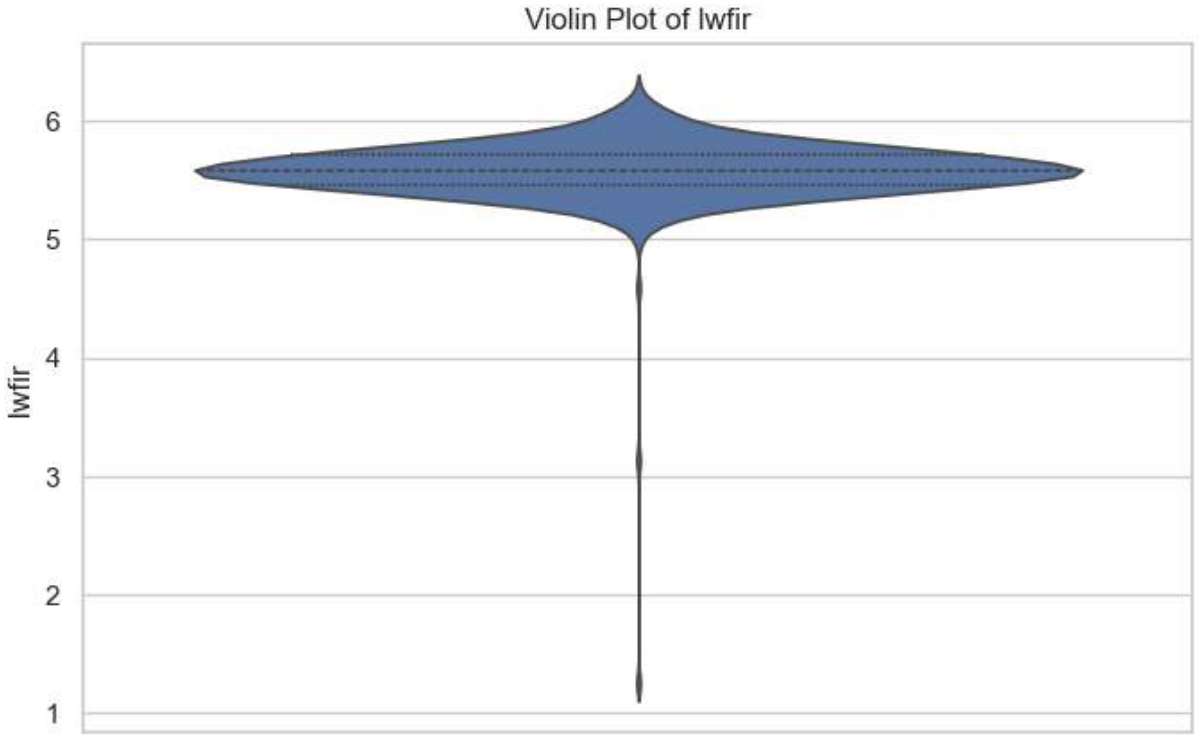

Violin Plot of lpolpc
lpolpc
−3
−4
−5
−6
−7
−8

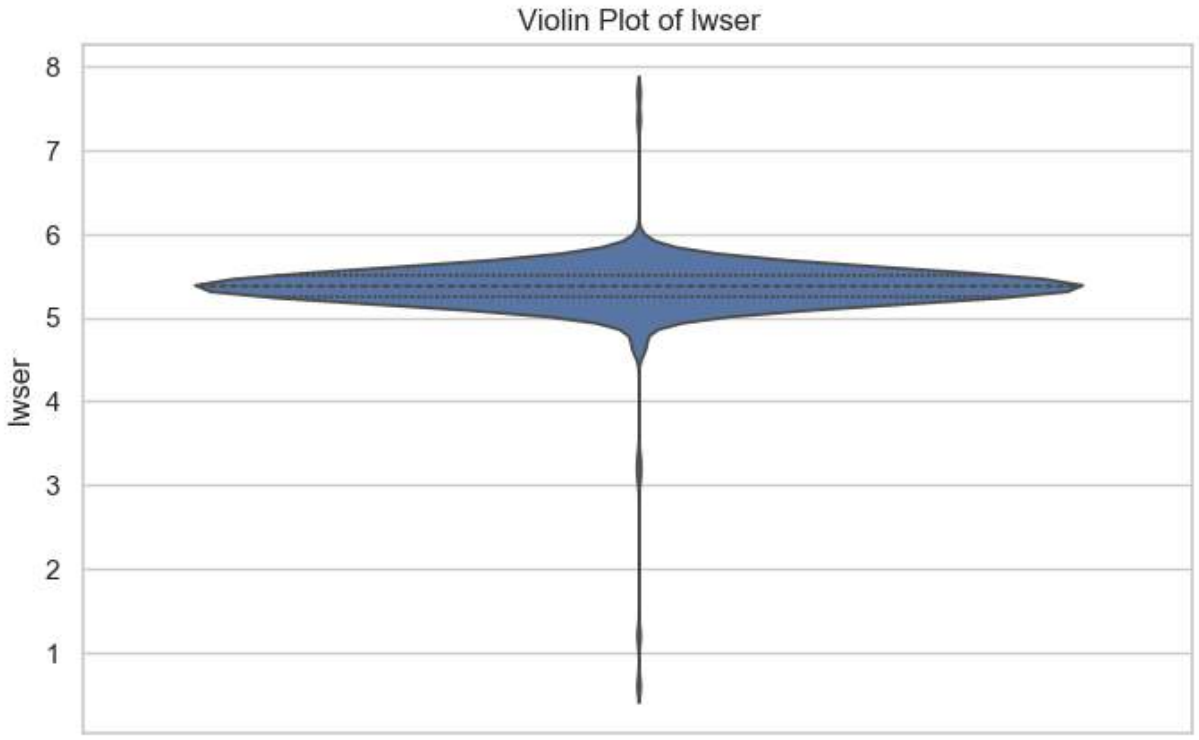

Violin Plot of ldensity
ldensity
2
1
0
−1
−2

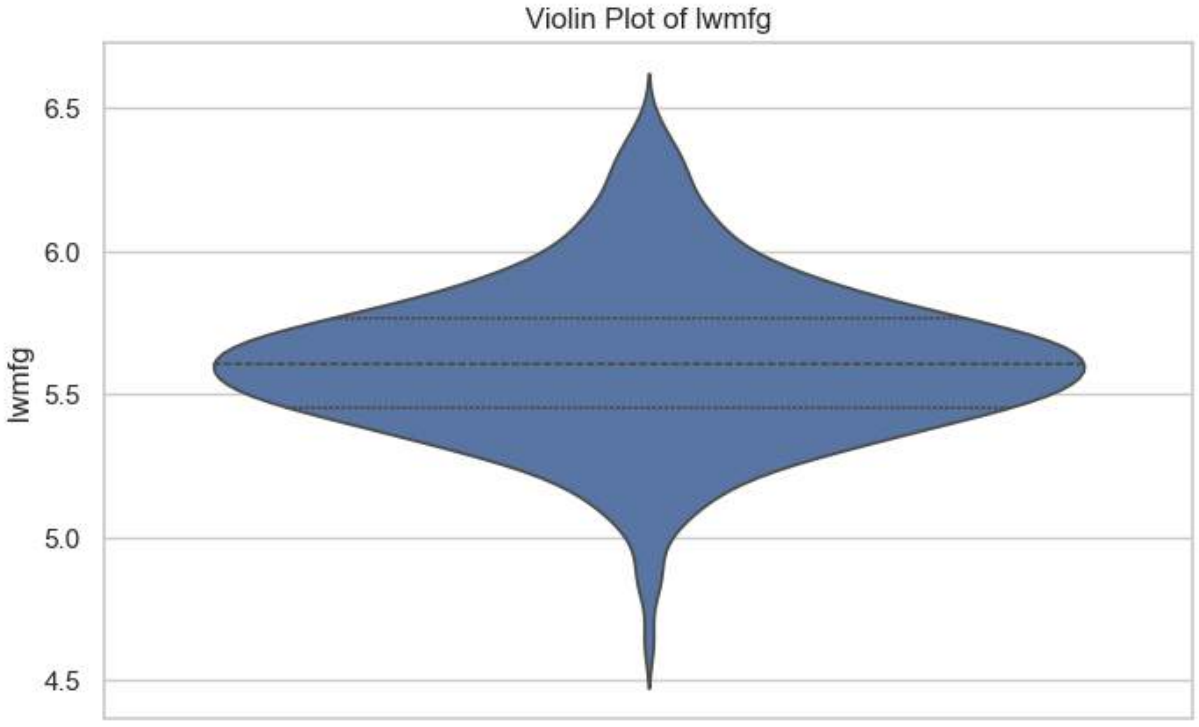

Violin Plot of lwcon
lwcon
8.0
7.5
7.0
6.5
6.0
5.5
5.0
4.5
4.0

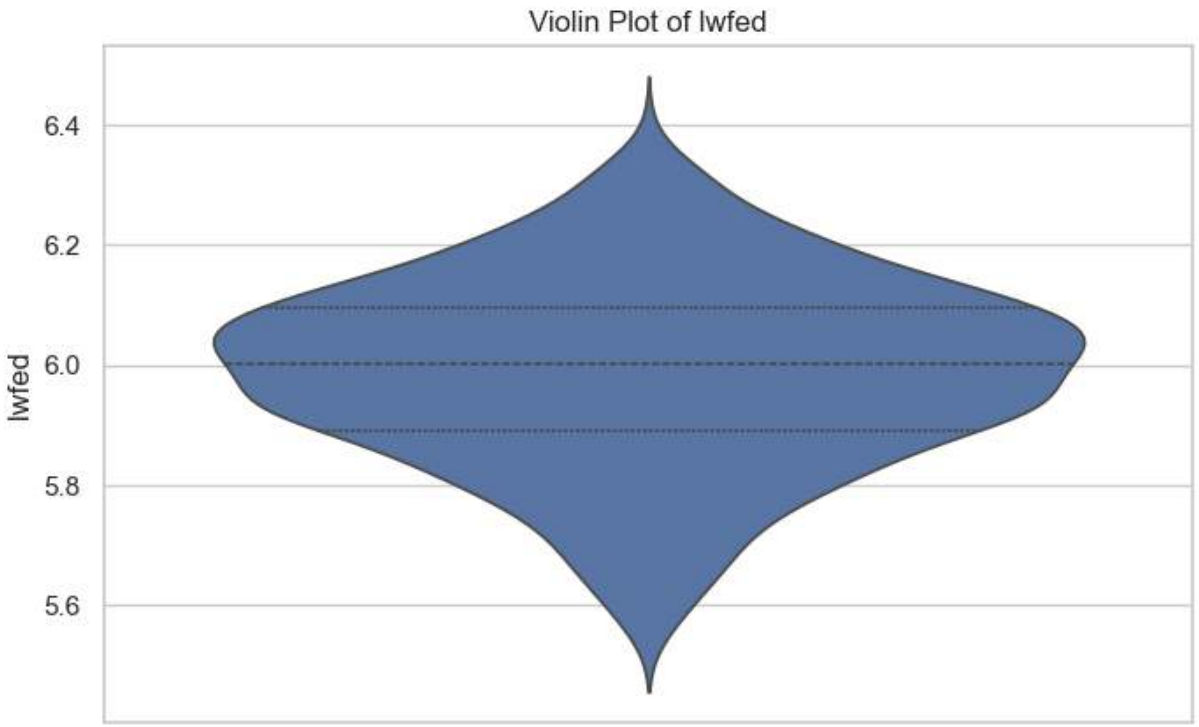

Violin Plot of lwtuc
lwtuc
8
7
6
5
4
3

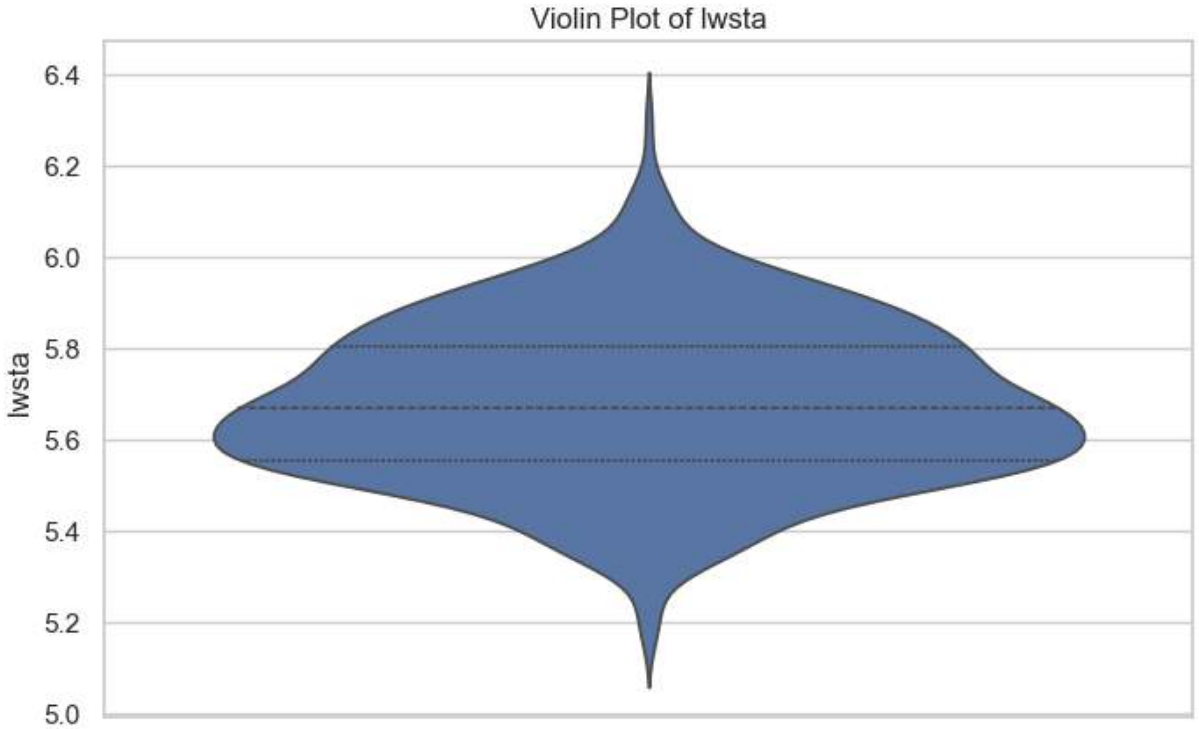

Violin Plot of lwtrd
lwtrd
8
7
6
5
4
3

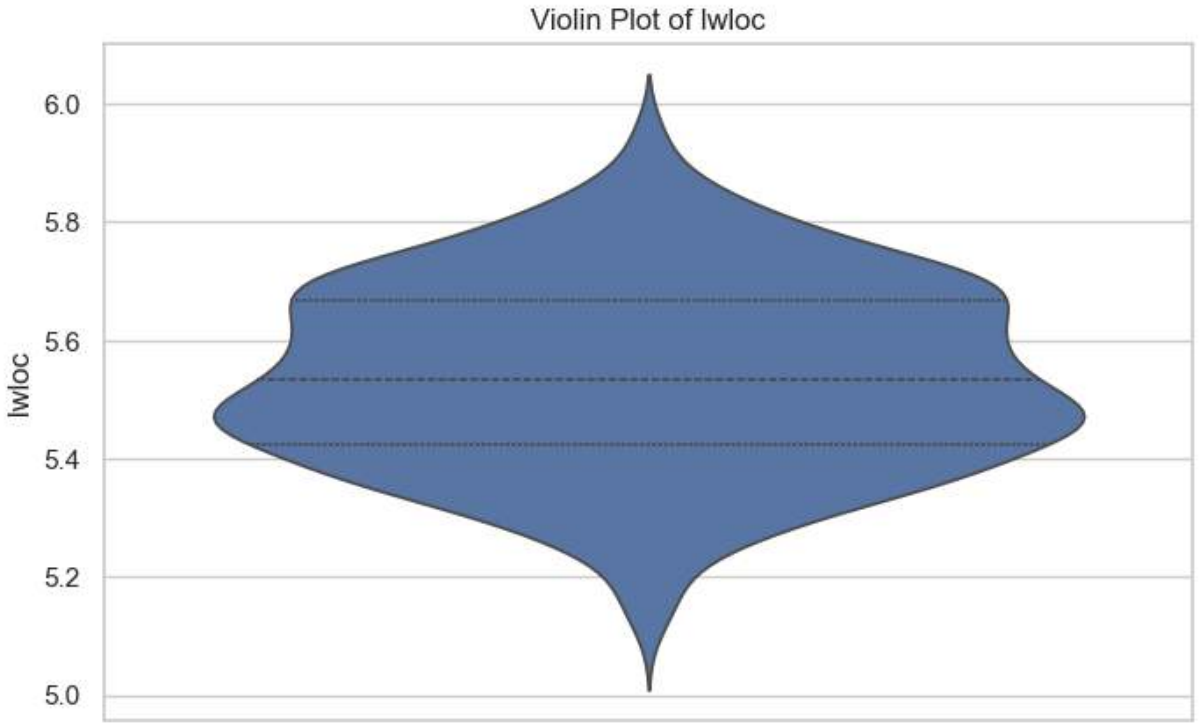

Violin Plot of lwfir
lwfir
6
5
4
3
2
1

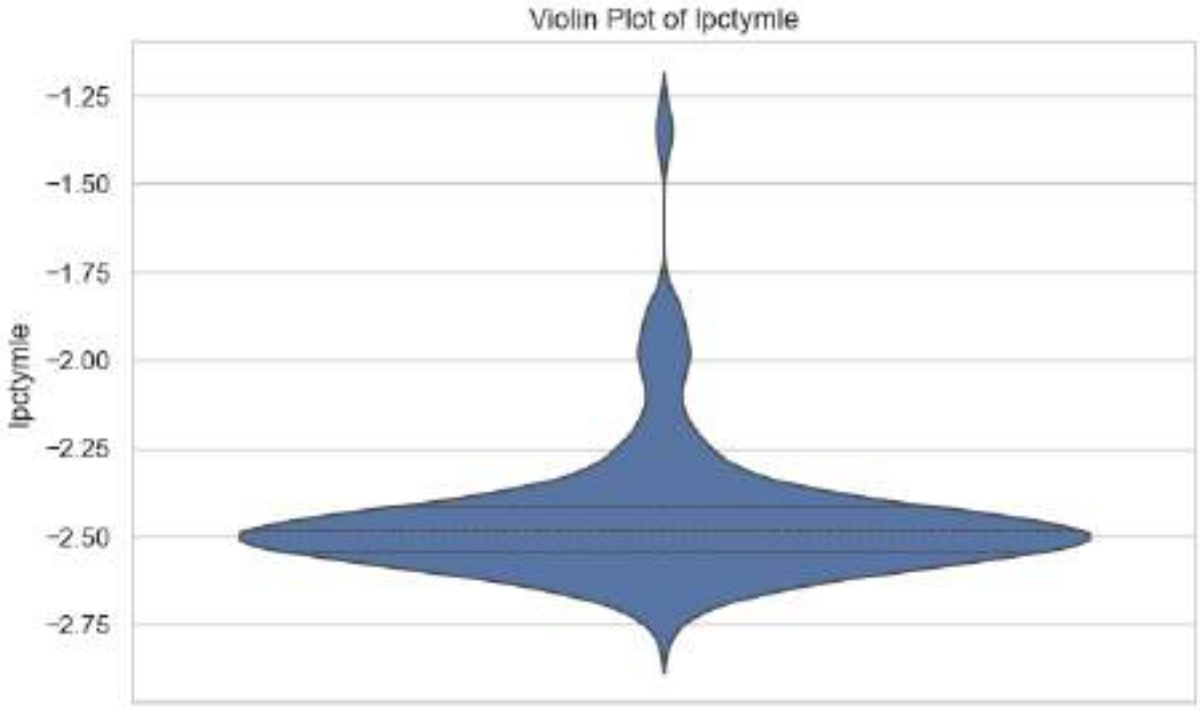

Violin Plot of lwser
lwser
8
7
6
5
4
3
2
1

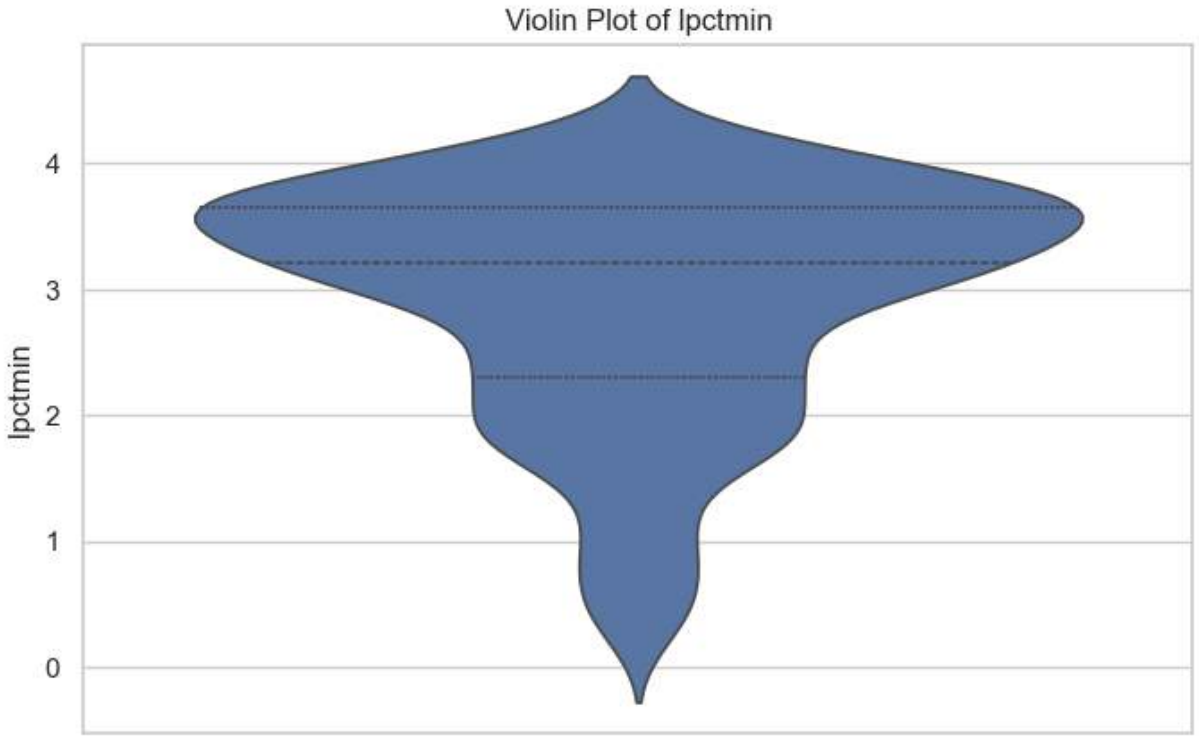

Violin Plot of lwmfg
lwmfg
6.5
6.0
5.5
5.0
4.5

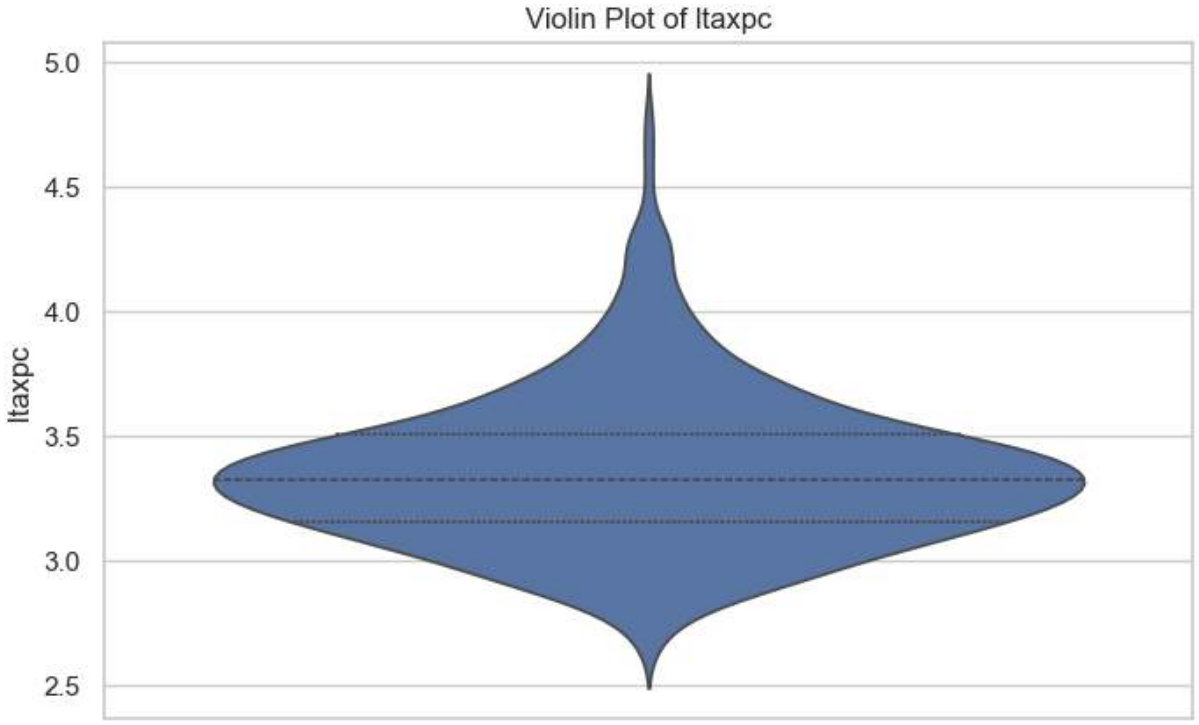

Violin Plot of lwfed
lwfed
6.4
6.2
6.0
5.8
5.6

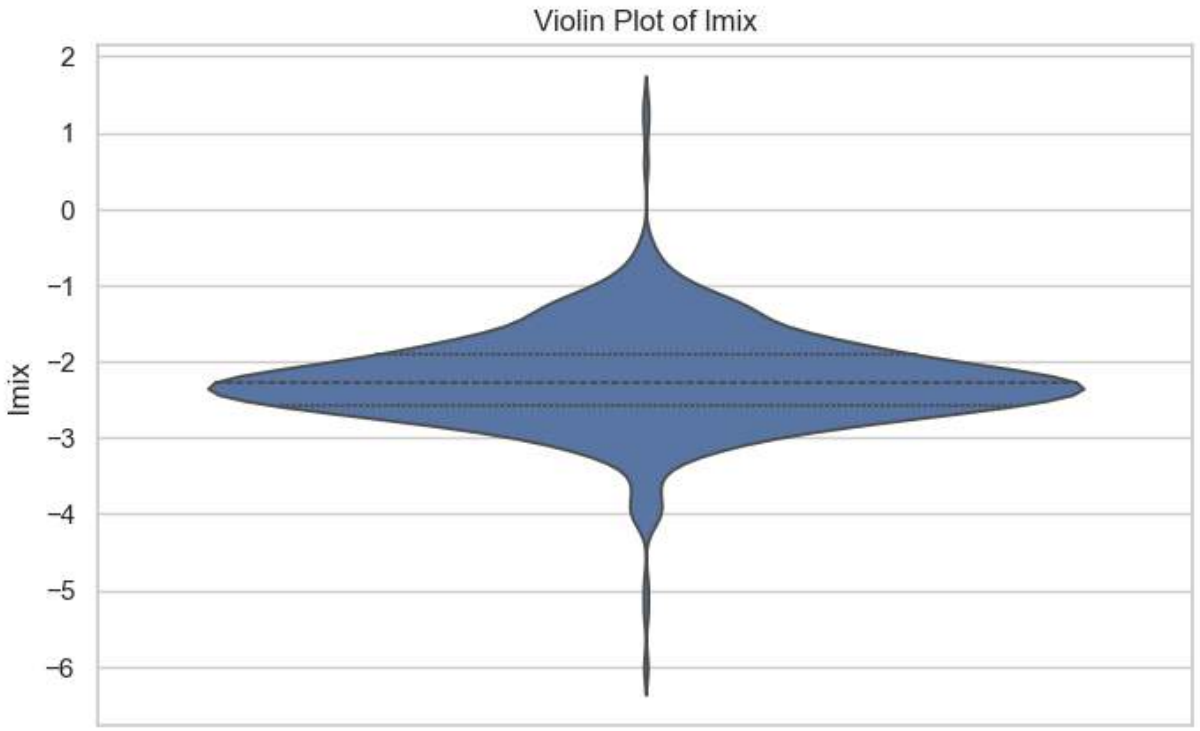

Violin Plot of lwsta
lwsta
6.4
6.2
6.0
5.8
5.6
5.4
5.2
5.0

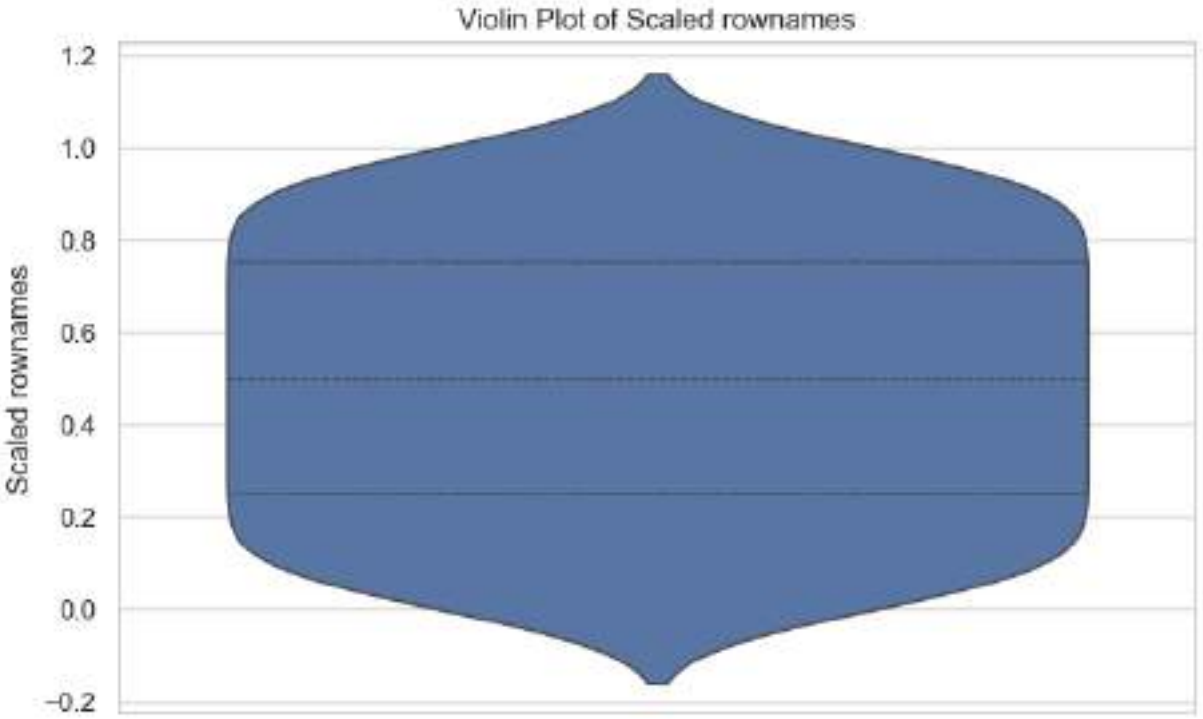

Violin Plot of lwloc
lwloc
6.0
5.8
5.6
5.4
5.2
5.0


Violin Plot of lpctymle
lpctymle
−1.25
−1.50
−1.75
−2.00
−2.25
−2.50
−2.75


Violin Plot of lpctmin
lpctmin
4
3
2
1
0


Violin Plot of ltaxpc
ltaxpc
5.0
4.5
4.0
3.5
3.0
2.5





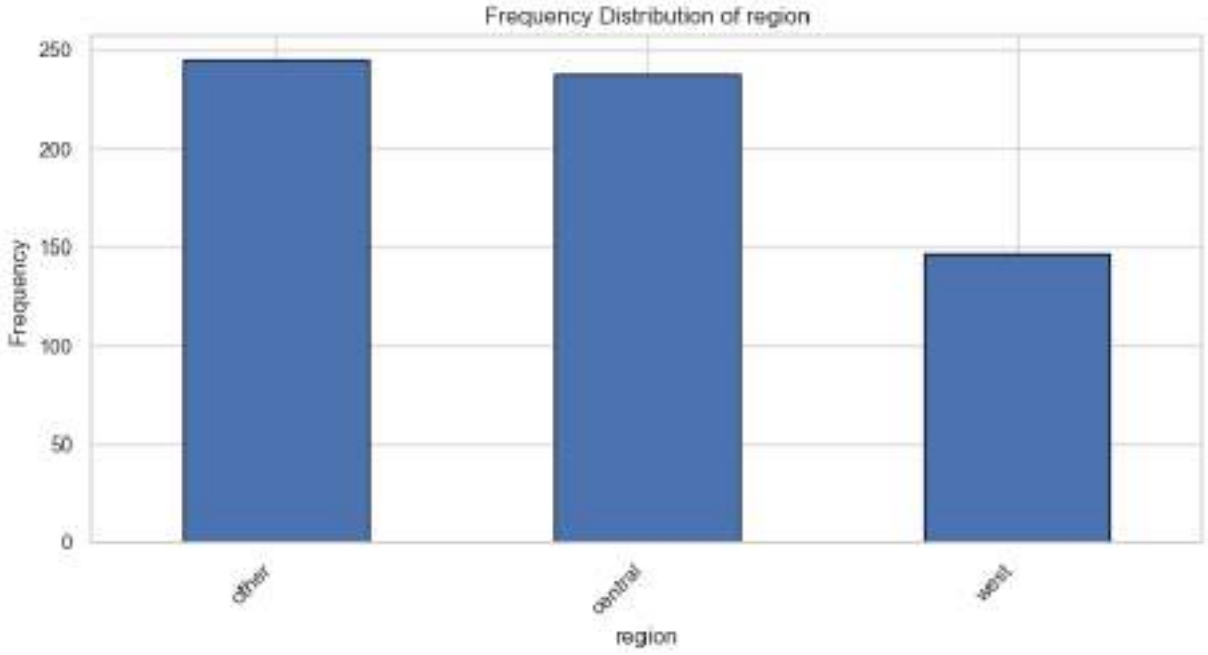
In [29]:
```
categorical = clean.select_dtypes(include=["object", "category"])

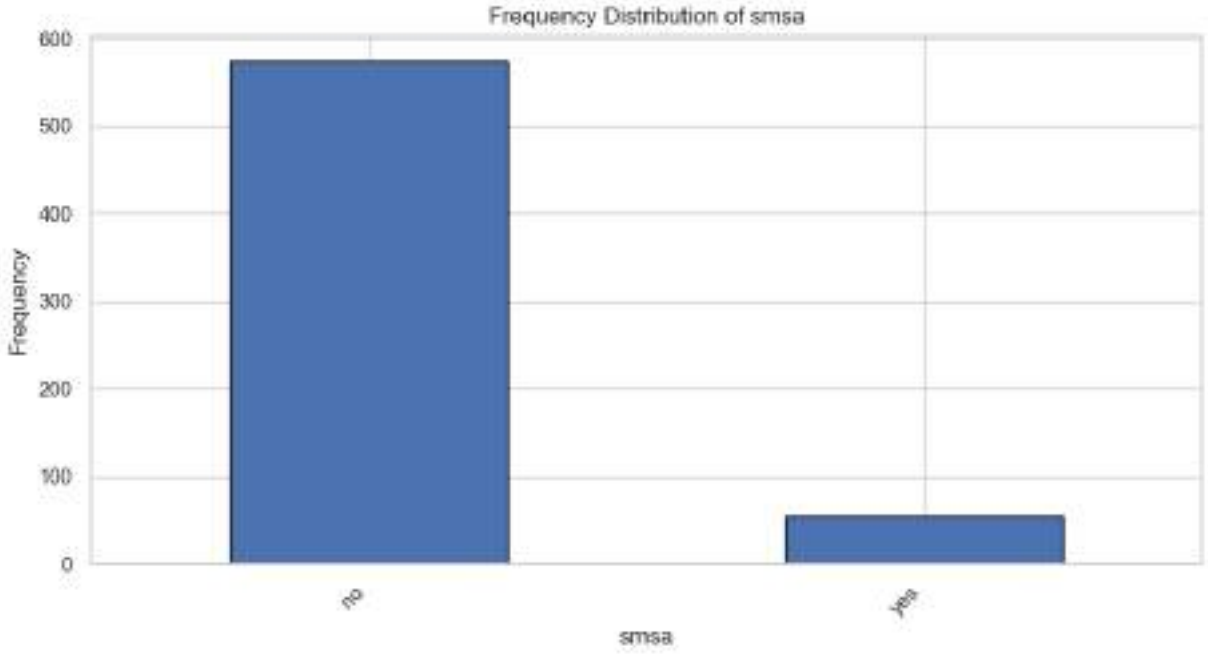
for column in categorical.columns:
    counts = clean[column].value_counts().head(15)

    plt.figure(figsize=(9, 5))
    counts.plot(kind="bar", edgecolor="black")
    plt.title("Frequency Distribution of " + column)
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.xticks(rotation=45, ha="right")
```

```
    plt.tight_layout()
    plt.show()
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\2440001326.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```





In [30]:
```
distribution = pd.DataFrame({
    "Mean": numeric.mean(),
    "Median": numeric.median(),
    "Standard Deviation": numeric.std(),
```

```
    "Skewness": numeric.skew(),
    "Kurtosis": numeric.kurtosis()
})

display(distribution)
```

	Mean	Median	Standard Deviation	Skewness	Kurtosis
rownames	315.500000	315.500000	182.009615	0.000000e+00	-1.200000
county	100.600000	103.000000	58.036270	-6.133162e-02	-1.221525
year	84.000000	84.000000	2.001589	0.000000e+00	-1.250393
crmrte	0.031588	0.028441	0.018121	1.600715e+00	5.043196
prbarr	0.307368	0.278240	0.171205	5.956328e+00	69.564300
prbconv	0.688618	0.474375	1.690345	1.725875e+01	349.314762
prbpris	0.425518	0.428571	0.087245	-3.496262e-01	0.520554
avgsen	8.954540	8.495000	2.658082	1.529076e+00	4.528210
polpc	0.001917	0.001451	0.002735	7.956826e+00	76.684154
density	1.386062	0.952595	1.439703	2.611805e+00	7.282483
taxpc	30.239194	27.792330	11.454695	2.808601e+00	13.099305
pctmin	25.712852	24.851550	16.903536	3.665350e-01	-0.787892
wcon	245.666079	236.462200	121.983720	1.340544e+01	216.095026
wtuc	406.102800	358.202400	266.513781	5.838147e+00	41.188268
wtrd	192.823065	185.477700	88.407259	1.991694e+01	461.084508
wfir	272.059255	264.423100	55.768084	4.410166e-01	2.392648
wser	224.670455	216.475300	104.866700	1.366532e+01	234.325724
wmfg	285.170095	271.595000	82.368065	1.280358e+00	2.538895
wfed	403.895889	403.980000	63.066692	1.482183e-01	-0.195064
wsta	296.907460	289.370000	53.431614	6.890717e-01	0.756478
wloc	257.976206	253.135000	41.358021	3.723024e-01	-0.340089
mix	0.139396	0.102089	0.219755	1.295252e+01	201.381011
pctymle	0.088974	0.083165	0.024349	4.524776e+00	26.513637
lcrmrte	-3.609225	-3.559938	0.572808	-4.313562e-01	1.314699
lprbarr	-1.274264	-1.279271	0.415897	3.837816e-01	2.325636
lprbconv	-0.692919	-0.745758	0.609595	1.561801e+00	6.900779
lprbpris	-0.878632	-0.847299	0.230514	-1.310374e+00	3.022812
lavgsen	2.153344	2.139477	0.273729	3.792367e-01	0.514685
lpolpc	-6.490637	-6.535785	0.526654	2.231947e+00	9.267324
ldensity	-0.015927	-0.048567	0.774735	6.193981e-01	0.045265

	Mean	Median	Standard Deviation	Skewness	Kurtosis
lwcon	5.462869	5.465789	0.248178	2.074103e+00	20.720376
lwtuc	5.915883	5.881098	0.370219	1.486021e+00	11.596394
lwtrd	5.232423	5.222934	0.214391	4.331911e-01	52.724230
lwfir	5.579433	5.577550	0.277204	-7.085851e+00	102.543828
lwser	5.364625	5.377477	0.360098	-6.120316e+00	83.018083
lwmfg	5.615181	5.604311	0.272747	1.733652e-01	0.898130
lwfed	5.988757	6.001365	0.158761	-2.689353e-01	-0.155638
lwsta	5.677787	5.667706	0.176131	1.667852e-01	-0.079157
lwloc	5.540139	5.533923	0.159691	1.820612e-02	-0.505745
lpctymle	-2.443015	-2.486930	0.196784	2.778591e+00	10.614598
lpctmin	2.913361	3.212685	0.954615	-9.614943e-01	0.129742
ltaxpc	3.356385	3.324759	0.308224	9.377160e-01	1.889388
lmix	-2.234233	-2.281916	0.649565	1.002882e-01	5.547617
Scaled rownames	0.500000	0.500000	0.289363	1.169309e-16	-1.200000

In [31]:
```
print("Assessment 4 – Bivariate analysis")
```

```
Assessment 4 – Bivariate analysis
```

In [32]:
```
numeric = clean.select_dtypes(include=np.number)

correlation = numeric.corr()

display(correlation)

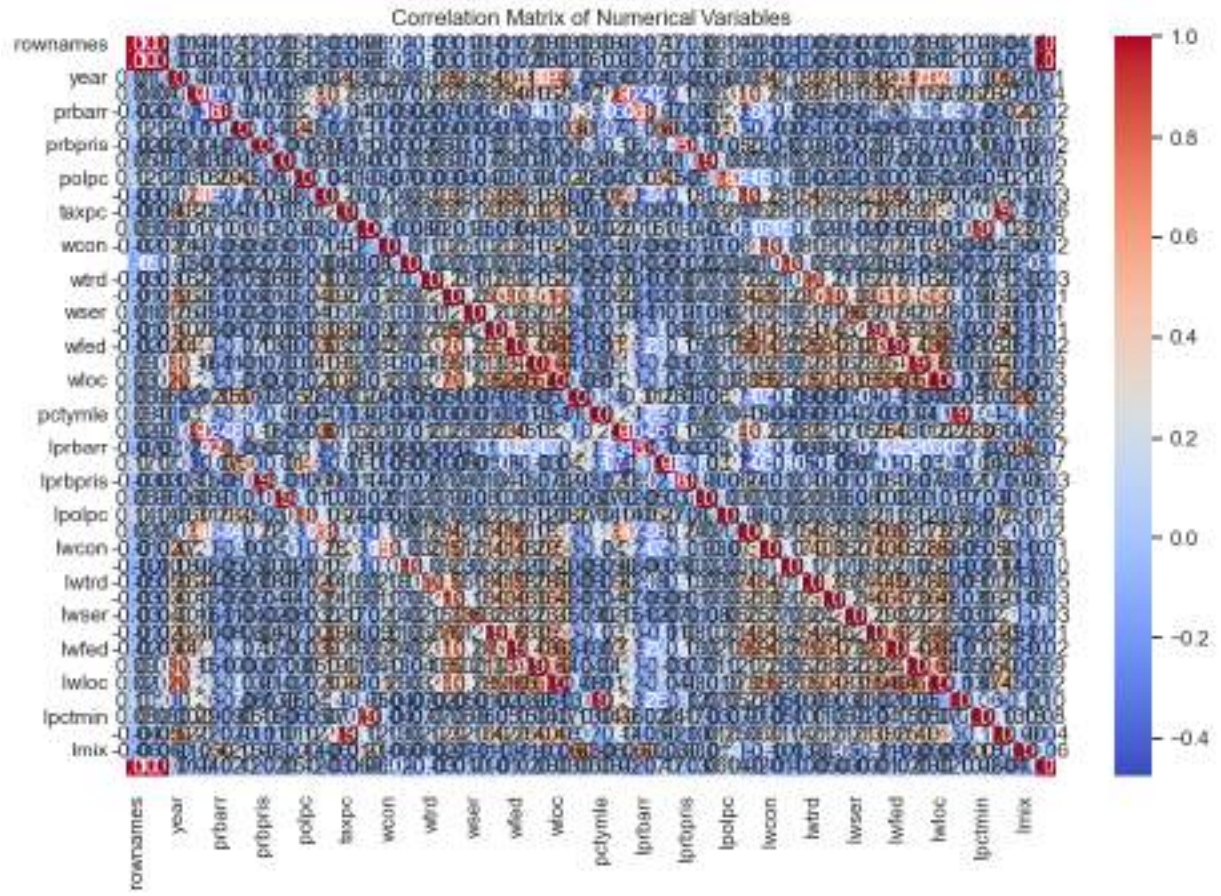
plt.figure(figsize=(10, 7))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix of Numerical Variables")
plt.tight_layout()
plt.show()
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpri
rownames	1.000000	9.994803e-01	1.099716e-02	0.037209	-0.020628	0.119920	-0.02418
county	0.999480	1.000000e+00	2.479813e-14	0.041679	-0.018569	0.118875	-0.02486
year	0.010997	2.479813e-14	1.000000e+00	0.002392	-0.028067	0.006527	-0.09608
crmrte	0.037209	4.167863e-02	2.392457e-03	1.000000	-0.358553	-0.113033	0.13553
prbarr	-0.020628	-1.856917e-02	-2.806708e-02	-0.358553	1.000000	0.035569	-0.07489
prbconv	0.119920	1.188753e-01	6.527245e-03	-0.113033	0.035569	1.000000	-0.03734
prbpris	-0.024187	-2.486140e-02	-9.608535e-02	0.135537	-0.074893	-0.037340	1.00000
avgsen	0.049026	5.053367e-02	-7.525718e-02	0.032416	0.034031	0.015305	-0.00429
polpc	0.122540	1.237755e-01	2.536452e-02	0.184826	0.290581	0.449635	-0.05745
density	-0.026558	-2.580562e-02	2.347055e-02	0.694072	-0.271228	-0.115555	0.16466
taxpc	-0.058097	-6.176276e-02	4.263984e-01	0.230685	-0.041217	0.006733	-0.11206
pctmin	0.058527	6.231841e-02	3.330427e-15	0.169021	0.100050	0.105077	0.13072
wcon	-0.015448	-1.897445e-02	2.371506e-01	0.070500	-0.088110	-0.045082	-0.02921
wtuc	-0.147590	-1.501372e-01	3.164835e-02	-0.002451	-0.022224	-0.023258	-0.01930
wtrd	-0.025822	-2.838055e-02	1.640346e-01	0.232684	-0.074539	-0.049324	0.05663
wfir	-0.005749	-1.292808e-02	5.627879e-01	0.256157	-0.100232	-0.064391	0.00783
wser	0.012649	8.726151e-03	2.486292e-01	0.092340	-0.098007	-0.017110	-0.00504
wmfg	-0.007828	-1.415905e-02	3.994417e-01	0.279718	-0.077343	-0.070292	0.00151
wfed	-0.018385	-2.203766e-02	4.435469e-01	0.437157	-0.185477	-0.072332	0.13457
wsta	0.090430	8.282573e-02	6.887883e-01	0.163967	-0.113617	-0.006010	-0.07158
wloc	0.028408	2.020241e-02	7.938777e-01	0.234601	-0.134148	-0.030400	-0.00217
mix	0.057149	5.688059e-02	-1.884485e-02	-0.119574	0.347530	0.506171	0.02936

	rownames	county	year	crmrte	prbarr	prbconv	prbpri
pctymle	0.092493	9.827258e-02	-1.270499e-01	0.224795	-0.156273	-0.069298	-0.13324
lcrmrte	0.023098	2.656217e-02	-1.618459e-02	0.918616	-0.477639	-0.189442	0.15908
lprbarr	-0.068415	-6.754402e-02	-2.412379e-02	-0.417453	0.873081	-0.048559	-0.02406
lprbconv	0.168604	1.668116e-01	2.886936e-02	-0.351079	0.067288	0.600954	-0.15009
lprbpris	-0.025198	-2.594194e-02	-8.263118e-02	0.154085	-0.081841	-0.041737	0.98156
lavgsen	0.055181	5.601315e-02	-6.493201e-02	0.063388	0.005150	-0.010197	0.00630
lpolpc	0.138604	1.378293e-01	5.865608e-02	0.309255	0.232341	0.353395	-0.05098
ldensity	0.021652	2.155694e-02	2.862764e-02	0.656662	-0.338033	-0.168948	0.22253
lwcon	-0.012341	-1.819130e-02	4.668645e-01	0.211537	-0.185545	-0.103118	-0.03734
lwtuc	-0.097348	-1.011973e-01	1.801850e-01	0.082956	-0.049988	-0.045347	0.00482
lwtrd	-0.047426	-5.202811e-02	3.631093e-01	0.338487	-0.086186	-0.097893	0.08268
lwfir	-0.026191	-3.077255e-02	4.004590e-01	0.175438	-0.045783	-0.051362	-0.03874
lwser	-0.031544	-3.511456e-02	2.980535e-01	0.156418	-0.108968	-0.041474	-0.02474
lwmfg	-0.009238	-1.571425e-02	4.213892e-01	0.298015	-0.082256	-0.083755	0.03918
lwfed	-0.021410	-2.541192e-02	4.426130e-01	0.414733	-0.176566	-0.066186	0.14596
lwsta	0.087190	7.933925e-02	7.003245e-01	0.153715	-0.103448	-0.001660	-0.07228
lwloc	0.023774	1.573701e-02	7.933633e-01	0.234105	-0.138880	-0.027186	0.00706
lpctymle	0.096708	1.026238e-01	-1.785723e-01	0.259973	-0.170675	-0.083677	-0.12641
lpctmin	0.079925	8.453915e-02	4.838256e-15	0.294210	-0.091289	0.045294	0.16425
ltaxpc	-0.041196	-4.538220e-02	5.201255e-01	0.218594	-0.023602	0.014066	-0.10940
lmix	-0.060461	-5.880649e-02	1.013102e-02	-0.050270	0.423923	0.154635	0.07614

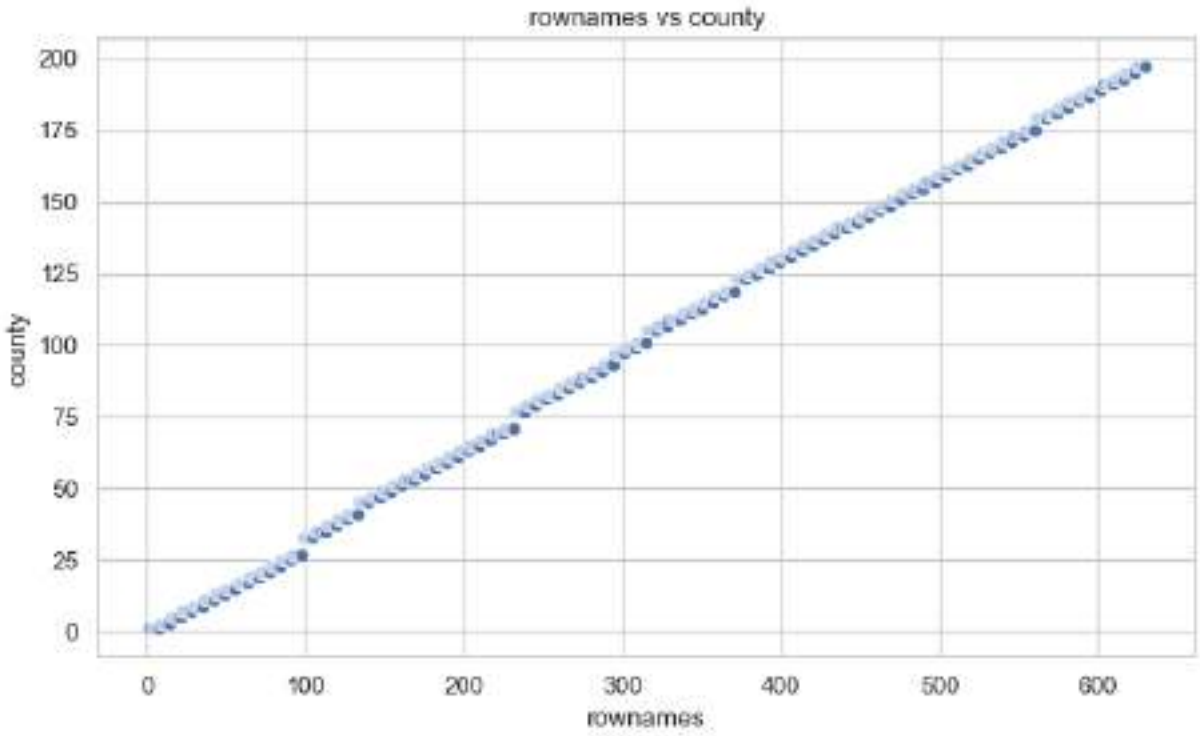
	rownames	county	year	crmrte	prbarr	prbconv	prbpri
Scaled rownames	1.000000	9.994803e-01	1.099716e-02	0.037209	-0.020628	0.119920	-0.02418

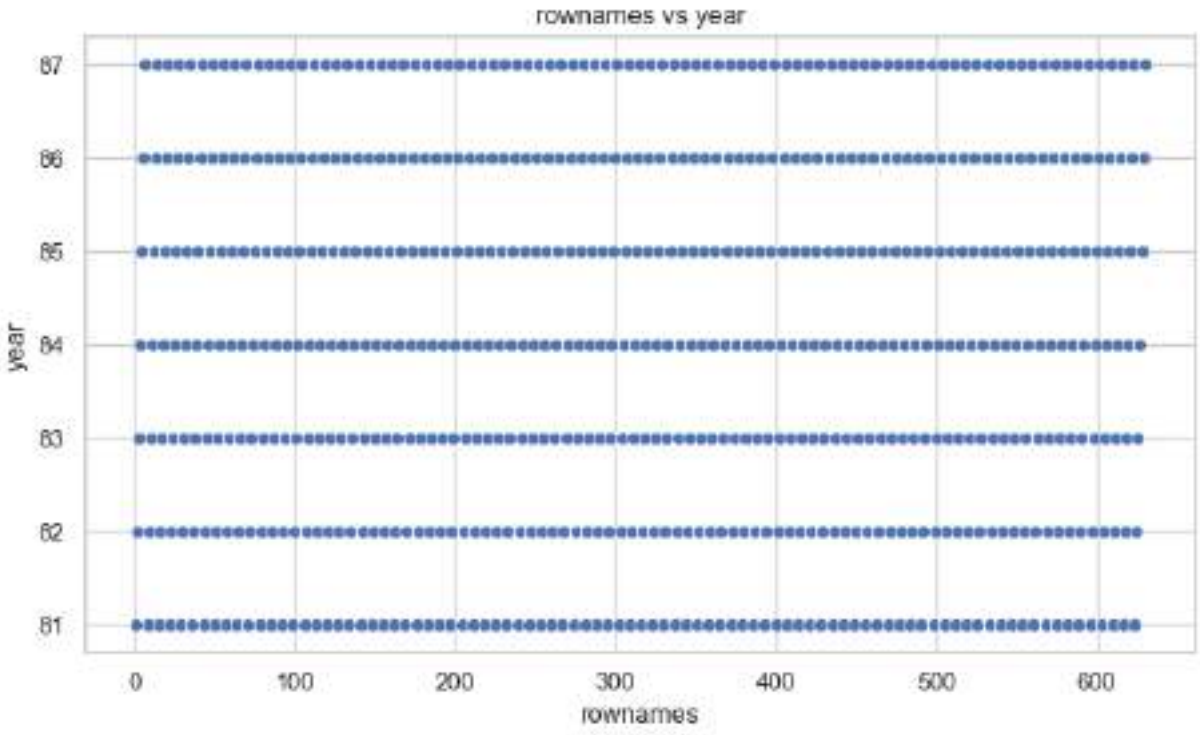
44 rows × 44 columns

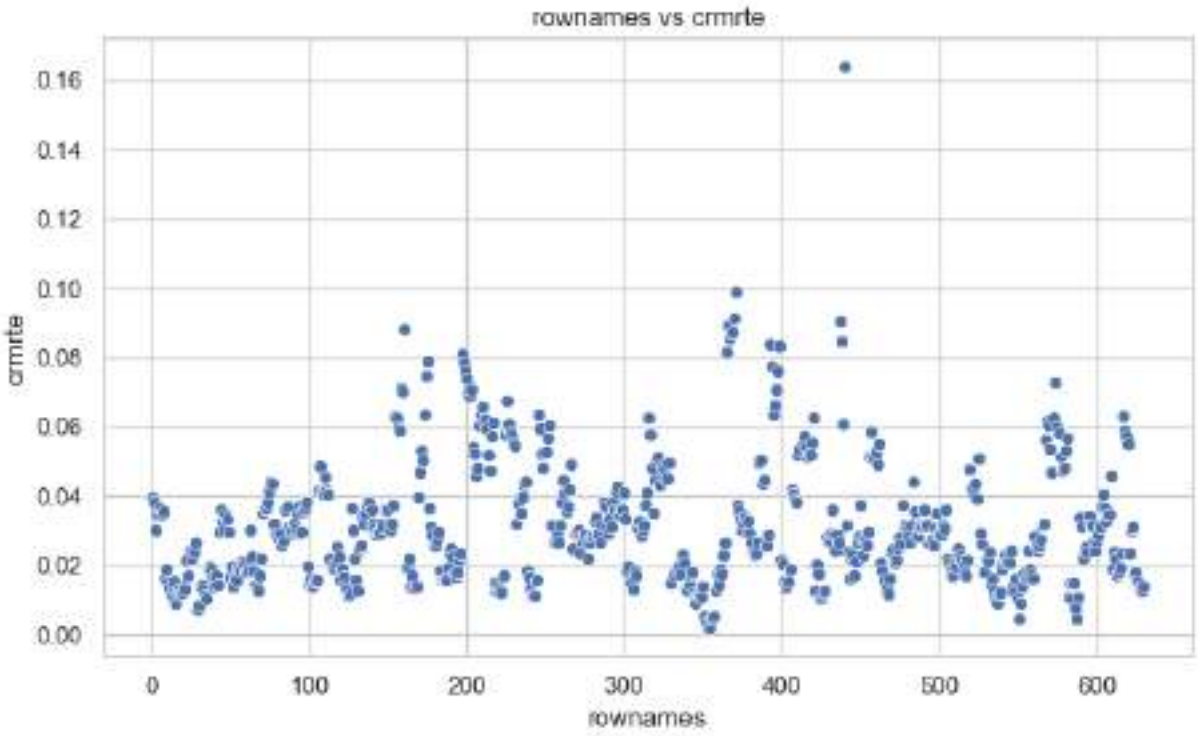


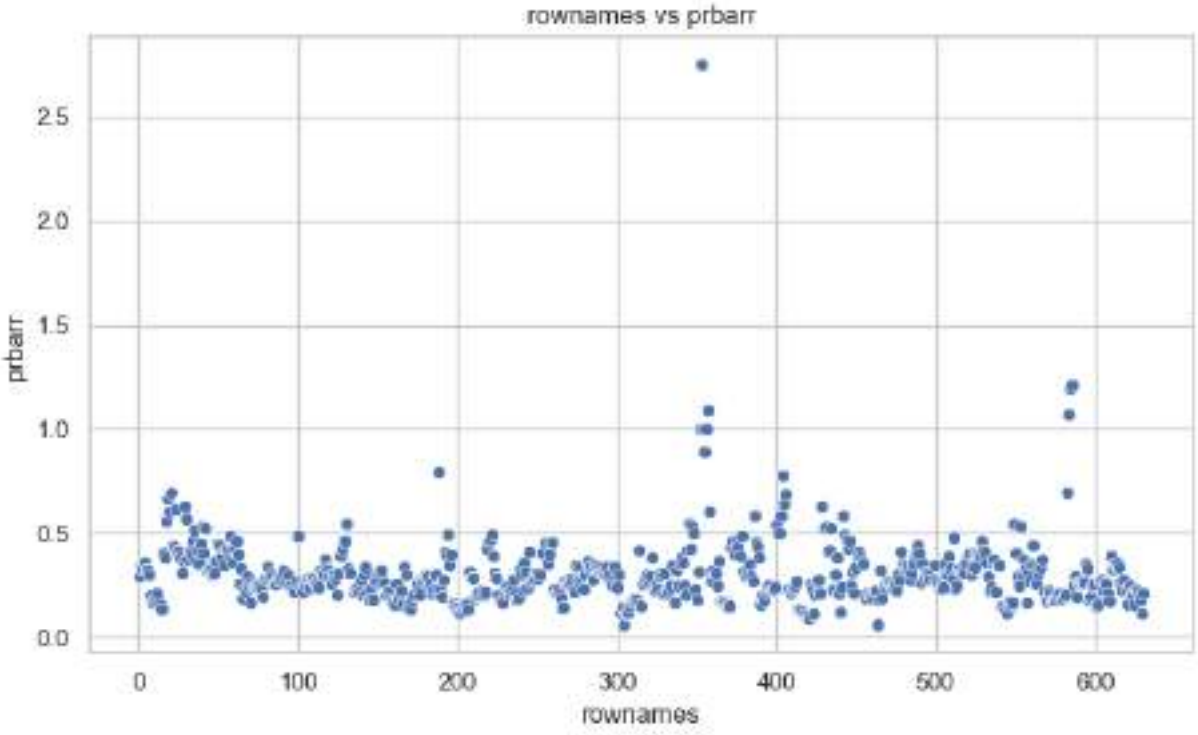
In [33]:
```
numeric = clean.select_dtypes(include=np.number)
columns = numeric.columns[:6]

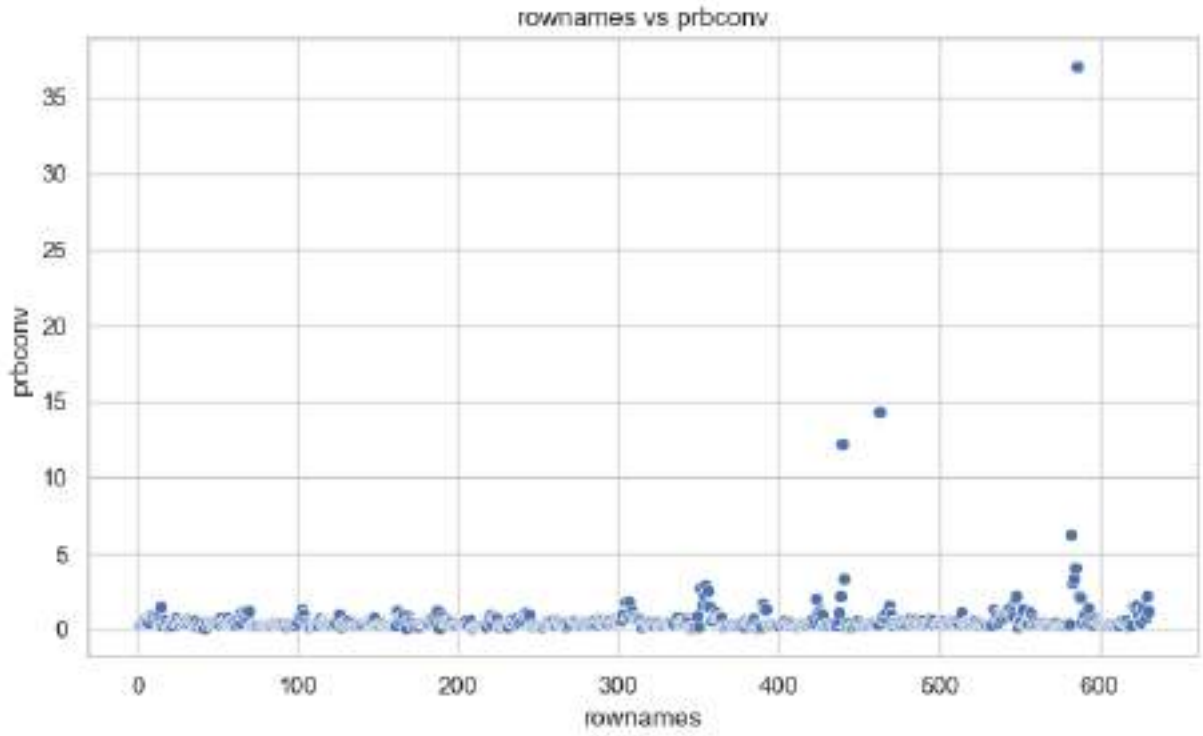
for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        plt.figure()
        sns.scatterplot(data=clean, x=columns[i], y=columns[j])
        plt.title(columns[i] + " vs " + columns[j])
        plt.tight_layout()
        plt.show()
```

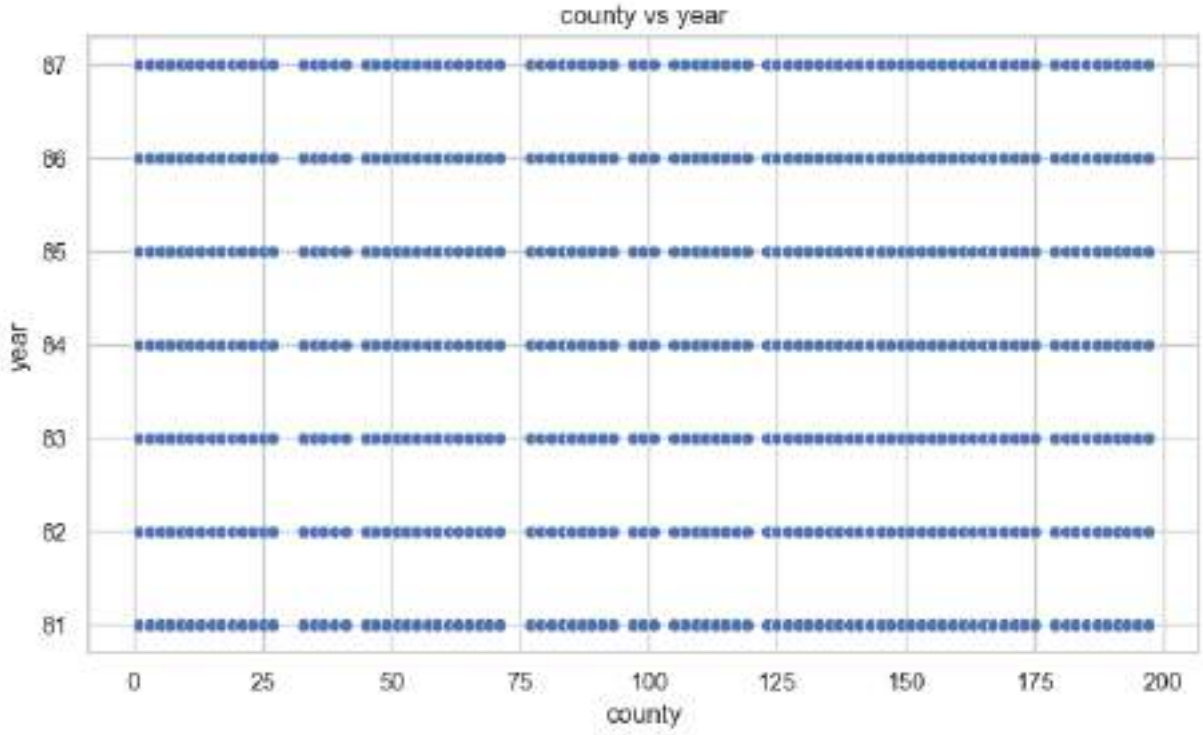

rownames vs county
county
rownames
0
25
50
75
100
125
150
175
200
0
100
200
300
400
500
600

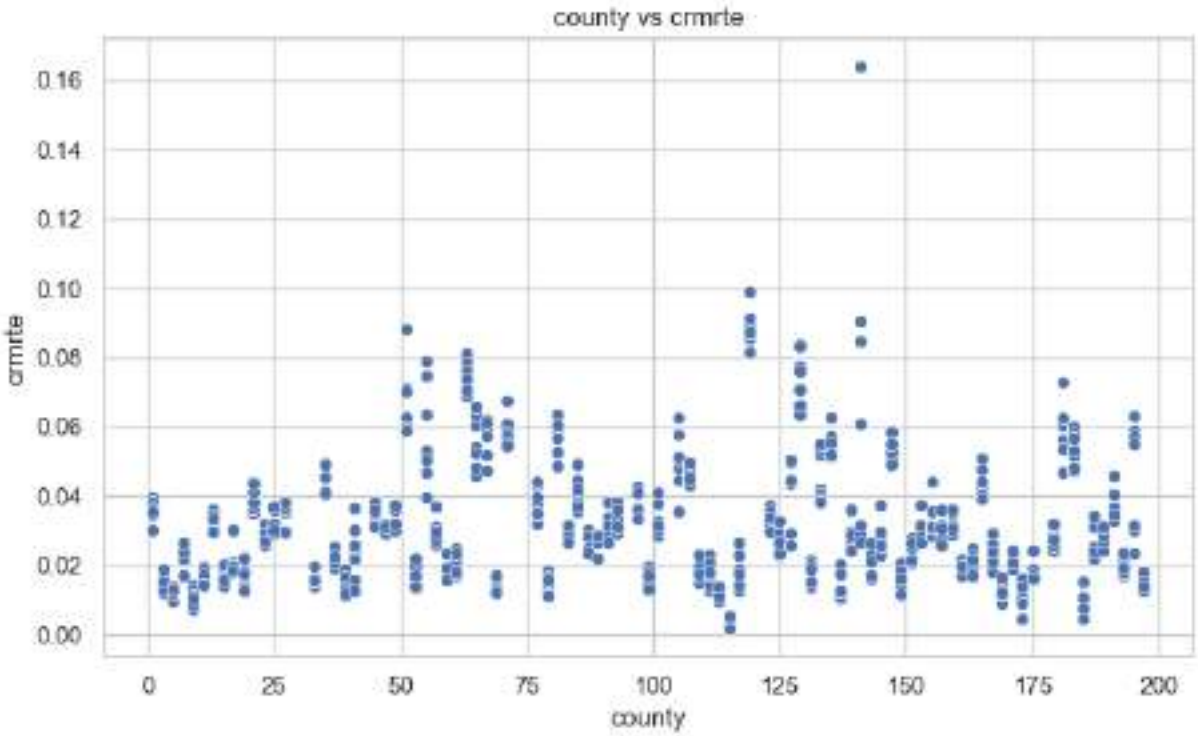

rownames vs year
year
rownames
81
82
83
84
85
86
87
0
100
200
300
400
500
600

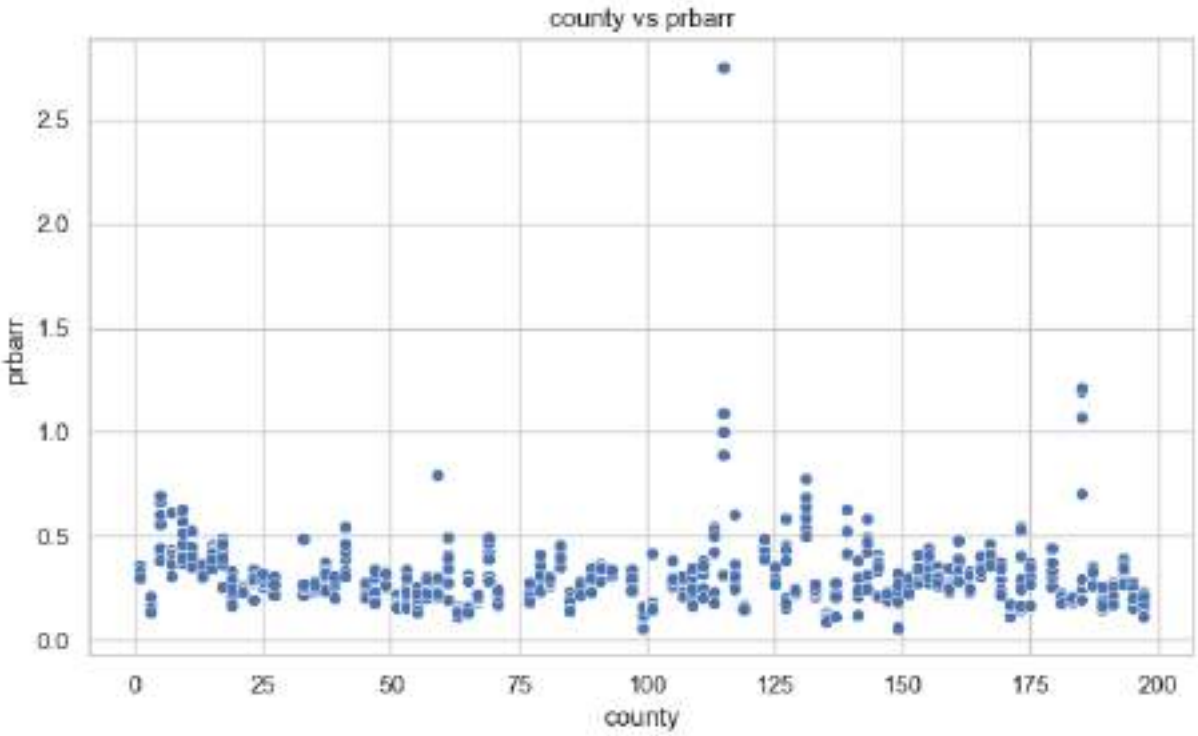

rownames vs crmrte
crmrte
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
0
100
200
300
400
500
600
rownames

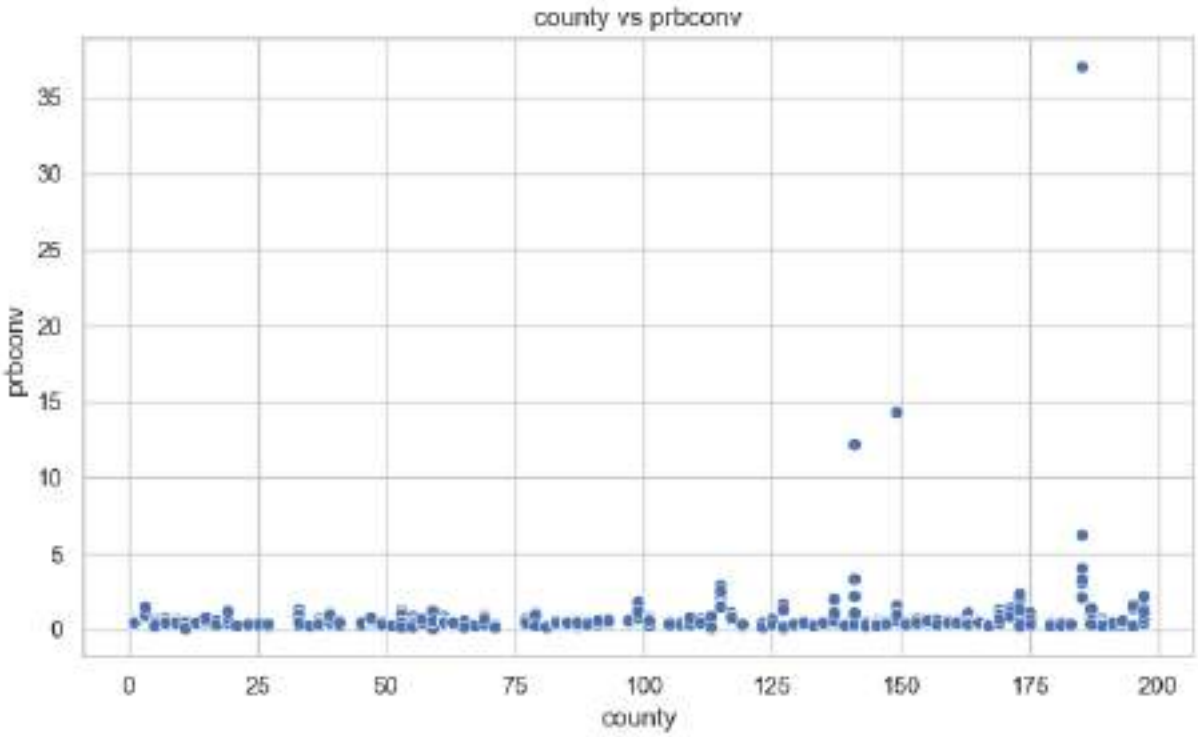

rownames vs prbarr
prbarr
2.5
2.0
1.5
1.0
0.5
0.0
0
100
200
300
400
500
600
rownames

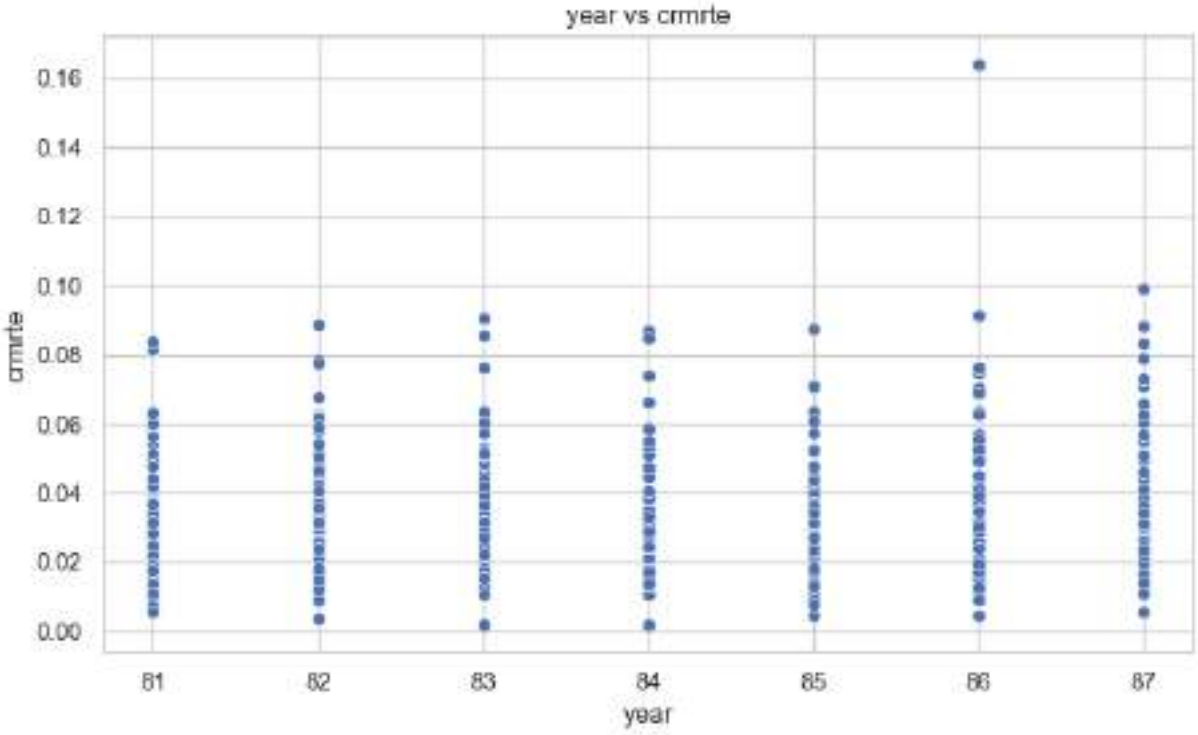

rownames vs prbconv
prbconv
rownames
0
5
10
15
20
25
30
35
0
100
200
300
400
500
600

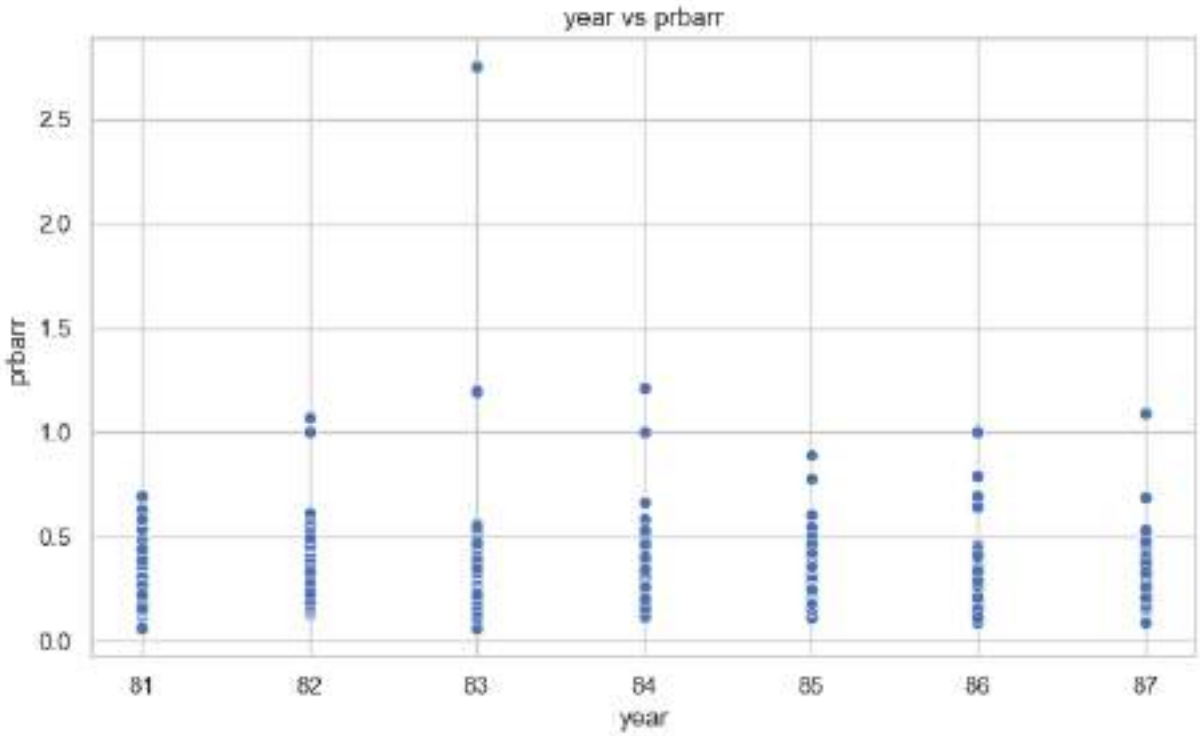

county vs year
year
county
81
82
83
84
85
86
87
0
25
50
75
100
125
150
175
200

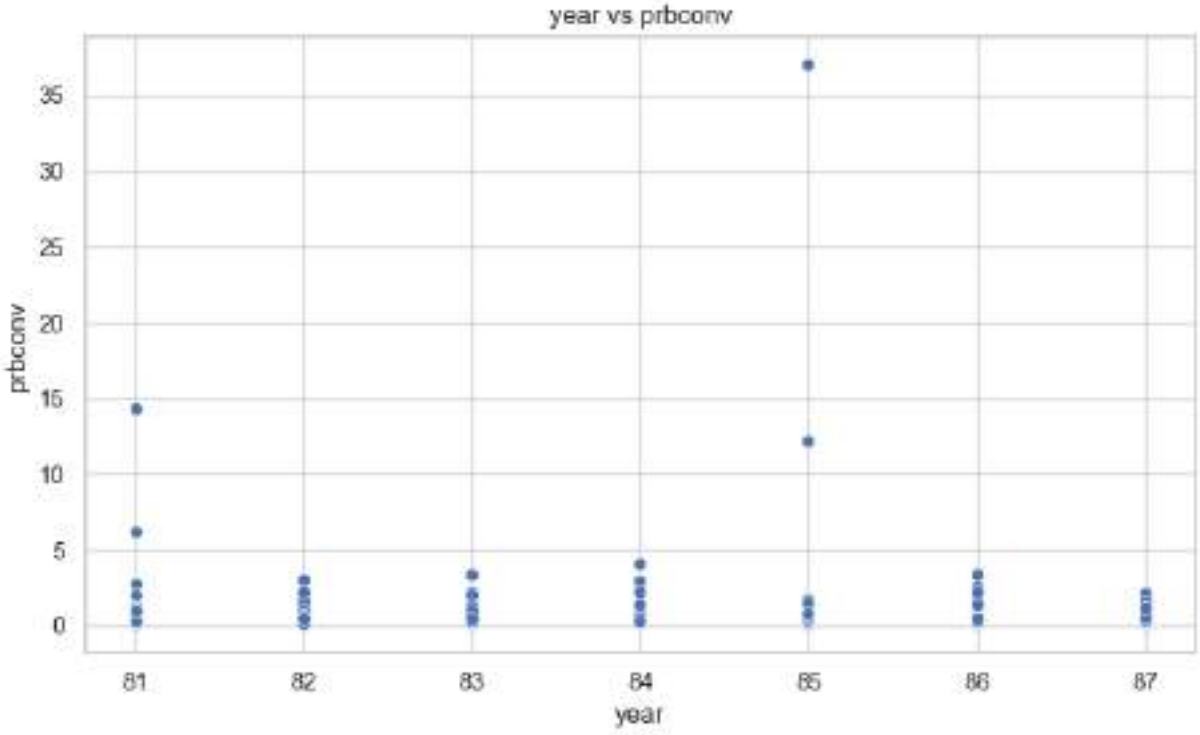

county vs crmrte
crmrte
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
0
25
50
75
100
125
150
175
200
county

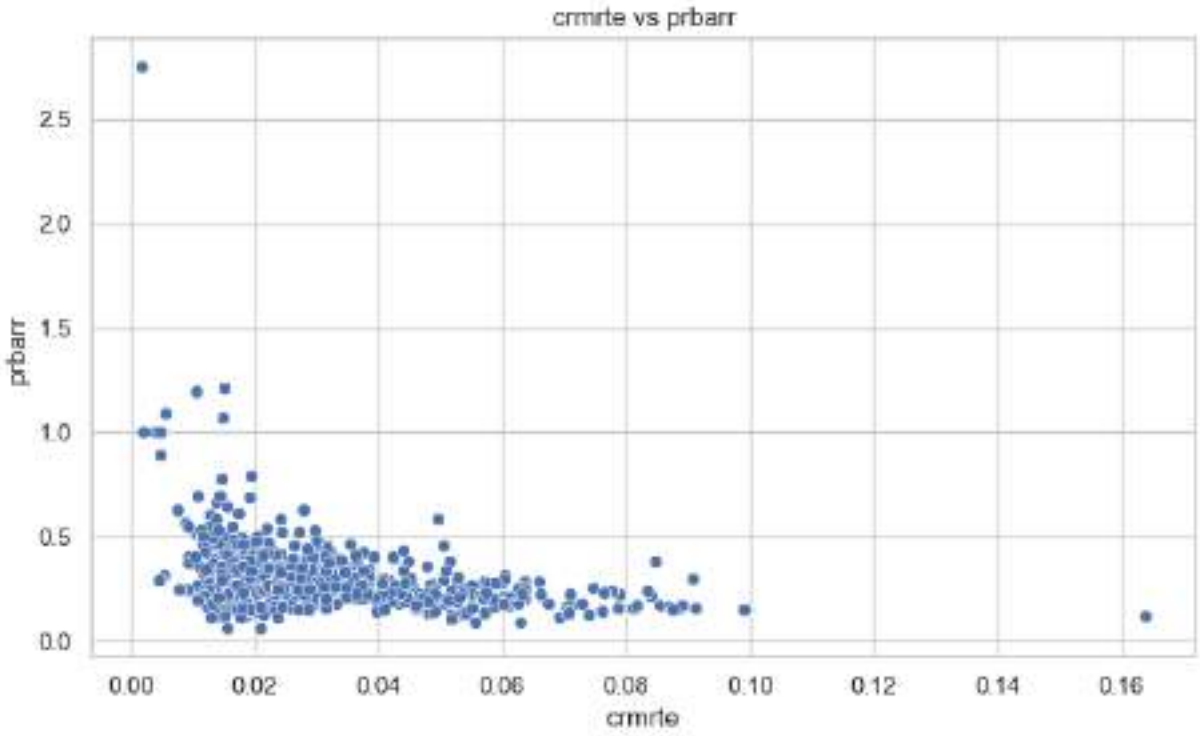

county vs prbarr
prbarr
2.5
2.0
1.5
1.0
0.5
0.0
0
25
50
75
100
125
150
175
200
county

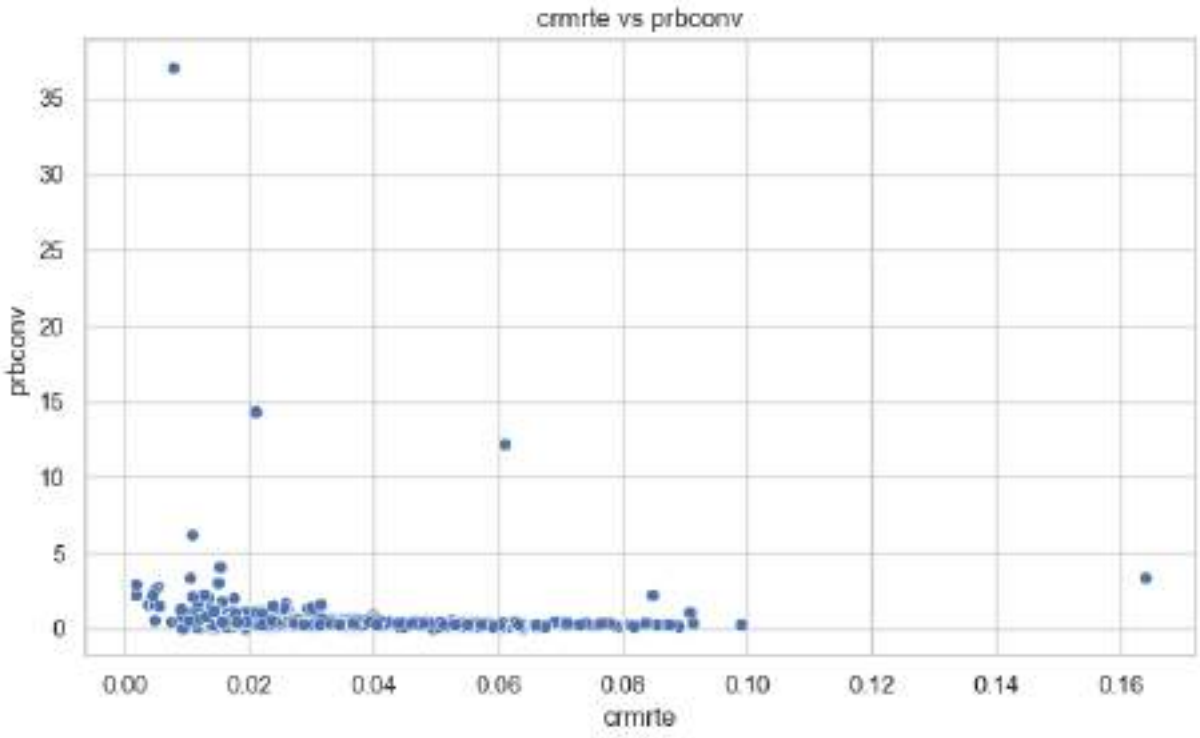

county vs prbconv
prbconv
35
30
25
20
15
10
5
0
0
25
50
75
100
125
150
175
200
county

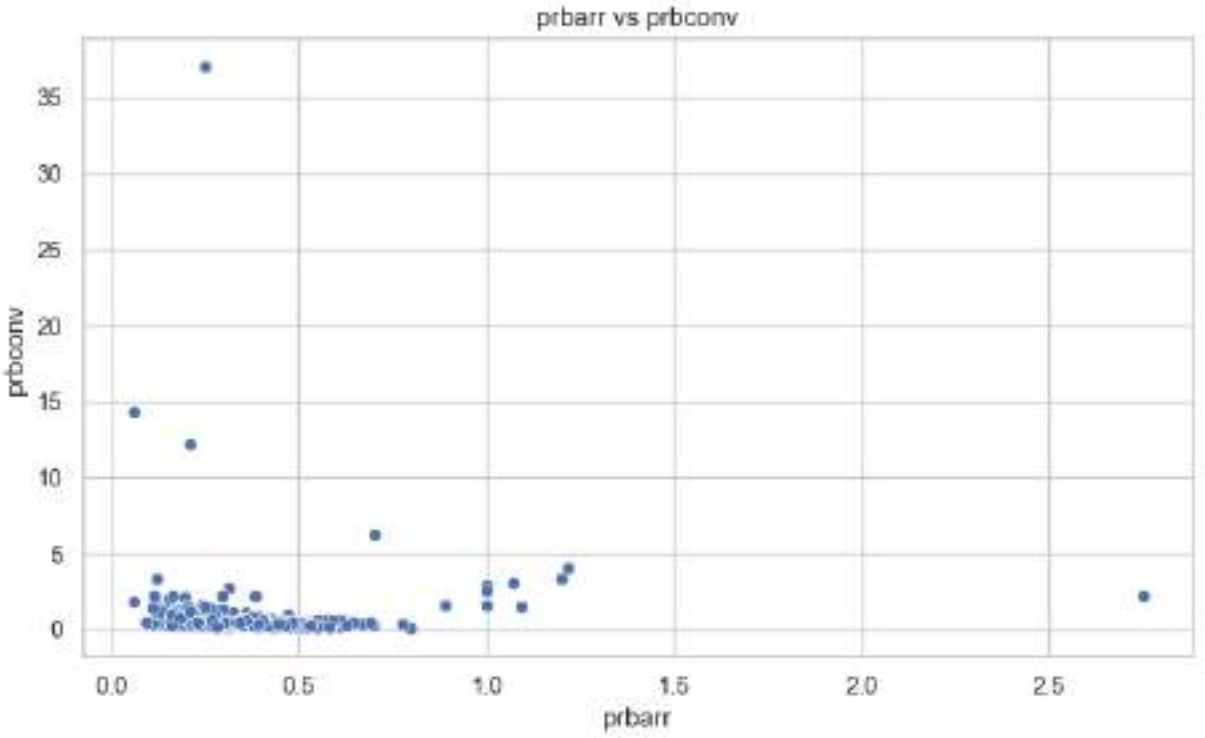

year vs crmrte
crmrte
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
81
82
83
84
85
86
87
year


year vs prbarr
prbarr
2.5
2.0
1.5
1.0
0.5
0.0
81
82
83
84
85
86
87
year


year vs prbconv
prbconv
35
30
25
20
15
10
5
0
81
82
83
84
85
86
87
year


crmrte vs prbarr
prbarr
crmrte
0.0
0.5
1.0
1.5
2.0
2.5
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16


crmrte vs prbconv
prbconv
crmrte
0
5
10
15
20
25
30
35
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16



In [34]:
```
from scipy.stats import pearsonr, spearmanr

numeric = clean.select_dtypes(include=np.number)
results = []

for i in range(len(numeric.columns)):
    for j in range(i + 1, len(numeric.columns)):
        pair = numeric.iloc[:, [i, j]].dropna()

        if len(pair) > 2 and pair.iloc[:, 0].nunique() > 1 and pair.iloc[:, 1].nuni
            pearson = pearsonr(pair.iloc[:, 0], pair.iloc[:, 1])
            spearman = spearmanr(pair.iloc[:, 0], pair.iloc[:, 1])

            results.append({
                "Variable 1": pair.columns[0],
                "Variable 2": pair.columns[1],
                "Pearson Correlation": pearson.statistic,
                "Pearson P-value": pearson.pvalue,
                "Spearman Correlation": spearman.statistic,
                "Spearman P-value": spearman.pvalue
            })

display(pd.DataFrame(results))
```

	Variable 1	Variable 2	Pearson Correlation	Pearson P-value	Spearman Correlation	Spearman P-value
0	rownames	county	0.999480	0.000000	0.999940	0.000000e+00
1	rownames	year	0.010997	0.782942	0.010997	7.829423e-01
2	rownames	crmrte	0.037209	0.351124	0.025264	5.267581e-01
3	rownames	prbarr	-0.020628	0.605301	-0.087346	2.836389e-02
4	rownames	prbconv	0.119920	0.002571	0.101095	1.111942e-02
...	...	...	...	...	...	...
941	lpctmin	lmix	0.181261	0.000005	0.274277	2.462663e-12
942	lpctmin	Scaled rownames	0.079925	0.044927	0.049575	2.140082e-01
943	ltaxpc	lmix	0.001832	0.963388	0.016897	6.720730e-01
944	ltaxpc	Scaled rownames	-0.041196	0.301890	-0.008620	8.290417e-01
945	lmix	Scaled rownames	-0.060461	0.129538	-0.100803	1.135575e-02

946 rows × 6 columns

In [35]:
```
categorical = clean.select_dtypes(include=["object", "category"])
numeric = clean.select_dtypes(include=np.number)

if not categorical.empty and not numeric.empty:
    category = categorical.columns[0]
    value = numeric.columns[0]

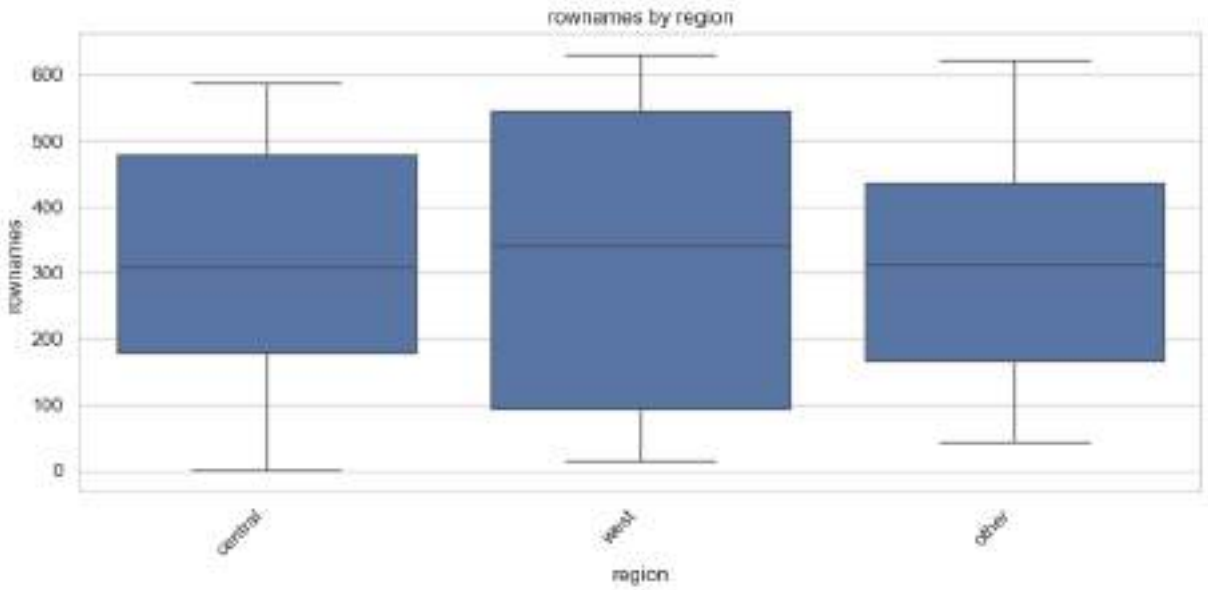
    comparison = clean.groupby(category, dropna=False)[value].agg(
        ["count", "mean", "median", "std"]
    )

    display(comparison)

    plt.figure(figsize=(10, 5))
    sns.boxplot(data=clean, x=category, y=value)
    plt.title(value + " by " + category)
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\1365109543.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

region	count	mean	median	std
central	238	310.558824	308.5	179.658615
other	245	314.800000	312.0	164.326434
west	147	324.666667	340.0	212.300489



In [36]:
```
categorical = clean.select_dtypes(include=["object", "category"])

if len(categorical.columns) >= 2:
    first = categorical.columns[0]
    second = categorical.columns[1]

    table = pd.crosstab(clean[first], clean[second])

    print("Contingency Table")
    display(table)

    print("Row-wise percentages")
    display(pd.crosstab(clean[first], clean[second], normalize="index").round(3) *
```

```
Contingency Table
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\3554291391.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

smsa	no	yes
region		
central	203	35
other	231	14
west	140	7

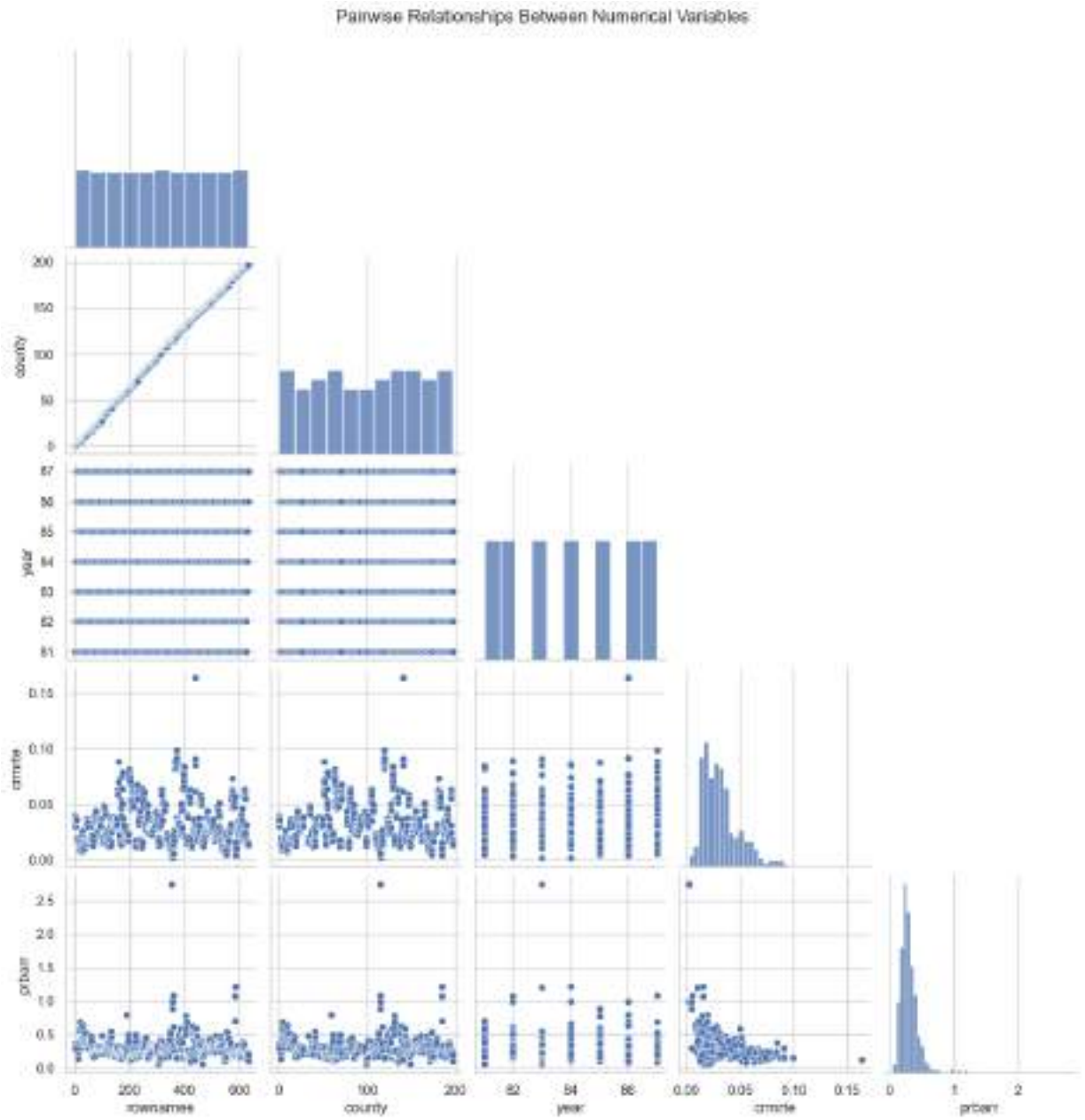
Row-wise percentages

smsa	no	yes
region		
central	85.3	14.7
other	94.3	5.7
west	95.2	4.8

In [37]:
```
numeric = clean.select_dtypes(include=np.number)

if len(numeric.columns) >= 2:
    selected = numeric.iloc[:, :5].copy()

    sns.pairplot(selected, corner=True, diag_kind="hist")
    plt.suptitle("Pairwise Relationships Between Numerical Variables", y=1.02)
    plt.show()
```



In [38]:
```
print("Assessment 5 – Multivariate analysis")
```

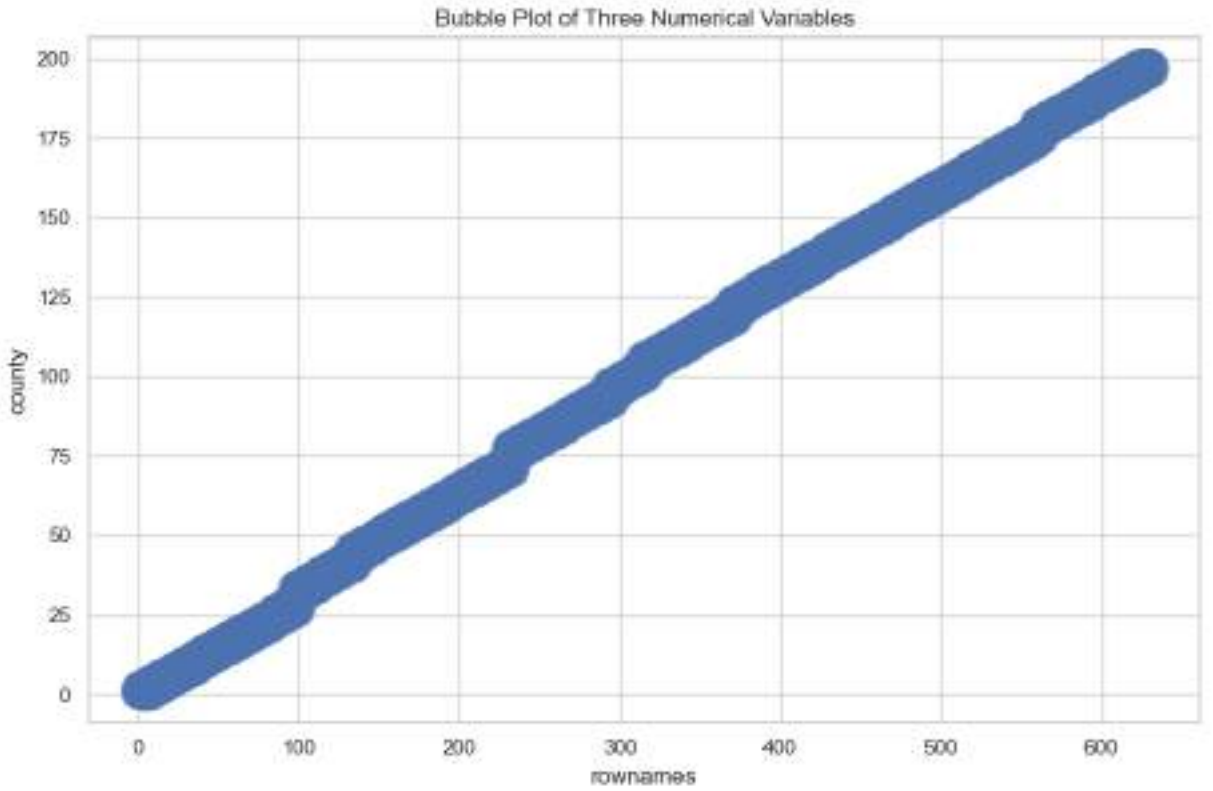
```
Assessment 5 – Multivariate analysis
```

In [39]:
```
numeric = clean.select_dtypes(include=np.number)
columns = numeric.columns[:3]

if len(columns) >= 3:
    plotdata = clean[list(columns)].dropna()

    plt.figure(figsize=(9, 6))
    plt.scatter(
        plotdata[columns[0]],
        plotdata[columns[1]],
        s=plotdata[columns[2]].abs() / (plotdata[columns[2]].abs().max() or 1) * 40
        alpha=0.6
    )
    plt.xlabel(columns[0])
```

```
    plt.ylabel(columns[1])
    plt.title("Bubble Plot of Three Numerical Variables")
    plt.tight_layout()
    plt.show()
```



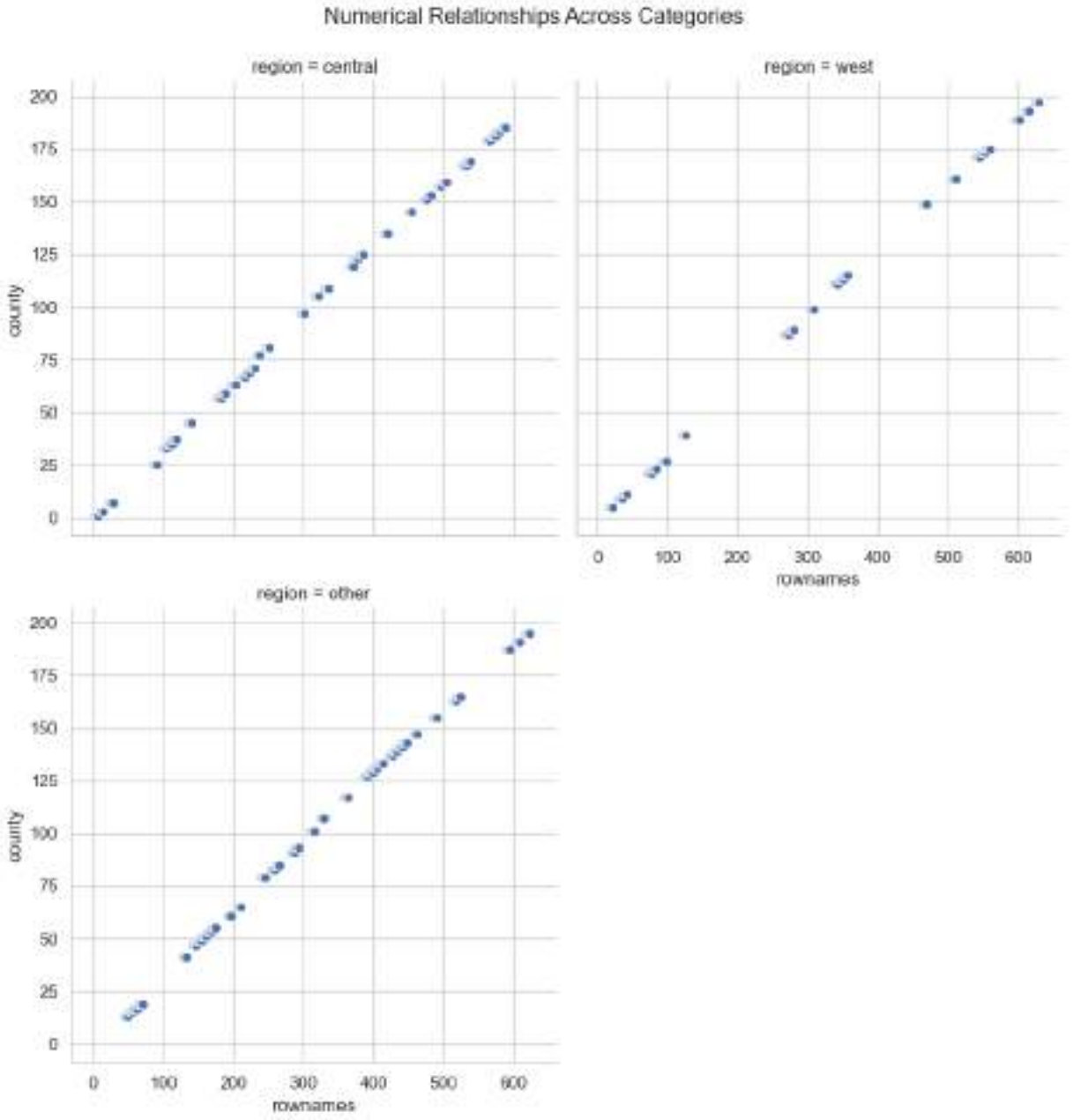
In [40]:
```
numeric = clean.select_dtypes(include=np.number)
categorical = clean.select_dtypes(include=["object", "category"])

if len(numeric.columns) >= 2 and len(categorical.columns) >= 1:
    x = numeric.columns[0]
    y = numeric.columns[1]
    category = categorical.columns[0]

    plotdata = clean[[x, y, category]].dropna()
    plotdata = plotdata[plotdata[category].isin(plotdata[category].value_counts().h

    if not plotdata.empty:
        graph = sns.relplot(
            data=plotdata,
            x=x,
            y=y,
            col=category,
            col_wrap=2,
            kind="scatter"
        )
        graph.fig.suptitle("Numerical Relationships Across Categories", y=1.03)
        plt.show()
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\1586313987.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```



In [41]:
```
from pandas.plotting import parallel_coordinates
from sklearn.preprocessing import StandardScaler

numeric = clean.select_dtypes(include=np.number)
categorical = clean.select_dtypes(include=["object", "category"])

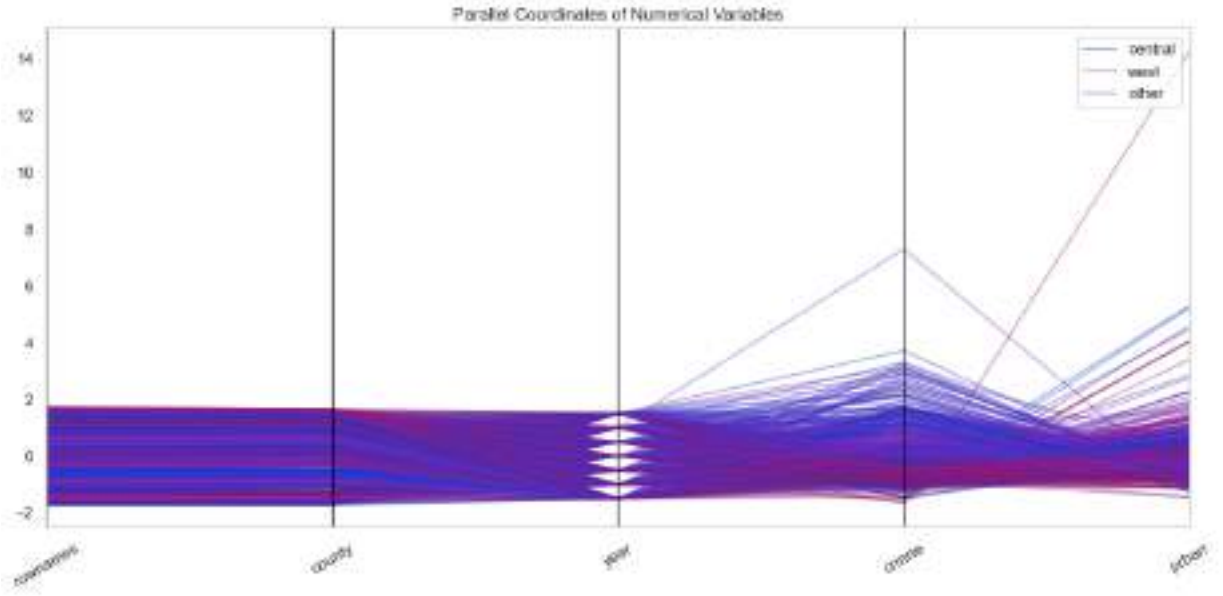
if len(numeric.columns) >= 2 and len(categorical.columns) >= 1:
    category = categorical.columns[0]
    selected = numeric.columns[:5]
```

```
    plotdata = clean[list(selected)].copy()
    plotdata = (plotdata - plotdata.mean()) / plotdata.std().replace(0, 1)
    plotdata[category] = clean[category].astype(str)

    plotdata = plotdata.dropna()
    common = plotdata[category].value_counts().head(5).index
    plotdata = plotdata[plotdata[category].isin(common)]

    if not plotdata.empty:
        plt.figure(figsize=(12, 6))
        parallel_coordinates(plotdata, category, alpha=0.4)
        plt.title("Parallel Coordinates of Numerical Variables")
        plt.xticks(rotation=30)
        plt.tight_layout()
        plt.show()
```

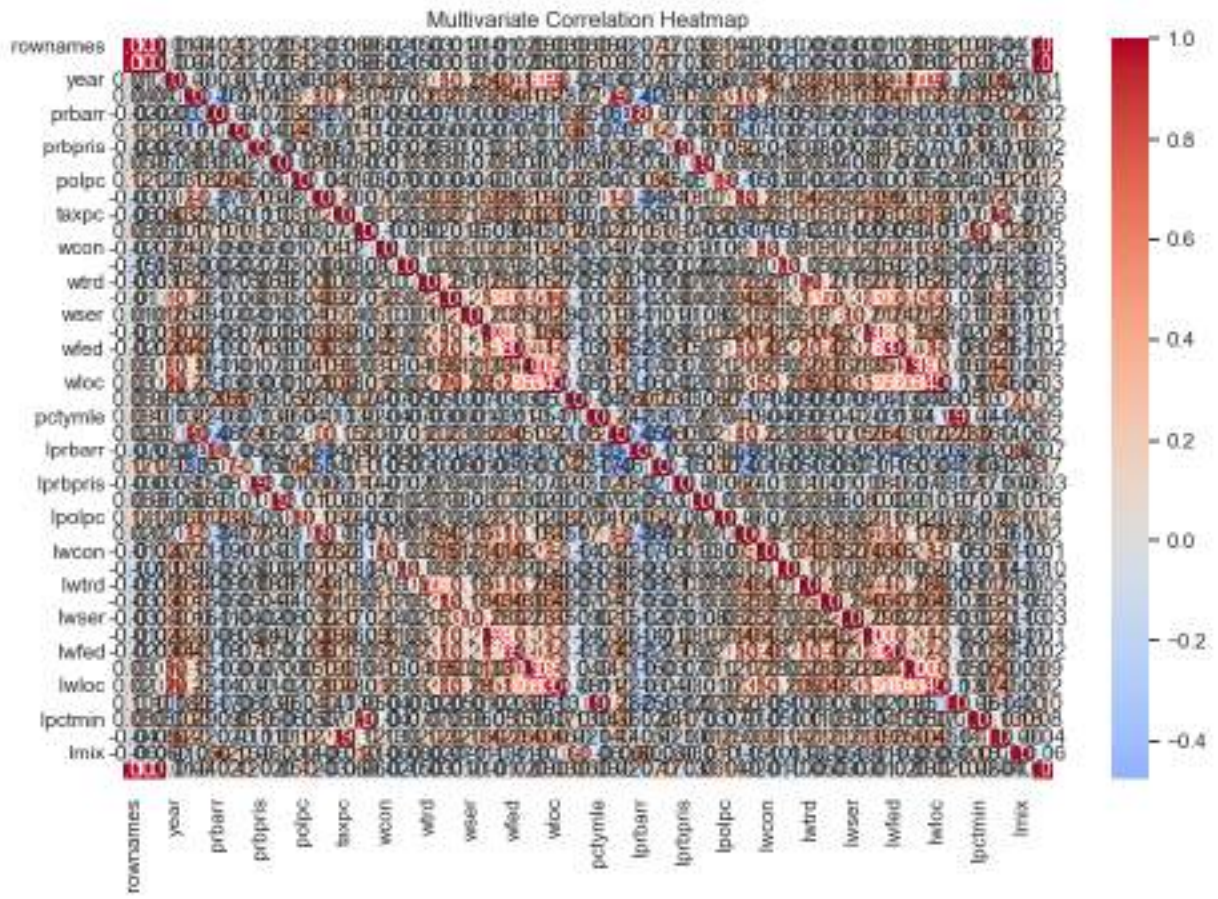
```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\266960030.py:5: Pandas4Warn
ing: For backward compatibility, 'str' dtypes are included by select_dtypes when 'ob
ject' dtype is specified. This behavior is deprecated and will be removed in a futur
e version. Explicitly pass 'str' to `include` to select them, or to `exclude` to rem
ove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migrat
ion-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```



In [42]:
```
numeric = clean.select_dtypes(include=np.number)

if len(numeric.columns) >= 2:
    plt.figure(figsize=(10, 7))
    sns.heatmap(
        numeric.corr(),
        annot=True,
        cmap="coolwarm",
        fmt=".2f",
        center=0
    )
    plt.title("Multivariate Correlation Heatmap")
```

```
    plt.tight_layout()
    plt.show()
```



In [43]:
```
numeric = clean.select_dtypes(include=np.number)
categorical = clean.select_dtypes(include=["object", "category"])

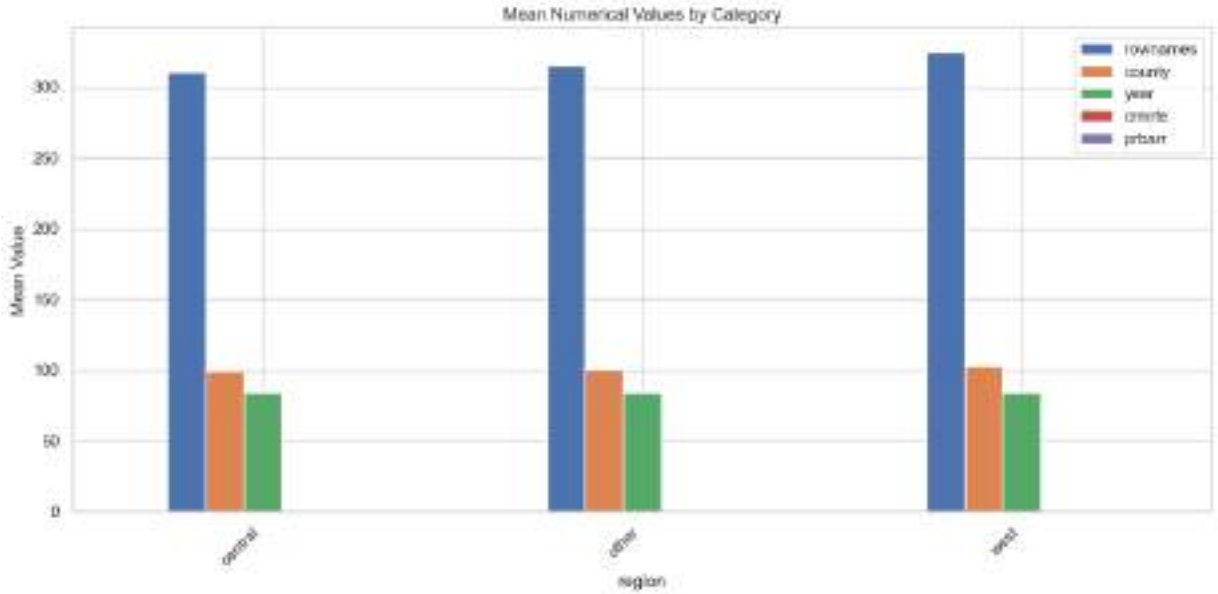
if len(numeric.columns) >= 2 and len(categorical.columns) >= 1:
    category = categorical.columns[0]
    selected = numeric.columns[:5]

    grouped = clean.groupby(category, dropna=False)[list(selected)].mean()
    display(grouped)

    grouped.plot(kind="bar", figsize=(12, 6))
    plt.title("Mean Numerical Values by Category")
    plt.xlabel(category)
    plt.ylabel("Mean Value")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\3871213033.py:2: Pandas4War
ning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'o
bject' dtype is specified. This behavior is deprecated and will be removed in a futu
re version. Explicitly pass 'str' to `include` to select them, or to `exclude` to re
move them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migrat
ion-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

region	rownames	county	year	crmrte	prbarr
central	310.558824	99.176471	84.0	0.035613	0.290156
other	314.800000	100.714286	84.0	0.034776	0.305547
west	324.666667	102.714286	84.0	0.019756	0.338270



In [44]:
```
print("Assessment 6 – Time-series analysis using AirPassengers")
```

```
Assessment 6 – Time-series analysis using AirPassengers
```

In [45]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passenge
air = pd.read_csv(url, parse_dates=["Month"])

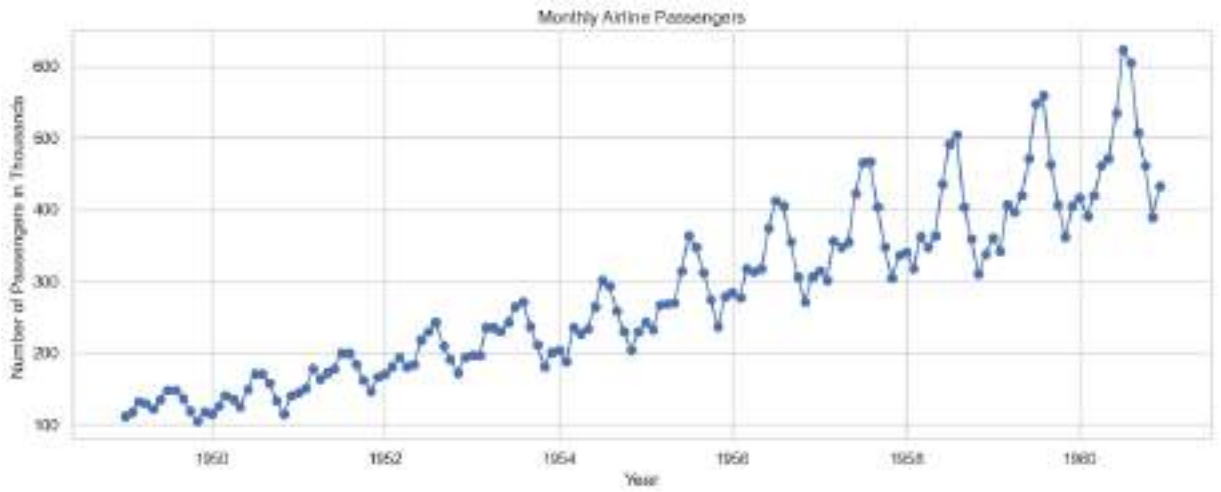
air.columns = ["Month", "Passengers"]
air = air.set_index("Month")

display(air.head())
display(air.tail())
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

In [46]:
```
plt.figure(figsize=(12, 5))
plt.plot(air.index, air["Passengers"], marker="o")
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers in Thousands")
plt.tight_layout()
plt.show()
```



In [47]:
```
display(air["Passengers"].describe().to_frame("Passenger Statistics"))

print("Mean:", air["Passengers"].mean())
print("Median:", air["Passengers"].median())
print("Standard deviation:", air["Passengers"].std())
```

```
print("Minimum:", air["Passengers"].min())
print("Maximum:", air["Passengers"].max())
```

	Passenger Statistics
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

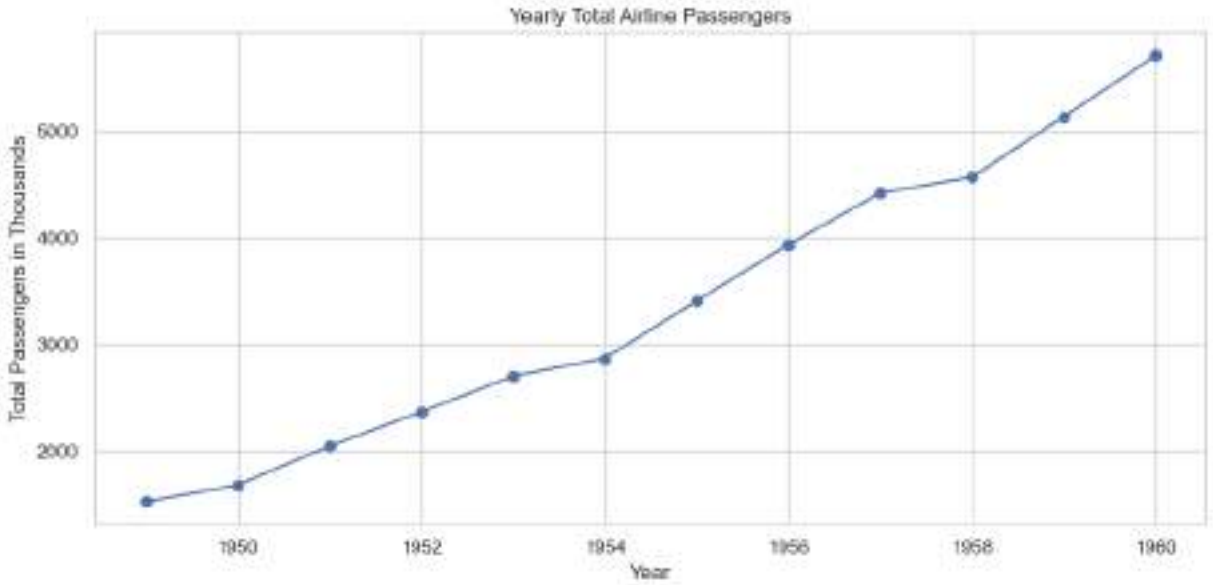
```
Mean: 280.2986111111111
Median: 265.5
Standard deviation: 119.9663169429432
Minimum: 104
Maximum: 622
```

In [48]:
```
yearly = air["Passengers"].resample("YE").sum()
yearly.index = yearly.index.year

display(yearly.to_frame("Total Passengers"))

plt.figure(figsize=(10, 5))
plt.plot(yearly.index, yearly.values, marker="o")
plt.title("Yearly Total Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Total Passengers in Thousands")
plt.tight_layout()
plt.show()
```

Month	Total Passengers
1949	1520
1950	1676
1951	2042
1952	2364
1953	2700
1954	2867
1955	3408
1956	3939
1957	4421
1958	4572
1959	5140
1960	5714



In [49]:
```
monthly = air.groupby(air.index.month)["Passengers"].mean()

month_names = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
]

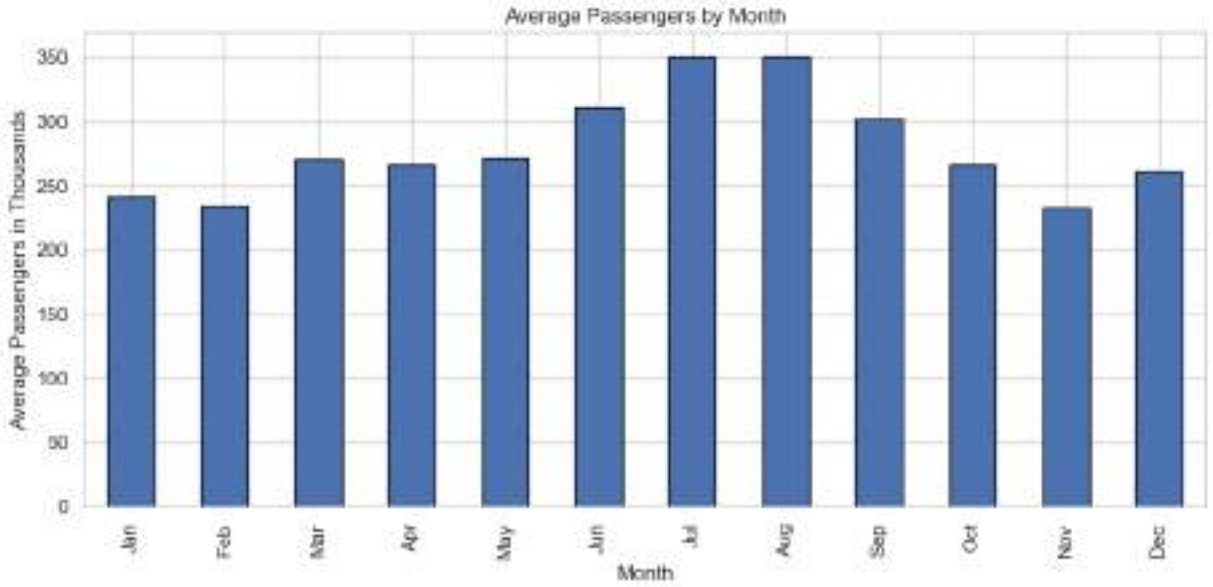
monthly.index = month_names

display(monthly.to_frame("Average Passengers"))

plt.figure(figsize=(10, 5))
```

```
monthly.plot(kind="bar", edgecolor="black")
plt.title("Average Passengers by Month")
plt.xlabel("Month")
plt.ylabel("Average Passengers in Thousands")
plt.tight_layout()
plt.show()
```

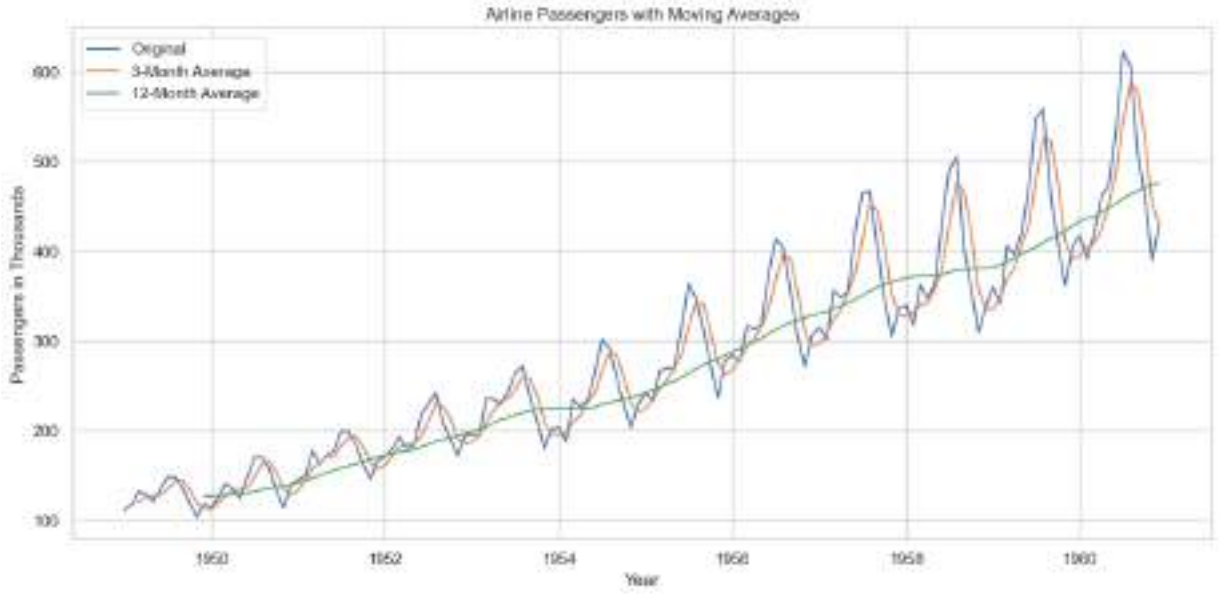
	Average Passengers
Jan	241.750000
Feb	235.000000
Mar	270.166667
Apr	267.083333
May	271.833333
Jun	311.666667
Jul	351.333333
Aug	351.083333
Sep	302.416667
Oct	266.583333
Nov	232.833333
Dec	261.833333



In [50]:
```
air["3 Month Moving Average"] = air["Passengers"].rolling(3).mean()
air["12 Month Moving Average"] = air["Passengers"].rolling(12).mean()

plt.figure(figsize=(12, 6))
plt.plot(air.index, air["Passengers"], label="Original")
plt.plot(air.index, air["3 Month Moving Average"], label="3-Month Average")
plt.plot(air.index, air["12 Month Moving Average"], label="12-Month Average")
```

```
plt.title("Airline Passengers with Moving Averages")
plt.xlabel("Year")
plt.ylabel("Passengers in Thousands")
plt.legend()
plt.tight_layout()
plt.show()
```



In [53]:
```
%pip install statsmodels
```

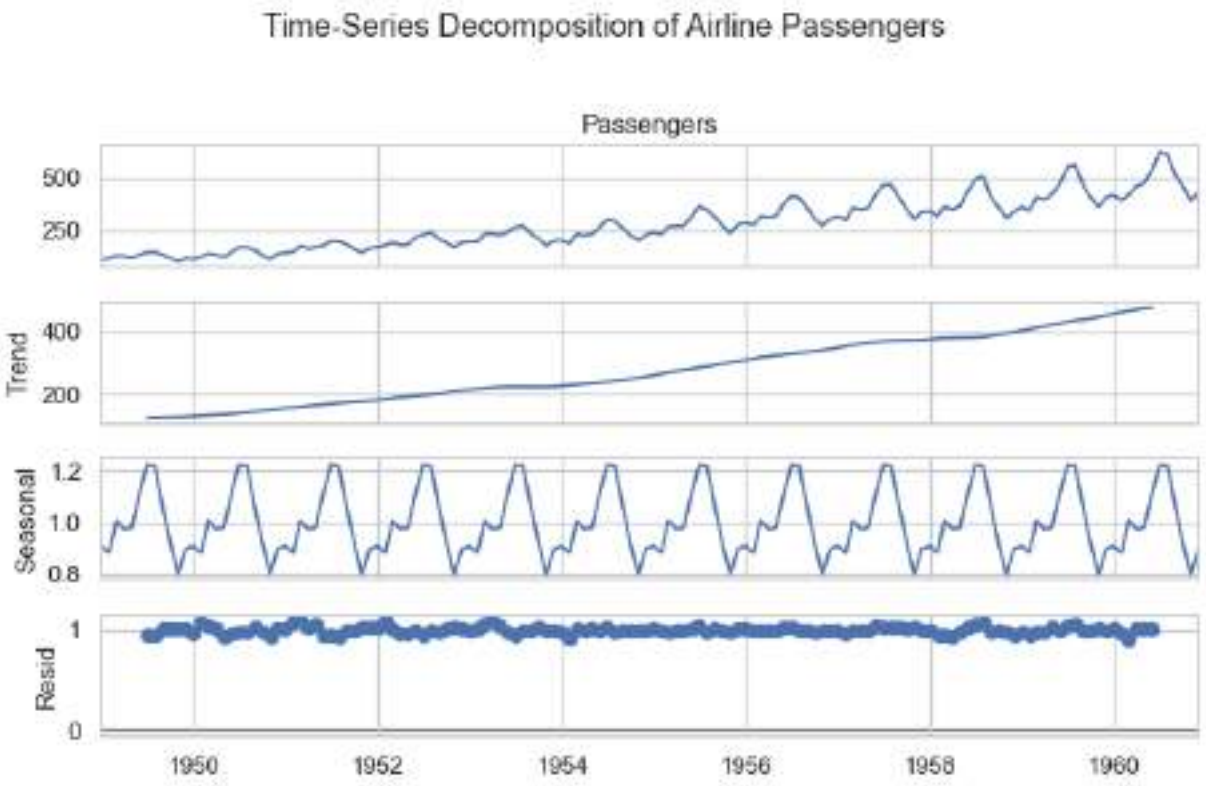
```
Requirement already satisfied: statsmodels in c:\users\abdrutgers234\appdata\local\p
rograms\python\python314\lib\site-packages (0.15.0)
Requirement already satisfied: numpy<3,>=1.23.5 in c:\users\abdrutgers234\appdata\lo
cal\programs\python\python314\lib\site-packages (from statsmodels) (2.4.2)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in c:\users\abdrutgers234\appdata
\local\programs\python\python314\lib\site-packages (from statsmodels) (1.17.0)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in c:\users\abdrutgers234\appdata
\local\programs\python\python314\lib\site-packages (from statsmodels) (3.0.1)
Requirement already satisfied: patsy>=0.5.6 in c:\users\abdrutgers234\appdata\local
\programs\python\python314\lib\site-packages (from statsmodels) (1.0.3)
Requirement already satisfied: packaging>=21.3 in c:\users\abdrutgers234\appdata\loc
al\programs\python\python314\lib\site-packages (from statsmodels) (26.0)
Requirement already satisfied: formulaic>=1.1.0 in c:\users\abdrutgers234\appdata\lo
cal\programs\python\python314\lib\site-packages (from statsmodels) (1.2.2)
Requirement already satisfied: interface-meta>=1.2.0 in c:\users\abdrutgers234\appda
ta\local\programs\python\python314\lib\site-packages (from formulaic>=1.1.0->statsmo
dels) (2.0.1)
Requirement already satisfied: narwhals>=1.17 in c:\users\abdrutgers234\appdata\loca
l\programs\python\python314\lib\site-packages (from formulaic>=1.1.0->statsmodels)
(2.27.1)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\abdrutgers234\ap
pdata\local\programs\python\python314\lib\site-packages (from formulaic>=1.1.0->stat
smodels) (4.15.0)
Requirement already satisfied: wrapt>=1.17.0rc1 in c:\users\abdrutgers234\appdata\lo
cal\programs\python\python314\lib\site-packages (from formulaic>=1.1.0->statsmodels)
(2.5.2rc1)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\abdrutgers234\appd
ata\local\programs\python\python314\lib\site-packages (from pandas!=2.1.0,>=1.4->sta
tsmodels) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\abdrutgers234\appdata\local\progra
ms\python\python314\lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.
3)
Requirement already satisfied: six>=1.5 in c:\users\abdrutgers234\appdata\local\prog
rams\python\python314\lib\site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,
>=1.4->statsmodels) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [54]:

```python
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    air["Passengers"],
    model="multiplicative",
    period=12
)

decomposition.plot()
plt.suptitle("Time-Series Decomposition of Airline Passengers", y=1.02)
plt.tight_layout()
plt.show()
```



In [55]:
```
air["Monthly Growth"] = air["Passengers"].pct_change() * 100

display(air[["Passengers", "Monthly Growth"]].head(15))

plt.figure(figsize=(12, 5))
plt.plot(air.index, air["Monthly Growth"])
plt.axhline(0, linestyle="--")
plt.title("Monthly Percentage Growth in Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Growth (%)")
plt.tight_layout()
plt.show()
```

Month	Passengers	Monthly Growth
1949-01-01	112	NaN
1949-02-01	118	5.357143
1949-03-01	132	11.864407
1949-04-01	129	-2.272727
1949-05-01	121	-6.201550
1949-06-01	135	11.570248
1949-07-01	148	9.629630
1949-08-01	148	0.000000
1949-09-01	136	-8.108108
1949-10-01	119	-12.500000
1949-11-01	104	-12.605042
1949-12-01	118	13.461538
1950-01-01	115	-2.542373
1950-02-01	126	9.565217
1950-03-01	141	11.904762



In [56]:
```
yearly_average = air["Passengers"].groupby(air.index.year).mean()

comparison = pd.DataFrame({
    "Yearly Average": [
        yearly_average.iloc[0],
        yearly_average.iloc[-1]
    ]
}, index=[yearly_average.index[0], yearly_average.index[-1]])
```

```
display(comparison)

growth = (
    (yearly_average.iloc[-1] - yearly_average.iloc[0])
    / yearly_average.iloc[0]
) * 100

print(f"Increase in yearly average from {yearly_average.index[0]} to {yearly_averag
```

	Yearly Average
1949	126.666667
1960	476.166667

```
Increase in yearly average from 1949 to 1960: 275.92%
```

In [57]:
```
print("Assessment 7 – Summary statistics and data comparison")
```

```
Assessment 7 – Summary statistics and data comparison
```

In [58]:
```
numeric = clean.select_dtypes(include=np.number)

summary = numeric.agg([
    "count", "mean", "median", "std", "min", "max", "var", "skew"
])

display(summary.T)
```

	count	mean	median	std	min	max	var
rownames	630.0	315.500000	315.500000	182.009615	1.000000	630.000000	33127.500000
county	630.0	100.600000	103.000000	58.036270	1.000000	197.000000	3368.208585
year	630.0	84.000000	84.000000	2.001589	81.000000	87.000000	4.006359
crmrte	630.0	0.031588	0.028441	0.018121	0.001812	0.163835	0.000328
prbarr	630.0	0.307368	0.278240	0.171205	0.058824	2.750000	0.029311
prbconv	630.0	0.688618	0.474375	1.690345	0.068376	37.000000	2.857265
prbpris	630.0	0.425518	0.428571	0.087245	0.148936	0.678571	0.007612
avgsen	630.0	8.954540	8.495000	2.658082	4.220000	25.830000	7.065402
polpc	630.0	0.001917	0.001451	0.002735	0.000458	0.035578	0.000007
density	630.0	1.386062	0.952595	1.439703	0.197719	8.827652	2.072746
taxpc	630.0	30.239194	27.792330	11.454695	14.302560	119.761500	131.210043
pctmin	630.0	25.712852	24.851550	16.903536	1.283650	64.348200	285.729534
wcon	630.0	245.666079	236.462200	121.983720	65.621580	2324.598000	14880.028000
wtuc	630.0	406.102800	358.202400	266.513781	28.857700	3041.958000	71029.595323
wtrd	630.0	192.823065	185.477700	88.407259	16.873760	2242.747000	7815.843364
wfir	630.0	272.059255	264.423100	55.768084	3.515680	509.465500	3110.079158
wser	630.0	224.670455	216.475300	104.866700	1.843794	2177.068000	10997.024707
wmfg	630.0	285.170095	271.595000	82.368065	101.830000	646.850000	6784.498180
wfed	630.0	403.895889	403.980000	63.066692	255.400000	597.950000	3977.407619
wsta	630.0	296.907460	289.370000	53.431614	173.020000	548.000000	2854.937328
wloc	630.0	257.976206	253.135000	41.358021	163.590000	388.090000	1710.485928
mix	630.0	0.139396	0.102089	0.219755	0.002457	4.000000	0.048292
pctymle	630.0	0.088974	0.083165	0.024349	0.062158	0.274358	0.000593
lcrmrte	630.0	-3.609225	-3.559938	0.572808	-6.313550	-1.808895	0.328109
lprbarr	630.0	-1.274264	-1.279271	0.415897	-2.833214	1.011601	0.172970
lprbconv	630.0	-0.692919	-0.745758	0.609595	-2.682732	3.610918	0.371606
lprbpris	630.0	-0.878632	-0.847299	0.230514	-1.904239	-0.387766	0.053137
lavgsen	630.0	2.153344	2.139477	0.273729	1.439835	3.251537	0.074928
lpolpc	630.0	-6.490637	-6.535785	0.526654	-7.687507	-3.336024	0.277364
ldensity	630.0	-0.015927	-0.048567	0.774735	-1.620910	2.177889	0.600215

	count	mean	median	std	min	max	var
lwcon	630.0	5.462869	5.465789	0.248178	4.183905	7.751303	0.061592
lwtuc	630.0	5.915883	5.881098	0.370219	3.362377	8.020257	0.137062
lwtrd	630.0	5.232423	5.222934	0.214391	2.825760	7.715457	0.045964
lwfir	630.0	5.579433	5.577550	0.277204	1.257233	6.233362	0.076842
lwser	630.0	5.364625	5.377477	0.360098	0.611825	7.685734	0.129671
lwmfg	630.0	5.615181	5.604311	0.272747	4.623305	6.472115	0.074391
lwfed	630.0	5.988757	6.001365	0.158761	5.542831	6.393507	0.025205
lwsta	630.0	5.677787	5.667706	0.176131	5.153407	6.306275	0.031022
lwloc	630.0	5.540139	5.533923	0.159691	5.097363	5.961237	0.025501
lpctymle	630.0	-2.443015	-2.486930	0.196784	-2.778080	-1.293320	0.038724
lpctmin	630.0	2.913361	3.212685	0.954615	0.249708	4.164309	0.911289
ltaxpc	630.0	3.356385	3.324759	0.308224	2.660439	4.785502	0.095002
lmix	630.0	-2.234233	-2.281916	0.649565	-6.008814	1.386294	0.421935
Scaled rownames	630.0	0.500000	0.500000	0.289363	0.000000	1.000000	0.083731

In [59]:
```
categorical = clean.select_dtypes(include=["object", "category"])

for column in categorical.columns:
    print("Frequency Distribution of", column)
    display(clean[column].value_counts().to_frame("Frequency"))
```

```
Frequency Distribution of region
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\3487119211.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

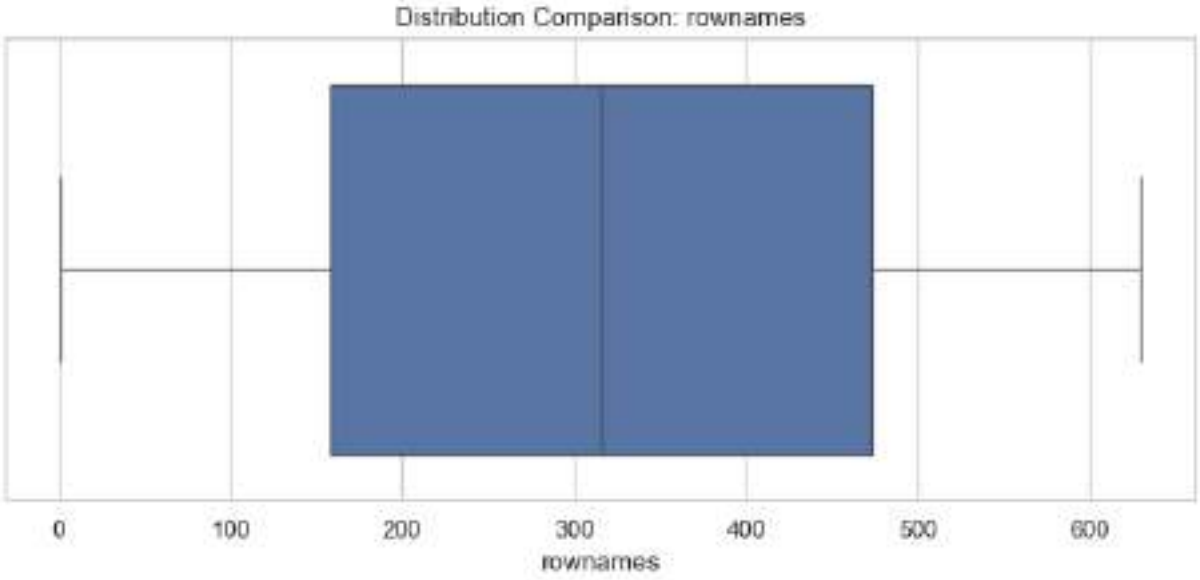
region	Frequency
other	245
central	238
west	147

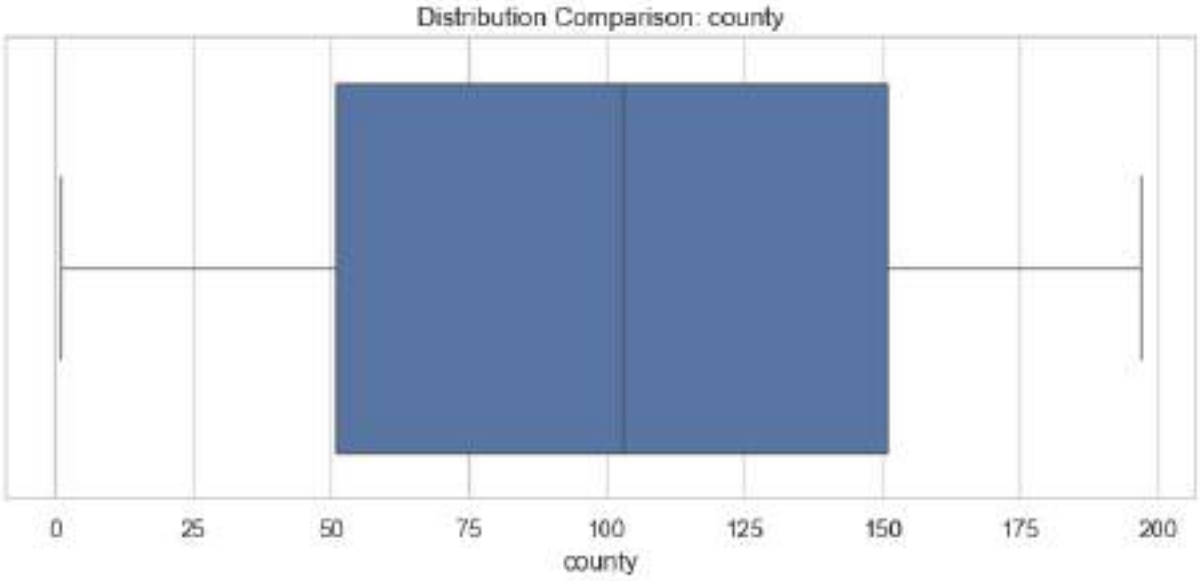
```
Frequency Distribution of smsa
```

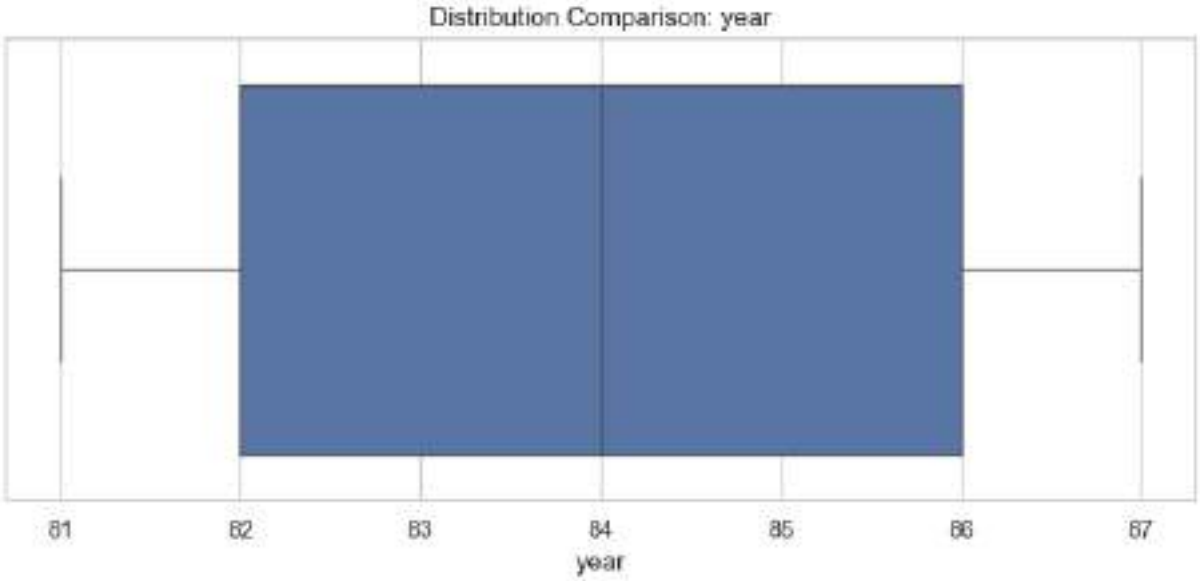
smsa	Frequency
no	574
yes	56

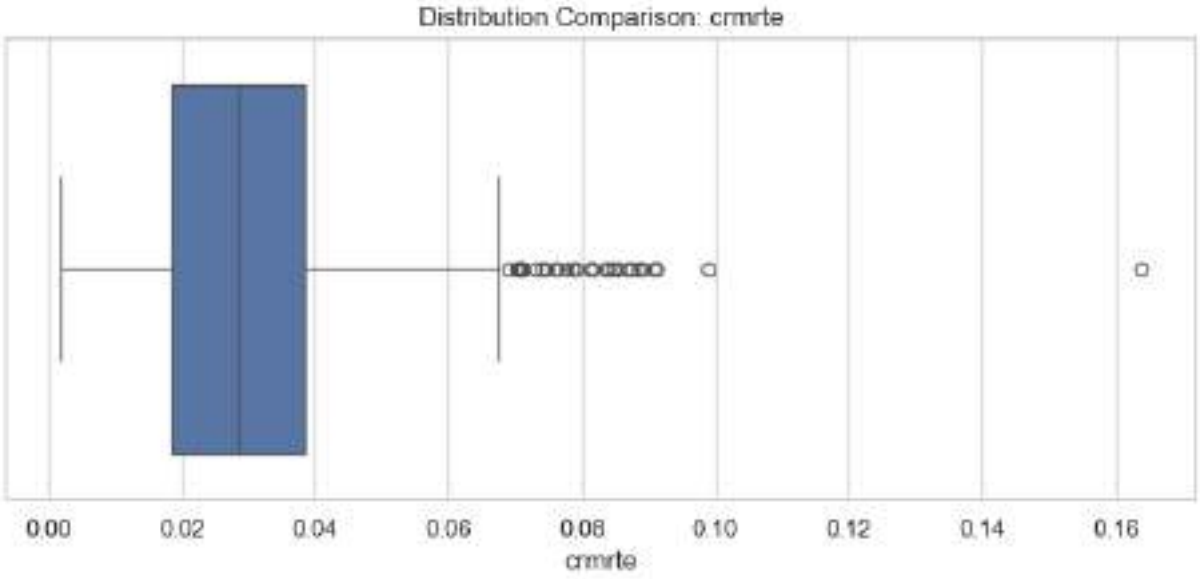
In [60]:
```
numeric = clean.select_dtypes(include=np.number)

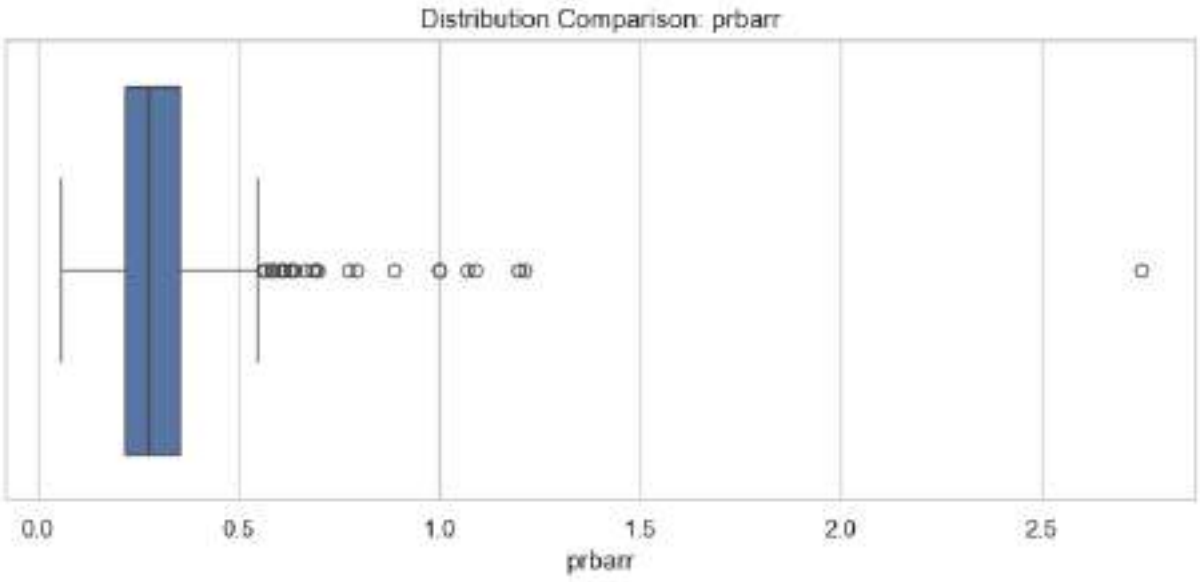
for column in numeric.columns[:5]:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x=clean[column])
    plt.title("Distribution Comparison: " + column)
    plt.xlabel(column)
    plt.tight_layout()
    plt.show()
```











In [61]:
```
numeric = clean.select_dtypes(include=np.number)
categorical = clean.select_dtypes(include=["object", "category"])
```

```
if not numeric.empty and not categorical.empty:
    category = categorical.columns[0]
    value = numeric.columns[0]

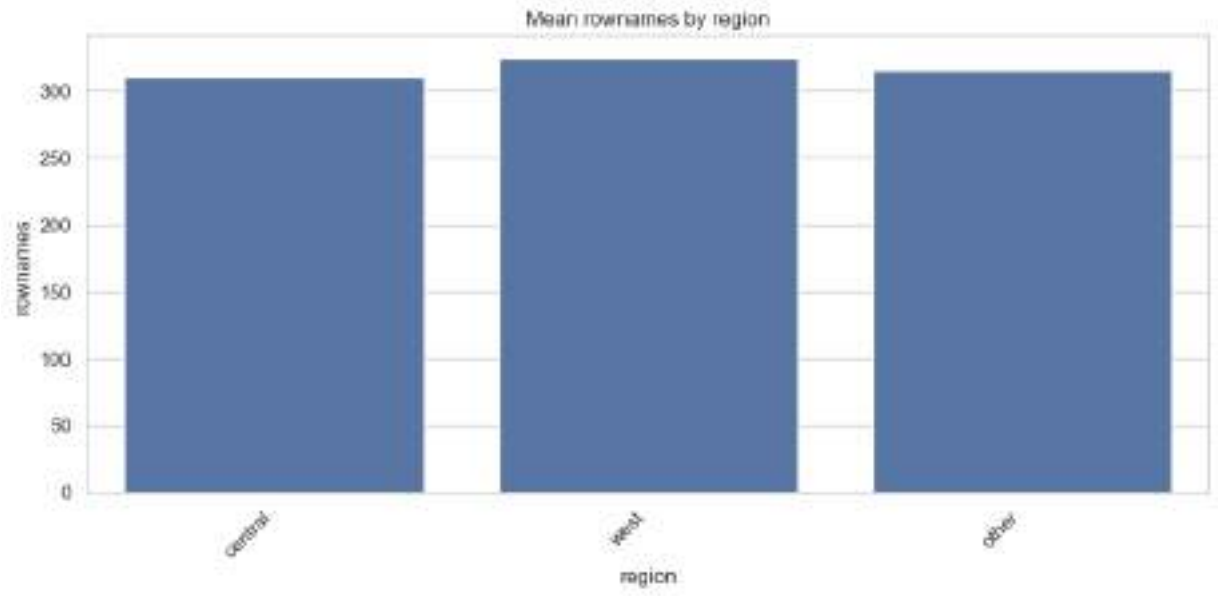
    comparison = clean.groupby(category, dropna=False)[value].agg(
        ["count", "mean", "median", "std", "min", "max"]
    )

    display(comparison)

    plt.figure(figsize=(10, 5))
    sns.barplot(data=clean, x=category, y=value, errorbar=None)
    plt.title("Mean " + value + " by " + category)
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```

```
C:\Users\abdrutgers234\AppData\Local\Temp\ipykernel_9300\719176114.py:2: Pandas4Warn
ing: For backward compatibility, 'str' dtypes are included by select_dtypes when 'ob
ject' dtype is specified. This behavior is deprecated and will be removed in a futur
e version. Explicitly pass 'str' to `include` to select them, or to `exclude` to rem
ove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migrat
ion-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical = clean.select_dtypes(include=["object", "category"])
```

region	count	mean	median	std	min	max
central	238	310.558824	308.5	179.658615	1	588
other	245	314.800000	312.0	164.326434	43	623
west	147	324.666667	340.0	212.300489	15	630



In [62]:
```
numeric = clean.select_dtypes(include=np.number)
```

```
display(numeric.corr())

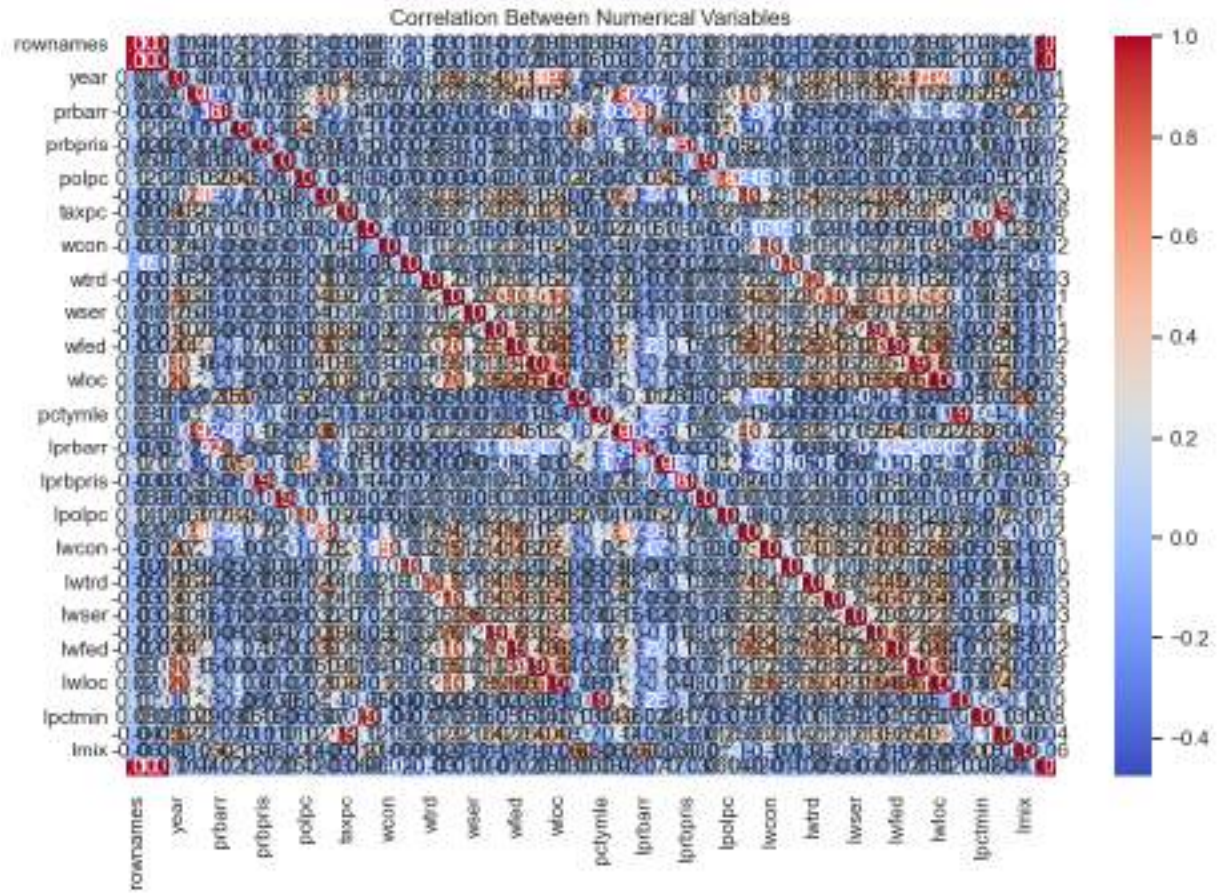
plt.figure(figsize=(10, 7))
sns.heatmap(numeric.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Between Numerical Variables")
plt.tight_layout()
plt.show()
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpri
rownames	1.000000	9.994803e-01	1.099716e-02	0.037209	-0.020628	0.119920	-0.02418
county	0.999480	1.000000e+00	2.479813e-14	0.041679	-0.018569	0.118875	-0.02486
year	0.010997	2.479813e-14	1.000000e+00	0.002392	-0.028067	0.006527	-0.09608
crmrte	0.037209	4.167863e-02	2.392457e-03	1.000000	-0.358553	-0.113033	0.13553
prbarr	-0.020628	-1.856917e-02	-2.806708e-02	-0.358553	1.000000	0.035569	-0.07489
prbconv	0.119920	1.188753e-01	6.527245e-03	-0.113033	0.035569	1.000000	-0.03734
prbpris	-0.024187	-2.486140e-02	-9.608535e-02	0.135537	-0.074893	-0.037340	1.00000
avgsen	0.049026	5.053367e-02	-7.525718e-02	0.032416	0.034031	0.015305	-0.00429
polpc	0.122540	1.237755e-01	2.536452e-02	0.184826	0.290581	0.449635	-0.05745
density	-0.026558	-2.580562e-02	2.347055e-02	0.694072	-0.271228	-0.115555	0.16466
taxpc	-0.058097	-6.176276e-02	4.263984e-01	0.230685	-0.041217	0.006733	-0.11206
pctmin	0.058527	6.231841e-02	3.330427e-15	0.169021	0.100050	0.105077	0.13072
wcon	-0.015448	-1.897445e-02	2.371506e-01	0.070500	-0.088110	-0.045082	-0.02921
wtuc	-0.147590	-1.501372e-01	3.164835e-02	-0.002451	-0.022224	-0.023258	-0.01930
wtrd	-0.025822	-2.838055e-02	1.640346e-01	0.232684	-0.074539	-0.049324	0.05663
wfir	-0.005749	-1.292808e-02	5.627879e-01	0.256157	-0.100232	-0.064391	0.00783
wser	0.012649	8.726151e-03	2.486292e-01	0.092340	-0.098007	-0.017110	-0.00504
wmfg	-0.007828	-1.415905e-02	3.994417e-01	0.279718	-0.077343	-0.070292	0.00151
wfed	-0.018385	-2.203766e-02	4.435469e-01	0.437157	-0.185477	-0.072332	0.13457
wsta	0.090430	8.282573e-02	6.887883e-01	0.163967	-0.113617	-0.006010	-0.07158
wloc	0.028408	2.020241e-02	7.938777e-01	0.234601	-0.134148	-0.030400	-0.00217
mix	0.057149	5.688059e-02	-1.884485e-02	-0.119574	0.347530	0.506171	0.02936

	rownames	county	year	crmrte	prbarr	prbconv	prbpri
pctymle	0.092493	9.827258e-02	-1.270499e-01	0.224795	-0.156273	-0.069298	-0.13324
lcrmrte	0.023098	2.656217e-02	-1.618459e-02	0.918616	-0.477639	-0.189442	0.15908
lprbarr	-0.068415	-6.754402e-02	-2.412379e-02	-0.417453	0.873081	-0.048559	-0.02406
lprbconv	0.168604	1.668116e-01	2.886936e-02	-0.351079	0.067288	0.600954	-0.15009
lprbpris	-0.025198	-2.594194e-02	-8.263118e-02	0.154085	-0.081841	-0.041737	0.98156
lavgsen	0.055181	5.601315e-02	-6.493201e-02	0.063388	0.005150	-0.010197	0.00630
lpolpc	0.138604	1.378293e-01	5.865608e-02	0.309255	0.232341	0.353395	-0.05098
ldensity	0.021652	2.155694e-02	2.862764e-02	0.656662	-0.338033	-0.168948	0.22253
lwcon	-0.012341	-1.819130e-02	4.668645e-01	0.211537	-0.185545	-0.103118	-0.03734
lwtuc	-0.097348	-1.011973e-01	1.801850e-01	0.082956	-0.049988	-0.045347	0.00482
lwtrd	-0.047426	-5.202811e-02	3.631093e-01	0.338487	-0.086186	-0.097893	0.08268
lwfir	-0.026191	-3.077255e-02	4.004590e-01	0.175438	-0.045783	-0.051362	-0.03874
lwser	-0.031544	-3.511456e-02	2.980535e-01	0.156418	-0.108968	-0.041474	-0.02474
lwmfg	-0.009238	-1.571425e-02	4.213892e-01	0.298015	-0.082256	-0.083755	0.03918
lwfed	-0.021410	-2.541192e-02	4.426130e-01	0.414733	-0.176566	-0.066186	0.14596
lwsta	0.087190	7.933925e-02	7.003245e-01	0.153715	-0.103448	-0.001660	-0.07228
lwloc	0.023774	1.573701e-02	7.933633e-01	0.234105	-0.138880	-0.027186	0.00706
lpctymle	0.096708	1.026238e-01	-1.785723e-01	0.259973	-0.170675	-0.083677	-0.12641
lpctmin	0.079925	8.453915e-02	4.838256e-15	0.294210	-0.091289	0.045294	0.16425
ltaxpc	-0.041196	-4.538220e-02	5.201255e-01	0.218594	-0.023602	0.014066	-0.10940
lmix	-0.060461	-5.880649e-02	1.013102e-02	-0.050270	0.423923	0.154635	0.07614

	rownames	county	year	crmrte	prbarr	prbconv	prbpri
Scaled rownames	1.000000	9.994803e-01	1.099716e-02	0.037209	-0.020628	0.119920	-0.02418

44 rows × 44 columns



In [63]:
```
numeric = clean.select_dtypes(include=np.number)

correlation = numeric.corr()

if len(correlation.columns) >= 2:
    pairs = correlation.where(
        np.triu(np.ones(correlation.shape), k=1).astype(bool)
    ).stack().reset_index()

    pairs.columns = ["Variable 1", "Variable 2", "Correlation"]
    pairs["Absolute Correlation"] = pairs["Correlation"].abs()

    display(pairs.sort_values("Absolute Correlation", ascending=False))
```

	Variable 1	Variable 2	Correlation	Absolute Correlation
43	rownames	Scaled rownames	1.000000	1.000000
87	county	Scaled rownames	0.999480	0.999480
1	rownames	county	0.999480	0.999480
918	wloc	lwloc	0.995340	0.995340
828	wfed	lwfed	0.994538	0.994538
...	...	...	...	...
1931	Scaled rownames	lpctymle	NaN	NaN
1932	Scaled rownames	lpctmin	NaN	NaN
1933	Scaled rownames	ltaxpc	NaN	NaN
1934	Scaled rownames	lmix	NaN	NaN
1935	Scaled rownames	Scaled rownames	NaN	NaN

1936 rows × 4 columns

In [64]:
```
print("Assessment 8 – Clustering analysis")
```

```
Assessment 8 – Clustering analysis
```

In [66]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

numeric = clean.select_dtypes(include=np.number)
clusterdata = numeric.dropna()

scaler = StandardScaler()
scaleddata = scaler.fit_transform(clusterdata)

print("Data shape for clustering:", scaleddata.shape)
```

```
Data shape for clustering: (630, 44)
```

In [67]:
```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

labels = kmeans.fit_predict(scaleddata)

clusterdata = clusterdata.copy()
clusterdata["Cluster"] = labels

display(clusterdata.head())
```

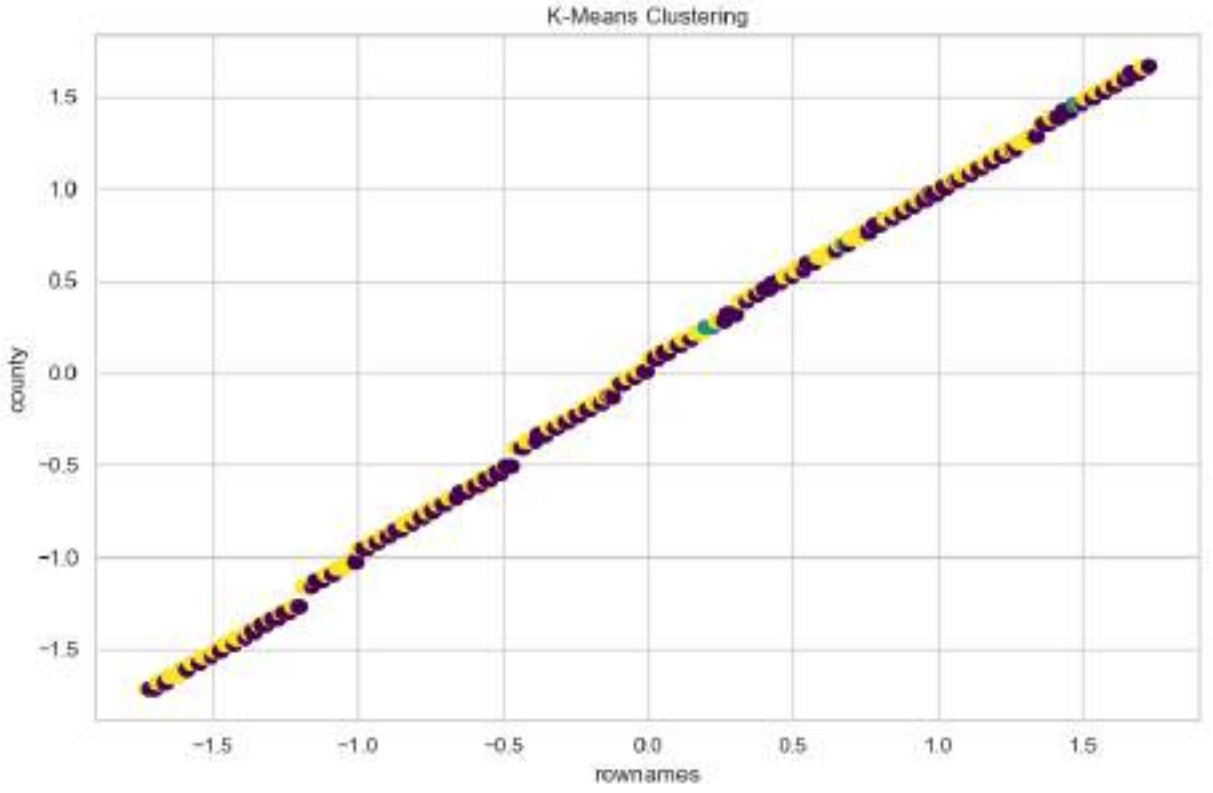
	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	dens
0	1	1	81	0.039885	0.289696	0.402062	0.472222	5.61	0.001787	2.307
1	2	1	82	0.038345	0.338111	0.433005	0.506993	5.59	0.001767	2.330
2	3	1	83	0.030305	0.330449	0.525703	0.479705	5.80	0.001836	2.341
3	4	1	84	0.034726	0.362525	0.604706	0.520104	6.89	0.001886	2.346
4	5	1	85	0.036573	0.325395	0.578723	0.497059	6.55	0.001924	2.364

5 rows × 45 columns

In [68]:
```
plt.figure(figsize=(9, 6))

plt.scatter(
    scaleddata[:, 0],
    scaleddata[:, 1],
    c=labels,
    cmap="viridis",
    s=50
)

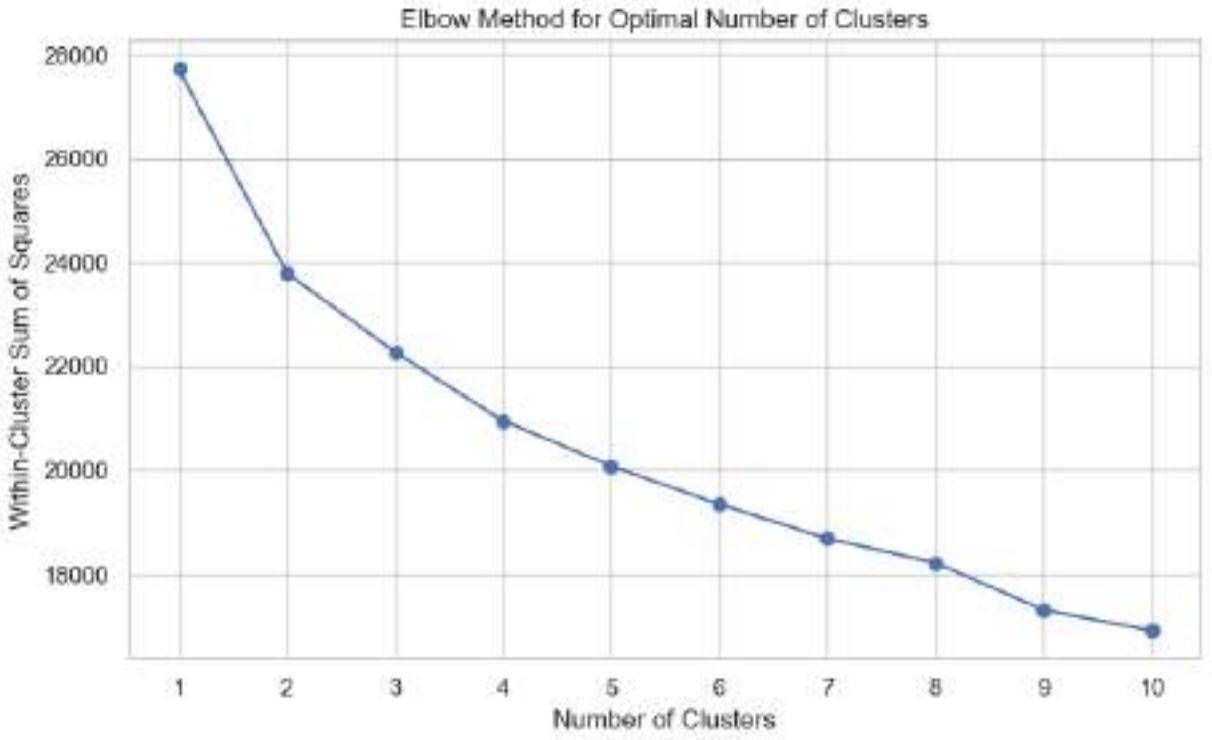
plt.xlabel(numeric.columns[0])
plt.ylabel(numeric.columns[1])
plt.title("K-Means Clustering")
plt.tight_layout()
plt.show()
```



In [69]:
```
inertia = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(scaleddata)
    inertia.append(model.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), inertia, marker="o")
plt.title("Elbow Method for Optimal Number of Clusters")
plt.xlabel("Number of Clusters")
plt.ylabel("Within-Cluster Sum of Squares")
plt.xticks(range(1, 11))
plt.tight_layout()
plt.show()
```



In [70]:
```
clusterdata = clusterdata.copy()

print("Number of observations in each cluster:")
display(clusterdata["Cluster"].value_counts().sort_index().to_frame("Count"))

print("Mean values by cluster:")
display(clusterdata.groupby("Cluster").mean())
```

```
Number of observations in each cluster:
```

Cluster	Count
0	280
1	16
2	334

Mean values by cluster:

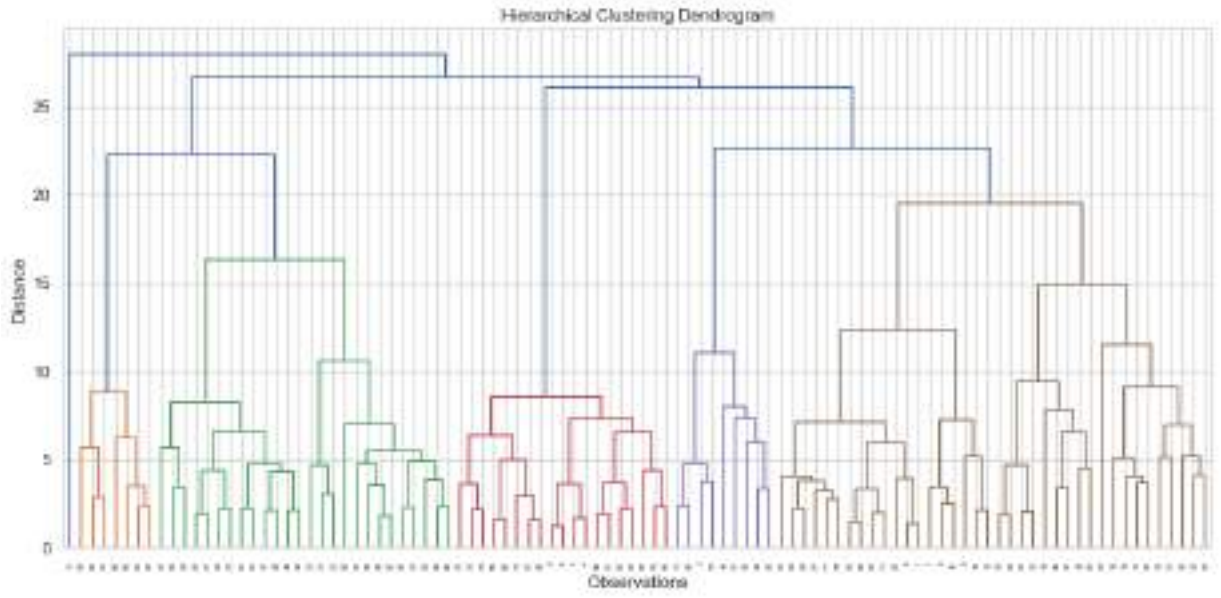
Cluster	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen
0	317.550000	100.750000	85.400000	0.038405	0.270830	0.498001	0.429762	8.946250
1	448.625000	143.875000	83.687500	0.026096	0.827790	6.331143	0.402764	10.845000
2	307.404192	98.401198	82.841317	0.026136	0.313068	0.578116	0.423051	8.870928

3 rows × 44 columns

In [71]:
```
sample = scaleddata[:min(100, len(scaleddata))]

linked = linkage(sample, method="ward")

plt.figure(figsize=(12, 6))
dendrogram(linked)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Observations")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```



In [72]:
```
hierarchicallabels = fcluster(linked, t=3, criterion="maxclust")

hierarchicaldata = clusterdata.iloc[:len(sample)].copy()
hierarchicaldata["Hierarchical Cluster"] = hierarchicallabels

display(hierarchicaldata.head())
display(hierarchicaldata.groupby("Hierarchical Cluster").mean())
```

	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgsen	polpc	dens
0	1	1	81	0.039885	0.289696	0.402062	0.472222	5.61	0.001787	2.307
1	2	1	82	0.038345	0.338111	0.433005	0.506993	5.59	0.001767	2.330
2	3	1	83	0.030305	0.330449	0.525703	0.479705	5.80	0.001836	2.341
3	4	1	84	0.034726	0.362525	0.604706	0.520104	6.89	0.001886	2.346
4	5	1	85	0.036573	0.325395	0.578723	0.497059	6.55	0.001924	2.364

5 rows × 46 columns

Hierarchical Cluster	rownames	county	year	crmrte	prbarr	prbconv	prbpris	avgs
1	53.212121	15.303030	84.000000	0.021547	0.365909	0.615964	0.428020	9.0475
2	49.727273	14.090909	83.909091	0.025426	0.338676	0.457272	0.449206	7.9889
3	12.000000	3.000000	85.000000	0.012035	0.206897	0.909091	0.366667	8.5900

3 rows × 45 columns

In [73]:
```
print("Assessment 9 – Principal Component Analysis (PCA)")
```

```
Assessment 9 – Principal Component Analysis (PCA)
```

In [74]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error

numeric = clean.select_dtypes(include=np.number)
pcadata = numeric.dropna()

scaler = StandardScaler()
scaleddata = scaler.fit_transform(pcadata)

print("Original data shape:", scaleddata.shape)
```

```
Original data shape: (630, 44)
```

In [75]:
```
pca = PCA()
scores = pca.fit_transform(scaleddata)

print("Principal component scores:")
display(pd.DataFrame(
    scores,
    columns=["PC" + str(i + 1) for i in range(scores.shape[1])],
    index=pcadata.index
).head())
```

Principal component scores:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	
0	-1.078450	2.315083	-2.243323	1.916561	0.394553	1.081855	-1.581796	-1.340335	-0.807
1	-0.349419	1.819462	-2.410834	2.128495	0.097121	0.731586	-1.480215	-1.501310	-0.700
2	0.722118	1.107510	-3.660103	1.482255	1.874125	-0.485380	-2.944233	-0.576768	3.051
3	1.389241	0.857201	-2.529012	1.952728	0.240995	0.054538	-1.093522	-0.982828	-0.923
4	2.731187	0.360866	-2.488829	1.776145	-0.101961	0.370542	-1.005423	-1.168872	-1.193

5 rows × 44 columns

In [76]:
```
variance = pca.explained_variance_ratio_

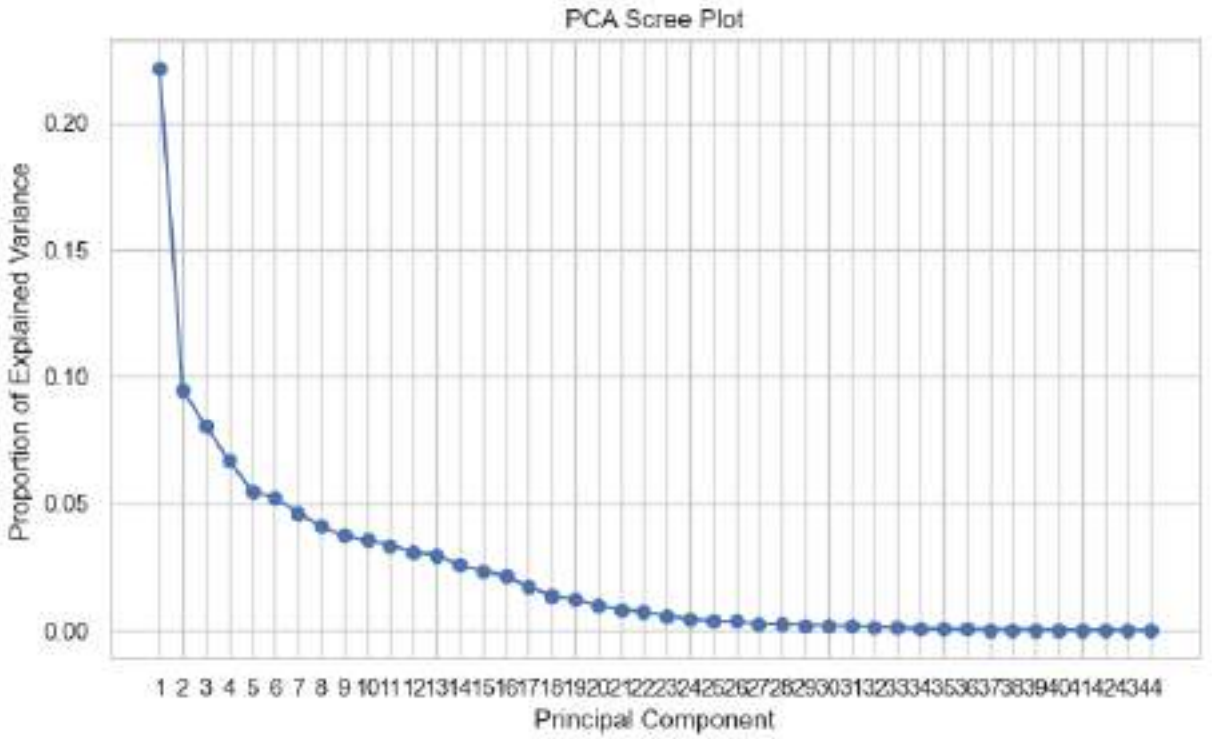
variance_table = pd.DataFrame({
    "Principal Component": ["PC" + str(i + 1) for i in range(len(variance))],
    "Explained Variance": variance,
    "Cumulative Variance": np.cumsum(variance)
})

display(variance_table)
```

	Principal Component	Explained Variance	Cumulative Variance
0	PC1	0.221184	0.221184
1	PC2	0.094653	0.315837
2	PC3	0.080546	0.396383
3	PC4	0.066924	0.463307
4	PC5	0.055061	0.518368
5	PC6	0.052150	0.570518
6	PC7	0.046426	0.616944
7	PC8	0.041079	0.658023
8	PC9	0.037330	0.695353
9	PC10	0.035726	0.731080
10	PC11	0.033539	0.764618
11	PC12	0.031055	0.795673
12	PC13	0.029757	0.825430
13	PC14	0.025850	0.851280
14	PC15	0.023567	0.874847
15	PC16	0.021598	0.896445
16	PC17	0.017261	0.913706
17	PC18	0.013556	0.927262
18	PC19	0.012525	0.939788
19	PC20	0.009937	0.949724
20	PC21	0.008256	0.957980
21	PC22	0.007363	0.965344
22	PC23	0.005893	0.971236
23	PC24	0.004521	0.975757
24	PC25	0.003888	0.979645
25	PC26	0.003586	0.983231
26	PC27	0.002745	0.985976
27	PC28	0.002447	0.988423
28	PC29	0.002259	0.990682
29	PC30	0.001854	0.992537

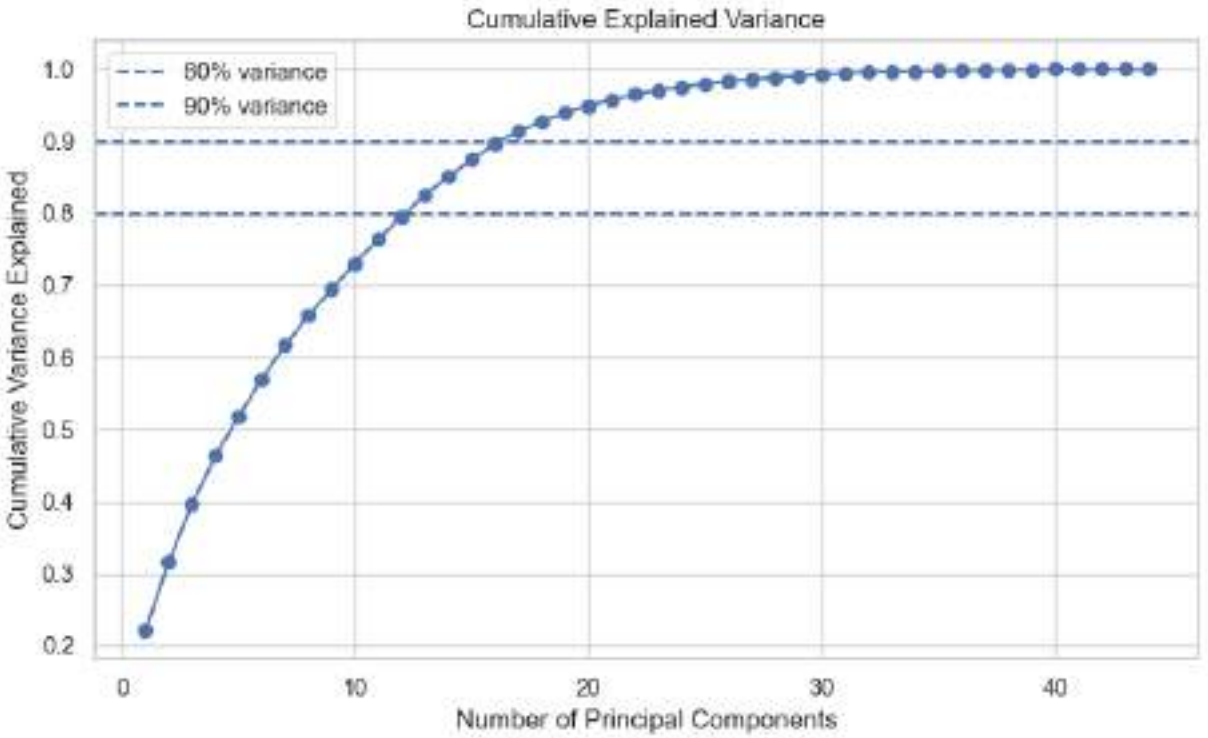
	Principal Component	Explained Variance	Cumulative Variance
30	PC31	0.001779	0.994316
31	PC32	0.001374	0.995690
32	PC33	0.001104	0.996795
33	PC34	0.000730	0.997525
34	PC35	0.000578	0.998103
35	PC36	0.000457	0.998560
36	PC37	0.000413	0.998972
37	PC38	0.000378	0.999351
38	PC39	0.000333	0.999683
39	PC40	0.000156	0.999839
40	PC41	0.000092	0.999931
41	PC42	0.000058	0.999989
42	PC43	0.000011	1.000000
43	PC44	0.000000	1.000000

In [77]:
```
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(variance) + 1), variance, marker="o")
plt.title("PCA Scree Plot")
plt.xlabel("Principal Component")
plt.ylabel("Proportion of Explained Variance")
plt.xticks(range(1, len(variance) + 1))
plt.tight_layout()
plt.show()
```



In [78]:
```
plt.figure(figsize=(8, 5))
plt.plot(
    range(1, len(variance) + 1),
    np.cumsum(variance),
    marker="o"
)
plt.axhline(0.80, linestyle="--", label="80% variance")
plt.axhline(0.90, linestyle="--", label="90% variance")
plt.title("Cumulative Explained Variance")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Variance Explained")
plt.legend()
plt.tight_layout()
plt.show()

print("Components for 80% variance:", np.argmax(np.cumsum(variance) >= 0.80) + 1)
print("Components for 90% variance:", np.argmax(np.cumsum(variance) >= 0.90) + 1)
```




```
Components for 80% variance: 13
Components for 90% variance: 17
```

In [79]:
```
loadings = pd.DataFrame(
    pca.components_.T,
    index=pcadata.columns,
    columns=["PC" + str(i + 1) for i in range(len(pcadata.columns))]
)

display(loadings)
```

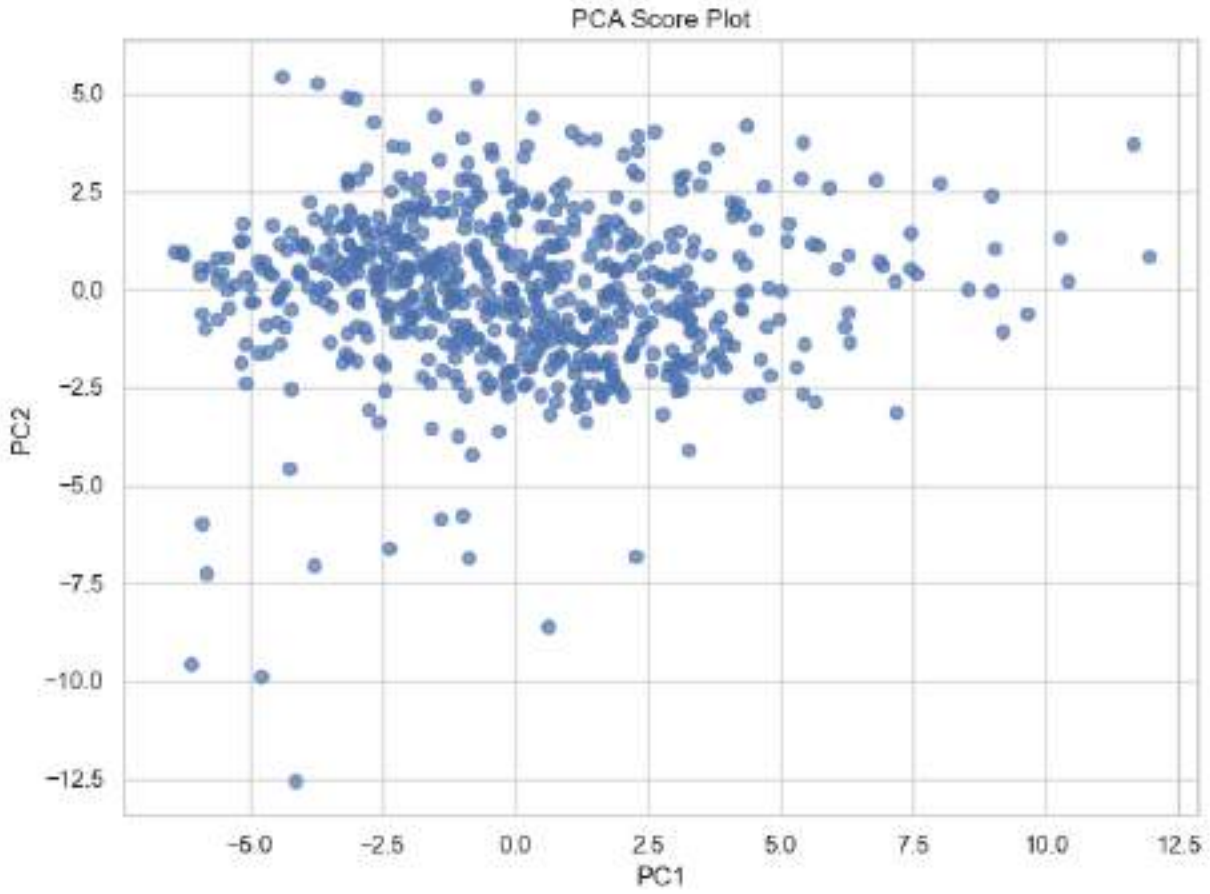
	PC1	PC2	PC3	PC4	PC5	PC6	PC7	P
rownames	-0.001686	-0.041036	0.443890	-0.150122	-0.145619	-0.221421	-0.015115	-0.037
county	-0.003915	-0.037460	0.445586	-0.147919	-0.144155	-0.219251	-0.013307	-0.038
year	0.207976	-0.254534	-0.038806	-0.092984	-0.125520	0.093060	0.033771	0.043
crmrte	0.169022	0.252846	0.160079	0.172749	0.152422	0.133041	-0.057561	0.010
prbarr	-0.092485	-0.259898	-0.039365	0.186982	0.056933	-0.104627	0.243444	-0.285
prbconv	-0.040594	-0.191850	0.181336	0.108987	0.157658	0.012954	-0.217871	0.062
prbpris	0.020970	0.139645	-0.010784	0.299486	-0.201296	-0.248993	-0.126477	0.126
avgsen	0.010148	0.026993	0.056275	-0.028432	0.251882	-0.237189	0.425487	0.361
polpc	-0.013347	-0.191533	0.208275	0.160573	0.334706	0.023390	-0.174999	-0.095
density	0.193635	0.227426	0.052792	0.121222	0.142998	-0.030048	-0.020733	-0.081
taxpc	0.143853	-0.158762	-0.005052	0.025116	0.130784	0.202445	-0.012900	0.243
pctmin	-0.005875	-0.030360	0.149258	0.324531	-0.216482	0.231588	0.101977	0.236
wcon	0.112658	-0.024798	-0.067402	-0.095102	-0.065489	-0.070619	0.016334	-0.082
wtuc	0.024503	-0.006090	-0.153432	-0.019716	0.226725	-0.123494	-0.191603	0.123
wtrd	0.133862	0.037006	-0.010512	0.094491	0.030540	-0.098783	0.060733	-0.069
wfir	0.258463	-0.077795	-0.028056	-0.005708	-0.028745	-0.069484	0.084601	-0.060
wser	0.112939	-0.032162	-0.003485	-0.045138	-0.098166	0.080746	-0.109286	-0.063
wmfg	0.228967	-0.046267	-0.017423	-0.004381	0.140945	-0.097386	0.020482	-0.047
wfed	0.268621	0.027537	-0.001693	0.090251	-0.031673	-0.090885	-0.038738	-0.056

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	P
wsta	0.201551	-0.150029	0.075030	-0.114234	-0.089281	0.168803	0.091387	0.057
wloc	0.266477	-0.146280	-0.012373	-0.046292	-0.103603	0.016343	0.019992	0.021
mix	-0.062070	-0.192285	0.124923	0.285369	0.053943	0.049962	0.036104	-0.140
pctymle	-0.006672	0.208603	0.173362	-0.132535	0.126829	0.279363	0.211148	-0.182
lcrmrte	0.161668	0.303766	0.126721	0.150856	0.048409	0.143962	-0.049189	0.025
lprbarr	-0.104415	-0.235121	-0.100547	0.209793	-0.055320	-0.100749	0.263093	-0.263
lprbconv	-0.069350	-0.255696	0.145388	-0.039864	0.095455	-0.017455	-0.255561	0.185
lprbpris	0.030123	0.141947	-0.010869	0.302845	-0.203997	-0.247246	-0.120257	0.114
lavgsen	0.019766	0.043984	0.058848	-0.024854	0.235041	-0.233813	0.437218	0.365
lpolpc	0.048460	-0.140286	0.217925	0.160356	0.391080	0.018946	-0.136409	-0.131
ldensity	0.205698	0.276786	0.047217	0.063022	0.072565	-0.090157	-0.026386	-0.158
lwcon	0.206529	-0.025683	-0.076864	-0.109030	-0.068914	-0.065537	0.034166	-0.070
lwtuc	0.089309	-0.015681	-0.142590	-0.030517	0.219822	-0.161839	-0.199963	0.088
lwtrd	0.211542	0.021461	-0.035766	0.114936	-0.003129	-0.120550	0.076646	-0.092
lwfir	0.193514	-0.070216	-0.034392	-0.004917	-0.015105	-0.048287	0.104856	-0.079
lwser	0.142406	-0.018650	-0.025674	-0.048304	-0.051026	0.085623	-0.102231	-0.117
lwmfg	0.239490	-0.034451	-0.022345	0.006230	0.113697	-0.111492	0.025996	-0.062
lwfed	0.262320	0.022431	-0.004986	0.094732	-0.042757	-0.095066	-0.046345	-0.052
lwsta	0.200005	-0.156602	0.071728	-0.110487	-0.093528	0.166983	0.089498	0.055

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	I
lwloc	0.265495	-0.143363	-0.014255	-0.043771	-0.109148	0.016374	0.012284	0.018
lpctymle	-0.002755	0.226810	0.181714	-0.123329	0.130751	0.281050	0.202995	-0.180
lpctmin	0.034215	0.047316	0.154675	0.295178	-0.234400	0.222929	0.043181	0.267
ltaxpc	0.163047	-0.184811	0.004327	0.033226	0.098453	0.209280	-0.008217	0.235
lmix	-0.052879	-0.135766	0.022663	0.350220	-0.031822	0.037887	0.220742	-0.171
Scaled rownames	-0.001686	-0.041036	0.443890	-0.150122	-0.145619	-0.221421	-0.015115	-0.037

44 rows × 44 columns

In [80]:
```
if scores.shape[1] >= 2:
    plt.figure(figsize=(8, 6))
    plt.scatter(scores[:, 0], scores[:, 1], alpha=0.7)
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("PCA Score Plot")
    plt.tight_layout()
    plt.show()
```



In [81]:
```
componentcount = min(2, scaleddata.shape[1])

reducedpca = PCA(n_components=componentcount)
reducedscores = reducedpca.fit_transform(scaleddata)
reconstructed = reducedpca.inverse_transform(reducedscores)

error = mean_squared_error(scaleddata, reconstructed)

print("Number of components:", componentcount)
print("Reconstruction Mean Squared Error:", error)
print("Variance retained:", reducedpca.explained_variance_ratio_.sum())
```

```
Number of components: 2
Reconstruction Mean Squared Error: 0.6841629437798263
Variance retained: 0.31583705622017355
```

In [82]:
```
print("Original number of numerical variables:", scaleddata.shape[1])
print("Reduced number of components:", componentcount)
print("Original data dimensions:", scaleddata.shape)
print("Reduced data dimensions:", reducedscores.shape)
```

```
Original number of numerical variables: 44
Reduced number of components: 2
Original data dimensions: (630, 44)
Reduced data dimensions: (630, 2)
```